

M. Cristina C. Cunha

MÉTODOS NUMÉRICOS

EDITOR A UNICAMP

FICHA CATALOGRÁFICA ELABORADA PELA
BIBLIOTECA CENTRAL DA UNICAMP

Cunha, Cristina

C914m Métodos numéricos / M. Cristina C. Cunha. – 2ª ed. rev.
2ª ed. ampl. – Campinas, SP: Editora da UNICAMP, 2003.

(Coleção Livro-Texto)

1. Métodos numéricos. 2. MATLAB (programa de computador).
3. Engenharia – Modelos matemáticos. 4. Tecnologia – Modelos
matemáticos. 5. Análise numérica I. Título.

CDD – 620.00151
– 001.6424
– 620.0042
– 519.4

ISBN: 85-268-0636-X

Índices para catálogo sistemático:

1. Métodos numéricos	620.00151
2. MATLAB (programa de computador)	001.6424
3. Engenharia – Modelos matemáticos	620.0042
4. Tecnologia – Modelos matemáticos	519.4

Copyright © by Editora da UNICAMP, 2003

Nenhuma parte desta publicação pode ser gravada, armazenada em
sistema eletrônico, fotocopiada, reproduzida por meios mecânicos ou
outros quaisquer sem autorização prévia do editor.



Dedico este trabalho aos amigos com os quais compartilho
esta incrível aventura que é viver...

Em especial,
nas origens: aos meus pais Miguel e Oiléa;
no presente: ao companheiro Euclides;
sempre: aos filhotes Paulinho, Dudu e Gustavo.

A stylized handwritten signature, possibly reading 'Thaí', enclosed within a large, loopy oval.

10/09/2018
Belém/PA

SUMÁRIO

Algoritmos	11
Notações e Definições	13
Prefácio	17
1 Erros, Representação de Números	19
1.1 Origem das Incertezas	19
1.2 Erros e Representação de Números	20
1.2.1 Representação de Inteiros	21
1.2.2 Representação de Números Reais	21
1.2.3 Aritmética Ponto Flutuante	21
1.2.4 Medidas de Erros	22
1.3 Condicionamento de Algoritmos	23
1.4 Matlab	24
2 Métodos Diretos para Solução de Sistemas de Equações Lineares	29
2.1 Introdução	29
2.2 Sistemas Triangulares	30
2.3 Método de Eliminação Gaussiana	32
2.4 Estratégia do Pivoteamento	36
2.5 Matrizes Tridiagonais	38
2.6 Fatoração LU	39
2.7 Refinamento da Solução	42
2.8 Condicionamento da Matriz e Estimativas de Erro	42
2.9 Sistemas Indeterminados, Sobredeterminados e Malcondicionados: a Decomposição em Valores Singulares (SVD)	44
2.9.1 Decomposição em Valores Singulares	44
2.9.2 Sistemas Quadrados	45
2.9.3 Sistemas Retangulares	48
2.10 Uso do Matlab na Solução de Sistemas Lineares: Métodos Diretos	49
2.10.1 Sistemas Indeterminados e Sobredeterminados	51

3	Métodos Iterativos para Sistemas Lineares	57
3.1	Métodos de Jacobi e Gauss-Seidel	58
3.1.1	Método de Jacobi	58
3.1.2	Método de Gauss-Seidel	60
3.1.3	Convergência dos Métodos Jacobi e Gauss-Seidel	61
3.2	Métodos SOR	63
3.3	Método Gradiente Conjugado	64
3.3.1	Método dos Gradientes Conjugados	66
3.3.2	Pré-condicionamento e Matrizes Não Simétricas	68
3.4	Uso do Matlab na Solução de Sistemas Lineares: Matrizes Esparsas e Gradientes Conjugados	70
4	Equações Não-Lineares	73
4.1	Introdução	73
4.2	Aproximação Inicial e o Método da Bissecção	74
4.3	Método de Newton	76
4.4	Método das Secantes	78
4.5	A Convergência dos Métodos de Newton e da Secante	80
4.6	Método de Newton para Sistemas Não-Lineares	81
4.7	Uso do Matlab para Encontrar Raízes de Equações	86
5	Aproximação	93
5.1	Introdução	93
5.2	Interpolação Polinomial: Formas de Lagrange e Newton	94
5.2.1	Forma de Newton para Polinômio Interpolador	97
5.3	Interpolação por Partes	101
5.4	Quadrados Mínimos Lineares	103
5.4.1	Funções Ortogonais	109
5.5	Quadrados Mínimos Não-Lineares	113
5.5.1	Método de Gauss-Newton	117
5.5.2	Método de Levenberg-Marquardt	118
5.5.3	Métodos com Aproximações Quase-Newton	119
5.6	Interpolação e Ajuste usando o Matlab	120
6	Splines e Aplicações	125
6.1	Introdução	125
6.2	Definições	126
6.3	Bases	127
6.4	Uso de Splines na Interpolação: Interpolação por Partes	131
6.5	Ajuste de Curvas usando Splines	135

57	6.6	Exemplos de Aplicação de Splines em Equações Diferenciais: os Métodos da	
58		Colocação e Elementos Finitos	137
58	6.6.1	Colocação: um Exemplo	137
60	6.6.2	Elementos Finitos: um Exemplo	140
61	6.7	Splines com Ajuda do Matlab	141
63			
64	7	Integração Numérica	147
66	7.1	Introdução	147
68	7.2	Fórmulas de Newton-Cotes	148
	7.2.1	Fórmula dos Trapézios: $n = 1$	149
70	7.2.2	Fórmula de Simpson: $n = 2$	150
	7.2.3	Fórmulas para $n = 3$ e $n = 4$	151
73	7.2.4	Fórmulas Repetidas	153
73	7.3	Integração de Romberg	157
74	7.3.1	Fórmulas Gaussianas	160
76	7.3.2	Integrais Múltiplas	165
78	7.4	Uso do Matlab na Integração Numérica	167
80			
81	8	Soluções Aproximadas para Equações Diferenciais Ordinárias	173
86	8.1	Introdução	173
	8.2	Método das Diferenças Finitas	174
93	8.3	Método da Série de Taylor	180
93	8.4	Métodos de Runge-Kutta	185
94	8.5	Conceito de Estabilidade	189
97	8.6	Uso do Matlab em Equações Diferenciais Ordinárias	193
101			
103	9	Equações Diferenciais Parciais: Métodos para Soluções Suaves	199
109	9.1	Introdução	199
113	9.2	Definição da Malha e Discretização do Laplaciano	204
117	9.3	Equação de Poisson com Dados de Contorno	208
118	9.4	A Equação do Calor	218
119	9.5	A Equação da Onda	233
120			
125	10	Métodos para Equações Diferenciais Parciais com Predominância Con- vectiva	241
125	10.1	Introdução	241
126	10.2	Método das Características	241
127	10.3	Métodos de Discretização para Equações Lineares	246
131	10.4	Soluções Descontínuas: Equações Modificadas	249
135	10.5	Problemas Não-Lineares: Leis de Conservação	251
	10.6	Método TVD: um Exemplo de Métodos de Alta Resolução	255

Referências Bibliográficas

261

Anexo

267

ALGORITMOS

1.1	Precisão da máquina	26
2.1	Solução de sistema triangular superior	31
2.2	Eliminação Gaussiana	35
2.3	Eliminação Gaussiana com pivoteamento	38
2.4	Solução de sistema tridiagonal	39
2.5	Fatoração LU	40
2.6	Fatoração de Cholesky	41
3.1	Método de Jacobi	59
3.2	Método de Gauss-Seidel	61
3.3	Método SOR	63
3.4	Método Gradiente	66
3.5	Método Gradiente Conjugado	67
3.6	Método Gradiente Conjugado com Pré-Condicionamento	69
4.1	Método da Bissecção	75
4.2	Método de Newton	78
4.3	Método das Secantes	79
4.4	Método de Newton para sistemas	84
5.1	Tabela de diferenças divididas	100
5.2	Método de Gauss-Newton	117
7.1	Integração por trapézios repetidos	154
7.2	Integração por Simpson repetida	154
8.1	Método de Diferenças Finitas para a equação de Sturm-Liouville	179
8.2	Runge-Kutta de segunda ordem	187
8.3	Runge-Kutta de terceira ordem	187
8.4	Runge-Kutta de quarta ordem	187
8.5	Runge-Kutta de quarta ordem para $y'' = f(x, y, y')$	190
9.1	Discretização da equação de Poisson	213
9.2	Método explícito para a equação do calor com condição de Dirichlet	220
9.3	Método de Crank-Nicolson	225
9.4	Discretização da equação para pressão no <i>five-spot</i>	232
9.5	Método Explícito para equação da onda	237

NOTAÇÕES E DEFINIÇÕES

$i = 1 : N$ é equivalente a $i = 1, 2, \dots, N$.

$A = [a_{ij}]$ matriz cujo elemento da i -ésima linha e j -ésima coluna é a_{ij} (maiúsculas em negrito denotam matrizes).

$A_{m \times n}$ matriz com m linhas e n colunas.

$\mathbf{x}$ vetor coluna com n componentes $x_1, x_2, \dots, x_n$ (minúsculas em negrito denotam vetores).

A^T transposta da matriz A , isto é, $A^T = [a_{ji}]$.

$\mathbf{x}^T$ vetor linha com n componentes $x_1, x_2, \dots, x_n$.

A^{-1} inversa da matriz A ($AA^{-1} = A^{-1}A = I$)

I matriz identidade ($a_{ii} = 1$ e $a_{ij} = 0, i \neq j$).

$\text{diag}(\sigma_1, \dots, \sigma_n)$ = matriz diagonal, com elementos σ_i .

$f(\mathbf{x})$ função escalar de uma variável vetorial $f(\mathbf{x}) = f(x_1, x_2, \dots, x_n)$.

$\mathbf{f}(\mathbf{x})$ função vetorial de variável vetorial $\mathbf{f}(\mathbf{x}) = (f_1(\mathbf{x}), \dots, f_m(\mathbf{x}))$.

$u_{x_i} = \frac{\partial u}{\partial x_i}$ derivada parcial de $u(x_1, x_2, \dots, x_n)$ com relação a x_i .

$J(\mathbf{x})$ matriz Jacobiana de $\mathbf{f}(\mathbf{x})$: $J_{ij} = \frac{\partial f_i(\mathbf{x})}{\partial x_j}$.

$\nabla f = \text{grad } f(\mathbf{x})$ vetor com componentes $\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n}$.

$\|x\|$ norma de um vetor x . Algumas normas usuais:

$$\|x\|_2 = (x_1^2 + x_2^2 + \dots + x_n^2)^{\frac{1}{2}} = \left(\sum_{i=1}^n x_i^2 \right)^{\frac{1}{2}}$$

$$\|x\|_\infty = \max\{|x_1|, |x_2|, \dots, |x_n|\} = \max_{1 \leq i \leq n} |x_i|$$

$$\|x\|_1 = |x_1| + |x_2| + \dots + |x_n| = \sum_{i=1}^n |x_i|.$$

$\|A\|$ norma de uma matriz A . Algumas normas usuais:

$$\|A\|_\infty = \max_{1 \leq i \leq n} \{|a_{i1}| + |a_{i2}| + \dots + |a_{in}|\} = \max_{1 \leq i \leq n} \left(\sum_{j=1}^n |a_{ij}| \right)$$

$$\|A\|_2 = \max_{\|x\|_2=1} \|Ax\|_2$$

$$\|A\|_1 = \max_{1 \leq j \leq n} \{|a_{1j}| + |a_{2j}| + \dots + |a_{nj}|\} = \max_{1 \leq j \leq n} \left(\sum_{i=1}^n |a_{ij}| \right)$$

Autovalor λ de A : $Ax = \lambda x$ para algum autovetor $x \neq 0$.

$\rho(A)$ = raio espectral de $A = \max\{|\lambda| \text{ onde } \lambda \text{ é autovalor de } A\}$.

$\mathcal{O}(\bullet)$ ordem de aproximação ou de convergência: $f = \mathcal{O}(g)$ quando $x \rightarrow a$ significa $|f| \leq k|g|$ quando $x \rightarrow a$, para alguma constante k .

$\text{cond}(A) = \|A\| \|A^{-1}\|$ = número de condição de uma matriz não singular A relativo à norma $\|A\|$.

Obs.: $\text{cond}(A) \leq 1$. Se $\text{cond}(A) \approx 1$, A é bem condicionada e, se $\text{cond}(A)$ é muito grande, dizemos que A é mal condicionada.

$\langle f, g \rangle$ = produto escalar entre duas funções:

$$\langle f, g \rangle = \sum_{i=1}^m w(x_i) f(x_i) g(x_i) \quad \text{caso discreto}$$

$$\langle f, g \rangle = \int_a^b w(x) f(x) g(x) \quad \text{caso contínuo}$$

onde $w(x) > 0$ é a função peso.

(u, v) = produto escalar entre vetores:

$$(u, v) = \sum_{i=1}^n u_i v_i.$$

Usamos a notação de calculadoras científicas para os números reais, com ponto separando a parte decimal. Assim,

$$2.354e(-2) = 2.354 \times 10^{-2} = 0.02354,$$

que representa o número 0,02354, na convenção brasileira.

□ = símbolo usado no final de exemplos.

PREFÁCIO

Por que escrever um livro se existem tantos prontos? Imagino que esta pergunta persegue todos que resolvem se dedicar a um projeto deste. Talvez a resposta esteja no desejo de transmitir uma experiência pessoal, na expectativa de que ela seja útil para alguém.

Este livro foi escrito com base na experiência de lecionar anualmente Métodos Numéricos no curso de Pós Graduação em Engenharia do Petróleo, desde 1987.

A primeira edição é de 1993. Reescrevê-lo é uma tentativa de publicar uma versão melhorada em alguns aspectos. Em primeiro lugar, espero ter diminuído o número de erros de digitação e de conteúdo. Também, ao passar para um editor mais moderno, melhoramos a apresentação.

Outras modificações aparecem nesta nova versão.

Em cada capítulo, introduzimos uma seção sobre uso e aplicação do pacote computacional Matlab. O objetivo é tentar colaborar na divulgação e utilização de pacotes computacionais como ferramenta auxiliar em tarefas que demandam métodos numéricos. A combinação de métodos numéricos com métodos matemáticos é uma fonte de desafios e descobertas. O aperfeiçoamento dos pacotes tem nos permitido realizar cálculos rápidos, testar com facilidade a influência dos parâmetros presentes nos modelos, testar simplificações, diferentes discretizações etc. Não é por acaso que se tornam mais freqüentes expressões como experimentos computacionais, laboratórios computacionais e matemática numérica. Alguns exemplos foram resolvidos usando programas na linguagem Matlab; incluímos alguns destes programas no Apêndice, ou nas seções correspondentes à utilização do Matlab no assunto do capítulo.

Ampliamos o tópico "Métodos numéricos para equações diferenciais parciais", incluindo alguns métodos destinados às equações cujas soluções apresentam descontinuidades ou variações bruscas, cada vez mais freqüentes nas aplicações. O assunto foi então desdobrado em dois capítulos. Também desdobramos o capítulo de métodos para sistemas de equações lineares em dois, separando os métodos diretos dos métodos iterativos.

Algumas notas históricas foram incluídas com o objetivo de fornecer parte da evolução histórica dos métodos numéricos. Gostaríamos de ter mais informações históricas, mas não as conseguimos no tempo disponível. Como veremos, alguns métodos são muito antigos, como por exemplo o método de Newton (1687), e outros são mais recentes, como o método TVD, para leis de conservação, que é da década de 80 deste século.

O objetivo do livro continua o mesmo: apresentar os métodos que têm sido mais usados e que devem continuar úteis num futuro próximo. Como a utilização de um método numérico depende de sua eficiência, o desenvolvimento tecnológico dos computadores tem feito a

seleção dos melhores métodos.

Tentamos apresentar nosso texto da maneira mais objetiva e clara possível. Evitamos, sempre que possível, a notação matemática carregada, na tentativa de ampliar o público alvo: usuários de métodos numéricos em suas tarefas profissionais. Poucas demonstrações são apresentadas, mas nas referências os interessados podem encontrar mais detalhes matemáticos do assunto. Muitos algoritmos foram incluídos para permitir uma programação rápida do método.

Seria impossível agradecer a todas as pessoas que me ajudaram e incentivaram neste trabalho. Mas seria injusto deixar de agradecer algumas pessoas. Agradeço aos colegas do Departamento de Engenharia do Petróleo, em especial Bonet e Antônio Cláudio, aos colegas do Departamento de Matemática Aplicada, em especial Valéria, Silvio e Sônia, Anamaria e Suely, sempre me incentivando nesta tarefa. Agradeço ao Euclides pelas inúmeras e frutíferas discussões no dia-a-dia desta versão. Não teria palavras para agradecer a Cantão e Luiza pelo trabalho e dedicação na digitação; sem eles, esta versão do livro dificilmente sairia. Agradeço ao Pronex do Petróleo pelo suporte financeiro. Também devo agradecimentos à Fátima, secretária do Departamento de Matemática Aplicada, que digitou a primeira edição, fonte para a atual, e que me auxiliou em muitas tarefas desta segunda edição.

Campinas, setembro de 2000.

Capítulo 1

ERROS, REPRESENTAÇÃO DE NÚMEROS

1.1 Origem das Incertezas

A utilização de simuladores matemáticos ou numéricos requer a execução de uma sequência de etapas que procuramos descrever brevemente nesta seção.

Inicialmente, é necessário equacionar o fenômeno físico que pretendemos estudar, isto é, decidir quais são as variáveis relevantes e como elas se relacionam. Por exemplo, para descrever o movimento de fluidos, em geral são usadas as componentes da velocidade e a pressão em cada ponto da região considerada. As leis de conservação (da massa, da quantidade de movimento, da energia etc.), bem como as equações constitutivas, são requisitadas para estabelecer as relações que existem entre as variáveis escolhidas. Como nem tudo que influi no fenômeno pode ser levado em consideração, algumas simplificações são introduzidas na modelagem já neste estágio.

Uma vez estabelecidas as equações que descrevem matematicamente o nosso objeto de estudo, o passo seguinte será resolvê-las. Aqui, a aspiração maior seria obter soluções através de expressões analíticas, de preferência envolvendo poucos cálculos na sua avaliação. Infelizmente, nem sempre no estágio de desenvolvimento da matemática as técnicas disponíveis nos permitem atingir este alvo. Devemos então optar por simplificações no equacionamento ou utilizar métodos numéricos para calcular uma aproximação para a solução desejada. Na escolha da solução numérica, abre-se um novo leque de procedimentos alternativos, e a cada um deles está associada uma nova fonte de erros.

Uma vez decidido o método numérico adequado, torna-se necessário sua implementação computacional para que tenhamos resultados palpáveis. Como a máquina trabalha com um número finito de dígitos, insuficientes para representar os números reais, novos erros serão introduzidos na simulação.

O conjunto de todas estas incertezas certamente vai contaminar o resultado final. Se esta contaminação é desprezível, ou se ela compromete o resultado, é uma análise necessária na validação de todo processo adotado. Assim, torna-se necessário que tenhamos algum controle sobre os erros que foram sendo introduzidos no processo de simulação e como eles se propagam.

Como nestas notas pretendemos abordar apenas alguns métodos numéricos, na seção seguinte caracterizaremos alguns tipos de erros que ocorrem com frequência na solução

numérica de problemas matemáticos.

1.2 Erros e Representação de Números

Podemos caracterizar os erros que aparecem no processo de simulação numérica localizando suas fontes. Chamamos *erro inicial* à soma das incertezas introduzidas no equacionamento do problema, na medição dos parâmetros, nas condições iniciais etc. A influência destas perturbações no resultado final vai depender da estabilidade do problema. Em algumas situações, conhecidas como problemas malpostos ou malcondicionados, sua influência pode ser desastrosa e inviabilizar completamente soluções numéricas. Neste caso, são necessárias técnicas especializadas, objeto de uma área da matemática que se encontra em pleno progresso: *Problemas Inversos*.

Chamamos de *erro de truncamento* aquele associado ao truncamento de um processo infinito. Explicando melhor: como um processo infinito não se conclui, em geral somos forçados a adotar uma aproximação obtida após a execução de número finito de passos.

Vamos apresentar dois exemplos clássicos que ilustram fontes de erro de truncamento: a utilização de séries no cálculo de funções e o uso de diferenças finitas para aproximar derivadas.

Exemplo 1.2.1. Para calcular o valor de $e^{0.5}$ podemos lançar mão da série de Taylor da função exponencial

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \cdots + \frac{x^n}{n!} + \cdots$$

Entretanto, no cálculo efetivo de $e^{0.5}$, precisamos *truncar* a série, usando apenas um número finito de termos desta série. Por exemplo, usando os cinco primeiros termos como aproximação, o erro de truncamento será $\alpha^6/6!$, $0 \leq \alpha \leq 0.5$, como prediz a expressão do erro da série de Taylor. $\square$

Exemplo 1.2.2. A derivada de uma função $f(x)$ pode ser calculada pela diferença centrada:

$$f'(x) \cong \frac{f(x+h) - f(x-h)}{2h}$$

Veremos que o erro de truncamento da fórmula de diferenças centradas é $h^2 f''(\alpha)$, $x-h < \alpha < x+h$. $\square$

Definimos como *erro de arredondamento* a soma das incertezas associadas à representação do sistema de numeração na máquina calculadora. Para melhor entender como ocorre o erro de arredondamento, vejamos como se dá a representação dos números no computador.

Os cálculos nos computadores são efetuados com base nos pulsos elétricos. Assim, só dois estados podem ocorrer: presença e ausência de corrente elétrica. Assim, é conveniente

a representação binária de números, uma vez que na base 2 só dígitos 0 e 1 são utilizados. Também por conveniência os números inteiros são representados de forma diferente daquela dos números reais.

1.2.1 Representação de Inteiros

Um número inteiro é representado pelos coeficientes de sua expansão binária. Dessa forma, consideremos a expansão binária do número inteiro N :

$$N = a_n 2^n + a_{n-1} 2^{n-1} + \dots + a_1 2^1 + a_0 2^0,$$

onde $a_n = 1$ e $a_{n-1}, \dots, a_0$ são 0 ou 1. A representação binária de N será

$$N = (a_n a_{n-1} \dots a_1 a_0).$$

Por exemplo, como

$$12 = 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 0 \times 2^0,$$

este número será representado na máquina por 1100.

Como a representação binária exige muitos dígitos e sua passagem para a base octal é relativamente simples, alguns computadores usam a representação octal (ou qualquer outra potência de 2).

1.2.2 Representação de Números Reais

Se o número real x tem parte inteira x_i , sua parte fracionária $x_f = x - x_i$ pode ser escrita como uma soma de frações binárias

$$x_f = b_1 2^{-1} + b_2 2^{-2} + b_3 2^{-3} + \dots$$

Assim, o número real será representado juntando as partes inteira e fracionária, ou seja,

$$x = (a_n a_{n-1} \dots a_1 a_0 b_1 b_2 \dots).$$

Embora este tipo de representação de números pareça complicado por estarmos habituados à base decimal, existem algoritmos simples que fazem as transformações binárias-decimais sem grandes custos para a máquina.

1.2.3 Aritmética Ponto Flutuante

Em geral, a *aritmética ponto flutuante* é usada nos cálculos científicos em computadores. A representação em ponto flutuante do número x , na base β , é definida por

$$fl(x) = \pm (d_1 d_2 \dots d_n) \times \beta^e$$

onde a mantissa $(.d_1 d_2 \dots d_n)$ é a representação fracionária com n dígitos na base β , com $d_1 \neq 0$ (ou $d_i = 0$ para todo i), e o expoente e é um número inteiro, $-M < e < M$ para algum $M > 0$.

Por exemplo, na notação ponto flutuante com cinco dígitos,

$.25312 \times 10^{-1}$ representa o número .025312 na base 10

e

$.25312 \times 10^2$ representa o número 25.312 na base 10.

As limitações no expoente e na mantissa introduzem erros de arredondamento na representação dos números reais.

Se for ultrapassado o limite permitido para o expoente na representação de um número, teremos *underflow* para números muito pequenos e *overflow* para números muito grandes. Em alguns sistemas, a execução do programa é suspensa caso seja detectado *underflow* ou *overflow*. Em outros sistemas há mensagens de alerta.

Nos cálculos que exigem mais precisão, pode-se adotar o recurso da *precisão dupla*. A maioria dos computadores dispõe deste recurso e, em geral, a precisão dupla permite que a mantissa seja representada com o dobro do número de dígitos da precisão simples. Assim, os efeitos dos erros de arredondamento são amenizados.

Com a representação em ponto flutuante, a aritmética do computador se processa num conjunto finito de pontos da reta. Embora as parcelas carreguem o erro associado às operações, realizadas em ponto flutuante, as calculadoras são programadas de modo que o resultado apresentado seja a representação em ponto flutuante do valor da operação realizada com mais dígitos na mantissa. Dessa maneira, procura-se diminuir o efeito dos erros de arredondamento nas operações.

1.2.4 Medidas de Erros

Tendo em vista que, na aplicação dos métodos numéricos, trabalhamos com aproximações, precisamos estabelecer algumas maneiras de medir ou delimitar o erro de valores aproximados.

Se v é uma aproximação do valor exato u , definimos o *erro absoluto* da aproximação

$$|v - u|$$

e o *erro relativo* da aproximação

$$\left| \frac{v - u}{u} \right|$$

É comum apresentar o erro relativo em percentual, o que é obtido multiplicando esta última expressão por 100.

Embora o erro relativo forneça mais informações sobre o "peso" do erro, seu cálculo envolve o conhecimento do valor exato u . Se o erro relativo é pequeno, ele é próximo de

$|(v - u)/v|$, que pode ser calculado se tivermos uma limitação para o erro absoluto. Isto pode ser verificado chamando $\alpha = (u - v)/u$ e verificando que $(u - v)/v = \alpha/(1 - \alpha) \approx \alpha$, se $\alpha \approx 0$.

Concluimos esta seção destacando as fontes de erros nos cálculos numéricos:

- E_T : truncamento do processo de cálculo;
- E_R : arredondamento das operações realizadas no cálculo.

As influências destas duas fontes de erros no resultado final são conflitantes. Ao diminuir o erro de truncamento do processo, em geral aumentamos o número de operações ou trabalhamos com valores muito próximos. Em ambos os casos, será inevitável o aumento no erro de arredondamento no resultado final.

O equilíbrio entre as influências dos erros de truncamento e arredondamento depende do problema e da habilidade humana.

1.3 Condicionamento de Algoritmos

Nesta seção, apresentamos algumas palavras sobre bom/mau condicionamento de algoritmos.

A noção de problemas bem-postos foi formalizada pelo matemático Hadamard [Hadamard, 1902], no início do século XX. Segundo esta noção, se um problema tem uma solução única e se pequenas perturbações nos dados de entrada provocam pequenas perturbações nos resultados, então este problema é bem-posto. Esta última condição é chamada estabilidade do problema com relação aos dados.

O termo *estabilidade* tem um significado análogo quando usado para qualificar um método numérico: a estabilidade (ou instabilidade) nos dá informações sobre a sensibilidade do método aos erros de arredondamento acumulados nos cálculos.

Dizemos que um método é *estável* se pequenas perturbações nos dados conduzem a soluções próximas. Se isso não ocorre, dizemos que o método é *instável*. Algumas definições mais precisas são usadas em tópicos específicos, como é o caso das equações diferenciais ordinárias.

Pode acontecer o caso do problema matemático ser bem-posto e, no cálculo de soluções aproximadas, usarmos algoritmos instáveis. Neste caso, podemos obter maus resultados, apesar do problema matemático ser bem-posto. Como exemplo consideremos o cálculo de raízes de equações do segundo grau, descrito a seguir.

Exemplo 1.3.1. As raízes da equação $ax^2 - bx + c = 0$, onde a e b são números positivos, podem ser calculadas por

$$x_1 = \frac{b + \sqrt{b^2 - 4ac}}{2a} \quad x_2 = \frac{b - \sqrt{b^2 - 4ac}}{2a}.$$

Se $b^2 \gg 4ac$, o cálculo de x_2 envolverá a diferença de dois números próximos, o que provoca perda de dígitos significativos. A resposta sofrerá influência desta perda. Uma alternativa pode ser calcular x_1 como proposto acima e usar a propriedade de raízes de equações do segundo grau: $x_1 x_2 = c/a$.

Como ilustração numérica consideremos a equação

$$x^2 - 100.22x + 1.2371 = 0.$$

Usando aritmética ponto flutuante com cinco dígitos temos

$$\begin{aligned} b^2 &= 10044 \\ b^2 - 4ac &= 10039 \\ \sqrt{b^2 - 4ac} &= 100.19. \end{aligned}$$

Calculando as raízes pelas fórmulas apresentadas no primeiro procedimento temos

$$\begin{aligned} x_1 &= (100.22 + 100.19)/2 = 100.20 \\ x_2 &= (100.22 - 100.19)/2 = 0.015. \end{aligned}$$

Por outro lado, usando o valor de x_1 e o segundo procedimento temos

$$x_2 = \frac{c}{a x_1} = 1.2371/100.20 = 0.012346.$$

Numa calculadora com mais dígitos (16, por exemplo) e usando a segunda alternativa, teríamos

$$x_1 = 100.2076546 \quad x_2 = 0.012345364.$$

Para quantificar a diferença entre os dois procedimentos, observamos que o erro relativo da aproximação x_2 , usando o primeiro processo, é de 21.5%, ao passo que, usando o segundo procedimento, teríamos uma aproximação com erro relativo de 0.0052%. $\square$

1.4 Matlab

A partir da década de 70, o acesso aos computadores pessoais se tornou mais fácil e estas calculadoras se tornaram cada vez mais amigáveis e potentes. Uma das mais frutíferas consequências foi o surgimento e difusão de vários pacotes computacionais ambientados para micros e especialmente projetados para o cálculo científico nas engenharias e ciências aplicadas. Muitos destes pacotes integram parte gráfica e cálculos de alta precisão. Eles dispõem de funções matemáticas especializadas, implementações de métodos numéricos, feitas por especialistas, e também de linguagem própria, tornando fácil a programação usando seus recursos na computação científica.

O acesso a um pacote computacional nos permite testar rapidamente várias possibilidades, a observação gráfica imediata dos resultados e a interface com outras linguagens.

Destacando estas facilidades de uma ferramenta computacional deste porte, gostaríamos de deixar registrada uma mensagem que reforce a importância de engenheiros, físicos, matemáticos e químicos conhecerem e se familiarizarem com pelo menos um pacote destinado à computação científica.

Nestas notas vamos usar o pacote Matlab como ferramenta auxiliar nos cálculos, na comparação de resultados e em gráficos. Também apresentaremos alguns comentários de sua utilização em situações cuja programação pode ser complexa, e até mesmo inviável. A escolha do Matlab se deve a sua comprovada utilização na Matemática Aplicada, Física, Química e Engenharia. Este pacote é amplamente usado em universidades e indústrias de várias partes do mundo. O pacote está bem documentado em vários livros da literatura especializada, dos quais citamos alguns em nossa bibliografia.

O Matlab foi originalmente escrito por Cleve Moler, da MathWorks, Inc., para permitir fácil acesso ao *software* matricial desenvolvido nos projetos LINPACK e EISPACK. A versão inicial, do início da década de 70, era destinada a cursos de Álgebra Computacional. A partir desta versão, sofisticações na linguagem matricial foram sendo incorporadas e exploradas em novas funções do pacote. As últimas versões incorporam o cálculo simbólico e capacidade gráfica de alto nível. No momento em que escrevemos estas notas, a versão mais recente é a 5.3. Entretanto a maioria de nossas aplicações podem ser realizadas na versão 4.0 e seguintes, inclusive a versão estudantil.

Exercícios

1. Seja o número real $x = 10.128$ (representação decimal). Escreva sua representação em ponto flutuante, com cinco dígitos, na base 2.
2. (a) Verifique se as duas expressões abaixo podem ser usadas para calcular a abscissa da interseção da reta, que passa pelos pontos (x_0, y_0) e (x_1, y_1) , com o eixo- x :

$$x = \frac{x_0 y_1 - x_1 y_0}{y_1 - y_0} \quad \text{e} \quad x = x_0 - \frac{(x_1 - x_0) y_0}{y_1 - y_0}.$$

- (b) Usando os pontos $(1.31, 3.24)$ e $(1.93, 4.76)$ e três dígitos nos cálculos, calcule a interseção com o eixo- x usando as duas expressões. Qual método é melhor? Justifique.

3. Considere o sistema de equações

$$\begin{cases} 31.69x + 14.31y = 45.00 \\ 13.11x + 5.89y = 19.00 \end{cases}$$

cujas únicas soluções são $x = 7.2$ e $y = -12.8$. Resolva o sistema usando quatro dígitos e os métodos que você conhece. Compare e justifique os resultados.

4. Os números abaixo são fornecidos a um computador decimal que trabalha com ponto flutuante e quatro dígitos:

$$(a) 0.4523 \times 10^4 \quad (b) 0.2116 \times 10^{-3} \quad (c) 0.2583 \times 10^1$$

Nesta máquina, as operações têm arredondamento no corte dos dígitos, isto é, se o primeiro dígito a ser desprezado for maior ou igual a 5, arredondar o último dígito representativo para cima.

Qual é o resultado das seguintes operações, calculadas nesta máquina:

$$\begin{array}{ll} \text{(i)} & (a)+(b)+(c) \\ \text{(ii)} & (a)/(c) \\ \text{(iii)} & (a)-(b) \end{array} \quad \begin{array}{ll} \text{(iv)} & (a)-(b)-(c) \\ \text{(v)} & (a) (b)/(c) \\ \text{(vi)} & (b)/(c) (a) \end{array}$$

Calcule os erros relativos e absolutos destas aproximações.

5. Encontre a menor raiz (em módulo) da equação

$$x^2 + 0.4002x + 0.8 \times 10^{-4} = 0$$

usando quatro dígitos e as fórmulas apresentadas no exemplo 1.3.1. Compare com a solução exata (calculada, por exemplo, usando 16 dígitos).

6. Usando seis termos da série de Taylor da função e^x , com seis dígitos e $x = -12$, calcule e^{-12} . Compare com o valor $\frac{1}{e^{12}}$, calculado tomando $x = 12$ ainda nos seis primeiros termos da série de Taylor. Você pode explicar seus resultados? Qual método é melhor?

7. O algoritmo abaixo pode ser utilizado para calcular com quantos dígitos uma máquina trabalha.

Algoritmo 1.1 Precisão da máquina

- 1: $\epsilon = 1$
 - 2: Para $j = 1, 2, \dots$ faça
 - 3: $\epsilon = \epsilon/2$
 - 4: Se $1 + \epsilon > 1$ então
 - 5: Volte a 3
 - 6: Caso contrário
 - 7: Escreva o valor de j
-

Explique por que o algoritmo funciona.

8. Qual é a representação do número $x = 517314618350$ em um computador que usa sete dígitos, se:

- (a) Ele for representado como um número inteiro na base 10.
- (b) Ele for representado em ponto flutuante, precisão simples, isto é, sete dígitos.
- (c) Se ele for representado em ponto flutuante, precisão dupla, isto é, com 14 dígitos.

9. Qual é o maior número positivo representado corretamente no Matlab? Qual é o menor número positivo representado corretamente no Matlab?

Capítulo 2

MÉTODOS DIRETOS PARA SOLUÇÃO DE SISTEMAS DE EQUAÇÕES LINEARES

2.1 Introdução

A necessidade de resolver sistemas de equações lineares aparece numa grande quantidade de problemas científicos. Existem estimativas que apontam que, a cada quatro problemas de simulação em matemática, três convertem-se em solução de sistemas de equações.

A título de exemplo, uma fonte substancial de sistemas de equações, lineares ou não-lineares, conforme o caso, é a solução de equações diferenciais por via dos métodos de discretização, como diferenças finitas ou elementos finitos. Em geral, os sistemas que resultam da discretização das equações diferenciais são muito grandes e têm características de esparsidade (muitos elementos nulos na matriz) que auxiliam em sua solução numérica. Outras fontes de sistemas são problemas de ajuste de dados, de minimização de funções e problemas inversos.

Nestas notas, estaremos admitindo que o leitor tenha uma certa familiaridade com conceitos básicos de álgebra linear. Trabalharemos com matrizes, inversas de matrizes, operações envolvendo matrizes e vetores e determinantes de matrizes.

Respeitando a distinção dos métodos, é comum catalogá-los em dois grupos. Os *Métodos Diretos* são aqueles que conduzem à solução, exata a menos de erros de arredondamento introduzidos pela máquina, após um número finito de passos. Os *Métodos Iterativos* são aqueles que se baseiam na construção de seqüências de aproximações. Em um método iterativo, a cada passo, os valores calculados anteriormente são usados para melhorar a aproximação. É claro que o método iterativo será útil se a seqüência das aproximações construídas pelo método convergir para a solução do sistema.

Neste capítulo, apresentamos alguns métodos diretos para a solução de sistemas lineares. A escolha do método a ser utilizado depende de cada caso. Se os métodos diretos têm a vantagem de fornecer solução após um número finito de passos e não dependem de condições de convergência, eles podem ser inviáveis quando o sistema é muito grande ou malcondicionado. De fato, como veremos neste capítulo, o número de operações realizadas nos métodos diretos cresce com o cubo da dimensão da matriz dos coeficientes e isto pode

ser prejudicial quando a propagação de erros contamina os cálculos.

Quanto à notação, adotaremos a usual em álgebra linear. Um sistema com n equações lineares nas n incógnitas será representado por

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n = b_1 \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n = b_2 \\ \vdots \quad \quad \quad \vdots \quad \quad \quad \vdots \quad \quad \quad \vdots \\ a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn}x_n = b_n \end{cases} \quad (2.1)$$

Os coeficientes das incógnitas compõem uma matriz quadrada com n linhas e n colunas, $A = [a_{ij}]$:

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \cdots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{bmatrix}.$$

Assim, definindo os vetores colunas $\mathbf{x} = (x_1, x_2, \dots, x_n)^T$ e $\mathbf{b} = (b_1, b_2, \dots, b_n)^T$, podemos representar o sistema (2.1) na forma matricial

$$A\mathbf{x} = \mathbf{b}. \quad (2.2)$$

Se admitimos que A é uma matriz inversível (isto é, se existe A^{-1} tal que $A^{-1}A = AA^{-1} = I$), então o sistema (2.1) tem uma única solução $\mathbf{x} = A^{-1}\mathbf{b}$, onde A^{-1} representa a inversa da matriz A .

2.2 Sistemas Triangulares

Observe que o sistema

$$\begin{aligned} 2x_1 - x_2 + x_3 &= 2 \\ x_2 + 2x_3 &= 3 \\ x_3 &= 1 \end{aligned}$$

pode ser resolvido, facilmente, por substituição das variáveis. De fato, substituindo $x_3 = 1$ na segunda equação, tiramos $x_2 = 1$. Levando estes dois valores para a primeira equação, teremos finalmente $x_1 = 1$. Este é um exemplo simples de *sistema triangular*. Nele, a matriz A é triangular superior – todos os elementos localizados abaixo da diagonal são nulos, isto é, $a_{ij} = 0$ se $i > j$. Como vimos no exemplo, a solução de um sistema triangular superior é obtida por substituição retrocedida (algo análogo se obtém para sistemas triangulares inferiores).

Para generalizar, tomemos o sistema triangular:

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1,n-1}x_{n-1} + a_{1n}x_n = b_1 \\ a_{22}x_2 + \cdots + a_{2,n-1}x_{n-1} + a_{2n}x_n = b_2 \\ \vdots \\ a_{n-1,n-1}x_{n-1} + a_{n-1,n}x_n = b_{n-1} \\ a_{nn}x_n = b_n \end{cases}$$

Admitindo que $a_{ii} \neq 0, i = 1 : n$, podemos calcular as incógnitas $x_1, x_2, \dots, x_n$ fazendo as seguintes operações:

$$\begin{cases} x_n = b_n/a_{nn} \\ x_{n-1} = (b_{n-1} - a_{n-1,n}x_n)/a_{n-1,n-1} \\ \vdots \\ x_1 = (b_1 - a_{1n}x_n - \cdots - a_{13}x_3 - a_{12}x_2)/a_{11}. \end{cases}$$

Dessa maneira, estabelecemos o Algoritmo 2.1 para resolução de sistemas triangulares. Nesta versão do algoritmo, estamos supondo que $a_{ii} \neq 0, i = 1 : n$.

Algoritmo 2.1 Solução de sistema triangular superior

Dados $a_{ij}, j \geq i, b_i, 1 \leq i, j \leq n$

Faça $x_n = \frac{b_n}{a_{nn}}$, soma = 0

1: Para $k = n - 1 : 1$, faça

2: soma = b_k

3: Para $j = k + 1 : n$, faça

4: soma = soma - $a_{kj}x_j$

5: $x_k = \frac{\text{soma}}{a_{kk}}$

Fazendo a contabilidade de operações vemos, diretamente do algoritmo, que a resolução de um sistema triangular com n equações envolve

n	divisões
$\sum_{j=1}^n j = \frac{n(n-1)}{2}$	adições
$\sum_{j=1}^n j = \frac{n(n-1)}{2}$	multiplicações.

Nas duas últimas passagens usamos a soma de uma progressão aritmética.

Assim, o número total de operações realizadas é n^2 . Observe que este é o número de operações realizadas quando se multiplica a matriz triangular A por um vetor com n componentes.

Se um dos elementos da diagonal de A se anula (isto é, $a_{ii} = 0$ para algum i), então o sistema pode ter uma infinidade de soluções ou não admitir solução.

2.3 Método de Eliminação Gaussiana

Os métodos diretos mais comuns têm como base as seguintes propriedades elementares de sistemas de equações lineares.

Propriedade: A solução do sistema $Ax = b$ não se altera se o submetemos a uma seqüência de operações do tipo:

- i. multiplicação de uma equação por constante não-nula;
- ii. soma do múltiplo de uma equação à outra;
- iii. troca da ordem das equações.

Veremos a seguir que é possível definir, de forma sistemática, uma seqüência conveniente de operações dos tipos (i, ii, iii) de modo a transformar um sistema linear num outro equivalente, escrito na forma triangular.

A solução de sistemas lineares usando uma eliminação ordenada de suas variáveis é atribuída ao matemático alemão Carl Friedrich Gauss (1777-1855).

É importante que a eliminação seja feita de forma sistemática, pois assim podemos elaborar algoritmos para os programas computacionais, nosso objetivo. Apresentamos inicialmente um exemplo bem simples do método denominado *Eliminação Gaussiana*. A idéia será posteriormente generalizada.

Considere o sistema linear

$$\begin{array}{rcll} x_1 & & + x_3 & = 0 & l_1 = 1^a \text{ equação} \\ x_1 & + & x_2 & = 1 & l_2 = 2^a \text{ equação} \\ 2x_1 & + & 3x_2 & + x_3 = 1 & l_3 = 3^a \text{ equação.} \end{array}$$

O primeiro passo no sentido da triangularização será "eliminar" os coeficientes da primeira coluna que estão abaixo da diagonal, o que corresponde a eliminar x_1 das duas últimas equações. Podemos fazê-lo subtraindo, da segunda e terceira equações, um múltiplo adequado da primeira equação. Assim, fazendo $l'_2 = l_2 - l_1$ e $l'_3 = l_3 - 2l_1$, teremos o sistema reescrito na forma:

$$\begin{array}{rcll} x_1 & & + x_3 & = 0 & l_1 \\ & x_2 & - x_3 & = 1 & l'_2 = l_2 - \frac{a_{21}}{a_{11}} l_1 \\ & 3x_2 & - x_3 & = 1 & l'_3 = l_3 - \frac{a_{31}}{a_{11}} l_1 \end{array}$$

Para chegar à forma triangular, basta eliminar o elemento da segunda coluna que está abaixo da diagonal, o que corresponde à eliminação de x_3 na última equação. Para isso, tomamos $l_3'' = l_3' - 3l_2'$ (observe que agora estamos usando a segunda equação na eliminação) e teremos o pretendido sistema triangular

$$\begin{aligned} x_1 + x_3 &= 0 \\ x_2 - x_3 &= 1 \\ x_3 &= -1 \quad l_3'' = l_3' - \frac{a_{32}}{a_{22}} l_2' \end{aligned}$$

cujas soluções são $x_3 = -1$, $x_2 = 0$, $x_1 = 1$.

Podemos generalizar o procedimento do exemplo acima, para estabelecer o processo sistemático de triangularização de um sistema com n equações e n incógnitas. Para simplificar de notação, consideramos todos os coeficientes, dados do sistema, tabulados numa matriz retangular com n linhas e $n + 1$ colunas. Assim, montamos a matriz ampliada

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} & \dots & a_{1n} & b_1 \\ a_{21} & a_{22} & a_{23} & \dots & a_{2n} & b_2 \\ & \vdots & & & \vdots & \vdots \\ a_{i1} & a_{i2} & a_{i3} & \dots & a_{in} & b_i \\ & \vdots & & & \vdots & \vdots \\ a_{n1} & a_{n2} & a_{n3} & \dots & a_{nn} & b_n \end{bmatrix} \quad \begin{matrix} l_1^1 \\ l_2^1 \\ \vdots \\ l_i^1 \\ \vdots \\ l_n^1 \end{matrix}$$

na qual cada linha l_i^1 , $i = 1 : n$ representa uma equação do sistema linear que queremos triangularizar.

Eliminação da Primeira Coluna

Supondo $a_{11} \neq 0$, para eliminar a incógnita x_1 das $n - 1$ últimas equações vamos subtrair a primeira linha multiplicada pelo fator

$$m_{i1} = a_{i1}/a_{11}$$

de todas as outras linhas l_i , $i = 2 : n$.

Dessa maneira, estamos modificando os elementos das $n - 1$ últimas linhas da seguinte maneira:

$$\begin{aligned} \text{Para } i = 2 : n \quad a_{ij}^2 &= a_{ij} - m_{i1} a_{1j}, \quad j = 2 : n \\ b_i^2 &= b_i - m_{i1} b_1. \end{aligned}$$

O índice superior 2 em a_{ij}^2 e b_i^2 indica que vamos usar um segundo valor para a_{ij} e b_i .

No fim deste estágio, os coeficientes da matriz aumentada foram modificados de modo que a matriz assume a seguinte configuração:

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} & \dots & a_{1n} & b_1 \\ 0 & a_{22}^2 & a_{23}^2 & \dots & a_{2n}^2 & b_2^2 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ 0 & a_{n2}^2 & a_{n3}^2 & \dots & a_{nn}^2 & b_n^2 \end{bmatrix}$$

Eliminação da Segunda Coluna

Para eliminar a incógnita x_2 das $n - 2$ últimas equações repetimos o procedimento anterior tomando, agora, a segunda linha como auxiliar no processo de eliminação, isto é, faremos

$$l_i^3 = l_i^2 - m_{i2} l_2^2, \quad i = 3 : n, \quad \text{onde} \quad m_{i2} = \frac{a_{i2}}{a_{22}}, \quad i = 3 : n.$$

Nesta etapa, estamos supondo que $a_{22} \neq 0$.

Os coeficientes serão modificados segundo as relações

$$\begin{aligned} \text{Para } i = 3 : n \quad a_{ij}^3 &= a_{ij}^2 - m_{i2} a_{2j}^2, \quad j = 3 : n \\ b_i^3 &= b_i^2 - m_{i2} b_2^2. \end{aligned}$$

Continuamos eliminando os elementos das colunas que estão abaixo da diagonal, repetindo a idéia descrita acima.

No processo de eliminação, os elementos a_{11} , a_{22}^2 , a_{33}^3 , $\dots$, a_{jj}^j que aparecem na diagonal de A são chamados *pivôs*. Se eles não se anulam, após a eliminação chegamos à forma triangularizada do sistema:

$$\begin{aligned} a_{11} x_1 + a_{12} x_2 + \dots + a_{1n} x_n &= b_1 \\ a_{22}^2 x_2 + \dots + a_{2n}^2 x_n &= b_2^2 \\ &\vdots \\ a_{nn}^n x_n &= b_n^n \end{aligned}$$

Se, no processo de eliminação um dos pivôs se anular, devemos trocar linhas (sempre escolhendo aquelas abaixo da diagonal para não perder a eliminação anterior), de modo que escolhamos elementos não nulos para pivôs.

A sistemática do Método de Eliminação Gaussiana é descrita pelo Algoritmo 2.2.

Esforço Computacional

Uma maneira usual de avaliar o custo computacional de um algoritmo numérico é através do volume de operações realizadas, ou seja, medindo seu *esforço computacional*.

Algoritmo 2.2 Eliminação Gaussiana

Dados matriz $A_{n \times n}$ e o vetor coluna b

- 1: Para $k = 1 : n - 1$ faça
- 2: Encontre $i \geq k$ tal que $a_{ik} \neq 0$
- 3: Se $a_{ii} = 0$ para todo $i \geq k$, então A não é inversível.
- 4: Troque a linha k com a linha i
- 5: Para $i = k + 1 : n$ faça
- 6: $m = m_{ik} = \frac{a_{ik}}{a_{kk}}$
- 7: $b_i = b_i - m b_k$
- 8: Para $j = k + 1 : n$ faça
- 9: $a_{ij} = a_{ij} - m a_{kj}$

Como vimos anteriormente, se o sistema é triangular, a solução pela substituição retrocessiva requer n^2 operações. Neste caso, o esforço computacional é n^2 .

Esta contabilidade também pode ser feita no caso do método de eliminação, como veremos a seguir.

Para cada valor j do terceiro laço do Algoritmo 2.2 (o laço em j) são realizadas duas operações: uma multiplicação e uma adição. Assim, neste laço são necessárias

$$\sum_{j=k+1}^{n+1} 2 = 2(n - k + 1).$$

No segundo laço do Algoritmo 2.2 (o laço em i), além das operações contabilizadas acima, para cada i realizamos uma divisão; assim, o número de operações, no laço correspondente a i , será:

$$\sum_{i=k+1}^n [1 + 2(n - k + 1)] = [1 + 2(n - k + 1)](n - k).$$

Finalmente, para obter o número total de operações fazemos a soma em k , correspondente ao laço externo do Algoritmo 2.2, isto é,

$$\begin{aligned} \sum_{k=1}^{n-1} (n - k) + 2(n - k + 1)(n - k) &= \sum_{k=1}^{n-1} (n - k) + 2 \sum_{k=1}^{n-1} (n - k + 1)(n - k) = \\ &= \frac{n(n-1)}{2} + 2 \frac{n^3 - n}{3} = \\ &= \frac{2}{3}n^3 + \frac{n^2}{2} - \frac{7}{6}n. \end{aligned}$$

Nos cálculos acima usamos, além da soma de uma progressão aritmética, o seguinte resultado:

$$\sum_{k=1}^{n-1} k^2 = \frac{(n-1)n(2n-1)}{6}.$$

Para contabilizar o número total de operações no Método de Eliminação Gaussiana, precisamos somar o número de operações necessárias para resolver um sistema triangular, deduzido na seção anterior. Assim, na aplicação do Método de Eliminação na resolução de um sistema com n equações e n incógnitas, o número total de operações é

$$\frac{2}{3}n^3 + \frac{n^2}{2} - \frac{7}{6}n + n^2 = \frac{2}{3}n^3 + \frac{3}{2}n^2 - \frac{7}{6}n,$$

que é um número próximo de $\frac{2}{3}n^3$ para n grande.

Por exemplo, para resolver um sistema triangular com cem equações, são necessárias 10 mil operações. Se o sistema não é triangular, no método de eliminação são necessárias 681 550 operações, isto é, mais que 68 vezes o exigido no caso triangular.

O peso maior no esforço computacional do método deve-se às operações realizadas na eliminação, pois o processo de eliminação é responsável pelo termo cúbico do esforço computacional. Isso deve ser explorado quando for necessário resolver dois ou mais sistemas com a mesma matriz dos coeficientes. Assim, se vamos resolver os sistemas $Ax = b$ e $Ax = d$, procedemos à eliminação apenas uma vez e resolvemos dois sistemas triangulares.

Um caso particular desta natureza é o cálculo da inversa de uma matriz. Para inverter uma matriz A devemos resolver uma equação matricial

$$AC = I$$

onde I é a matriz identidade. Se chamamos c_i o vetor que representa a i -ésima coluna da inversa, para calculá-lo resolvemos o sistema

$$Ac_i = e_i = (0, 0, \dots, 0, 1, 0, \dots, 0)^T,$$

onde o elemento 1 está na i -ésima posição do vetor e_i . Assim, o cálculo da inversa envolverá a resolução de n sistemas com a mesma matriz de coeficientes, A , e a eliminação pode ser realizada numa única vez.

2.4 Estratégia do Pivoteamento

Com o exemplo que apresentamos a seguir, ilustramos a importância da troca na ordem das equações do sistema, de modo que o pivô (elemento da diagonal que será usado na eliminação) seja, em valor absoluto, o maior possível.

Exemplo 2.4.1. Vamos resolver o sistema

$$\begin{aligned} 0.004x_1 + 15.73x_2 &= 15.77 \\ 0.423x_1 - 24.72x_2 &= -20.49. \end{aligned}$$

Trabalhando com quatro dígitos na representação em ponto flutuante e arredondamento ao desprezar o quinto dígito, procedemos a eliminação de x_1 na segunda equação, com $m = 105.8$:

$$\begin{aligned} 0.004x_1 + 15.73x_2 &= 15.77 \\ -1689x_2 &= -1688. \end{aligned}$$

Dessa maneira, a solução obtida é

$$x_1 = 12.50 \text{ e } x_2 = 0.9994.$$

Podemos verificar que a solução exata é $x_1 = 10.0$ e $x_2 = 1.0$. Sendo assim, a aproximação obtida para x_1 tem erro relativo 0.25, isto é, 25% de erro. Vamos agora inverter a ordem das equações no sistema, isto é, consideremos

$$\begin{aligned} 0.423x_1 - 24.72x_2 &= -20.49 \\ 0.004x_1 + 15.73x_2 &= 15.77. \end{aligned}$$

Ainda trabalhando com 4 dígitos, a eliminação de x_1 é realizada com $m = 0.9456 \cdot 10^{-2}$, resultando no sistema triangularizado

$$\begin{aligned} 0.423x_1 - 24.72x_2 &= -20.49 \\ 15.96x_2 &= 15.96, \end{aligned}$$

cujas soluções são $x_1 = 10.0$ e $x_2 = 1.0$, que é a solução exata do sistema. $\square$

Chamamos de *estratégia do pivoteamento* a troca sistemática de linhas, de modo que o pivô seja o maior elemento, em valor absoluto, da coluna que estamos eliminando. Com esta estratégia, no k -ésimo passo procuramos o elemento de maior valor absoluto, dentre aqueles que estão na mesma coluna, abaixo da diagonal. No Algoritmo 2.2 devemos introduzir, para cada k , a busca do r , tal que

$$|a_{rk}| = \max_{k \leq i \leq n} |a_{ik}|,$$

e trocar as linhas k e r . O esquema final será o descrito no Algoritmo 2.3.

Existem dois casos nos quais o método de eliminação pode ser aplicado sem pivoteamento: sistemas nos quais a matriz dos coeficientes é diagonalmente dominante ou simétrica positiva definida.

Uma matriz é simétrica e positiva definida se

$$A^T = A \quad \text{e} \quad x^T A x > 0, \quad \text{para todo vetor } x \neq 0.$$

Uma matriz é diagonalmente dominante se seus elementos satisfazem a $|a_{ii}| > \sum_{j \neq i} |a_{ij}|$, $i = 1 : n$, para todo i .

Algoritmo 2.3 Eliminação Gaussiana com pivoteamento

Dados matriz A e vetor b

- 1: Para $k = 1 : n - 1$ faça
 - 2: $w = |a_{kk}|$
 - 3: Para $j = k : n$ faça
 - 4: Se $|a_{jk}| > w$ então
 - 5: $w = |a_{jk}|$ e $r = j$
 - 6: Trocar linhas k e r
 - 7: Para $i = k + 1 : n$ faça
 - 8: $m = m_{ik} = \frac{a_{ik}}{a_{kk}}$
 - 9: $b_i = b_i - m_{ik} b_k$
 - 10: Para $j = k + 1 : n$ faça
 - 11: $a_{ij} = a_{ij} - m_{ik} a_{kj}$
-

2.5 Matrizes Tridiagonais

Na solução numérica de equações diferenciais, assim como em outras aplicações, a matriz associada à discretização é *esparsa*, isto é, a quantidade de elementos não-nulos é pequena, quando comparada com o número total de elementos da matriz. Se, além de esparsa, a matriz tem os elementos não-nulos concentrados em torno da diagonal, ela é chamada *matriz de banda*.

Podemos formalizar a definição dizendo que $A = [a_{ij}]$ é uma matriz de banda $p + q + 1$, se $a_{ij} = 0$ se $i > j + q$ ou $i < j - p$.

A estrutura de banda oferece vantagens na triangularização, pois em cada coluna teremos de eliminar, no máximo, q elementos. Se isto for incorporado ao algoritmo, haverá uma substancial redução no esforço computacional.

Tomemos como exemplo o caso particular dos sistemas tridiagonais.

Nos sistemas tridiagonais, $p = q = 1$, isto é, os elementos não-nulos estão nas três diagonais centrais. Neste caso, a eliminação em cada coluna será de um elemento. Além disso, o sistema tridiagonal pode ser armazenado usando quatro vetores, um para a diagonal principal, dois para as diagonais secundárias e um para o termo independente, como ilustrado a seguir.

$$\begin{bmatrix} d_1 & c_1 & & & \\ a_2 & d_2 & c_2 & & \\ & \ddots & \ddots & \ddots & \\ & & a_{n-1} & d_{n-1} & c_{n-1} \\ & & & a_n & d_n \end{bmatrix} \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_{n-1} \\ b_n \end{bmatrix}$$

Observamos que o armazenamento de matrizes tridiagonais em quatro vetores é bas-

taute significativo. A título de ilustração, na matriz associada a um sistema com duzentas incógnitas serão necessárias 800 posições em vez das 40 200 posições utilizadas no procedimento com a matriz não esparsa.

A versão simplificada do Algoritmo 2.2, no caso de sistemas tridiagonais, é apresentada no Algoritmo 2.4.

Algoritmo 2.4 Solução de sistema tridiagonal

Dados vetores a, b, c, d

- 1: Para $k = 1 : n - 1$ faça
 - 2: $d_{k+1} = d_{k+1} - (a_{k+1}/d_k) c_k$
 - 3: $b_{k+1} = b_{k+1} - (a_{k+1}/d_k) b_k$
 - 4: $x_n = b_n/d_n$
 - 5: Para $k = n - 1 : 1$ faça
 - 6: $x_k = (b_k - c_k x_{k+1})/d_k$
-

Veremos mais tarde que os sistemas resultantes da discretização de equações diferenciais, unidimensionais na variável espacial, são tridiagonais e diagonalmente dominantes. Esta importante aplicação de sistemas tridiagonais motiva esta seção.

2.6 Fatoração LU

Como comentamos anteriormente, o método de eliminação pode ser usado economicamente quando precisamos resolver vários sistemas com a mesma matriz dos coeficientes. Em algumas situações práticas, os diversos termos independentes, b , não estão disponíveis simultaneamente ou podem depender da própria solução do sistema associada a outro b .

Uma opção seria guardar os coeficientes m_{ij} calculados no processo de eliminação e usá-los na atualização de novos termos independentes, b . Outra alternativa computacionalmente equivalente é o que chamamos *fatoração* ou *decomposição LU* da matriz A . A base do método também está apoiada na simplicidade de resolução de sistemas triangulares.

Suponhamos que seja possível fatorar a matriz A num produto de uma matriz triangular inferior, (com os elementos da diagonal principal iguais a 1) L , e uma matriz triangular superior, U , isto é,

$$A = LU.$$

Nestas condições, o sistema $Ax = b$ pode ser reescrito na forma $LUx = b$, o que permite o desmembramento em dois sistemas triangulares

$$Ly = b \quad \text{e} \quad Ux = y.$$

Resolvendo o primeiro sistema, calculamos y que, usado no segundo sistema, fornecerá o vetor procurado x .

Dessa maneira, conhecidas L e U , o sistema será resolvido com $2n^2$ operações (dois sistemas triangulares), o que representa um ganho substancial comparado com as $2n^3/3$ operações do método de eliminação.

A questão básica da existência dos fatores L e U é tratada em livros especializados em análise numérica, por exemplo [Golub-van Loan, 1989]. Dada uma matriz A , os fatores L e U são únicos se exigirmos que todos os elementos da diagonal de L sejam iguais a 1.

Podem-se encontrar diretamente os elementos de L e U a partir da definição de produtos de matrizes, montando-se um sistema com n^2 equações e n^2 incógnitas, que será resolvido progressivamente a partir de valores anteriormente calculados. Dessa maneira, se chamarmos m_{ij} os elementos de L , e de u_{ij} os elementos de U , obtemos expressões que viabilizam uma implementação eficiente do método em questão (lembrando que $m_{ii} = 1$ para todo i):

$$u_{ij} = a_{ij} - \sum_{k=1}^{i-1} m_{ik} u_{kj} \quad \text{para } i \leq j$$

e

$$m_{ij} = \left(a_{ij} - \sum_{k=1}^{j-1} m_{ik} u_{kj} \right) / u_{jj} \quad \text{para } i > j.$$

Estas expressões são usadas no Algoritmo 2.5.

Algoritmo 2.5 Fatoração LU

Dados matriz $A = [a_{ij}]$

1: Para $i = 1 : n$, faça

2: Para $j = i : n$, faça

3: $u_{ij} = a_{ij} - \sum_{k=1}^{i-1} m_{ik} u_{kj}$

4: Para $j = i + 1 : n$, faça

5: $m_{ji} = \left(a_{ji} - \sum_{k=1}^{i-1} m_{jk} u_{ki} \right) / u_{ii}$

Pode-se também mostrar que os coeficientes m_{ij} calculados no processo de eliminação formam a matriz L (desde que não seja necessária nenhuma troca de linhas), e a matriz triangular superior do método de eliminação é a própria U . No caso da troca de linhas (pivoteamento), na eliminação também teremos uma fatoração triangular, mas $LU = A'$, onde A' é obtida da troca de linhas de A . Esta equivalência é mostrada, por exemplo, em [Ruggiero-Lopes, 1997].

Exemplo 2.6.1. Seja

$$A = \begin{bmatrix} 1 & 2 & -1 \\ 2 & 3 & -2 \\ 1 & -2 & 1 \end{bmatrix}.$$

Calculando os m_{ij} e u_{ij} pelas fórmulas apresentadas nesta seção teremos:

$$U = \begin{bmatrix} 1 & 2 & -1 \\ 0 & -1 & 0 \\ 0 & 0 & 2 \end{bmatrix} \quad L = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 1 & 4 & 1 \end{bmatrix}.$$

Para resolvermos o sistema $Ax = (2, 3, 0)^T$, resolvemos $Ly = b$

$$\begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 1 & 4 & 1 \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix} = \begin{bmatrix} 2 \\ 3 \\ 0 \end{bmatrix} \Rightarrow y_1 = 2, y_2 = -1 \text{ e } y_3 = 2$$

e, com estes valores, calculamos x através de $Ux = y$:

$$\begin{bmatrix} 1 & 2 & -1 \\ 0 & -1 & 0 \\ 0 & 0 & 2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 2 \\ -1 \\ 2 \end{bmatrix} \Rightarrow x_1 = 1, x_2 = 1 \text{ e } x_3 = 1.$$

□

Em algumas aplicações, a matriz A é simétrica ($A = A^T$) e positiva definida ($x^T Ax > 0$ para todo $x \neq 0$). Neste caso, pode-se mostrar que a fatoração triangular é

$$A = LDL^T$$

onde L é uma matriz triangular inferior (com 1 na diagonal) e D é uma matriz diagonal. Esta é a *decomposição de Cholesky*, e o Algoritmo 2.6 apresenta uma sistemática para obtê-la.

Algoritmo 2.6 Fatoração de Cholesky

Dados matriz $A = [a_{ij}]$, simétrica e positiva definida

1: Para $j = 1 : n$, faça

2: $d_j = a_{jj} - \sum d_k l_{jk}$

3: Para $i = j + 1 : n$, faça

4: $l_{ij} = \left(a_{ij} - \sum_{k=1}^{j-1} d_k l_{ik} l_{jk} \right) / d_j$

A existência da decomposição de Cholesky é uma condição necessária e suficiente para que uma matriz simétrica seja positiva definida. Assim, o Algoritmo acima também pode ser usado para verificarmos se uma matriz simétrica é positiva definida. Entretanto, os erros de arredondamento que ocorrem na aplicação do Algoritmo podem comprometer o resultado final.

2.7 Refinamento da Solução

Mesmo que se proceda o pivoteamento no método de eliminação, os erros de arredondamento interferem no resultado final. A influência desta perturbação pode ser verificada pelos seguintes cálculos: chamando y o vetor "solução" obtido após a aplicação do método no sistema $Ax = b$, calculamos o vetor $c = Ay$. Se c não está suficientemente próximo de b , a "solução" y não está boa. Neste caso, podemos refinar a solução, procedimento que não altera muito o custo computacional total.

Para refinar a solução, observamos que se x é a solução e y a aproximação obtida pelo método, então

$$A(x - y) = Ax - Ay = b - c.$$

Chamemos $\epsilon = x - y$ e $\beta = b - c$. Como já triangularizamos o sistema, usamos os mesmos m_{ij} (que são os elementos da matriz triangular inferior L) para resolver o sistema

$$A\epsilon = \beta,$$

calculando ϵ . Uma aproximação refinada será obtida por

$$x = y + \epsilon.$$

Observe que podemos repetir o refinamento até que a diferença $b - c$ atenda a um critério de aproximação satisfatório.

Como usamos os coeficientes na eliminação, cada refinamento demanda a resolução de um sistema triangular e, portanto, o esforço computacional aumenta de n^2 operações, onde n é a dimensão do sistema.

2.8 Condicionamento da Matriz e Estimativas de Erro

Para analisar a influência que perturbações dos dados de entrada podem provocar na solução do sistema, vamos usar a seguinte

Definição: O *número de condição* de uma matriz $A_{n \times n}$ é

$$\text{cond}(A) = \|A\| \|A^{-1}\|,$$

onde $\|A\|$ é alguma medida da matriz A , denominada norma da matriz. (No glossário apresentamos algumas normas usuais para matrizes).

Suponhamos que, em vez de com a matriz exata A , vamos trabalhar com uma aproximação $\tilde{A}$. Sejam x e y as soluções exata e aproximada, respectivamente, isto é,

$$Ax = b \quad \text{e} \quad \tilde{A}y = b.$$

Observe que

$$x = A^{-1}b = A^{-1}\tilde{A}y = A^{-1}(A + \tilde{A} - A)y = y + A^{-1}(\tilde{A} - A)y.$$

Assim, se chamarmos $\delta A = \tilde{A} - A$ teremos $x - y = A^{-1}\delta A y$.

Uma propriedade de normas de produto de matrizes nos diz que $\|Ax\| \leq \|A\| \|x\|$ e $\|AB\| \leq \|A\| \|B\|$. Usando duas vezes esta propriedade no último termo do desenvolvimento acima, temos:

$$\|x - y\| \leq \|A^{-1}\| \|\delta A\| \|y\| = \|A\| \|A^{-1}\| \frac{\|\delta A\|}{\|A\|} \|y\|.$$

Lembrando que $\|A\| \|A^{-1}\| = \text{cond}(A)$, esta desigualdade fornece uma delimitação para o erro relativo (com relação a y) da solução, em termos do erro relativo dos dados de entrada, uma vez que ela pode ser reescrita como

$$\frac{\|x - y\|}{\|y\|} \leq \text{cond}(A) \frac{\|\delta A\|}{\|A\|}.$$

Se o vetor independente b também contém erro, pode-se mostrar que [Isaacson-Keller, 1967]:

$$\frac{\|x - y\|}{\|x\|} \leq \frac{\text{cond}(A)}{1 - \text{cond}(A) \frac{\|\delta A\|}{\|A\|}} \left\{ \frac{\|\delta A\|}{\|A\|} + \frac{\|\delta b\|}{\|b\|} \right\},$$

onde δb representa o erro absoluto do termo independente.

As duas últimas desigualdades ilustram o papel do número de condição da matriz A . Se o número de condição é grande, os erros nos dados, aqui representados por δA e δb , serão ampliados, pois o limitante do erro relativo será grande. Neste caso, dificilmente conseguiremos bons resultados.

Um exemplo clássico de matriz malcondicionada é a matriz de Hilbert (David Hilbert, 1862-1943), cujos elementos são definidos por

$$H_{ij} = \frac{1}{i+j-1}, \quad 1 \leq i, j \leq n.$$

Quando $n = 10$, a matriz de Hilbert tem o número de condição aproximadamente 1.6×10^{13} . Nem com precisão dupla conseguiremos resolver numericamente sistemas com esta matriz, uma vez que o número de condição destrói a precisão da máquina.

Uma regra prática nos diz que, se estamos trabalhando com uma calculadora que usa p dígitos e a matriz A tem $\text{cond}(A) = 10^q$, então a solução pode ter apenas $p - q$ dígitos significativos confiáveis (o termo "pode" é usado porque estamos lidando com limitantes para o erro).

O número de condição de uma matriz pode ser calculado usando-se o Matlab. Por exemplo, para calcular o número de condição da matriz de Hilbert, definida acima, podemos usar:

```
>> n = 10; for i = 1:n, for j = 1:n, h(i, j) = 1/(i+j-1); end; end;  
>> cond(h)
```

```
ans =
```

```
1.6025e+013.
```

No caso da matriz de Hilbert poderíamos usar `cond(hilb(10))`, uma vez que este é um tipo especial de matriz oferecida pelo pacote.

2.9 Sistemas Indeterminados, Sobredeterminados e Malcondicionados: a Decomposição em Valores Singulares (SVD)

2.9.1 Decomposição em Valores Singulares

Se num sistema linear o número de equações é igual ao número de incógnitas, temos chance de encontrar uma única solução aplicando um dos métodos descritos anteriormente.

Mas sabemos, da Álgebra Linear, que é possível perder a existência e/ou unicidade quando a matriz do sistema é singular, isto é, não tem inversa. Neste caso, uma ou mais linhas (ou colunas) podem ser escritas como combinação linear das outras.

Do ponto de vista numérico, podem acontecer os agravantes decorrentes da forma de representação dos números na máquina, do acúmulo de erros de arredondamento.

Para lidar com sistemas de equações cujas matrizes são singulares, ou quase singulares, os algoritmos que apresentamos anteriormente nem sempre funcionam a contento. De fato, como vimos na seção anterior, o erro da solução numérica de um sistema de equações, obtida pelo método de eliminação, é delimitado usando-se o número de condição da matriz A . Quando comparado com o número de dígitos significativos da máquina, o número de condição grande pode comprometer a confiabilidade dos cálculos.

Uma técnica numérica que tem se mostrado eficiente na álgebra linear de problemas malcondicionados é conhecida como *Decomposição em Valores Singulares*, ou pela sigla SVD, herdada da denominação em inglês. Existem também outras razões para a popularidade da SVD: ela nos permite tratar adequadamente problemas sobredeterminados (mais equações que incógnitas), nos dá um diagnóstico de onde está a dificuldade de resolver o problema e também pode nos fornecer “soluções”, em algum sentido, quando a solução convencional não existe.

Algumas destas virtudes estão no resumo do assunto que apresentamos a seguir. De fato, dentro do escopo deste livro, esta seção tem como objetivo semear o assunto. Para aprofundar, pode-se recorrer às referências [Golub-van Loan, 1989] e [Björk, 1996].

As *matrizes ortogonais* desempenham um papel importante nesta seção.

Definição: Dizemos que uma matriz U é *ortogonal* se

$$U^T U = I,$$

onde I é a matriz identidade.

Observe que, se uma matriz é ortogonal, sua inversa é calculada pela simples operação de transposição.

O embasamento teórico do método SVD está no resultado que apresentamos a seguir, cuja demonstração pode ser encontrada, por exemplo, em [Golub-van Loan, 1989].

Teorema 2.1. *Seja $A_{m \times n}$, com $m \geq n$. Então existem matrizes ortogonais $U_{m \times m}$ e $V_{n \times n}$ tais que*

$$A = U E V^T$$

onde

$$E_{m \times n} = \begin{bmatrix} \sigma_1 & & & & \\ & \sigma_2 & & & \\ & & \ddots & & \\ & & & \ddots & \\ & & & & \sigma_p \\ 0 & \dots & \dots & \dots & 0 \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ 0 & \dots & \dots & \dots & 0 \end{bmatrix}$$

com $p = n$ e $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_p \geq 0$.

Os números σ_i são chamados valores singulares de A .

Existem diversas técnicas numéricas para encontrar as matrizes U , E e V , algumas delas são apresentadas em [Golub-van Loan, 1989]. Por outro lado, os pacotes computacionais em divulgação no meio científico têm, em geral, recursos para calcular estas matrizes. Por exemplo, no Matlab elas podem ser obtidas usando-se a função `svd`, como veremos na seção 2.10.

2.9.2 Sistemas Quadrados

Embora a potencialidade da SVD seja muito mais ampla, nos restringiremos inicialmente a sua aplicação na resolução de sistemas quadrados, isto é, no caso em que a matriz A é quadrada, com dimensão n . Neste caso U , E e V são também matrizes quadradas de dimensão n .

Se A é uma matriz não-singular, teremos

$$\sigma_1 \geq \sigma_2 \geq \sigma_3 \geq \dots \geq \sigma_n > 0,$$

e a matriz E pode ser invertida. Sua inversa, E^{-1} , é uma matriz diagonal com elementos $\frac{1}{\sigma_i}, i = 1 : n$. Usando a decomposição SVD da matriz A podemos escrever a solução do sistema na forma

$$x = A^{-1}b = (UEV^T)^{-1}b = VE^{-1}U^T b. \quad (2.3)$$

Na última passagem usamos a propriedade $(AB)^{-1} = B^{-1}A^{-1}$, e pela ortogonalidade de U e V , $U^{-1} = U^T$ e $V^{-1} = V^T$.

Vejamos agora o que acontece quando A é singular. Neste caso, procedida a SVD obteremos U , V e matriz "diagonal"

$$E = \begin{bmatrix} \sigma_1 & & & & & \\ & \sigma_2 & & & & \\ & & \ddots & & & \\ & & & \sigma_p & & \\ & & & & 0 & \\ & & & & & \ddots \\ & & & & & & 0 \end{bmatrix} = \begin{bmatrix} E_p & 0 \\ 0 & 0 \end{bmatrix}$$

onde $p < n$ é chamado posto da matriz A e $E_p = \text{diag}(\sigma_1, \sigma_2, \dots, \sigma_p)$ é uma matriz cuja diagonal contém os valores singulares não nulos.

Usando o Teorema 2.1, o sistema $Ax = b$ pode ser reescrito na forma

$$UEV^T x = b.$$

Chamando $V^T x = y$, pré-multiplicando a equação acima por U^T (lembrando que $U^T U = I$), temos

$$Ey = U^T b \quad (2.4)$$

que podemos representar na forma

$$Ey = \begin{bmatrix} \sigma_1 & & & & & \\ & \sigma_2 & & & & \\ & & \ddots & & & \\ & & & \sigma_p & & \\ & & & & 0 & \\ & & & & & \ddots \\ & & & & & & 0 \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_p \\ y_{p+1} \\ \vdots \\ y_n \end{bmatrix} = \begin{bmatrix} b'_1 \\ b'_2 \\ \vdots \\ b'_p \\ b'_{p+1} \\ \vdots \\ b'_n \end{bmatrix}$$

onde b'_i são componentes do vetor $U^T b$.

Existem duas possibilidades para o vetor $U^T b$.

Caso A: $b'_{p+1} = b'_{p+2} = \dots = b'_n = 0$. Neste caso, a solução para (2.4) será

$$y_1 = \frac{b'_1}{\sigma_1}, \dots, y_p = \frac{b'_p}{\sigma_p}, y_{p+1}, \dots, y_n,$$

quaisquer que sejam $y_{p+1}, y_{p+2}, \dots, y_n$. Em particular, é comum usar $y_{p+1} = y_{p+2} = \dots = y_n = 0$, isto é, a solução é

$$y = (y_1, y_2, \dots, y_p, 0, \dots, 0).$$

Chamando de E^+ a matriz diagonal cujos elementos são

$$E_{ii}^+ = \begin{cases} \frac{1}{\sigma_i} & 1 \leq i \leq p \\ 0 & p < i \leq n \end{cases}$$

podemos reescrever o vetor y usando E^+ :

$$y = E^+ U^T b.$$

Assim, a solução no Caso A é obtida lembrando que $V^T x = y$ e, sendo V ortogonal:

$$x^+ = Vy = VE^+ U^T b. \quad (2.5)$$

Esta é a solução que tem a menor norma (comprimento), entre todas as soluções possíveis (veja [Press et al., 1990], por exemplo).

Definição: Podemos escrever (2.5) na forma $x = A^+ b$, onde $A^+ = VE^+ U^T$ é chamada pseudo-inversa de A .

Caso B: Se algum dos b'_i , $i = p+1 : n$ é não-nulo o sistema (2.4) não tem solução. Mas é possível apresentar uma "solução" desde que tomemos outro sentido para o termo.

Como foi feito no Caso A, definimos uma modificação da expressão (2.3), usando E^+ :

$$x^+ = VE^+ U^T b. \quad (2.6)$$

É possível mostrar ([Press et al., 1990], por exemplo), que x^+ é o vetor que minimiza o resíduo do sistema $r = \|Ax - b\|$.

Ainda no caso de sistemas quadrados, existe a situação intermediária entre os que têm matrizes não-singulares e aqueles cujas matrizes são singulares: sistemas malcondicionados.

Como até agora não introduzimos o conceito de posto de uma matriz, vamos fazê-lo através de seus valores singulares. Dizemos que A tem posto r se $\sigma_r \neq 0$ e $\sigma_{r+1} = \sigma_{r+2} = \dots = \sigma_n = 0$. Embora existam várias outras implicações deste conceito, não as abordaremos.

Vamos detalhar um pouco mais como as coisas ocorrem em cálculos práticos. As perturbações, erros de arredondamento e outras fontes de erros fazem com que dificilmente se obtenham valores singulares zero nas aplicações práticas. Define-se posto numérico de A com respeito ao erro $\tau > 0$ como o número de valores singulares estritamente maiores que τ . Assim, se os valores singulares da matriz A satisfazem a

$$\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > \tau > \sigma_{r+1} \dots$$

então dizemos que o posto numérico de A é r .

Estabelecido o posto numérico da matriz, pode-se falar do procedimento de truncamento da decomposição em valores singulares (TSVD) de forma natural, desprezando os valores singulares menores que τ e procedendo como foi descrito no Caso B.

Observamos que a definição do valor τ nem sempre é tão óbvia; maiores detalhes sobre o assunto em [Hansen, 1987].

2.9.3 Sistemas Retangulares

Podemos usar as idéias desenvolvidas para sistemas cujas matrizes são quadradas para definir soluções para sistemas nos quais as matrizes têm número de linhas diferente do número de colunas.

Se o sistema tem menos equações que incógnitas, não podemos esperar unicidade de solução. Neste caso, a matriz do sistema tem m linhas e n colunas, com $m < n$.

Por outro lado, se o sistema tem mais equações do que número de incógnitas, sistemas sobredeterminados, não existe vetor x tal que $Ax - b = 0$. Este caso, $m > n$, é comum no método dos quadrados mínimos, como veremos mais tarde.

Nestes casos, é possível procurar soluções em um sentido diferente do clássico e que tenham utilidade do ponto de vista das aplicações.

A decomposição de valores tem papel importante neste novo conceito de solução. A idéia é usar pseudo-inversas, que generalizam a solução definida na equação (2.3). Como vimos pelo teorema 2.1, a SVD existe para matrizes $A_{m \times n}$:

$$A_{m \times n} = U_{m \times m} E_{m \times n} V_{n \times n}^T$$

A matriz E pode ser escrita na forma

$$E = \begin{bmatrix} E_p & 0 \\ 0 & 0 \end{bmatrix}$$

onde $E_p = \text{diag}(\sigma_1, \sigma_2, \dots, \sigma_p)$ é a matriz que tem como diagonal os valores singulares não-nulos e $p = \text{posto}(A)$. Supondo que $p \leq \min\{m, n\}$, vamos definir E^+ no caso $m \neq n$ por

$$E_{n \times m}^+ = \begin{bmatrix} E_p^{-1} & 0 \\ 0 & 0 \end{bmatrix}.$$

Como E_p^{-1} é uma matriz com dimensão $p \times p$, blocos com elementos nulos são introduzidos para completar a dimensão de E^+ , $n \times m$.

Com esta definição podemos definir a pseudo-inversa por

$$A_{n \times m}^+ = V_{n \times n} E_{n \times m}^+ U_{m \times m}^T.$$

Desta maneira as soluções (2.5) e (2.6) ficam também definidas para o caso $m \neq n$. O sentido destas soluções continua o mesmo do caso $m = n$. Assim, se $m < n$, (2.5) é a solução de norma mínima e se $m > n$, (2.6) é a solução que minimiza a norma do resíduo $r = Ax - b$. Maiores detalhes sobre este assunto podem ser encontrados em [Björk, 1996] e [Press et al., 1990].

Na seção seguinte, apresentamos exemplos de sistemas indeterminados e sobredeterminados "resolvidos" pela SVD e pelo Matlab.

2.10 Uso do Matlab na Solução de Sistemas Lineares: Métodos Diretos

Consideremos o sistema $Ax = b$, associado à matriz A e ao vetor b , definidos no Matlab da seguinte forma:

```
>> A=[3 -2 2; 1 1 1; 2 1 -1]
```

```
A = 3 -2 2
     1 1 1
     2 1 -1
```

```
>> b=[ -3 -4 -3]'
```

```
b =
    -3
    -4
    -3
```

Observe que o vetor coluna b foi definido pela transposição de um vetor usando uma aspa, após a entrada do vetor linha. Isto é equivalente à defini-lo por $b = [-3;-4;-3]$.

Apresentamos a seguir algumas funções definidas no Matlab relacionadas com o assunto deste capítulo. Alguns comentários aparecem após o símbolo %, seguindo a convenção usada no pacote.

Mais informações podem ser obtidas pelo recurso help seguido da função de interesse. Por exemplo, usando help det obtemos detalhes sobre a função que calcula o determinante de uma matriz.

```
>> x = A\b; % Calcula a solução do sistema Ax = b
```

O ponto e vírgula no final da linha significa que não queremos que a resposta seja apresentada na tela. Para apresentar a resposta num vetor linha fazemos

```
>> x'
ans =
1.2857   -1.5714   -1.1429
```

O ponto e vírgula também pode ser usado para separar comandos, como vemos a seguir.

```
>> format long; x' % A partir deste comando os resultados serão
                    % apresentados com 15 dígitos
```

```
ans =
1.28571428571429   -1.57142857142857   -1.14285714285714
```

```
>> format short      % Os resultados voltam a ser apresentados
                    % com cinco dígitos
```

```
>> IA = inv(A)      % Cálculo da inversa da Matriz A; A inversa
                    % será denotada IA
```

```
IA =
0.1429    0.0000    0.2857
0.2143    0.5000    0.0714
0.0714    0.5000   -0.3571
```

```
>> IA*A              % Cálculo do produto das duas matrizes IA e A.
                    % Deve haver compatibilidade de dimensões para
                    % tornar o produto entre as matrizes possível.
```

```
ans =
1.0000    0.0000    0.0000
0         1.0000    0.0000
0.0000    0         1.0000
```

```
>> d = det(A);      % Cálculo do determinante da matriz A
>> [V,D] = eig(A);  % Cálculo dos autovalores e autovetores da matriz A
```

► Exercício: Calcule a solução do sistema usando a inversa de A: $x_1 = \text{inv}(A) * b = A^{-1}b$.

```
>> [L,U,P] = lu(A)    % Cálculo da Fatoração LU da matriz A
```

P é a matriz de permutação do pivoteamento usado no cálculo dos fatores L e U . A matriz P multiplicada por A tem o efeito das trocas de linhas efetuadas quando o pivoteamento é realizado. Neste exemplo, isto é, usando a matriz definida no início desta seção, vemos que a matriz P é obtida da troca das duas últimas linhas da matriz identidade. Isto significa que, no pivoteamento, a segunda linha da matriz A foi trocada com a terceira.

► Exercício:

- Verifique que $PA = LU$.
- Verifique que, em $[L, U] = \text{lu}(A)$, a matriz L é apresentada com as trocas de linhas correspondentes ao pivoteamento.
- Como resolver o sistema usando as matrizes apresentadas em (b)?

```
>> [U,E,V] = svd(A);      % Cálculo da Decomposição em Valores
                           % Singulares da matriz
```

Observe que os valores singulares compõem, de forma decrescente, a matriz diagonal E . Para verificarmos que U é uma matriz ortogonal realizamos o produto abaixo, cujo resultado será a matriz identidade.

```
>> U'*U;
```

► Exercício: Verifique que $s = \text{svd}(A)$ fornece apenas os valores singulares da decomposição.

2.10.1 Sistemas Indeterminados e Sobredeterminados

Consideremos agora o sistema linear $Ax = b$, no qual a matriz A é definida por:

```
>> A = [.96 1.72 .76; 2.28 .96 -1.32; 1.32 -.76 -2.08]
```

Pode-se verificar se a matriz tem linhas linearmente independentes através da forma escalonada da matriz, que no Matlab é obtida pela função *Reduced Row Echelon Form*, que também pode ser aplicada em matrizes retangulares:

```
>> rref(A);
```

Na forma escalonada da matriz A , a última linha é nula, informando-nos que só duas das três linhas da matriz A são linearmente independentes. Em outras palavras, o posto da matriz é 2, o que pode ser verificado usando $\text{rank}(A)$ no Matlab. Usando a função $\text{det}(A)$ obtemos o determinante da matriz A : $6.66\text{e-}16$. Como no Matlab os números são representados usando 16 dígitos, o determinante da matriz é nulo. Segundo aprendemos na Álgebra Linear, não existe uma única solução para o sistema, qualquer que seja o vetor b . Vamos resolver o sistema para o caso $b = [1 \ 2 \ 1]'$ usando o Matlab.

```
>> x1 = (A\b)'
```

Warning: Matrix is close to singular or badly scaled.
Results may be inaccurate. RCOND = 1.623133e-017.

```
x1 = 1.4517   -0.5050    0.6250
```

Assim, apesar da mensagem de alerta sobre erros, foi apresentada uma solução para o sistema.

Num procedimento alternativo, vamos usar a decomposição em valores singulares para “resolver” o sistema, seguindo o que foi apresentado na seção 2.9. Os valores singulares da matriz A são

```
>> s = svd(A)'; s = [3.5315    2.5485    0.0000].
```

Calculando o vetor $U^T b$, termo independente do sistema (2.3), temos

```
>> [U,E,V] = svd(A); b1 = (U'*b)'
```

```
b1 = [2.3064   -0.8250   -0.0000]
```

c, portanto, o sistema se classifica no caso A da seção 2.9 e a solução de norma mínima é calculada usando eq. (2.5). Calculamos inicialmente a matriz E^+ :

```
>> EE(1,1) = 1/E(1,1); EE(2,2) = 1/E(2,2); EE(3,3) = 0;
```

que, levada à expressão (2.5), nos dará a solução de menor norma:

```
>> x = V*EE*U'*b
```

```
x =
```

```
    0.5911
```

```
    0.3556
```

```
   -0.2356
```

Realizando a operação $A*x_1$, ($x_1 = A\b$), e $A*x$, verificamos que os dois vetores são soluções para o sistema $Ax = b$. Também podemos verificar que a norma de x é menor do que a norma de x_1 :

```
[norm(x) norm(x1)] = [0.7289    1.6592]
```

► **Exercício:** Verifique, com este exemplo, que a solução definida no caso A pode também ser obtida usando, no Matlab, a operação $\text{pinv}(A)*b$, onde $\text{pinv}(A)$ é a pseudo inversa da matriz A (veja [Golub-van Loan, 1989]) para maiores detalhes.

Consideremos agora um novo vetor para o sistema: $b = [3 \ 2 \ 6]'$. Realizando a operação $U^T b$, onde U é a mesma matriz ortogonal obtida anteriormente, temos um novo vetor para o lado direito em (2.3): $b1 = [5.7116 \ 0.2098 \ 4.0415]'$. Sendo assim, com este novo vetor b passamos ao Caso B descrito na seção 2.9. A “solução” definida em (2.6) é o vetor que minimiza a norma do resíduo do sistema, neste caso,

```
>> x = V*EE*U'*b
```

```
x =
    1.2207
    0.1704
   -1.0504
```

O resíduo associado à esta solução pode ser calculado usando

```
>> (A*x-b) '
ans =
   -2.3333    2.3333   -2.3333
```

e sua norma é $\text{norm}(A*x-b) = 4.0415$.

► Exercício:

- Verifique que esta mesma solução pode ser obtida usando, no Matlab, a operação $\text{pinv}(A)*b$, onde $\text{pinv}(A)$ é a pseudo-inversa da matriz A citada anteriormente.
- Use a função $A \backslash b$ do exemplo acima e verifique que, neste caso, a solução não tem sentido.

Para exemplificar o cálculo de soluções para sistemas retangulares pelo Matlab consideremos o sistema $Ax = b$,

```
>> A = [2 1 -1; -1 3 1]; b = [3 2]'; % Definindo matriz e vetor
                                     % independente do sistema
```

Inicialmente vamos usar a função que resolve sistemas no Matlab:

```
>> x = A\b

x' = [ 1.0000    1.0000    0]
```

Pode-se verificar diretamente que x é solução do sistema e que sua norma é $\text{norm}(x) = 1.4142$.

Neste exemplo, o sistema tem infinitas soluções e a solução apresentada pelo Matlab é o vetor que tem o maior número de componentes nulas, dentre as possíveis soluções.

► Exercício: Verifique que o vetor $x_n = \text{pinv}(A)*b$ também é solução para o sistema acima. Verifique que a norma deste vetor é menor que a norma da solução acima. Este procedimento que oferece a solução de norma mínima é a generalização da eq. (2.5) para sistemas indeterminados, $m < n$. Neste caso, a norma de x é 1.4142 e a norma de x_n é 1.3652, calculadas usando a função $\text{norm}(x)$.

Para finalizar, consideramos um sistema $Ax = b$, com mais equações do que incógnitas:

```
>> A = [2 1;3 1;5 6]; b = [3;2;4]; % Definição da matriz e vetor  
                                     % independente do sistema
```

Usando a função solução de sistemas do Matlab, calculamos uma solução:

```
>> x = A\b  
x =  
    0.9178  
   -0.0822
```

A norma do resíduo associado a esta solução, calculada por $\text{norm}(A*x-b)$, é 1.4190.

► Exercício: Verifique que esta é a mesma solução calculada por $\text{pinv}(A)*b$.

Resumimos a solução de sistemas de equações lineares $Ax = b$, usando o Matlab, nos itens:

1. Se $m = n$, há compatibilidade entre número de equações e incógnitas. A função $x = A\b$ utiliza a fatoração LU apresentando a única solução quando este for o caso. Também é apresentada uma solução quando o sistema é indeterminado. Neste caso, a solução apresentada é diferente daquela calculada usando $x = \text{pinv}(A)*b$, sendo esta última a solução de norma mínima, Caso A da seção 2.9. Se o sistema não tem solução, Caso B da seção 2.9, o resultado fornecido pela função $A\b$ fica comprometido, pois há divisão por zero. Neste caso, usando $x_m = \text{pinv}(A)*b$, obtemos a "solução" que minimiza a norma do resíduo $Ax - b$.
2. Se $m < n$, menos equações do que incógnitas, o sistema é indeterminado. A função $x = A\b$ fornece uma solução com o maior número de componentes nulas possível. Por outro lado, $x_m = \text{pinv}(A)*b$ é a solução de mínima norma. As duas soluções são diferentes.
3. Se $m > n$, mais equações do que incógnitas, o sistema não tem solução no sentido tradicional. Neste caso, a função $x = A\b$ fornecerá a solução no sentido dos quadrados mínimos, isto é, aquela que minimiza a norma do resíduo $Ax - b$. Esta solução é igual à obtida com $x = \text{pinv}(A)*b$.

Exercícios

1. Programe os Métodos de Eliminação Gaussiana e decomposição LU. Aplique-os na resolução dos sistemas

a.

$$\begin{aligned} 7x_1 + 2x_2 - 5x_3 &= -18 \\ x_1 + 5x_2 - 3x_3 &= -40 \\ 2x_1 - x_2 - 9x_3 &= -26. \end{aligned}$$

b. $Ax = b$ onde,

$$A_{ij} = \frac{1}{i+j-1}, \quad b_i = \frac{1}{i}, \quad i, j = 1:n.$$

Use $n = 5, 10$ e 15 . Você pode explicar seus resultados? Observe que a solução exata é $(1, 0, 0, \dots, 0)$.

2. Mostre que a matriz

$$A = \begin{bmatrix} 2 & 2 & 1 \\ 1 & 1 & 1 \\ 3 & 2 & 1 \end{bmatrix}$$

é inversível e que A não pode ser escrita como o produto de uma matriz triangular inferior por uma matriz triangular superior ($A = LU$) sem pivoteamento.

3. Elabore um algoritmo de inversão da matriz A com base na decomposição LU de A . Qual é o esforço computacional para a inversão de A ?

4. Elabore o programa correspondente ao Algoritmo 2.4. Use seu programa para resolver o sistema

$$\begin{aligned} -2(1+h^2)x_1 + x_2 &= 1 \\ x_{i-1} - 2(1+h^2)x_i + x_{i+1} &= 0 \quad i = 2:n-1 \\ x_{n-1} - 2(1+h^2)x_n &= 1 \end{aligned}$$

tomando $n = 30$ e $h = 0.1$.

5. Conte o número de operações necessárias à resolução de um sistema linear com n equações e n incógnitas usando o método de decomposição LU. Com base neste esforço computacional, este método é competitivo com a Eliminação Gaussiana?

6. Resolva os sistemas abaixo, com e sem pivoteamento, usando quatro dígitos e aritmética com arredondamento. Qual técnica deu melhores resultados?

$$\begin{array}{ll} \text{a.} & \begin{aligned} 58.09x_1 + 1.003x_2 &= 68.12 \\ 321.8x_1 + 5.550x_2 &= 377.3 \end{aligned} \\ \text{b.} & \begin{aligned} 321.8x_1 + 5.550x_2 &= 377.3 \\ 100.3x_1 + 5809x_2 &= 6812 \end{aligned} \end{array}$$

7. Encontre a fatoração LDL^T das matrizes abaixo

$$A = \begin{bmatrix} 4 & 1 & 1 & 1 \\ 1 & 3 & -1 & 1 \\ 1 & -1 & 2 & 0 \\ 1 & 1 & 0 & 2 \end{bmatrix} \quad A = \begin{bmatrix} 4 & 1 & -1 & 1 \\ 1 & 1 & -1 & 0 \\ -1 & -1 & 5 & 2 \\ 0 & 0 & 2 & -4 \end{bmatrix}$$

8. Complete o Algoritmo 2.6 para resolver os sistemas com matrizes A do exercício anterior, tomando $b = (0.65, 0.05, 0.05, 0)^T$ e $b = (7, 8, -4, 6)^T$, respectivamente.

9. É possível mostrar que uma matriz simétrica A é positiva definida se e somente se A puder ser fatorada na forma LDL^T . Assim, a menos de erro de arredondamento introduzido na fatoração, o Algoritmo 2.6 pode ser usado para testar se uma matriz é positiva definida. Verifique se as matrizes abaixo são positivas definidas

$$A = \begin{bmatrix} -1 & 2 & 0 & 1 \\ 2 & -3 & 2 & -1 \\ 0 & 2 & 5 & 6 \\ 1 & -1 & 6 & 12 \end{bmatrix} \quad A = \begin{bmatrix} 2 & -2 & 4 & -4 \\ -2 & 3 & -4 & 5 \\ 4 & -4 & 10 & -10 \\ -4 & 5 & -10 & 14 \end{bmatrix}$$

$$A = \begin{bmatrix} 3 & -6 & 9 \\ -6 & 14 & -20 \\ 9 & -20 & 29 \end{bmatrix}.$$

10. Considere o sistema $Ax = b$, com quatro equações e três incógnitas:

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 0 \\ 2 & 5 & 8 \end{bmatrix} \quad b = \begin{bmatrix} 366 \\ 804 \\ 351 \\ 514 \end{bmatrix}.$$

a. Encontre a "solução" usando $x = A \backslash b$ no Matlab.

b. Encontre a "solução" usando a seção 2.9.

c. Compare os resultados.

11. Seja $B = A^T$, onde A é a matriz definida no exercício anterior. Repita os itens (a), (b) e (c) do exercício 10, tomando o sistema $Bx = b$, onde

$$b = \begin{bmatrix} 366 \\ 804 \\ 351 \end{bmatrix}.$$

Capítulo 3

MÉTODOS ITERATIVOS PARA SISTEMAS LINEARES

No capítulo anterior abordamos métodos diretos que, a menos de erros de arredondamento, fornecem a solução de sistemas de equações lineares após a realização de um número finito de passos. Neste capítulo, apresentamos métodos que usam iterações para calcular sucessivas aproximações para a solução do sistema, num processo cuja convergência depende da matriz dos coeficientes A .

Os métodos iterativos são convenientes para sistemas grandes e esparsos que aparecem com frequência na discretização de equações diferenciais. Por exemplo, veremos mais tarde, a discretização da equação de Laplace, no caso bidimensional e usando diferenças finitas, fornece uma matriz com estrutura apresentada na figura 3.1. Nesta figura, as linhas cheias representam as diagonais não-nulas, isto é, os elementos fora destas diagonais são nulos.

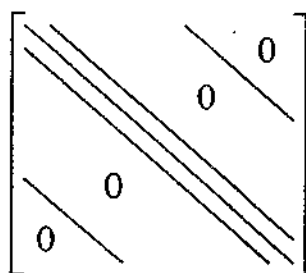


Figura 3.1: Estrutura de uma matriz esparsa.

Este exemplo mostra que o Método de Eliminação Gaussiana pode destruir faixas de zero que existem na matriz. Este preenchimento (*fill-in* em inglês) com elementos não-nulos pode comprometer a utilização dos métodos diretos no caso de matrizes esparsas. Neste caso os métodos iterativos podem ser mais eficientes, uma vez que nenhum elemento não-nulo será introduzido. Mas, como nem tudo é perfeito, existem algumas condições para garantir a convergência da sequência de aproximações, calculadas por métodos iterativos.

3.1 Métodos de Jacobi e Gauss-Seidel

Se no sistema $Ax = b$ os elementos da diagonal são diferentes de zero, $a_{ii} \neq 0, i = 1 : n$, então podemos explicitar x_1 usando a primeira equação, x_2 usando a segunda equação e assim sucessivamente. Ou seja, reescrevemos o sistema numa forma que é conveniente para os métodos iterativos:

$$\begin{aligned} x_1 &= \left(b_1 - \sum_{j=2}^n a_{1j} x_j \right) / a_{11} \\ &\vdots \\ x_i &= \left(b_i - \sum_{j \neq i}^n a_{ij} x_j \right) / a_{ii} \\ &\vdots \\ x_n &= \left(b_n - \sum_{j=1}^{n-1} a_{nj} x_j \right) / a_{nn}. \end{aligned} \quad (3.1)$$

3.1.1 Método de Jacobi

Carl Gustav Jacobi (1804-1851) apresentou o método iterativo que recebe seu nome em 1845.

No método de Jacobi, as equações (3.1) são usadas para calcular uma seqüência de vetores aproximações $x^{(1)}, x^{(2)}, \dots, x^{(n)}$. Conhecida uma aproximação inicial, que chamaremos $x^{(0)}$, usamos o primeiro termo à direita da primeira equação para definir uma nova aproximação para x_1 :

$$x_1^{(1)} = \left(b_1 - \sum_{j=2}^n a_{1j} x_j^{(0)} \right) / a_{11}.$$

Repetindo a idéia no cálculo de novas aproximações para as componentes, isto é, $x_2^{(1)}, \dots, x_n^{(1)}$, teremos

$$x_i^{(1)} = \left(b_i - \sum_{j=1, j \neq i}^n a_{ij} x_j^{(0)} \right) / a_{ii}, \quad i = 1 : n. \quad (3.2)$$

Usamos agora o vetor $x^{(1)}$ nas equações (3.1) para calcular o novo vetor das aproximações, $x^{(2)} = (x_1^{(2)}, x_2^{(2)}, \dots, x_n^{(2)})$.

Em resumo, o método de Jacobi consiste em calcularmos as componentes dos vetores $x^{(1)}, x^{(2)}, \dots$ usando (3.1).

O Algoritmo 3.1 sintetiza o processo de cálculo do Método de Jacobi.

Algoritmo 3.1 Método de Jacobi

Dados $A_{n \times n}$, b_n , $x^{(0)}$, max, tol

- 1: Para $k = 0 : \text{max}$, faça
- 2: Para $i = 1 : n$, faça
- 3:
$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1, j \neq i}^n a_{ij} x_j^{(k)} \right)$$
- 4: Se $\max_{1 \leq i \leq n} |x_i^{(k+1)} - x_i^{(k)}| < \text{tol}$ então
- 5: $x = x^{(k+1)}$
- 6: Caso contrário Se $k = \text{max}$ então
- 7: Não houve convergência

A aplicação do método terá êxito se a sequência de vetores construída sucessivamente convergir para a solução do sistema. Isto ocorre se $\max_{1 \leq i \leq n} |x_i^{(k+1)} - x_i^{(k)}|$ se aproxima de zero quando k cresce. Esta condição gera o critério de parada que foi adotado no algoritmo:

$$\max_{1 \leq i \leq n} |x_i^{(k+1)} - x_i^{(k)}| < \text{tol}.$$

Para iniciar o processo iterativo foi necessário fornecer uma aproximação inicial $x^{(0)}$. Na falta de melhores informações sobre a solução, podemos tomar $x^{(0)} = 0$, isto é, $x_i = 0$, $i = 1 : n$.

Por precaução devemos prefixar o número máximo de iterações, denotada por max no algoritmo.

Exemplo 3.1.1. Seja o sistema

$$\begin{aligned} 4.00 x_1 + 0.24 x_2 - 0.08 x_3 &= 8.00 \\ 0.09 x_1 + 3.00 x_2 - 0.15 x_3 &= 9.00 \\ 0.04 x_1 - 0.08 x_2 + 4.00 x_3 &= 20.00. \end{aligned}$$

Tomamos como condição inicial o vetor $x^{(0)} = (0, 0, 0)$, e as iterações do Método de Jacobi são calculadas por

$$\begin{aligned} x_1^{(k+1)} &= 2 - 0.06 x_2^{(k)} + 0.02 x_3^{(k)} \\ x_2^{(k+1)} &= 3 - 0.03 x_1^{(k)} + 0.05 x_3^{(k)} \\ x_3^{(k+1)} &= 5 - 0.01 x_1^{(k)} + 0.02 x_2^{(k)}. \end{aligned}$$

Na tabela 3.1, apresentamos os resultados das três primeiras iterações do método de Jacobi. Na última coluna desta tabela, apresentamos o erro absoluto da terceira iteração.

$x^{(1)}$	$x^{(2)}$	$x^{(3)}$	$ x^{(3)} - \bar{x} $
2	1.92	1.909	0.2×10^{-3}
3	3.19	3.1944	0.6×10^{-3}
5	5.04	5.0446	0.2×10^{-3}

Tabela 3.1: Iterações para o exemplo 3.1.1.

□

Lançando mão da notação matricial, poderemos reescrever o método de Jacobi numa forma mais concisa. A forma matricial será útil quando abordarmos as condições de convergência dos métodos iterativos.

Consideremos a decomposição da matriz A , $A = L + D + U$, onde:

- L é uma matriz triangular estritamente inferior com $l_{ij} = a_{ij}$ se $i > j$, e $l_{ij} = 0$ se $i \leq j$;
- D uma matriz diagonal com $d_{ii} = a_{ii}$;
- U é uma matriz triangular estritamente superior com $u_{ij} = a_{ij}$ se $i < j$ e $u_{ij} = 0$ se $i \geq j$;

Nesta notação, as iterações de Jacobi são definidas pela equação matricial

$$Dx^{(k+1)} = b - (L + U)x^{(k)}.$$

Como D é uma matriz diagonal, sua inversa é calculada facilmente e teremos

$$x^{(k+1)} = D^{-1} [b - (L + U)x^{(k)}],$$

na qual a inversa da matriz diagonal é

$$D^{-1} = \begin{bmatrix} 1 \\ d_{ii} \end{bmatrix}.$$

3.1.2 Método de Gauss-Seidel

Gauss apresentou seu método iterativo para resolver sistemas de equações provenientes do Método dos Quadrados Mínimos. Seidel foi aluno de Jacobi e publicou a versão hoje usada em 1874.

O método de Gauss-Seidel pode ser olhado como uma modificação do Método de Jacobi. Nele, as iterações serão calculadas usando as equações (3.1), mas vamos aproveitar os cálculos já atualizados, de outras componentes, para atualizar a componente que está sendo calculada. Assim, o valor recém-calculado para $x_1^{(k+1)}$ será usado no cálculo de $x_2^{(k+1)}$ e assim sucessivamente. Dessa maneira as iterações serão definidas pelo Algoritmo 3.2. Para simplificar, omitiremos o critério de parada neste algoritmo, uma vez que podemos usar o mesmo adotado no Algoritmo 3.1.

Algoritmo 3.2 Método de Gauss-Seidel

Dados $A, b, x^{(0)}, \max$

1: Para $k = 0 : \max$, faça

2: Para $i = 1 : n$, faça

$$3: \quad x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{(k+1)} - \sum_{j=i+1}^n a_{ij} x_j^{(k)} \right)$$

Exemplo 3.1.2. Tomando as mesmas equações do Exemplo (3.1.1), as iterações, partindo de $x^{(0)} = (0, 0, 0)$, são calculadas por

$$x_1^{(k+1)} = 2 - 0.06 x_2^{(k)} + 0.02 x_3^{(k)}$$

$$x_2^{(k+1)} = 3 - 0.03 x_1^{(k+1)} + 0.05 x_3^{(k)}$$

$$x_3^{(k+1)} = 5 - 0.01 x_1^{(k+1)} + 0.02 x_2^{(k+1)}$$

Na tabela 3.1.2, apresentamos os resultados das três primeiras iterações do Método de Gauss-Seidel e o erro absoluto da terceira iteração.

$x^{(1)}$	$x^{(2)}$	$x^{(3)}$	$ x^{(3)} - \bar{x} $
2	1.924376	1.909240	0.4×10^{-4}
2.94	3.194209	3.194955	0.96×10^{-5}
5.0388	5.044640	5.044807	0.6×10^{-6}

Tabela 3.2: Iterações para o exemplo 3.1.2.

□

3.1.3 Convergência dos Métodos Jacobi e Gauss-Seidel

Como no Método de Gauss-Seidel usamos a atualização imediata dos dados, é de se esperar que sua convergência seja mais rápida, como sugerem os resultados dos exemplos apresentados. Embora isso ocorra com frequência, o fato não pode ser generalizado, posto que existem exemplos para os quais o Método de Gauss-Seidel diverge e há convergência do Método de Jacobi [Varga, 1962]. No momento, o Método de Jacobi volta à popularidade devido às arquiteturas em paralelo para computadores.

Voltando à notação matricial introduzida no Método de Jacobi, podemos escrever o Método de Gauss-Seidel na forma

$$(L + D) x^{(k+1)} = b - Ux^{(k)},$$

ou ainda

$$x^{(k+1)} = D^{-1} [b - Lx^{(k+1)} - Ux^{(k)}].$$

A análise de convergência dos dois métodos iterativos apresentados pode ser feita colocando-os na forma geral dos métodos iterativos:

$$x^{(k+1)} = Bx^{(k)} + c,$$

onde B é chamada matriz de iteração. Nos métodos de Jacobi e Gauss-Seidel, as matrizes de iterações são, respectivamente,

$$B^J = D^{-1}(L + U) \quad \text{e} \quad B^{GS} = (L + D)^{-1}U.$$

A convergência dos dois métodos é estudada através dos autovalores de B (dizemos que λ_i , $i = 1 : n$ é autovalor da matriz B se $Bu = \lambda_i u$ para algum vetor $u \neq 0$). O teorema abaixo estabelece a condição necessária e suficiente para a convergência: os autovalores da matriz de iteração devem ser, em módulo, menores do que 1. A demonstração pode ser encontrada em [Burden-Faires, 1988], por exemplo.

Teorema 3.1. *O método iterativo $x^{(k+1)} = Bx^{(k)} + c$ converge se e somente se $\max_{1 \leq i \leq n} |\lambda_i| < 1$.*

O critério de convergência do teorema acima tem mais interesse teórico do que prático, uma vez que a determinação dos autovalores de uma matriz é um problema mais trabalhoso que a solução do sistema. Mas ele nos permite estabelecer outras condições de convergência de verificação mais simples. Por exemplo:

- i. Os métodos iterativos de Jacobi e Gauss-Seidel convergem se a matriz A (do sistema $Ax = b$) é diagonalmente dominante (veja a definição na seção 2.4);
- ii. os métodos iterativos de Jacobi e Gauss-Seidel convergem se A é uma matriz positiva definida (isto é, $y^T A y > 0$ para todo $y \neq 0$).

Em alguns casos, informações teóricas prévias nos indicam que a condição ii é satisfeita. Por exemplo, é possível mostrar que a matriz correspondente à aplicação do método de elementos finitos em problemas elíticos é positiva definida. Observamos também que a condição de dominância da diagonal não é necessária à convergência dos métodos iterativos. Isso pode ser verificado, por exemplo, aplicando o Método de Gauss-Seidel a um sistema com matriz

$$A = \begin{bmatrix} 8 & 2 & 1 \\ 5 & 4 & 1 \\ 4 & 3 & 4 \end{bmatrix}.$$

Calculando os autovalores da matriz de iteração, obtemos $\lambda_1 = 0.9072$, $\lambda_2 = -0.5414$ e $\lambda_3 = -0.3658$, o que nos mostra que o Método de Gauss-Seidel converge apesar de a matriz não ser diagonal-dominante. Em alguns casos, pode-se obter uma matriz diagonalmente dominante através de uma reordenação adequada das equações.

3.2 Métodos SOR

Uma técnica de aceleração da convergência dos métodos iterativos é conhecida com o nome de relaxação sucessiva, ou ainda, abusando do anglicismo, métodos SOR. Nestes métodos, definimos a aproximação na iteração $(k+1)$ como uma média entre o valor $x^{(k)}$ obtido na iteração (k) e o valor $x^{(k+1)}$, que seria obtido pelo Método de Gauss-Seidel.

As iterações, associadas ao parâmetro ω do Método SOR, são definidas por

$$x_{\text{SOR}}^{(k+1)} = (1 - \omega) x_{\text{SOR}}^{(k)} + \omega x_{\text{GS}}^{(k+1)},$$

onde as subslgas se referem às aproximações obtidas pelos métodos SOR e Gauss-Seidel, respectivamente. Lembrando a expressão que define o Método de Gauss-Seidel, algoritmo 3.2, podemos descrever o Método SOR pelo Algoritmo 3.3.

Algoritmo 3.3 Método SOR

Dados $A, b, x^{(0)}, \omega, \max, \text{tol}$

1: Para $k = 0 : \max$, faça

2: Para $i = 1 : n$, faça

$$3: \quad x_i^{(k+1)} = (1 - \omega) x_i^{(k)} + \frac{\omega}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{(k+1)} - \sum_{j=i+1}^n a_{ij} x_j^{(k)} \right)$$

4: Se $\max_i |x_i^{(k+1)} - x_i^{(k)}| \leq \text{tol}$ então

5: $x = x^{(k+1)}$

No Método SOR, ω é chamado parâmetro de sobre-relaxação. Pode-se observar que quando $\omega = 1$ temos o Método de Gauss-Seidel. A escolha $1 < \omega < 2$ caracteriza os métodos de sobre-relaxação, ao passo que os métodos de sub-relaxação são obtidos por valores $0 < \omega < 1$. A prática mostra que melhores resultados são obtidos na sobre-relaxação.

Dentre os resultados de convergência dos métodos SOR destacamos:

- O método só converge se $0 < \omega < 2$;
- se A é simétrica e positiva definida, então o método converge para qualquer $0 < \omega < 2$; em particular, como vimos anteriormente, neste caso o Método de Gauss-Seidel é convergente.

Em alguns casos particulares, de importância prática, é possível estabelecer um valor ótimo para ω . Por exemplo, se A é uma matriz bloco tridiagonal e ρ é o maior autovalor da matriz de iteração do Método de Jacobi, pode-se mostrar que o parâmetro ótimo é [Blum, 1972]

$$\omega = \frac{2}{\sqrt{1 + (1 - \rho^2)}}.$$

Exemplo 3.2.1. Veremos no capítulo 9 que a discretização de equações diferenciais parciais cujas incógnitas são funções com duas variáveis espaciais, $u(x, y)$, nos leva a uma matriz que tem estrutura como a da figura 3.1. Vamos ilustrar a influência do parâmetro de relaxação do Método SOR tomando como exemplo matrizes com esta estrutura.

Consideremos um sistema linear $Ax = b$, no qual a matriz A tem uma estrutura pentadiagonal como a ilustrada na figura 3.1. Os elementos da diagonal principal são definidos por $a_{11} = -2$, $a_{nn} = -2$ e $a_{ii} = -4$, $i = 2 : n - 1$. Os elementos das diagonais secundárias são $a_{21} = a_{n,n-1} = 1$, $a_{i,i+1} = a_{i-1,i} = 1$, $i = 2 : n - 1$. As diagonais mais afastadas são definidas por $a_{i,i+r} = 1$, $i = 1 : n - r$ e $a_{i-r,i} = 1$, $i = r + 1 : n$.

Escolhemos o termo independente, de modo que a solução seja conhecida $\underline{x}_i = 1$, $i = 1 : n$. A aproximação inicial do método iterativo será $x_i^{(0)} = 0$. Neste exemplo, vamos fixar $r = 6$ e tomar como critério de parada do método $|x_i^{(k)} - \underline{x}_i| < 10^{-5}$, $i = 1 : n$.

Elaboramos um programa na linguagem Matlab para o Método SOR, `sor.m`. Assim, é possível testar o método com vários parâmetros de relaxação, w , para várias dimensões da matriz, n . Apresentamos na Tabela 3.3 o número de iterações que foram realizadas para atingir a precisão, com diferentes valores de n e w . Observamos que, no sistema com $n = 15$, o melhor resultado foi obtido quando $w = 1.35$, para o qual foram necessárias 15 iterações do Método SOR. Este número representa menos da metade do número de iterações do Método de Gauss-Seidel, que, como vimos neste capítulo, corresponde ao SOR com $w = 1$. Quando o sistema tem 20 equações, o melhor parâmetro de relaxação é $w = 1.4$, para o qual são necessárias 41 iterações, contra as 81 necessárias para atingir a mesma precisão usando o Método Gauss-Seidel. Finalmente, quando o sistema tem 30 equações, o melhor parâmetro é $w = 1.3$, são necessárias 62 iterações para atingir a precisão desejada e o método não converge, se usamos $w = 1.6$.

$\begin{matrix} w \rightarrow \\ \downarrow n \end{matrix}$	1.0	1.1	1.2	1.3	1.35	1.4	1.6
15	41	33	27	21	15	21	25
20	81	68	57	48	44	41	62
30	103	87	74	62	63	154	não converge

Tabela 3.3: Número de iterações usando SOR.

□

3.3 Método Gradiente Conjugado

Os métodos gradientes conjugados têm sido estudados a partir da década de 50, motivados pela solução de sistemas provenientes da discretização de equações diferenciais parciais e viabilizados pela entrada dos computadores nos cálculos científicos. Vários autores citam o trabalho de Hestenes e Stiefel, de 1952, como a base do que foi desenvolvido posteriormente.

Para apresentar o Método Gradiente Conjugado, que denotaremos por GC, vamos admitir que a matriz do sistema linear seja simétrica e positiva definida, isto é, $A^T = A$ e $x^T A x > 0$, para qualquer x não-nulo.

Os métodos do tipo gradiente, para resolver o sistema $Ax = b$, têm como idéia básica minimizar a função de $x = (x_1, x_2, \dots, x_n)$

$$F(x) = \frac{1}{2} x^T A x - b^T x. \quad (3.3)$$

Para simplificar, quando abrirmos as equações matriciais, o faremos supondo $n = 2$, embora o desenvolvimento seja o mesmo para um número arbitrário de componentes. Por exemplo, para verificar que $F(x)$ é uma função quadrática nas componentes do vetor x , tomamos $n = 2$ em (3.3):

$$F(x) = \frac{1}{2} (a_{11}x_1^2 + 2a_{12}x_1x_2 + a_{22}x_2^2) - b_1x_1 - b_2x_2,$$

onde foi usada a simetria da matriz A .

As curvas de nível da função $F(x)$ são definidas por $F(x_1, x_2) = \text{constante}$ e, portanto, são elipses. Assim, o gráfico de $F(x)$ é um parabolóide e esta função tem um único mínimo.

O mínimo da função é atingido no vetor x que anula o gradiente da função,

$$\text{grad } F(x_1, x_2) = \begin{bmatrix} \frac{\partial F}{\partial x_1} \\ \frac{\partial F}{\partial x_2} \end{bmatrix} = \begin{bmatrix} a_{11}x_1 + a_{12}x_2 - b_1 \\ a_{21}x_1 + a_{22}x_2 - b_2 \end{bmatrix} = Ax - b. \quad (3.4)$$

Sendo assim, $\text{grad } F(x) = 0$ significa $Ax = b$, isto é, a solução do sistema de equações lineares minimiza a função quadrática e vice-versa.

Como é conhecido do Cálculo Diferencial, o vetor gradiente aponta a direção de crescimento máximo da função. Portanto, é natural que, nos passos de busca do mínimo, caminhemos na direção contrária ao gradiente, isto é:

$$x^{(k+1)} = x^{(k)} - s_k \text{grad } F(x^{(k)}). \quad (3.5)$$

Assim, a aproximação num passo $k + 1$ é calculada a partir da aproximação no passo anterior, caminhando na direção contrária ao gradiente. O parâmetro real s_k regula o tamanho do passo na k -ésima iteração.

Usando (3.4), temos que a direção de descida é $-\text{grad } F(x^{(k)}) = b - Ax^{(k)} = r^{(k)}$, que é o resíduo associado à aproximação $x^{(k)}$.

O parâmetro s_k em (3.5), tamanho do passo, será usado na minimização do resíduo associado à aproximação que está sendo calculada. Desta maneira, vamos calcular o valor de s que minimiza

$$F(x + sr) = \frac{1}{2} (x + sr)^T A (x + sr) - b^T (x + sr) \quad (3.6)$$

Fixado $\mathbf{x}$, derivamos a função com relação à variável s , aplicando as regras da cadeia e derivação de produto:

$$\frac{dF}{ds}(\mathbf{x} + s\mathbf{r}) = \frac{1}{2}\mathbf{r}^T \mathbf{A}(\mathbf{x} + s\mathbf{r}) + \frac{1}{2}(\mathbf{x} + s\mathbf{r})^T \mathbf{A}\mathbf{r} - \mathbf{b}^T \mathbf{r}.$$

Usando a propriedade distributiva nas operações e as propriedades de transposição de vetores e matrizes para mostrar que $\mathbf{x}^T \mathbf{A}\mathbf{r} = (\mathbf{A}\mathbf{r})^T \mathbf{x} = \mathbf{r}^T \mathbf{A}\mathbf{x}$, uma vez que $\mathbf{A}$ é simétrica, temos

$$\frac{dF}{ds}(\mathbf{x} + s\mathbf{r}) = s\mathbf{r}^T \mathbf{A}\mathbf{r} + \mathbf{r}^T (\mathbf{A}\mathbf{x} - \mathbf{b}) = s\mathbf{r}^T \mathbf{A}\mathbf{r} - \mathbf{r}^T \mathbf{r}.$$

Finalmente, igualando a derivada a zero, obtemos o valor de s que minimiza a função.

$$s = \frac{\mathbf{r}^T \mathbf{r}}{\mathbf{r}^T \mathbf{A}\mathbf{r}}.$$

Com este valor de s em (3.6) e lembrando que $-\text{grad } F(\mathbf{x}) = \mathbf{r}^{(k)}$, o Método Gradiente pode ser resumido no Algoritmo 3.4.

Algoritmo 3.4 Método Gradiente

Dados $\mathbf{A}$, $\mathbf{b}$, max, tol

- 1: $\mathbf{x}^{(0)} = \mathbf{0}$
 - 2: Para $k = 0 : \text{max}$, faça
 - 3: $\mathbf{r} = \mathbf{b} - \mathbf{A}\mathbf{x}$
 - 4: $s = \frac{\mathbf{r}^T \mathbf{r}}{\mathbf{r}^T \mathbf{A}\mathbf{r}}$
 - 5: $\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + s\mathbf{r}$
 - 6: Se $\mathbf{r}^T \mathbf{r} < \text{tol}$ então
 - 7: Saída com $\text{sol} = \mathbf{x}^{(k+1)}$
-

3.3.1 Método dos Gradientes Conjugados

No Método dos Gradientes Conjugados, GC, o método do tipo gradiente mais usado, ainda supomos que $\mathbf{A}$ seja uma matriz simétrica positiva definida. A base do método GC é a seguinte:

Propriedade: É possível escolher n direções linearmente independentes, $\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_n$ e, por meio da minimização da função $F(\mathbf{x}^{(k)} + s\mathbf{v}_k)$, em cada uma das direções separadamente, construir uma seqüência de aproximações que forneça o mínimo da função (3.3) após n passos (n é o número de equações do sistema).

Neste método, cada novo vetor $\mathbf{v}_j$ é gerado depois de completo um ciclo de minimização, como descrevemos a seguir. Dada uma aproximação inicial $\mathbf{x}^{(0)}$, tomamos a primeira direção $\mathbf{v}_1 = -\text{grad } F(\mathbf{x}^{(0)})$. Conhecidas as direções $\mathbf{v}_1, \dots, \mathbf{v}_j$, e as aproximações $\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(j)}$, tomamos:

1. v_{j+1} tal que $v_j^T A v_{j+1} = 0$.
2. s_{j+1} é o número real que minimiza $F(x^{(j)} + s v_{j+1})$, isto é, o mínimo de F ao longo da reta que passa por $x^{(j)}$ e tem a direção de v_{j+1} .
3. $x^{(j+1)} = x^{(j)} + s_{j+1} v_{j+1}$.

Existem várias maneiras de construir vetores que satisfaçam o item 1. No Algoritmo 3.5, que poderá ser usado na implementação do método GC, apresentamos um destes procedimentos. Quanto ao item 2, repetimos as mesmas contas realizadas ao minimizar a função (3.6) e podemos mostrar que

$$s_{j+1} = \frac{r_j^T v_{j+1}}{v_{j+1}^T A v_{j+1}}$$

onde, conforme a notação anterior, $r_j = b - Ax^{(j)}$.

Algoritmo 3.5 Método Gradiente Conjugado

Dados A , b , $\max$, tol

- 1: $x^{(0)} = 0$, $r = b$, $\text{aux} = r^T r$, $v = b$
 - 2: Para $k = 0 : \max$, faça
 - 3: $z = Av$
 - 4: $s = \frac{\text{aux}}{v^T z}$
 - 5: $x^{(k+1)} = x^{(k)} + sv$
 - 6: $r = r - sz$
 - 7: $\text{aux1} = r^T r$
 - 8: Se $\text{aux1} < \text{tol}$ então
 - 9: Saída com $\text{sol} = x^{(k+1)}$
 - 10: $m = \text{aux1}/\text{aux}$; $\text{aux} = \text{aux1}$
 - 11: $v = r + mv$
-

O custo computacional de cada ciclo do método envolve um produto da matriz A por um vetor, dois produtos internos e três somas de vetores. Na realização do produto da matriz pelo vetor, podemos também explorar a esparsidade da matriz, como é o caso de matrizes oriundas de discretizações de equações diferenciais parciais.

Exemplo 3.3.1. Vamos aplicar o Método Gradientes Conjugados ao sistema usado no Exemplo 3.2.1. Usando o programa `gracon.m` listado no Anexo, obtivemos os seguintes números de iterações necessárias para atingir uma precisão $\text{tol} = 10^{-5}$.

Sistema com 15 equações ($n = 15$)	8 iterações
Sistema com 20 equações	10 iterações
Sistema com 30 equações	12 iterações

Comparando estes resultados, observamos que o número de iterações realizadas, para atingir a mesma precisão, foi menor que no Método SOR como parâmetro de relaxação otimizado.

Também verificamos que o número de iterações necessárias foi menor do que a dimensão da matriz A , número de iterações previsto pela teoria do método. Como os resultados teóricos supõem que as operações e resultados sejam exatos, a diferença se deve ao fato de estarmos exigindo menos da aproximação, $\text{tol} = 10^{-5}$. Assim, bons resultados podem ser obtidos com um número de iterações menor que o previsto teoricamente. Se exigimos mais das aproximações, pode haver variações no número de iterações. Por exemplo, se tomarmos $\text{tol} = 10^{-15}$, os números de iterações serão 8 ($n = 15$), 10 ($n = 20$) e 14 ($n = 30$) e, se tomarmos $\text{tol} = 10^{-16}$, não há convergência dos resultados. $\square$

3.3.2 Pré-condicionamento e Matrizes Não Simétricas

O Método dos Gradientes Conjugados pode não convergir quando aplicado a um sistema cuja matriz é a matriz de Hilbert ($a_{ij} = 1/(i + j - 1)$) mesmo que a dimensão não seja muito grande. Por exemplo, tomando $\text{tol} = 10^{-10}$ num sistema com 15 equações, não houve convergência. O número de condição da matriz de Hilbert de dimensão 15 é da ordem de 10^{17} , o que faz com que se percam os algarismos significativos no Matlab. Este é mais um exemplo das dificuldades de se trabalhar, com aritmética ponto flutuante, em problemas malcondicionados.

Usamos a matriz de Hilbert no parágrafo acima para ilustrar como o mal condicionamento da matriz do sistema pode prejudicar a aplicação do Método Gradientes Conjugados. Isto se deve à ampliação dos erros de arredondamento quando a matriz é mal condicionada, como foi visto anteriormente. O mal condicionamento de uma matriz ocorre quando seus autovalores se distribuem num intervalo muito grande ou quando existem autovalores muito próximos de zero.

Em alguns casos, é possível acelerar a convergência do Método GC usando um pré-condicionamento do sistema. O pré-condicionamento consiste em substituir o sistema $Ax = b$ por outro equivalente, $M^{-1}Ax = M^{-1}b$, onde M é chamada matriz pré-condicionadora. O bom pré-condicionamento ocorre quando M é uma matriz "próxima" de A que satisfaz às seguintes propriedades:

1. M é simétrica positiva definida;
2. $M^{-1}A$ é bem condicionada;
3. a equação $Mx = b$ é fácil de ser resolvida.

Vasta atividade de pesquisa em pré-condicionamento de sistemas tem sido realizada nos últimos anos. Entretanto, até o presente não há uma metodologia geral para encontrar matrizes pré-condicionadoras. Existem recomendações para alguns casos particulares; citamos alguns a seguir. Quando os elementos da diagonal variam num intervalo grande,

o pré-condicionamento pela matriz diagonal, $M = D$, pode ser usado. Outras escolhas recaem em matrizes bloco-diagonais, fatorização de Cholesky incompleta e fatorização LU incompleta.

Uma vez escolhida a matriz pré-condicionadora, o Algoritmo 3.6 pode ser usado na implementação do método.

Algoritmo 3.6 Método Gradiente Conjugado com Pré-Condicionamento

Dados A , b , M (matriz pré-condicionadora), $\max$, tol

1: $x^{(0)} = 0$, $r = b$, $v = M^{-1}b$, $y = M^{-1}r$, $\text{aux} = y^T r$

2: Para $k = 0 : \max$, faça

3: $z = Av$

4: $s = \frac{\text{aux}}{v^T z}$

5: $x^{(k+1)} = x^{(k)} + sv$

6: $r = r - sz$

7: $y = M^{-1}r$

8: Se $r^T r < \text{tol}$ então

9: Saída com $\text{sol} = x^{(k+1)}$

10: $\text{aux1} = y^T r$

11: $m = \text{aux1}/\text{aux}$; $\text{aux} = \text{aux1}$

12: $v = y + mv$

O Método Gradiente Conjugado não pode ser usado em sistemas cujas matrizes não são simétricas, uma vez que os argumentos usados nesta seção não mais se aplicam. Uma primeira saída para esta dificuldade pode ser pré-multiplicar, à esquerda, o sistema linear:

$$A^T A x = A^T b.$$

Como $A^T A$ é uma matriz simétrica positiva definida, o Método GC pode ser usado neste sistema equivalente ao inicial, $Ax = b$. Este caminho pode não ser satisfatório quando a matriz A for mal condicionada. Os autovalores das matrizes mal condicionadas se distribuem num intervalo grande ou são próximos de zero. Como os autovalores de $A^T A$ são os autovalores de A elevados ao quadrado, o problema se agrava com a nova forma do sistema.

Existem outras alternativas recomendadas na literatura especializada: os métodos GMRES (*generalized minimum residual*) e QMR (*quasi-minimum residuals*), entre outros. Entretanto, parece não haver consenso de um método que seja claramente melhor. Como veremos na próxima seção, alguns destes métodos fazem parte do pacote computacional Matlab, o que facilita sua utilização em problemas práticos.

3.4 Uso do Matlab na Solução de Sistemas Lineares: Matrizes Esparças e Gradientes Conjugados

Para eliminar o desperdício de memória ao armazenar os zeros de uma matriz esparsa, podem-se guardar apenas os seus elementos não-nulos, usando um par de números que identifique a posição do elemento e seu valor. Do mesmo modo, para evitar operações envolvendo elementos nulos, alguns algoritmos especiais são aplicados no tratamento de sistemas lineares esparsos no Matlab. Tomemos a matriz usada no Exemplo 3.2.1. Para definir a matriz, no caso $n = 8$:

```
>> n = 8; r = 3;
>> a(1,1) = -2; a(n,n) = -2; a(1,2) = 1; a(n,n-1) = 1;
>> for i = 2:n-1; a(i,i) = -4; end
>> for i = 2:n-1; a(i,i+1) = 1; a(i,i-1) = 1; end
>> for i = 1:n-r; a(i,i+r) = 1; end
>> for i = r+1:n; a(i,i-r) = 1; end
```

a =

-2	1	0	1	0	0	0	0
1	-4	1	0	1	0	0	0
0	1	-4	1	0	1	0	0
1	0	1	-4	1	0	1	0
0	1	0	1	-4	1	0	1
0	0	1	0	1	-4	1	0
0	0	0	1	0	1	-4	1
0	0	0	0	1	0	1	-2

Se queremos guardar apenas os elementos não-nulos da matriz a, usamos

```
>> ae = sparse(a);      % Posições e valores dos elementos não
                        % nulos da matriz a
```

Se queremos saber quantos elementos não-nulos tem a matriz:

```
>> nnz(a)
```

ans =

32

Vamos definir o termos independente de modo que a solução do sistema $ax = b$ seja $x_i = 1$:

```
>> for i = 1:n; b(i) = sum(a(i,:)); end
```

A função que resolve sistemas quadrados pelo Método Gradiente Conjugado no Matlab é

```
>> xx = cgs(a,b,tol);    % Gradiente Conjugado com erro menor que tol
```

No exemplo acima, tomando $\text{tol} = 3 \times 10^{-15}$ e $n = 15$, houve convergência após 8 iterações, como foi obtido num programa mais simples deste capítulo. Para $n = 20$ foram necessárias 10 iterações para atingir a tolerância, ao passo que foram necessárias 15 iterações no caso da matriz de dimensão $n = 20$. Em resumo, os resultados foram semelhantes aos obtidos no Exemplo 3.3.1.

Citamos, a seguir, outros recursos do Matlab programados para matrizes esparsas. Maiores detalhes podem ser obtidos usando o help de cada uma destas funções.

bic	% Método Gradiente biconjugado
find	% Encontra os índices dos elementos não-nulos
full	% Converte uma matriz esparsa numa cheia
gmres	% Metodo Resíduo Mínimo Generalizado
luinc	% Fatoração LU incompleta
normest	% Estimativa da norma-2
qmr	% Metodo Resíduo Quase Mínimo
randperm	% Permutação randômica
spfun	% Aplica função nos elementos não-nulos
spy	% Visualiza a estrutura da esparsidade
svds	% Decomposição Valores Singulares

Exercícios

1. Resolva o sistema definido por

$$\begin{aligned} -2(1+h^2)x_1 + x_2 &= 1 \\ x_{i-1} - 2(1+h^2)x_i + x_{i+1} &= 0 & i = 2:n-1 \\ x_{n-1} - 2(1+h^2)x_n &= 1 \end{aligned}$$

com $n = 30$ e $h = 0.1$, usando os métodos iterativos de Jacobi, Gauss Seidel, SOR (com valores $\omega = 0.2i$, $i = 1:10$) e Gradientes Conjugados. Use tolerância 10^{-4} . Compare seus resultados.

2. Quantas operações são realizadas em cada iteração dos métodos de Jacobi, Gauss Seidel e SOR quando

- A é uma matriz $n \times n$?
- A é uma matriz de banda $p + q + 1$?

3. Levando em conta o exercício anterior, em quais condições os métodos iterativos serão competitivos com o Método de Eliminação Gaussiana (capítulo 2)?

4. Use o método dos gradientes conjugados para resolver os sistemas definidos pelas matrizes

$$A = \begin{bmatrix} 4 & 1 & 1 & 1 \\ 1 & 3 & -1 & 1 \\ 1 & -1 & 2 & 0 \\ 1 & 1 & 0 & 2 \end{bmatrix} \quad A = \begin{bmatrix} 4 & 1 & -1 & 1 \\ 1 & 1 & -1 & 0 \\ -1 & -1 & 5 & 2 \\ 0 & 0 & 2 & 4 \end{bmatrix}.$$

com respectivos vetores $b = (0.65, 0.05, 0.05, 0)^T$ e $b = (7, 8, -4, 6)^T$. Compare com os resultados obtidos pelo Matlab.

5. Quantas operações são realizadas em cada iteração do Método dos Gradientes Conjugados? Em quais condições este método será competitivo com o Método de Eliminação Gaussiana?

6. Usando os Algoritmos 3.5 e 3.6, compare o esforço computacional do Método Gradiente Conjugado sem e com pré-condicionamento.

7. Use o Algoritmo 3.5 para resolver o exercício 1; não deixe de explorar a esparsidade da matriz no seu programa. Houve a convergência após 30 iterações, como prevê a teoria? Caso negativo, como você explica esta contradição?

8. Esboce algumas curvas de nível do funcional quadrático

$$f(y) = \frac{1}{50} y^T \begin{bmatrix} 93 & 24 \\ 24 & 107 \end{bmatrix} y - \frac{1}{5} \begin{bmatrix} 42 \\ 31 \end{bmatrix}^T y - \frac{23}{2}, \quad y = (y_1, y_2)^T.$$

Capítulo 4

EQUAÇÕES NÃO-LINEARES

com resultados de convergência?

4.1 Introdução

Neste capítulo apresentaremos alguns métodos numéricos destinados à solução numérica de equações não-lineares. Em primeiro lugar, trataremos o caso escalar, isto é, queremos encontrar raízes de uma única equação não-linear. A seguir, generalizamos o Método de Newton para o caso de sistemas de equações não-lineares.

Embora exista uma grande quantidade de métodos destinados às equações não-lineares, trataremos de dois que são aplicados com maior frequência: os métodos de Newton e da Secante. Estes métodos apresentam bons resultados de convergência e são relativamente simples; estas são as características que lhes asseguram uma grande popularidade.

A formulação matemática do problema é a seguinte: conhecida uma função não-linear $f(x)$, procuramos valores de x^* , tais que:

$$f(x^*) = 0.$$

Graficamente, procuramos os pontos de interseção do gráfico $y = f(x)$ com o eixo dos x , como indica a figura 4.1.

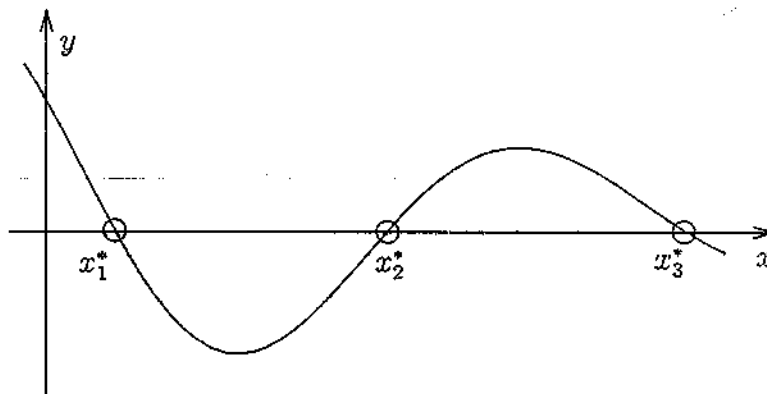


Figura 4.1: Solução gráfica de $f(x) = 0$.

No caso de sistemas de equações não-lineares, conhecemos uma função vetorial, não-linear, que denotaremos por $F(x)$, e procuramos os vetores x^* tais que, $F(x^*) = 0$.

Várias são as situações práticas que envolvem problemas desta natureza. A determinação de raízes de equações polinomiais e discretização de equações diferenciais não-lineares são dois exemplos mais comuns.

4.2 Aproximação Inicial e o Método da Bisseção

Os métodos apresentados neste capítulo são *métodos iterativos*: estabelecemos uma expressão (função de iteração) que, aplicada repetidas vezes, a partir de uma aproximação inicial conhecida, produz uma sequência de aproximações que (para ser atingido o objetivo do método) convergem para a solução do problema.

Formalizando, a idéia é encontrar uma função de iteração $\varphi(x)$ e uma aproximação inicial x_0 de modo que a sequência de aproximações $x_{n+1} = \varphi(x_n)$ convirja para a raiz x^* .

Assim, de partida devemos encontrar uma aproximação inicial, x_0 .

Nem sempre a aproximação inicial é facilmente encontrada. No caso particular de uma única equação, $f(x) = 0$, um resultado básico do Cálculo Diferencial e Integral é útil:

Teorema 4.1. *Se $y = f(x)$ é uma função contínua e muda de sinal no intervalo $[a, b]$ (isto é, se $f(a)f(b) < 0$), então existe pelo menos um ponto $\bar{x} \in [a, b]$ tal que $f(\bar{x}) = 0$. Além disso, se $f'(x)$ não muda de sinal em $[a, b]$ então $\bar{x}$ é a única raiz de $f(x)$ nesse intervalo.*

Nas condições do resultado acima, podemos tomar o ponto médio de $[a, b]$, $x_M = \frac{a+b}{2}$, como aproximação inicial. A figura 4.2 ilustra essa escolha.

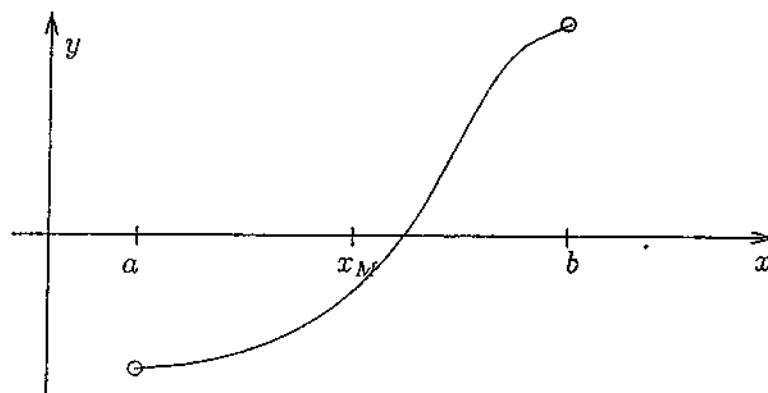


Figura 4.2: Ponto médio do intervalo $[a, b]$.

Exemplo 4.2.1. Consideremos a equação $x^3 - 2x - 5 = 0$. Sendo $f(2) = -1 < 0$ e $f(3) = 16 > 0$, uma vez que $f'(x) = 3x^2 - 2$ não se anula em $[2, 3]$, existe uma única raiz de $f(x) = 0$ neste intervalo. $\square$

Se a função troca de sinal em $[a, b]$ e x_M é o ponto médio deste intervalo, a troca de sinal será em $[a, x_M]$ ou em $[x_M, b]$. Temos então um novo intervalo que contém a raiz e tem comprimento $\frac{b-a}{2}$, isto é, metade do comprimento anterior. Podemos repetir esta idéia até obter um intervalo suficientemente pequeno que contenha a raiz. Este é o *Método da Bissecção*, resumido no Algoritmo 4.1.

Algoritmo 4.1 Método da Bissecção

Dados $f(x)$, a e b tais que $f(a)f(b) < 0$, $\max$, tol

1: Para $k = 0 : \max$, faça

2: $x = \frac{a+b}{2}$

3: Se $f(a)f(x) < 0$ então

4: $b = x$

5: Caso contrário

6: $a = x$

7: Se $|b-a| < \text{tol}$ então $x^* = \frac{b-a}{2}$, pare

Embora o Método da Bissecção seja simples, ele sofre de uma séria desvantagem em se tratando de métodos numéricos: é lento. De fato, podemos verificar que, ao fim de n iterações do Método da Bissecção, teremos uma aproximação para a raiz cujo erro absoluto é menor que 2^{-n} , isto é,

$$|x - x^*| \leq \frac{b-a}{2^n}.$$

Por exemplo, se $a = 0$ e $b = 2$, a aproximação da quinta iteração, $n = 5$, satisfaz a

$$|x - x^*| \leq 0.0625,$$

o que não é uma boa estimativa de erro. Por outro lado, as aproximações fornecidas pelo método permanecem dentro do intervalo inicial, o que pode representar uma vantagem em aplicações, por exemplo, quando existem restrições nos valores da variável x .

Exemplo 4.2.2. A raiz quadrada de 2 pode ser calculada resolvendo a equação $f(x) = x^2 - 2 = 0$. Como $f(1) = -1$ e $f(2) = 2$, podemos usar o Método da Bissecção no intervalo $[1, 2]$. Na primeira coluna da tabela 4.2.2, apresentamos os resultados das cinco primeiras iterações obtidas com este método. Nesta tabela, também, apresentamos as aproximações obtidas com os métodos de Newton e da secante, apresentados a seguir.

Bisseção	Newton	Secante
1.500000000000000	1.500000000000000	1.333333333333333
1.250000000000000	1.416666666666667	1.400000000000000
1.375000000000000	1.41421568627451	1.41463414634146
1.437500000000000	1.41421356237469	1.41421143847487
1.406250000000000	1.41421356237310	1.41421356205732

Tabela 4.1: Cálculo de $\sqrt{2}$ usando três métodos.

□

4.3 Método de Newton

Isaac Newton (1642–1727) publicou seu método para encontrar raízes de equações não lineares em 1687. Este método também é conhecido como Newton-Raphson, devido à sistematização apresentada por Joseph Raphson em 1690.

O *Método de Newton* combina duas idéias básicas comuns nas aproximações numéricas: linearização e iteração.

Na linearização, procuramos substituir (numa certa vizinhança) um problema complicado por sua aproximação linear que, por via de regra, é facilmente resolvida. A aproximação linear pode ser obtida tomando os primeiros termos do desenvolvimento em série de Taylor da função em questão.

Por outro lado, um processo iterativo, ou aproximações sucessivas, consiste na repetição sistemática de um procedimento até que seja atingido o grau de aproximação desejado.

No caso do Método de Newton, a linearização consiste na “substituição” da curva $y = f(x)$ por sua reta tangente.

Seja x_0 uma aproximação inicial da raiz, como ilustra a figura 4.3. Aproximando a curva $y = f(x)$ por sua reta tangente traçada no ponto $(x_0, f(x_0))$ obtemos a aproximação linear. Encontrando o ponto de intersecção desta reta com o eixo dos x , obteremos uma nova aproximação para a raiz, o ponto x_1 da figura.

O processo iterativo será obtido pela repetição do procedimento. A nova aproximação, representada na figura 4.3 pelo ponto x_2 , foi calculada a partir da reta tangente à curva $y = f(x)$ no ponto $(x_1, f(x_1))$.

Para estabelecer expressões analíticas que permitam o cálculo de $x_1, x_2, \dots$ observamos que a tangente do ângulo θ pode ser obtida tanto da definição da função trigonométrica tangente quanto pela derivada de $f(x)$ no ponto x_0 (inclinação da reta tangente). Assim, ainda olhando a figura 4.3 temos:

$$\tan \theta = \frac{f(x_0)}{x_0 - x_1} = f'(x_0).$$

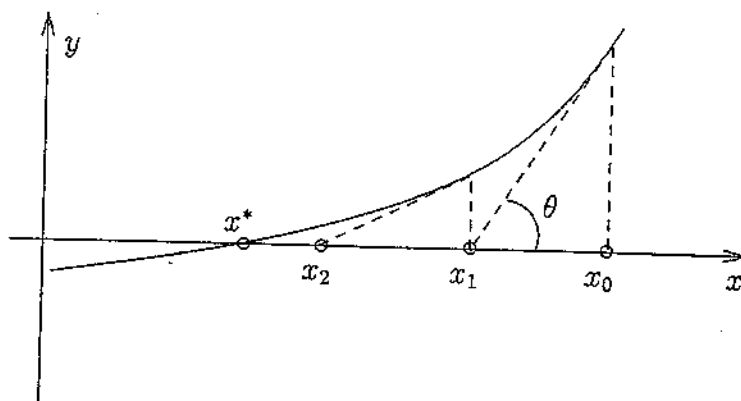


Figura 4.3: Retas tangentes em $(x_i, f(x_i))$.

Da igualdade à direita explicitamos x_1 :

$$x_1 = x_0 - \frac{f(x_0)}{f'(x_0)}.$$

Na repetição, uma nova iteração, calculamos x_2 a partir de x_1 por

$$x_2 = x_1 - \frac{f(x_1)}{f'(x_1)},$$

e assim sucessivamente.

Genericamente, o processo consiste em evoluir da aproximação x_k para a aproximação x_{k+1} usando a fórmula

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}, \quad k = 0, 1, 2, \dots \quad (4.1)$$

O lado direito desta expressão define a *Função de Iteração* do método.

Prosseguimos repetindo os cálculos até que a precisão desejada seja atingida. Podemos parar, por exemplo, quando o valor de $f(x_k)$ for suficientemente próximo de zero. Podemos também estabelecer como critério de parada a falta de progresso nos cálculos. Isto ocorre quando a diferença de dois iterados torna-se muito pequena.

O Método de Newton está esquematizado no Algoritmo 4.2.

Exemplo 4.3.1. A raiz quadrada de um número positivo A pode ser calculada resolvendo a equação $f(x) = x^2 - A = 0$. Na aplicação do Método de Newton para este problema, a função de iteração será

$$\phi(x) = x - \frac{f(x)}{f'(x)} = x - \frac{x^2 - A}{2x}.$$

Assim, as iterações serão obtidas a partir de uma aproximação inicial ($x_0 = \frac{A}{2}$, por exemplo), por

$$x_{k+1} = x_k - \frac{x_k^2 - A}{2x_k} = 0.5 \left(x_k + \frac{A}{x_k} \right).$$

No caso particular $A = 2$, tomando $x_0 = 1$, a sequência das aproximações de $\sqrt{2}$ definidas pelo Método de Newton são apresentadas na segunda coluna da tabela 4.2.2.

Observamos que a partir de x_5 o valor se repete porque os nove dígitos não se modificam. Comparando estes resultados com os apresentados no Exemplo 4.2.2, verificamos que o resultado da segunda iteração do Método de Newton é melhor que o valor da quinta iteração do Método da Bissecção. A comparação ilustra a diferença na taxa de convergência dos dois métodos. $\square$

Algoritmo 4.2 Método de Newton

Dados $x_0, f(x), f'(x), \epsilon_1$ e/ou ϵ_2

- 1: Para $k = 1, 2, \dots$, faça
 - 2: $x_k = x_{k-1} - \frac{f(x_{k-1})}{f'(x_{k-1})}$
 - 3: Se $|f(x_k)| < \epsilon_1$ então $x^* = x_k$
 - 4: ou ...
 - 5: Se $|x_k - x_{k-1}| < \epsilon_2$ então $x^* = x_k$
-

4.4 Método das Secantes

Alternativamente, podemos usar retas secantes como aproximações lineares locais da função, em vez das tangentes. Neste caso, são necessárias duas aproximações para iniciarmos o processo. Chamaremos estas aproximações iniciais de x_0 e x_1 .

No Método da Secante, tomamos a reta que passa pelos pontos $(x_0, f(x_0))$ e $(x_1, f(x_1))$ como uma aproximação linear da curva $y = f(x)$, como indica a figura 4.4.

Para estabelecer a relação de recorrência do Método das Secantes, usamos a semelhança de triângulos ABC e AED :

$$\frac{f(x_0)}{x_0 - x_2} = \frac{f(x_1)}{x_1 - x_2}$$

onde x_2 é o ponto denotado por A na figura. Explicitado o valor da incógnita x_2 teremos

$$x_2 = \frac{x_0 f(x_1) - x_1 f(x_0)}{f(x_1) - f(x_0)}.$$

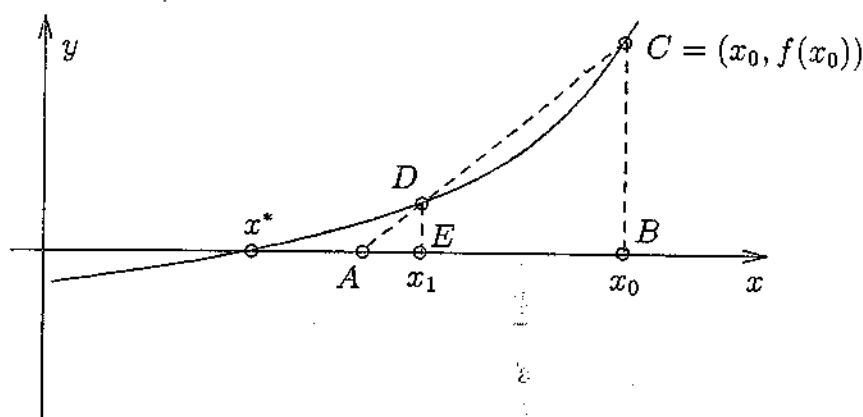


Figura 4.4: Reta secante passando por C e D .

Generalizando, no método das secantes usamos duas aproximações, x_{k-1} e x_k , para calcular uma nova aproximação x_{k+1} , através da fórmula

$$x_{k+1} = \frac{x_{k-1} f(x_k) - x_k f(x_{k-1})}{f(x_k) - f(x_{k-1})}. \quad (4.2)$$

Somando e subtraindo $x_k f(x_k)$ ao numerador de (4.2) e agrupando convenientemente obtemos

$$x_{k+1} = x_k - \frac{f(x_k)}{\frac{f(x_k) - f(x_{k-1})}{x_k - x_{k-1}}}.$$

Observamos então que a diferença entre este método e o Método de Newton, equação (4.1), está na aproximação da derivada pela secante, isto é,

$$f'(x_k) \approx \frac{f(x_k) - f(x_{k-1})}{x_k - x_{k-1}}.$$

O procedimento está sistematizado no Algoritmo 4.3. Observe que, neste algoritmo, usamos o mesmo critério de parada empregado no Método de Newton.

Algoritmo 4.3 Método das Secantes

Dados $x_0, x_1, f(x), \epsilon_1$ e/ou ϵ_2

1: Para $k = 1, 2, \dots$, faça

2: $x_{k+1} = \frac{x_{k-1} f(x_k) - x_k f(x_{k-1})}{f(x_k) - f(x_{k-1})}.$

3: Se $|f(x_{k+1})| < \epsilon_1$ então $x^* = x_{k+1}$

4: ou ...

5: Se $|x_{k+1} - x_k| < \epsilon_2$ então $x^* = x_{k+1}$

Exemplo 4.4.1. Considere, mais uma vez, o cálculo de raízes quadradas de números positivos. Para calcular a raiz quadrada de 2, podemos aplicar o Método das Secantes na equação $f(x) = x^2 - 2 = 0$. Usando as aproximações iniciais $x_0 = 1$ e $x_1 = 2$, obtemos as aproximações apresentadas na terceira coluna da tabela 4.2.2. A função de iteração usada neste exemplo é:

$$x_{k+1} = \frac{x_k(x_{k+1}^2 - A) - x_{k+1}(x_k^2 - A)}{x_{k+1}^2 - x_k^2} = \frac{A + x_k x_{k-1}}{x_{k+1} + x_k},$$

onde $A = 2$. □

Observe que no Método das Secantes não é necessário o cálculo de valores da derivada de $f(x)$, como era o caso do Método de Newton. Os valores de $f(x_k)$ e $f(x_{k-1})$, que de qualquer modo seriam calculados, são usados. Isso também representa um ganho, quando as derivadas forem complicadas ou suas expressões envolverem cálculos custosos.

4.5 A Convergência dos Métodos de Newton e da Secante

Pode-se mostrar que, se a aproximação inicial está próxima da raiz procurada e a função $f(x)$ tem regularidade suficiente (derivadas contínuas), então há garantia da convergência dos métodos de Newton e da Secante. No que diz respeito à evolução do erro da aproximação, podemos citar o seguinte resultado, cuja demonstração pode ser encontrada na bibliografia especializada ([Conte-deBoor, 1981], por exemplo).

Teorema 4.2. *Sejam x^* a raiz da equação $f(x) = 0$ e x_k uma aproximação de x^* , calculada na k -ésima iteração do Método de Newton ou da Secante, conforme o caso. Chamemos de e_k o erro absoluto dessa aproximação. Então, existem α_1 e α_2 (próximos de x^* , dependentes de k e do método usado), tais que:*

$$e_{k+1} = -\frac{f''(\alpha_1)}{2f'(\alpha_2)} e_k^2 \quad \text{no Método de Newton;} \quad (4.3)$$

$$e_{k+1} = -\frac{f''(\alpha_1)}{2f'(\alpha_2)} e_k e_{k-1} \quad \text{no Método da Secante.} \quad (4.4)$$

Este resultado é a explicação teórica da velocidade de convergência dos dois métodos. Observe que, se uma iteração do Método de Newton fornece uma aproximação com o erro da ordem de 10^{-4} , na iteração seguinte o erro será da ordem de 10^{-8} . Algo semelhante acontece com os erros das aproximações obtidas por via de secantes. Por esse motivo, os dois métodos convergem em quatro ou cinco iterações (sendo o das secantes ligeiramente mais lento).

Dizemos que uma sequência x_k converge para x^* com a ordem de convergência α se

$$\lim_{k \rightarrow \infty} \frac{e_{k+1}}{e_k^\alpha} = \lambda,$$

sendo λ independente de k e $e_k = |x_k - x^*|$, o erro absoluto da aproximação x_k . Com base nesta definição, (4.3) nos diz que a convergência do Método de Newton é quadrática ($\alpha = 2$).

Por outro lado, embora (4.4) não nos garanta também uma convergência quadrática para o Método das Secantes, ela nos diz que a convergência é mais que linear ($\alpha > 1$). É comum usar o termo convergência superlinear quando $1 < \alpha < 2$.

A tabela do exemplo 4.2.2 ilustra a velocidade de convergência dos métodos apresentados.

4.6 Método de Newton para Sistemas Não-Lineares

Nesta seção abordamos o problema um pouco mais complicado: resolver um conjunto de equações não-lineares acopladas entre si.

Para formular o problema consideremos dadas n funções $f_1, f_2, \dots, f_n$, onde cada uma delas é função das n variáveis $x_1, x_2, \dots, x_n$. Queremos encontrar os valores de $x_1^*, x_2^*, \dots, x_n^*$, tais que

$$\begin{cases} f_1(x_1^*, x_2^*, \dots, x_n^*) = 0 \\ f_2(x_1^*, x_2^*, \dots, x_n^*) = 0 \\ \vdots \\ f_n(x_1^*, x_2^*, \dots, x_n^*) = 0. \end{cases} \quad (4.5)$$

Por exemplo, no sistema não-linear

$$\begin{cases} -x_1(x_1 + 1) + 2x_2 = 18 \\ (x_1 - 1)^2 + (x_2 - 6)^2 = 25 \end{cases}$$

procuramos x_1 e x_2 , tais que

$$f_1(x_1^*, x_2^*) = -x_1^*(x_1^* + 1) + 2x_2^* - 18 = 0$$

$$f_2(x_1^*, x_2^*) = (x_1^* - 1)^2 + (x_2^* - 6)^2 - 25 = 0.$$

Na tentativa de simplificar, usaremos a notação vetorial: $\mathbf{x}$ é um vetor com n componentes e $\mathbf{F}(\mathbf{x})$ é uma função vetorial de variável vetorial, isto é,

$$\mathbf{x} = (x_1, x_2, \dots, x_n)$$

$$\mathbf{F}(\mathbf{x}) = (f_1(\mathbf{x}), f_2(\mathbf{x}), \dots, f_n(\mathbf{x})).$$

Nesta notação, o sistema (4.5) será representado por uma única equação vetorial,

$$\mathbf{F}(\mathbf{x}) = \mathbf{0}.$$

Como foi feito no caso de um equação não-linear, a idéia da linearização será usada no caso de sistemas. Como sabemos, a série de Taylor é útil na aproximação de funções. No caso escalar, onde $f(x)$ e x são números reais, a série de Taylor em torno de x_k é

$$f(x) = f(x_k) + f'(x_k)(x - x_k) + \frac{f''(x_k)}{2!}(x - x_k)^2 + \dots + \frac{f^{(n)}(x_k)}{n!}(x - x_k)^n + \frac{f^{(n+1)}(\alpha)}{(n+1)!}(x - x_k)^{n+1},$$

onde $x \leq \alpha \leq x_k$. O último termo desta expressão representa o erro da aproximação de $f(x)$ pelo polinômio de grau n , o polinômio de Taylor, formado pelos $n+1$ primeiros termos da expansão em série de Taylor.

Observe que, se tomarmos a aproximação linear (usando os dois primeiros termos da série de Taylor), teremos

$$f(x) \approx f(x_k) + f'(x_k)(x - x_k).$$

Se usarmos esta aproximação linear para encontrar a raiz de $f(x) = 0$,

$$f(\bar{x}) \approx 0 \quad \text{se} \quad f(x_k) + f'(x_k)(\bar{x} - x_k) = 0,$$

ou ainda, explicitando $\bar{x}$ temos

$$\bar{x} = x_k - \frac{f(x_k)}{f'(x_k)}. \quad (4.6)$$

Assim, a série de Taylor é uma outra maneira de se obter a função de iteração do método de Newton, estabelecida anteriormente na equação (4.1).

Repetindo esta idéia para resolver o sistema (4.5), procedemos à linearização através da série de Taylor. Neste caso, são necessárias algumas adaptações porque as entidades do problema matemático são diferentes. No caso anterior, tínhamos funções escalares de variáveis escalares, ao passo que agora temos funções vetoriais com variáveis vetoriais.

Sem maiores detalhes, que alongariam demasiadamente esta seção, a aproximação de Taylor no caso de sistemas será:

$$\begin{aligned} \mathbf{F}(\mathbf{x}) &= \mathbf{F}(\mathbf{x}^k) + \mathbf{F}'(\mathbf{x}^k)(\mathbf{x} - \mathbf{x}^k) + \mathbf{E} = \\ &= \text{aproximação linear} + \text{erro} \end{aligned} \quad (4.7)$$

onde $\mathbf{x}^k$ = vetor aproximação na k -ésima iteração e $\mathbf{E}$ é um vetor que representa o erro da aproximação linear. Analogamente à série de Taylor de funções de uma variável, $\|\mathbf{E}\| \leq C\|\mathbf{x} - \mathbf{x}^k\|^2$ para alguma constante C , quando $\mathbf{x}^k \rightarrow \mathbf{x}$.

Observe que mudamos um pouco a notação, $\mathbf{x}^k$ é o vetor obtido na k -ésima iteração porque os subíndices denotarão as componentes do vetor.

Em (4.7), $\mathbf{F}'(\mathbf{x})$ é a derivada de uma função vetorial com variáveis vetoriais. Como visto no Cálculo Diferencial e Integral, esta derivada é uma matriz que contém todas as derivadas parciais de todas as componentes da função $\mathbf{F}(\mathbf{x})$, a matriz *Jacobiana* de $\mathbf{F}(\mathbf{x})$,

$$\mathbf{J}(\mathbf{x}) = \mathbf{F}'(\mathbf{x}) = [J_{ij}] = \left[\frac{\partial f_i(\mathbf{x})}{\partial x_j} \right].$$

Para estabelecer o método iterativo, a aproximação na iteração $k+1$ será definida pelo vetor que anula a parte linear da equação (4.7), isto é, o vetor $\mathbf{x}^{k+1}$ é tal que:

$$\mathbf{F}(\mathbf{x}^k) + \mathbf{F}'(\mathbf{x}^k)(\mathbf{x}^{k+1} - \mathbf{x}^k) = 0. \quad (4.8)$$

Para explicitar $\mathbf{x}^{k+1}$, multiplicamos esta equação pela inversa da matriz Jacobiana e teremos

$$\mathbf{x}^{k+1} = \mathbf{x}^k - \mathbf{J}^{-1}(\mathbf{x}^k) \mathbf{F}(\mathbf{x}^k),$$

que é a adaptação de (4.6) ao caso dos sistemas não-lineares.

Como a inversão de matrizes é uma operação cara, podemos evitá-la trabalhando na expressão (4.8), isto é, encontramos a aproximação da iteração $k+1$ resolvendo um sistema linear para calcular $\mathbf{v} = \mathbf{x}^{k+1} - \mathbf{x}^k$:

$$\mathbf{J}(\mathbf{x}^k) \mathbf{v} = -\mathbf{F}(\mathbf{x}^k). \quad (4.9)$$

Assim, a nova aproximação será:

$$\mathbf{x}^{k+1} = \mathbf{x}^k + \mathbf{v}.$$

Exemplo 4.6.1. Vamos usar o método de Newton para resolver o sistema

$$f_1(x_1, x_2) = 2x_1^3 - x_2^2 - 1 = 0$$

$$f_2(x_1, x_2) = x_1 x_2^3 - x_2 - 4 = 0.$$

As derivadas parciais destas funções são

$$\frac{\partial f_1}{\partial x_1} = 6x_1^2 \quad \frac{\partial f_1}{\partial x_2} = -2x_2$$

$$\frac{\partial f_2}{\partial x_1} = x_2^3 \quad \frac{\partial f_2}{\partial x_2} = 3x_1 x_2^2 - 1$$

e, portanto, a matriz Jacobiana é

$$\mathbf{J}(x_1, x_2) = \begin{bmatrix} 6x_1^2 & -2x_2 \\ x_2^3 & 3x_1 x_2^2 - 1 \end{bmatrix}.$$

Tomando uma aproximação inicial $\mathbf{x}^0 = (1.2, 1.7)$ temos:

$$\mathbf{J}(\mathbf{x}^0) = \begin{bmatrix} 8.64 & -3.4 \\ 4.91 & 9.4 \end{bmatrix} \quad \text{e} \quad \mathbf{F}(\mathbf{x}^0) = \begin{bmatrix} -0.4340 \\ 0.1956 \end{bmatrix}.$$

Resolvendo o sistema (4.9) encontramos

$$\mathbf{v} = (0.03488, -0.03902)^T.$$

Assim,

$$\mathbf{x}^1 = (1.2349, 1.6610)^T.$$

Para o cálculo de uma nova aproximação recalculamos $F(\mathbf{x}^1)$ e a matriz Jacobiana em $\mathbf{x}^1$. Resolvendo o sistema correspondente a (4.9) temos

$$\mathbf{x}^2 = (1.2343, 1.6615)^T.$$

Podemos testar as aproximações obtidas calculando os respectivos f_1 e f_2 :

$$\begin{aligned} f_1(\mathbf{x}^0) &= -0.4340 & f_2(\mathbf{x}^0) &= 0.1956 \\ f_1(\mathbf{x}^1) &= 7.4607 \times 10^{-3} & f_2(\mathbf{x}^1) &= -1.9876 \times 10^{-3} \\ f_1(\mathbf{x}^2) &= -1.2779 \times 10^{-4} & f_2(\mathbf{x}^2) &= 3.202 \times 10^{-4} \end{aligned}$$

□

Pode-se mostrar ([Blum, 1972], por exemplo) que, também no caso de sistemas, a convergência do Método de Newton é quadrática, ou seja, o erro em cada iteração é da ordem do quadrado do erro na iteração anterior.

O Método de Newton é descrito no Algoritmo 4.4.

Algoritmo 4.4 Método de Newton para sistemas

Dados $\mathbf{x}^0$, f_i , $i = 1 : n$, $\partial f_i / \partial x_j$, $i, j = 1 : n$, ϵ_1 e ϵ_2

- 1: Para $k = 1, 2, \dots$, faça
 - 2: Para $i = 1 : n$, faça
 - 3: $F_i = f_i(\mathbf{x}^{k-1})$
 - 4: Para $j = 1 : n$, faça
 - 5: $J_{ij} = \frac{\partial f_i}{\partial x_j}(\mathbf{x}^{k-1})$
 - 6: Resolva o sistema $J\mathbf{v} = -\mathbf{F}$
 - 7: $\mathbf{x}^k = \mathbf{x}^{k-1} + \mathbf{v}$
 - 8: Se $\max_{1 \leq i \leq n} |f_i(\mathbf{x}^k)| < \epsilon_1$ então $\mathbf{x}^* = \mathbf{x}^k$
 - 9: ou ...
 - 10: Se $\max_{1 \leq i \leq n} |\mathbf{x}_i^k - \mathbf{x}_i^{k-1}| < \epsilon_2$ então $\mathbf{x}^* = \mathbf{x}^k$
-

Observando mais detalhadamente este algoritmo, vemos que ele inclui as seguintes etapas delicadas:

- i. Escolha da aproximação inicial;
- ii. cálculo da matriz Jacobiana $J(\mathbf{x}^k)$;
- iii. solução do sistema linear $J(\mathbf{x}^k)\mathbf{v} = -F(\mathbf{x}^k)$.

Para que a implementação computacional do Método de Newton seja eficiente, torna-se necessário um certo cuidado em cada uma dessas etapas; abordaremos alguns pontos.

Infelizmente, pouco progresso tem sido obtido na escolha de boas aproximações iniciais. Se não dispomos de alguma informação *a priori*, duas idéias podem ser aplicadas, com sucesso relativo. Na técnica de *continuação* tenta-se construir uma função que dependa de um só parâmetro, cuja raiz será uma aproximação inicial. No Método de Newton *amortecido*, não se permite que a nova aproximação forneça resultados piores que os anteriores: se esse for o caso, procura-se i tal que $\mathbf{x}^{k+1} = \mathbf{x}^k + \mathbf{v}/2^i$ com $\|\mathbf{F}(\mathbf{x}^{k+1})\| < \|\mathbf{F}(\mathbf{x}^k)\|$ (veja [Conte-deBoor, 1981] para maiores detalhes).

Em ii, pode-se evitar o trabalho, muitas vezes enfadonho, de calcular todas as derivadas parciais optando pelo cálculo da matriz Jacobiana por discretização das derivadas. Neste caso, as diferenças finitas são aproximações (os detalhes serão apresentados no capítulo de diferenciação numérica):

$$\frac{\partial f_i}{\partial x_j} \approx \frac{f_i(x_1, \dots, x_j + h, \dots, x_n) - f_i(x_1, \dots, x_j, \dots, x_n)}{h}$$

ou

$$\frac{\partial f_i}{\partial x_j} \approx \frac{f_i(x_1, \dots, x_j + h, \dots, x_n) - f_i(x_1, \dots, x_j - h, \dots, x_n)}{2h}.$$

Ainda no sentido de simplificação do cálculo da matriz Jacobiana, pode-se tentar imitar o Método das Secantes, apresentado na seção anterior. Lá usamos a aproximação

$$f'(x_{k+1}) \approx \frac{f(x_{k+1}) - f(x_k)}{x_{k+1} - x_k}.$$

No caso de sistemas, é possível encontrar aproximações da matriz Jacobiana em $\mathbf{x}^{k+1}$ a partir de atualizações adequadas de $\mathbf{J}_k = \mathbf{J}(\mathbf{x}^k)$. O Método de Broyden fornece expressões para esta atualização: se $\mathbf{x}^{k+1}$ e $\mathbf{x}^k$ são duas aproximações conhecidas, chamando os vetores coluna

$$\mathbf{s}_k = \mathbf{x}^{k+1} - \mathbf{x}^k \quad \text{e} \quad \mathbf{y}^k = \mathbf{F}(\mathbf{x}^{k+1}) - \mathbf{F}(\mathbf{x}^k).$$

podemos tomar

$$\mathbf{J}_{k+1} = \mathbf{J}_k + \frac{\mathbf{y}^k - \mathbf{J}_k \mathbf{s}_k}{\mathbf{s}_k^T \mathbf{s}_k} \mathbf{s}_k^T$$

como uma aproximação para a Jacobiana no vetor $\mathbf{x}^{k+1}$. Os métodos que usam aproximações deste tipo para a Jacobiana são chamados métodos Quase-Newton, um dos quais é o Método de Broyden. Estes métodos são usados com sucesso a partir da década de 1970. Maiores detalhes podem ser obtidos em [Dennis-Schnabel, 1996].

Na etapa da solução do sistema, iii, existem as alternativas apresentadas nos capítulos anteriores.

No caso de escolha de métodos iterativos, pode-se adotar um número fixo de iterações, para cada k , em vez de esperar a convergência completa. Este procedimento, conhecido

como *Newton inexato*, pode ser econômico e eficiente. Vários autores estudaram o comportamento do Método Newton inexato e, em 1982, Dembo, Eisenstat e Steihaug deram uma resposta satisfatória à questão do número conveniente de iterações a serem usadas. No critério desses autores, se v é qualquer incremento que satisfaça a

$$\|J(x^k)v + F(x^k)\| \leq \theta_k \|F(x^k)\|,$$

onde $0 < \theta_k < \theta < 1$ e $\|\bullet\|$ é alguma norma em $\mathbb{R}^n$, então a convergência linear do Método de Newton inexato está garantida. Em alguns casos, pode-se obter convergência superlinear.

Tendo em vista que a solução do sistema e o cálculo da Jacobiana são dispendiosos, podemos adotar uma matriz Jacobiana fixa em algumas iterações, para economizar os cálculos. Neste caso, a decomposição LU é adequada na solução do sistema, pois a mesma decomposição será usada em algumas iterações, atualizando-se apenas o termo independente do sistema.

Para interpretar esta alternativa, recorreremos ao caso escalar, isto é, $f(x) = 0$. Assim, em vez de procedermos aproximações lineares por via de retas tangentes, o fazemos durante algumas iterações, usando paralelas à tangente usada na iteração anterior, como ilustra a figura 4.5.

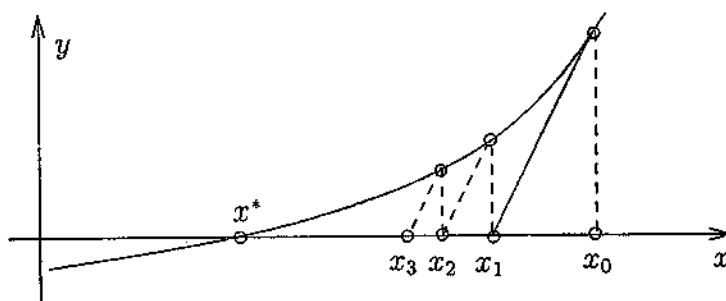


Figura 4.5: Newton modificado.

4.7 Uso do Matlab para Encontrar Raízes de Equações

Para a verificação gráfica de raízes de funções podemos usar alguns recursos do Matlab. Por exemplo,

```
>> x = linspace(0.3,1); % Gera a malha com 100 pontos igualmente espaçados
>> y = sin(1./x)./x; % Cálculo do vetor com os valores da função
                        % nos pontos da malha
>> plot(x,y) % Traça o gráfico
>> grid % Desenha a malha
>> title('Grafico da Funcao sen(1/x)/x') % Título da Figura
```

```
>> xlabel('Variavel x')           % Legenda do Eixo x
>> ylabel('Valores de y(x)')      % Legenda do Eixo y
```

Observando a figura gerada por estes comandos, verificamos que o gráfico da função corta o eixo dos x num ponto do intervalo $(0.3, 0.4)$ e, portanto, neste intervalo existe uma raiz para a função.

Com o recurso simbólico do Matlab, podemos obter o gráfico da função $y(x) = \sin(1/x)/x$ usando

```
>> fplot('sin(1/x)/x', [0.3, 1])
```

As raízes da função podem ser calculadas usando-se a função `fzero`. Para calcular raízes de uma equação, definimos inicialmente a função num arquivo, aqui denominado `funcao.m`, que deverá estar na área de trabalho:

```
function y = funcao(x)
y = sin(1/x)/x;
```

A raiz mais próxima de 0.3 pode ser calculada usando

```
>> xzero=fzero('funcao', .3)
Zero found in the interval: [0.276, 0.324].
xzero =
    0.3183
```

Pode-se verificar que, se usamos 0.25 em vez de 0.3, a resposta será diferente, `xzero = 0.1592`, que é outra raiz da função. Esta outra raiz pode ser observada pelo gráfico da função no intervalo $(0.1, 1)$, usando `fplot('sin(1/x)/x', [0.1, 1])`. De fato, ao ampliar o intervalo, surgiram mais duas raízes da função.

Podemos usar o Matlab para calcular máximos e mínimos de funções de uma ou mais variáveis. Os métodos numéricos usados são iterativos e, para iniciá-los, devemos fornecer os intervalos, se for o caso de função de uma variável, ou uma aproximação inicial para o vetor solução, no caso de funções de várias variáveis. Por exemplo, para encontrar o mínimo da função definida por uma `funcao.m` acima, usamos

```
>> fmin('funcao', 0.15, 0.2)

ans =

    0.2000
```

Para encontrar mínimos de funções de várias variáveis, usamos `fmins('g',x0)`, onde `g` é a função a ser minimizada e `x0` é um vetor com os valores iniciais do processo iterativo.

No Matlab, existem recursos específicos para o cálculo de raízes de polinômios. Um polinômio é definido por meio de um vetor que contém seus coeficientes, sendo o primeiro elemento do vetor o coeficiente do termo de maior grau do polinômio. A seguir, apresentamos alguns dos recursos do Matlab dedicados aos polinômios. Tomaremos como exemplo $p(x) = 3x^4 - 2x - 4$.

```
>> p = [3 0 0 -2 -4];      % Vetor que define o polinômio
>> z = polyval(p,2);      % Cálculo de p(2) = 40.
>> r = roots(p)           % Vetor das raízes do polinômio:
                           % duas reais e duas complexas

r =
    1.2094
   -0.1442 + 1.0846i
   -0.1442 - 1.0846i
   -0.9210

>> p1 = poly(r)           % Polinômio cujas raízes são dadas
                           % 0 coeficiente líder é 1

p1 =
    1.0000    0.0000    0.0000   -0.6667   -1.3333

>> pm = conv(p,p1)        % Multiplicação do polinômio p
                           % (vetor dos coeficientes) por p1

>> [q,r] = deconv(p,p1)   % Quociente e resto da divisão de p por p1

q =
     3

r =
    1.0e-014 *
         0   -0.0666   -0.1665   -0.7772   -0.7105
```

A seguir, apresentamos um exemplo de solução de um sistema não-linear com duas equações e duas incógnitas. O programa foi escrito para o Matlab com cálculo simbólico, que existe na Versão Estudantil ou no *Toolbox Symbolic*. Além do exercício de raízes de sistemas, com este exemplo ilustramos como podemos gerar programas na linguagem do Matlab, criando M-files que podem ser usados nas diversas aplicações. Vamos aplicar o Método de Newton para resolver o seguinte sistema não-linear de duas equações com duas

incógnitas

$$f_1(x, y) = 2x^3 - y^2 - 1 = 0$$

$$f_2(x, y) = xy^3 - y - 4 = 0.$$

O programa elaborado foi denominado `sis2.m`. Nele, podemos verificar como são definidas funções no *symbolic*, como são calculadas derivadas e como as funções são avaliadas. Os valores calculados nas iterações são guardados numa matriz `ms`, e os valores que as funções $f_1(x, y)$ e $f_2(x, y)$ assumem em cada iteração estão na matriz `mf`.

```
% sis2.m : Newton para sistema 2x2
% Neste programa usamos cálculo simbólico do Matlab

f1 = '2*x^3-y^2-1';           % Primeira Equação do Sistema
f2 = 'x*y^3-y-4';            % Segunda Equação do Sistema

f11 = diff(f1,'x'); f12 = diff(f1,'y'); % Derivadas Parciais de f1
f21 = diff(f2,'x'); f22 = diff(f2,'y'); % Derivadas Parciais de f2

x = 2; y = 3;                 % Aproximações Iniciais
for k = 1:6                    % Laço das Iterações
    J(1,1) = eval(f11);       % Cálculo da Jacobiana
    J(1,2) = eval(f12);
    J(2,1) = eval(f21);
    J(2,2) = eval(f22);

    f(1) = eval(f1);          % Valores de f1(x,y) e f2(x,y)
    f(2) = eval(f2);

    v = -J\f';                % Solução do Sistema Linear

    x = x + v(1);              % Atualizando a Aproximação
    y = y + v(2);

    ms(k,1) = x;               % Guardando as Aproximações na Iteração k
    ms(k,2) = y;
    mf(k,1) = f(1);            % Guardando os Valores das Funções nas Iterações
    mf(k,2) = f(2);
end
```

Executando este programa, foram obtidas as aproximações:

x_k	1.5816	1.3479	1.2522	1.2348	1.2343	1.2343	1.2343
y_k	2.3264	1.8951	1.7008	1.6628	1.6615	1.6615	1.6615

E os valores que as funções assumem nestas iterações são

$f_1(x_k, y_k)$	6.0000	1.5005	0.3067	0.0345	0.0008	0.0000	0.0000
$f_2(x_k, y_k)$	47.0000	13.5861	3.2782	0.4598	0.0147	0.0000	0.0000

Embora os cálculos tenham sido realizados com 16 dígitos, como o faz o Matlab, os resultados foram apresentados no format short.

O programa mostra que houve convergência, com a aproximação inicial (2,3), após cinco iterações. Mudando a aproximação inicial para $x = 0$ e $y = 1$, verificamos que a solução será obtida após 28 iterações. Se usamos, como aproximação inicial, $x = -1$ e $y = 0$, a solução será obtida após realizadas 53 iterações. Por outro lado, se usamos a aproximação inicial $x = 0$ e $y = 0$, não poderemos calcular nenhuma iteração, pois a matriz Jacobiana é singular, isto é, não tem inversa.

Exercícios

1. Use os métodos de Newton e da Secante para encontrar aproximações para as equações abaixo, nos intervalos indicados. Use como parada $\epsilon_2 = 10^{-4}$

a. $x^3 + 3x^2 - 1 = 0$, $[-4, 0]$,

b. $x - \cos x = 0$, $[0, \pi/2]$.

2. Use o método de Newton para encontrar aproximações para todas as soluções da equação abaixo. Use $\epsilon_1 = 10^{-5}$

$$3x^2 - e^x = 0.$$

3. Use o Método da Secante para aproximar, com precisão 10^{-4} , o ponto sobre a curva $y = \frac{1}{x}$ que está mais próximo de (2, 1).

4. Descreva um algoritmo para cálculo de raiz cúbica de um número x , usando o Método de Newton.

5. A função $f(x) = (4x - 7)/(x - 2)$ se anula em $x = 7/4$. Calcule as iterações do Método de Newton partindo das aproximações iniciais:

- a. $x_0 = 1.625$ b. $x_0 = 1.875$ c. $x_0 = 1.5$
d. $x_0 = 1.95$ e. $x_0 = 3$ f. $x_0 = 7$

Explique graficamente seus resultados.

6. Ainda com $f(x)$ definida no exercício anterior, verifique que $f(1.8)f(3) < 0$. É possível usar o método da bisseção para localizar raízes neste intervalo? Explique.

7. Encontre as soluções dos sistemas abaixo usando o método de Newton com parada quando:

$$\max_{1 \leq i \leq n} |x_i^k - x_i^{k-1}| < 10^{-4}.$$

$$\text{a. } \begin{cases} x_1^2 + x_2 - 37 & = 0 \\ x_1 - x_2^2 - 5 & = 0 \\ x_1 + x_2 + x_3 - 3 & = 0 \end{cases}$$

$$\text{b. } \begin{cases} x_1 + \cos(x_1 x_2 x_3) - 1 & = 0 \\ (1 - x_1)^{\frac{1}{4}} + x_2 + 0.05 x_3^2 - 0.15 x_3 - 1 & = 0 \\ -x_1^2 - 0.1 x_2^2 + 0.01 x_2 + x_3 - 1 & = 0 \end{cases}$$

$$\text{c. } \begin{cases} x_1^2 + 2 x_2^2 - x_2 - 2 x_3 & = 0 \\ x_1^2 - 8 x_2^2 + 10 x_3 & = 0 \\ \frac{x_1^2}{7 x_2 x_3} - 1 & = 0 \end{cases}$$

8. Compare os resultados obtidos aplicando os métodos de Newton e da Secante no sistema

$$\begin{cases} x_1 + x_2^3 - 5 x_2^2 - 2 x_2 & = 10 \\ x_1 + x_2^3 + x_2^2 - 14 x_2 & = 29 \end{cases}$$

partindo de $x_1 = 15$ e $x_2 = -2$.

9. Resolva os sistemas propostos no exercício 7 usando o método de Newton inexato e o número de iterações determinado pelo critério citado no texto.

10. Elabore um algoritmo para o método de Newton inexato para a solução de um sistema não-linear com n equações e n incógnitas.

11. Descreva um algoritmo obtido da modificação do algoritmo 4.4 pela introdução de um procedimento de cálculo aproximado da matriz Jacobiana.

12. Modifique o algoritmo 4.4 de modo que a matriz Jacobiana seja atualizada periodicamente, como sugere o último parágrafo deste capítulo. Aplique este procedimento na

solução do exercício 7.

13. Modifique o programa `sis2.m`, apresentado na seção 4.7, de modo a adaptá-lo ao Matlab sem recursos simbólicos.

Capítulo 5

APROXIMAÇÃO

5.1 Introdução

O problema de ler nas entrelinhas de dados tabelados ocorre com freqüência em aplicações. Também é comum nos depararmos com funções definidas implicitamente ou por expressões muito complicadas. Os métodos de aproximação são destinados a problemas desta natureza. Alguns exemplos introdutórios nos ajudam a perceber melhor como são esses problemas.

No primeiro exemplo, procuramos extrair informações de um conjunto de dados de censos demográficos, realizados a cada decênio. Como podemos estimar a população nos anos intermediários? Ou, ainda, é possível prever o crescimento populacional a partir dos censos de 1940, 1950, 1970 e 1980?

Considere agora o problema de determinação da constante de uma mola. A Lei de Hooke nos diz que o deslocamento de uma mola é proporcional à força nela aplicada. A questão é encontrar a constante de proporcionalidade (constante da mola) a partir de medições dos esforços aplicados na mola e dos correspondentes deslocamentos.

Num outro exemplo, como calcular valores de funções que não são definidas por expressões explícitas como, por exemplo

$$f(x) = \int_0^x \exp(-t^2) dt, \quad x \in [-1, 1],$$

uma vez que não é possível encontrar uma primitiva para o integrando?

Existem duas classes de métodos para aproximação de dados, e a distinção entre elas está em considerarmos, ou não, a existência de erros nos dados. No primeiro caso, consideramos que os dados são precisos e portanto pode-se exigir que a curva de ajuste passe pelos pontos dados. Resolveremos problemas desse tipo usando *Interpolação*. Como na maioria das vezes as funções escolhidas para o ajuste são polinômios, apresentaremos a técnica de *Interpolação Polinomial*.

No segundo grupo, levam-se em consideração possíveis erros introduzidos na obtenção dos dados. Neste caso, o *Método dos Quadrados Mínimos* tem sido o mais usado. Tomemos a determinação da constante de uma mola como exemplo. Podemos realizar vários experimentos medindo forças aplicadas à mola, e os respectivos deslocamentos. Com estes

valores, procuramos uma reta que aproxime esses dados, como ilustrado na figura 5.1. A inclinação desta reta nos dará a constante da mola, considerando os resultados de vários experimentos.

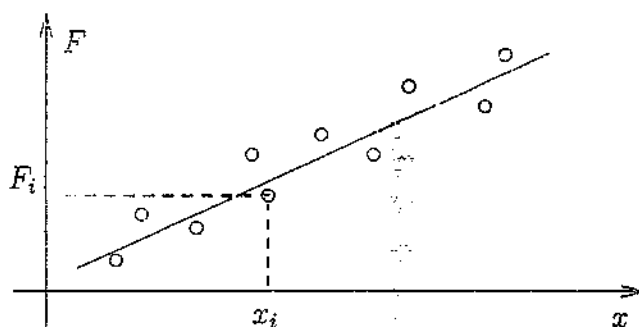


Figura 5.1: Lei de Hooke $F = kx$.

Outros tipos de funções, além das polinomiais, podem ser exploradas no método de quadrados mínimos para encontrar bons ajustes.

Neste capítulo, apresentamos inicialmente algumas técnicas usadas na interpolação polinomial. Em seguida, abordamos o método dos quadrados mínimos.

5.2 Interpolação Polinomial: Formas de Lagrange e Newton

Nesta seção, estaremos interessados em estabelecer métodos numéricos que permitam encontrar um polinômio que passe por $n + 1$ pontos conhecidos, denotados por $(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n)$ onde $x_0 \neq x_1 \neq \dots \neq x_n$.

Nessa classe de problemas, coloca-se a aproximação de funções que são conhecidas em pontos discretos, ou seja, conhecemos $y_0 = f(x_0), y_1 = f(x_1), \dots, y_n = f(x_n)$ e gostaríamos de encontrar um polinômio $p(x)$ tal que $p(x_0) = f(x_0), p(x_1) = f(x_1), \dots, p(x_n) = f(x_n)$. Este problema é chamado *Interpolação de Lagrange* (Joseph-Louis Lagrange, 1736–1813), e o polinômio $p(x)$ que satisfaz estas condições é chamado *polinômio interpolador* de $f(x)$ nos pontos $x_0, x_1, \dots, x_n$.

A figura 5.2 nos dá uma idéia gráfica do polinômio interpolador dos $(n + 1)$ pontos.

Poderíamos exigir mais do polinômio interpolador: que ele cruzasse a função, nos pontos de interpolação, com a mesma inclinação desta função (admitindo estas inclinações conhecidas), isto é,

$$p(x_i) = f(x_i) \quad \text{e} \quad p'(x_i) = f'(x_i) \quad \text{para } i = 0 : n.$$

Este tipo de interpolação é chamada *Hermitiana* (ver [Prenter, 1975], por exemplo). Nestas notas, trataremos apenas da interpolação Lagrangeana.

Uma vez que seja respeitada a relação entre o número de pontos e o grau do polinômio interpolador, o teorema abaixo nos fala da existência, da unicidade deste polinômio, além de nos indicar uma maneira de construí-lo.

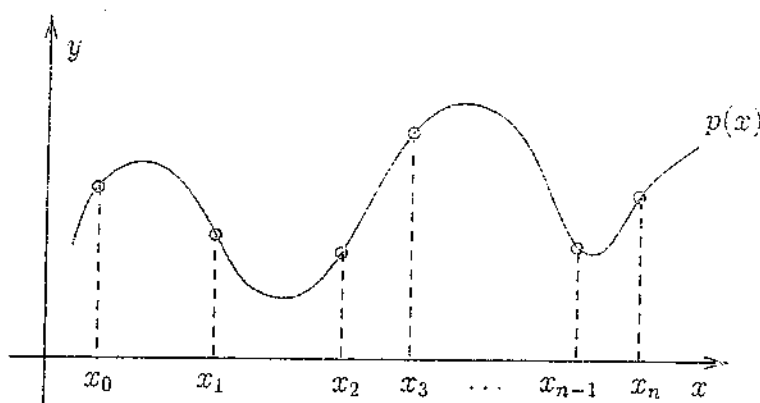


Figura 5.2: Exemplo de polinômio interpolador.

Teorema 5.1. *Seja $f(x)$ uma função conhecida nos $n + 1$ pontos distintos $x_0, x_1, \dots, x_n$. Existe um único polinômio $p(x)$, de grau menor ou igual a n , tal que*

$$p(x_i) = f(x_i) \quad \text{para } i = 0 : n.$$

Demonstração: Seja

$$\begin{aligned} p(x) &= f(x_0) l_0^n(x) + f(x_1) l_1^n(x) + \dots + f(x_n) l_n^n(x) = \\ &= \sum_{k=0}^n f(x_k) l_k^n(x) \end{aligned} \quad (5.1)$$

onde l_k^n são os polinômios de Lagrange definidos por:

$$l_k^n(x) = \frac{(x - x_0)(x - x_1) \dots (x - x_{k-1})(x - x_{k+1}) \dots (x - x_n)}{(x_k - x_0)(x_k - x_1) \dots (x_k - x_{k-1})(x_k - x_{k+1}) \dots (x_k - x_n)} = \prod_{i=0, i \neq k}^n \frac{(x - x_i)}{(x_k - x_i)}. \quad (5.2)$$

A verificação de que $p(x)$ é o polinômio interpolador segue diretamente da substituição dos pontos de interpolação, observando que:

$$l_k^n(x_k) = 1 \quad \text{e} \quad l_k^n(x_i) = 0 \quad \text{para } i \neq k.$$

A unicidade é consequência de ser $p(x) \equiv 0$ o único polinômio de grau n que se anula em $n + 1$ pontos distintos. De fato, supondo que existam dois polinômios interpoladores $p_1(x)$ e $p_2(x)$, usamos (5.1) com $p(x) = p_1(x) - p_2(x)$ para mostrar que $p(x) \equiv 0$ e portanto $p_1(x) \equiv p_2(x)$.

A expressão do polinômio interpolador dada por (5.1) é conhecida como *Fórmula de Lagrange*, e foi por ele estabelecida em 1794.

Exemplo 5.2.1. Encontrar o polinômio que interpola $f(x) = 1/x^2$ nos pontos $x_0 = 2$, $x_1 = 2.5$ e $x_2 = 4$.

Como serão usados três pontos, o polinômio interpolador tem grau 2. Pela substituição dos pontos de interpolação nas expressões de $l_k^2(x)$ (5.2), calculamos os polinômios de Lagrange:

$$l_0^2(x) = \frac{(x - x_1)(x - x_2)}{(x_0 - x_1)(x_0 - x_2)} = \frac{(x - 2.5)(x - 4)}{(2 - 2.5)(2 - 4)} = x^2 - 6.5x + 10$$

$$l_1^2(x) = \frac{(x - x_0)(x - x_2)}{(x_1 - x_0)(x_1 - x_2)} = \frac{(x - 2)(x - 4)}{(2.5 - 2)(2.5 - 4)} = (-4x^2 + 24x - 32)/3$$

$$l_2^2(x) = \frac{(x - x_0)(x - x_1)}{(x_2 - x_0)(x_2 - x_1)} = \frac{(x - 2)(x - 2.5)}{(4 - 2)(4 - 2.5)} = (x^2 - 4.5x + 5)/3.$$

Lembrando que $f(x) = 1/x^2$, $f(x_0) = 0.25$, $f(x_1) = 0.16$ e $f(x_2) = 0.0625$. Usando a expressão (5.1) teremos o polinômio interpolador $p_2(x)$:

$$\begin{aligned} p_2(x) &= 0.25(x^2 - 6.5x + 10) + \frac{0.16}{3}(-4x^2 + 24x - 32) + \frac{0.0625}{3}(x^2 - 4.5x + 5) \\ &= 0.0575x^2 - 0.4388x + 0.8975. \end{aligned}$$

A título de comparação, tomando $x = 3$ teremos $p_2(3) = 0.0986$, que é uma aproximação do valor $f(3) = 0.1111$, com erro 0.0125. $\square$

Exemplo 5.2.2. Seja $p_1(x)$ o polinômio que interpola $f(x)$ nos pontos x_0 e x_1 . Verificar que o erro da interpolação num ponto qualquer $\underline{x}$, com $x_0 < \underline{x} < x_1$, é dado por

$$E(\underline{x}) = f(\underline{x}) - p_1(\underline{x}) = \frac{f''(\xi)}{2}(\underline{x} - x_0)(\underline{x} - x_1),$$

onde $x_0 < \xi < x_1$.

Tomemos uma função auxiliar $G(x)$ definida em $[x_0, x_1]$ por

$$G(x) = f(x) - p_1(x) - C(x - x_0)(x - x_1)$$

sendo C uma constante tal que $G(\underline{x}) = 0$. Como $p_1(x)$ interpola $f(x)$ em $x = x_0$ e $x = x_1$, temos $G(x_0) = 0$ e $G(x_1) = 0$. Sendo assim, a função $G(x)$ se anula em três pontos, x_0 , $\underline{x}$ e x_1 . Pelo teorema do valor médio, $G'(x)$ se anula em dois pontos e existe pelo menos um ξ no intervalo $[x_0, x_1]$, tal que $G''(\xi) = 0$. Calculando a derivada $G''(x)$ em $x = \xi$, temos $C = \frac{f''(\xi)}{2}$. Como $G(\underline{x}) = 0$, obtemos a expressão do erro proposta no exemplo. $\square$

O exemplo acima confirma e quantifica que substituir uma função pelo seu polinômio interpolador não é uma operação impune: erros são introduzidos nesta aproximação. O resultado que apresentaremos a seguir generaliza o exemplo 5.2.2, dando-nos informação sobre a medida do erro do polinômio interpolador. Para demonstração ver [Conte-deBoor, 1981], por exemplo.

Teorema 5.2. *Seja $[a, b]$ um intervalo que contém os pontos $x_0, x_1, \dots, x_n$ e $f(x)$ uma função com $(n+1)$ derivadas contínuas em $[a, b]$. Pode-se mostrar que o erro do polinômio interpolador de $f(x)$ em $x_0, x_1, \dots, x_n$ é, no ponto $x \in [a, b]$, dado por*

$$E(x) = f(x) - p_n(x) = \frac{f^{(n+1)}(\alpha)}{(n+1)!} [(x-x_0)(x-x_1)\dots(x-x_n)], \quad (5.3)$$

onde a derivada de ordem $(n+1)$ é calculada num ponto $\alpha \in [a, b]$.

Embora o valor de α não possa ser determinado, a expressão acima é importante na estimativa e/ou delimitação do erro. De fato, se podemos limitar a derivada de ordem $(n+1)$ no intervalo $[a, b]$,

$$|f^{(n+1)}(\alpha)| \leq M = \max_{[a,b]} |f^{(n+1)}(x)|,$$

então, uma limitação para o erro na interpolação polinomial será:

$$|f(x) - p_n(x)| \leq \frac{M}{(n+1)!} |(x-x_0)(x-x_1)\dots(x-x_n)|.$$

No exemplo 5.2.1 $f^{(3)}(x) = \frac{24}{x^5} \leq \frac{24}{2^5} = 0.75$, no intervalo $[2, 4]$. Assim sendo, podemos encontrar um delimitante para o erro no ponto $x = 3$:

$$|f(3) - p(3)| \leq |(3-2)(3-2.5)(3-4)| \frac{0.75}{3!} = 0.0625,$$

que está de acordo com o erro que encontramos anteriormente (neste caso particular, a função a ser interpolada é conhecida):

$$|f(3) - p(3)| = 0.0125 < 0.0625.$$

5.2.1 Forma de Newton para Polinômio Interpolador

Nem sempre temos conhecimento, *a priori*, do grau adequado para o polinômio interpolador. Um teste razoável consiste em aumentar o número de pontos de interpolação, crescendo portanto o grau do polinômio interpolador, e testar se houve melhoria nos cálculos. Assim, é importante que o trabalho efetuado anteriormente possa ser utilizado para estabelecer a expressão do novo polinômio. Observando a fórmula de Lagrange (5.1), verificamos

que, se acrescentarmos um novo ponto de interpolação, os cálculos anteriores deverão ser refeitos.

Apresentamos a seguir a *Forma de Newton para o polinômio interpolador*, que tem a vantajosa característica de recorrência, isto é, o polinômio interpolador de grau $n+1$ pode ser calculado usando o polinômio interpolador de grau n e um novo ponto. O trabalho de Isaac Newton (1642-1727) em interpolação iniciou-se em 1675 e produziu a maioria das idéias do que hoje conhecemos como diferenças finitas.

Vamos procurar escrever o polinômio interpolador na forma

$$p_n(x) = c_0 + c_1(x - x_0) + c_2(x - x_0)(x - x_1) + \cdots + c_n(x - x_0)(x - x_1) \cdots (x - x_{n-1}), \quad (5.4)$$

onde as constantes $c_0, c_1, \dots, c_n$ serão determinadas pelas condições de interpolação.

Da expressão do erro do polinômio interpolador, (5.3), temos:

$$f(x) - p_n(x) = A(x)(x - x_0)(x - x_1) \cdots (x - x_n)$$

com

$$A(x) = \frac{f^{(n+1)}(\alpha(x))}{(n+1)!}.$$

Tomando $x = x_0$ em (5.4), só o primeiro termo do lado direito não se anula. Usando a condição de interpolação $p(x_0) = f(x_0)$ obtemos

$$c_0 = f(x_0).$$

No cálculo de c_1 definimos a diferença dividida de segunda ordem associada aos pontos x_0 e x ,

$$f[x_0, x] = \frac{f(x) - f(x_0)}{x - x_0},$$

usamos $c_0 = f(x_0)$ e reorganizamos os termos de (5.4), para estabelecer

$$\frac{p_n(x) - f(x_0)}{x - x_0} = c_1 + c_2(x - x_1) + \cdots + c_n(x - x_0)(x - x_1) \cdots (x - x_{n-1}).$$

Fazendo $x = x_1$ nesta última equação, o lado direito é igual a c_1 . Usando a condição de interpolação $p_n(x_1) = f(x_1)$ obtemos o valor de c_1 :

$$c_1 = f[x_0, x_1] = \frac{f(x_1) - f(x_0)}{x_1 - x_0}.$$

A idéia é repetir este procedimento até encontrarmos todos os coeficientes $c_0, c_1, \dots, c_n$. Para generalizar o procedimento, a relação de recorrência que segue será útil.

Definição de Diferenças Divididas: Seja $f[x_0] = f(x_0)$. Por recorrência definimos, para qualquer k ,

$$f[x_0, x_1, \dots, x_{k-1}, x_k, x] = \frac{f[x_0, x_1, \dots, x_{k-1}, x] - f[x_0, x_1, \dots, x_{k-1}, x_k]}{x - x_k}.$$

Por exemplo, no caso particular $k = 0$ e $x = x_1$

$$f[x_0, x_1] = \frac{f(x_1) - f(x_0)}{x_1 - x_0},$$

como foi usado no cálculo de c_1 .

Substituindo os valores de c_0 e c_1 em (5.4), temos

$$p_n(x) = f(x_0) + f[x_0, x_1](x - x_0) + c_2(x - x_0)(x - x_1) + c_3(x - x_0)(x - x_1)(x - x_2) + \dots$$

Fazendo $x = x_2$ e usando a condição de interpolação $p_n(x_2) = f(x_2)$, temos, após a divisão por $(x_2 - x_0)$:

$$\frac{f(x_2) - f(x_0)}{x_2 - x_0} = f[x_0, x_1] + c_2(x_2 - x_1).$$

Pela definição de diferença dividida de primeira ordem, temos

$$\frac{f(x_2) - f(x_0)}{x_2 - x_0} = f[x_0, x_2].$$

Assim, usamos $k = 1$ e $x = x_2$ na definição genérica de diferenças divididas para obter o valor de c_2 em termos de diferenças divididas:

$$c_2 = \frac{f[x_0, x_2] - f[x_0, x_1]}{x_2 - x_1} = f[x_0, x_1, x_2].$$

A esta altura mostramos que

$$c_0 = f[x_0], \quad c_1 = f[x_0, x_1] \quad \text{e} \quad c_2 = f[x_0, x_1, x_2].$$

Continuando, num processo análogo, mostramos que

$$c_k = f[x_0, x_1, \dots, x_k], \quad k = 0 : n. \quad (5.5)$$

Desta forma, a definição de diferenças divididas nos fornece a forma de Newton para o polinômio interpolador:

$$p_n(x) = f(x_0) + (x - x_0)f[x_0, x_1] + (x - x_0)(x - x_1)f[x_0, x_1, x_2] + \dots + (x - x_0)(x - x_1) \dots (x - x_{n-1})f[x_0, x_1, \dots, x_{n-1}, x_n]. \quad (5.6)$$

Um meio prático de calcular as diferenças divididas é através da *tabela de diferenças divididas*, que apresentamos a seguir.

A tabela pode ser construída usando uma propriedade de simetria que torna o cálculo das diferenças mais simples do que usando a definição.

Observando a expressão (5.5), vemos que o coeficiente de x^n no polinômio interpolador é $c_n = f[x_0, x_1, \dots, x_{n-1}, x_n]$. Como o polinômio interpolador é único, este coeficiente independe da ordem adotada nos pontos de interpolação. Assim, trocando a posição de x_0 na expressão que define diferenças divididas, temos

$$\begin{aligned} f[x_0, x_1, \dots, x_n, x] &= f[x_1, x_2, \dots, x_n, x, x_0] \\ &= \frac{f[x_0, x_1, \dots, x_n] - f[x_1, x_2, \dots, x_n, x]}{x_0 - x}. \end{aligned} \quad (5.7)$$

Tomando $x = x_{n+1}$ teremos:

$$f[x_0, x_1, \dots, x_n, x_{n+1}] = \frac{f[x_1, x_2, \dots, x_n, x_{n+1}] - f[x_0, x_1, \dots, x_{n-1}, x_n]}{x_{n+1} - x_0}.$$

Genericamente, a k -ésima diferença dividida relativa aos pontos $x_i, x_{i+1}, \dots, x_{i+j}$ é calculada por

$$f[x_i, x_{i+1}, \dots, x_{i+j}] = \frac{f[x_{i+1}, x_{i+2}, \dots, x_{i+j}] - f[x_i, x_{i+1}, \dots, x_{i+j-1}]}{x_{i+j} - x_i}, \quad (5.8)$$

isto é, usando diferenças divididas de ordem menor, relativas aos últimos pontos e aos primeiros pontos.

O algoritmo para calcular a tabela de diferenças divididas fica simples se definirmos uma matriz das diferenças

$$d_{ij} = f[x_i, x_{i+1}, \dots, x_{i+j}].$$

Com esta notação, os cálculos seguem o Algoritmo 5.1.

Algoritmo 5.1 Tabela de diferenças divididas

Dados x_i e $f(x_i)$ para $i = 0 : n$

1: $d_{i0} = f(x_i)$ para $i = 0 : n$

2: Para $k = 1 : n$, faça

3: Para $i = 0 : n - k$, faça

4: $d_{ik} = \frac{d_{i+1,k-1} - d_{i,k-1}}{x_{i+k} - x_i}$

Exemplo 5.2.3. O polinômio interpolador de $(1, 0)$, $(2, 2)$, $(4, 12)$ e $(5, 20)$ pode ser escrito a partir da tabela de diferenças divididas. No cálculo desta tabela, a expressão (5.8) é usada nos pontos de interpolação, lembrando que

$$\begin{aligned} f[x_i, x_{i+1}] &= \frac{f[x_{i+1}] - f[x_i]}{x_{i+1} - x_i}, \quad \text{para } i = 0, 1, 2 \\ f[x_i, x_{i+1}, x_{i+2}] &= \frac{f[x_{i+1}, x_{i+2}] - f[x_i, x_{i+1}]}{x_{i+2} - x_i}, \quad \text{para } i = 0, 1 \\ f[x_0, x_1, x_2, x_3] &= \frac{f[x_1, x_2, x_3] - f[x_0, x_1, x_2]}{x_3 - x_0}. \end{aligned}$$

Esses cálculos são organizados na tabela 5.1.

x_i	$f(x_i)$	$f[x_i, x_{i+1}]$	$f[x_i, x_{i+1}, x_{i+2}]$	$f[x_i, x_{i+1}, x_{i+2}, x_{i+3}]$
1	0	2	1	0
2	2	5	1	
4	12	8		
5	20			

Tabela 5.1: Tabela de diferenças divididas.

Usamos agora (5.5). Os coeficientes do polinômio interpolador são:

$$\begin{aligned} c_0 &= f(x_0) = 0 \\ c_1 &= f[x_0, x_1] = 2 \\ c_2 &= f[x_0, x_1, x_2] = 1 \\ c_3 &= f[x_0, x_1, x_2, x_3] = 0, \end{aligned}$$

que, usados na forma de Newton (5.6), nos fornecem o polinômio interpolador:

$$p(x) = 0 + 2(x - 1) + 1(x - 1)(x - 2) + 0(x - 1)(x - 2)(x - 4) = x(x - 1).$$

Observe que, neste exemplo, apesar de usarmos quatro pontos na interpolação (portanto esperávamos obter um polinômio de grau 3), o resultado foi um polinômio de grau 2. Isto se deve aos quatro pontos estarem sobre uma parábola. $\square$

5.3 Interpolação por Partes

Intuitivamente e, também, induzidos pela expressão do erro do polinômio interpolador (5.3), deduzimos que ao aumentar o número de pontos de interpolação, estaremos melhorando os resultados de aproximação. Entretanto, isto não é verdade, como ilustra o exemplo a seguir.

Exemplo 5.3.1. Seja

$$f(x) = \frac{1}{1+x^2}, \quad x \in (-5, 5).$$

Para N inteiro, vamos chamar $p_N(x)$ o polinômio interpolador de $f(x)$ nos pontos igualmente espaçados $x_0, x_1, \dots, x_N$, dados por $x_0 = -5$, $x_i = -5 + ih$, onde $h = 10/N$. Nas figuras 5.3 (a) e 5.3 (b), comparamos os gráficos de $f(x)$ com os polinômios de grau 5 e 21, respectivamente. Nesta figura o gráfico de $f(x)$ é a curva desenhada com pontos. Vemos que, ao aumentar o grau de polinômio, os resultados pioraram. E ficarão piores ainda se continuarmos a aumentar N .

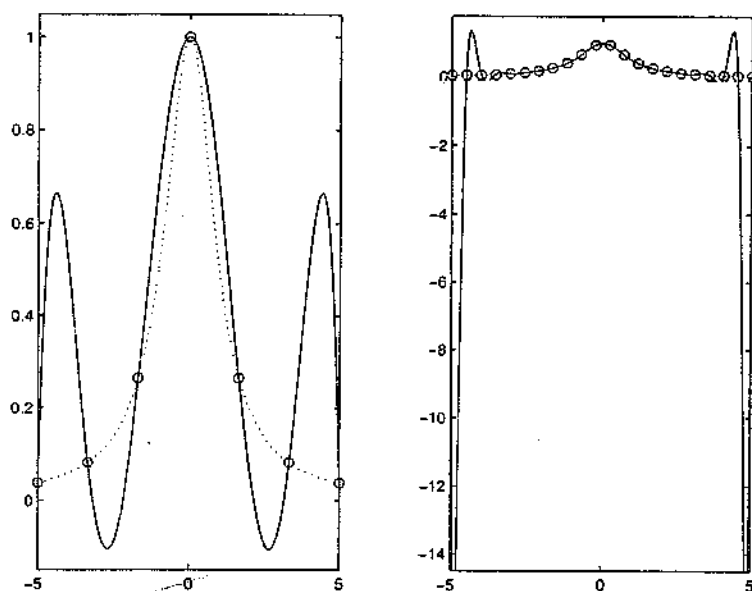


Figura 5.3: Polinômios interpoladores de graus 6 e 21.

□

Uma alternativa eficiente consiste em dividir o intervalo em uma coleção de subintervalos e construir polinômios interpoladores em cada subintervalo. Aproximações deste tipo são conhecidas como interpolações polinomiais por partes.

A interpolação linear por partes é o caso mais simples da interpolação por partes. A idéia é muito antiga, pois era usada por nossos antepassados para interpolar tabelas. Neste caso, os dados

$$\{(x_0, f_0), (x_1, f_1), \dots, (x_n, f_n)\}$$

são ligados por pedaços de retas como ilustra a figura 5.4

A desvantagem é que a função resultante não é diferenciável. Aumentando o grau do polinômio interpolador para 3, conseguimos suavidade. Voltaremos ao assunto no capítulo sobre splines.

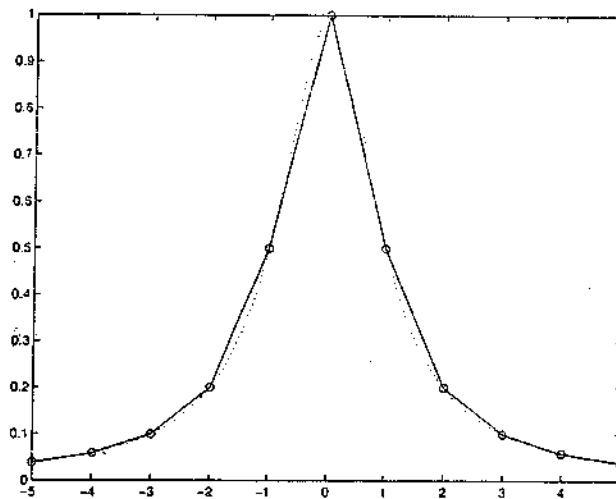


Figura 5.4: Exemplo de interpolação por partes.

5.4 Quadrados Mínimos Lineares

O Método dos Quadrados Mínimos é provavelmente a técnica de aproximação mais usada na análise numérica e em problemas práticos. Isto se deve tanto à sua simplicidade quanto ao fato de que, em geral, buscamos aproximações para dados que são medidas obtidas experimentalmente com um certo grau de incerteza. Este método foi publicado, pela primeira vez, por Adrien-Marie Legendre (1752-1833) em 1805, mas ele já era usado por Carl Friedrich Gauss (1777-1855), em seus cálculos na Astronomia, desde 1795. Nesta seção, apresentamos o Método dos Quadrados Mínimos, que contempla a possível existência de erros nos dados a serem aproximados; o critério de aproximação consiste em minimizar os resíduos.

Chamaremos de $f(x)$ a função que será convenientemente aproximada por outra função $g(x)$. No caso dos quadrados mínimos lineares, partimos da hipótese de que temos algumas informações sobre o comportamento de $g(x)$. Poderíamos saber, por exemplo, que $g(x)$ é uma reta, isto é,

$$g(x) = c_1 + c_2 x,$$

como é o caso da determinação da constante da mola usado como ilustração na introdução deste capítulo. A questão é encontrar qual é esta reta, isto é, quais são os valores de c_1 e c_2 que ajustam pontos dados.

Num outro exemplo, poderíamos saber se os dados se comportam como um parábola. Neste caso, vamos procurar valores para c_1 , c_2 e c_3 que tornam a função

$$g(x) = c_1 + c_2 x + c_3 x^2$$

uma boa aproximação dos dados.

Genericamente, no caso linear, estaremos supondo que os dados serão aproximados por uma função do tipo:

$$f(x) \cong g(x) = c_1\phi_1(x) + c_2\phi_2(x) + \cdots + c_n\phi_n(x) \quad (5.9)$$

onde as funções $\phi_1, \phi_2, \dots, \phi_n$ são preestabelecidas. As funções ϕ_i podem ser, por exemplo, os monômios x^i ou as funções trigonométricas $\sin(\pi ix)$. Para cada conjunto de coeficientes $c_i, i = 1 : n$, o desvio, ou resíduo da aproximação (5.9) no ponto x_k , será

$$\begin{aligned} r(x_k) &= f(x_k) - g(x_k) \\ r(x_k) &= f(x_k) - [c_1\phi_1(x_k) + c_2\phi_2(x_k) + \cdots + c_n\phi_n(x_k)] \\ &= r(x_k; c_1, c_2, \dots, c_n). \end{aligned} \quad (5.10)$$

Precisamos estabelecer critérios de aproximação para encaminhar o problema da determinação dos coeficientes $c_1, c_2, \dots, c_n$ que nos levarão à "melhor aproximação". No método dos quadrados mínimos, o critério é a minimização da soma dos quadrados dos resíduos, isto é, vamos minimizar

$$\sum_{i=1}^m r^2(x_i) = \sum_{i=1}^m (f(x_i) - g(x_i))^2,$$

onde $f(x_i), i = 1 : m$ são os dados que serão aproximados. Com este critério, procuramos tornar os resíduos pontuais tão pequenos quanto possível.

Até agora falamos em aproximação de dados discretos. Podemos ter o caso de aproximar $f(x)$, uma função conhecida para todo x , por expressões mais simples $g(x)$. Neste caso contínuo do método dos quadrados mínimos o natural é minimizar

$$\int_a^b r^2(x) dx = \int_a^b (f(x) - g(x))^2 dx.$$

Com a finalidade de "medir" os resíduos, englobando os casos discreto e contínuo, e por comodidade na notação, introduzimos uma noção de produto escalar entre funções comum em matemática.

Definição: O produto escalar entre duas funções $f(x)$ e $g(x)$ é definido por

$$\langle f, g \rangle = \langle f(x), g(x) \rangle = \begin{cases} \sum_{i=1}^m f(x_i)g(x_i), & \text{no caso discreto; isto é, se } f(x) \text{ e } g(x) \text{ são conhecidas em } x_i, i = 1 : m \\ \int_a^b f(x)g(x) dx, & \text{no caso contínuo, isto é, se } f(x) \text{ e } g(x) \text{ são conhecidas e integráveis em } (a, b). \end{cases} \quad (5.11)$$

Na definição acima, não há riscos de confusão, uma vez que estaremos trabalhando em apenas um dos dois casos.

Observe que esta "operação" entre funções tem as seguintes propriedades

- i. $\langle \alpha_1 g_1 + \alpha_2 g_2, h \rangle = \alpha_1 \langle g_1, h \rangle + \alpha_2 \langle g_2, h \rangle$, linearidade;
- ii. $\langle f, g \rangle = \langle g, f \rangle$, comutatividade;
- iii. $\langle g, g \rangle \geq 0$ e $\langle g, g \rangle = 0 \Leftrightarrow g \equiv 0$, positividade.

A estratégia do método dos quadrados mínimos consistirá na procura de parâmetros que minimizem a soma dos quadrados dos resíduos, no caso discreto, ou a integral (que também é uma soma) da função resíduo ao quadrado. Usando a notação introduzida em (5.11), estaremos minimizando $\langle r, r \rangle$, que é uma função de $c_1, c_2, \dots, c_n$.

Antes de estabelecer o procedimento genérico, vamos olhar como as coisas se processam em dois exemplos mais simples.

Exemplo 5.4.1. Regressão linear: Dada uma tabela com m valores $(x_i, f(x_i))$, $i = 1 : m$, queremos encontrar a reta que melhor ajusta esta tabela, no sentido dos quadrados mínimos. Como o ajuste será feito por uma reta, tomaremos $\phi_1(x) = 1$ e $\phi_2(x) = x$, isto é,

$$f(x) \cong g(x) = c_1 + c_2 x.$$

O resíduo, para cada par (c_1, c_2) e para cada x , será $r(c_1, c_2; x) = f(x) - c_1 - c_2 x$. Assim, pelo método dos quadrados mínimos devemos procurar c_1 e c_2 que minimizem a função

$$\langle r, r \rangle (c_1, c_2) = \langle f(x) - c_1 - c_2 x, f(x) - c_1 - c_2 x \rangle = \sum_{i=1}^m (f(x_i) - c_1 - c_2 x_i)^2.$$

Observe que estamos no caso discreto, pois $f(x)$ é uma função tabelada. Do Cálculo Diferencial sabemos que a condição necessária do ponto crítico é que as derivadas nele se anulem, isto é,

$$\frac{\partial}{\partial c_1} \langle r, r \rangle = \frac{\partial}{\partial c_2} \langle r, r \rangle = 0,$$

ou ainda, procedidas as respectivas derivações na expressão $\langle r, r \rangle$ temos

$$-\sum_{i=1}^m 2(f(x_i) - c_1 - c_2 x_i) = 0 \quad \text{e} \quad -\sum_{i=1}^m 2x_i(f(x_i) - c_1 - c_2 x_i) = 0.$$

Estas duas equações formam um sistema linear nas incógnitas c_1 e c_2 , que pode ser reescrito na forma

$$\begin{aligned} mc_1 + c_2 \sum_{i=1}^m x_i &= \sum_{i=1}^m f(x_i) \\ c_1 \sum_{i=1}^m x_i + c_2 \sum_{i=1}^m x_i^2 &= \sum_{i=1}^m x_i f(x_i) \end{aligned}$$

Para ilustrar este procedimento consideremos o ajuste da tabela abaixo por uma reta

x_i	0	0.25	0.5	0.75	1.00
$f(x_i)$	1.0000	1.2840	1.6487	2.1170	2.7183

Usando os valores da tabela temos:

$$\begin{aligned}\sum_{i=1}^5 x_i &= 2.5, & \sum_{i=1}^5 x_i^2 &= 1.875 \\ \sum_{i=1}^5 f(x_i) &= 8.768, & \sum_{i=1}^5 x_i f(x_i) &= 5.4514\end{aligned}$$

Assim, os valores de c_1 e c_2 da melhor reta (no sentido dos quadrados mínimos) são obtidos pelo sistema:

$$\begin{aligned}5c_1 + 2.5c_2 &= 8.768 \\ 2.5c_1 + 1.875c_2 &= 5.4514.\end{aligned}$$

Resolvendo o sistema, obtemos $c_1 = 0.89968$ e $c_2 = 1.70784$, ou seja, $g(x) = 0.89968 + 1.70784x$. Os valores de $g(x_i)$ e os respectivos resíduos estão na tabela abaixo.

x_i	0	0.25	0.5	0.75	1.00
$f(x_i)$	1.000	1.284	1.6487	2.1170	2.7183
$g(x_i)$	0.89968	1.32664	1.7536	2.18056	2.60752
$r(x_i)$	0.10032	-0.04264	-0.1049	-0.06356	-0.11078

Neste exemplo a soma dos quadrados dos resíduos é:

$$\sum_{i=1}^5 r^2(x_i) = 0.039198.$$

□

Exemplo 5.4.2. Neste exemplo, apresentamos uma aproximação contínua. Suponha que, por alguma razão, queremos substituir a função $f(x) = \sin \frac{\pi}{4}x$ por um polinômio de terceiro grau, no intervalo $(0, 1)$.

Como sabemos que $\sin \frac{\pi}{4}x$ é uma função ímpar, isto é, $f(-x) = -f(x)$, é natural que um polinômio ímpar seja usado no ajuste, isto é:

$$f(x) = \sin \frac{\pi}{4}x \cong c_1x + c_2x^3.$$

Isto significa que estamos escolhendo $\phi_1(x) = x$ e $\phi_2(x) = x^3$. Neste caso, sendo a aproximação no intervalo $(0, 1)$, devemos minimizar

$$\langle r, r \rangle = \int_0^1 \left(\sin \frac{\pi}{4}x - c_1x - c_2x^3 \right)^2 dx.$$

As condições de pontos críticos são

$$\frac{\partial}{\partial c_1} \langle r, r \rangle = - \int_0^1 2x \left(\sin \frac{\pi}{4} x - c_1 x - c_2 x^3 \right) dx = 0$$

$$\frac{\partial}{\partial c_2} \langle r, r \rangle = - \int_0^1 2x^3 \left(\sin \frac{\pi}{4} x - c_1 x - c_2 x^3 \right) dx = 0.$$

O sistema normal (esta é a denominação usual) será, neste caso,

$$c_1 \int_0^1 x^2 dx + c_2 \int_0^1 x^4 dx = \int_0^1 x \sin \frac{\pi}{4} x dx$$

$$c_1 \int_0^1 x^4 dx + c_2 \int_0^1 x^6 dx = \int_0^1 x^3 \sin \frac{\pi}{4} x dx$$

Resolvidas as integrais, teremos:

$$\frac{1}{3} c_1 + \frac{1}{5} c_2 = 0.246002$$

$$\frac{1}{5} c_1 + \frac{1}{7} c_2 = 0.145818,$$

cujas soluções são $c_1 = 0.784815$ e $c_2 = -0.078015$. Dessa maneira,

$$\sin \frac{\pi}{4} x \cong 0.784815x - 0.078015x^3,$$

sendo o lado direito a melhor aproximação em $(0, 1)$. Se calculamos a integral do resíduo

$$\langle r, r \rangle = \int_0^1 \left(\sin \frac{\pi}{4} x - 0.784815x + 0.078015x^3 \right)^2 dx = 0.116 \cdot 10^{-6},$$

que é o resíduo mínimo dentro das possíveis escolhas de c_1 e c_2 . □

O objetivo destes dois exemplos é motivar um procedimento geral a ser seguido na procura da melhor aproximação. Apresentamos o caso genérico a seguir.

Para determinar as constantes $c_1, c_2, \dots, c_n$ da aproximação definidas por (5.9), devemos minimizar a função:

$$\begin{aligned} M(c_1, c_2, \dots, c_n) &= \langle r, r \rangle = \\ &= \langle f(x) - c_1 \phi_1(x) - \dots - c_n \phi_n(x), f(x) - c_1 \phi_1(x) - \dots - c_n \phi_n(x) \rangle. \end{aligned}$$

Usando a linearidade do produto escalar, derivamos esta equação com relação a cada um dos parâmetros c_i . Observando que $\langle f, g \rangle = \langle g, f \rangle$, obtemos as condições de ponto crítico:

$$\frac{\partial M}{\partial c_j} = -2 \langle f(x) - c_1 \phi_1(x) - \dots - c_n \phi_n(x), \phi_j(x) \rangle = 0, \quad j = 1 : n. \quad (5.12)$$

Distribuindo o produto escalar, obtemos a equação genérica

$$c_1 \langle \phi_1, \phi_j \rangle + c_2 \langle \phi_2, \phi_j \rangle + \dots + c_n \langle \phi_n, \phi_j \rangle = \langle f, \phi_j \rangle, \quad j = 1 : n.$$

Tomando $j = 1, j = 2, \dots, j = n$, as equações acima definem o *Sistema Normal*

$$\begin{cases} \langle \phi_1, \phi_1 \rangle c_1 + \langle \phi_2, \phi_1 \rangle c_2 + \dots + \langle \phi_n, \phi_1 \rangle c_n = \langle f, \phi_1 \rangle \\ \langle \phi_1, \phi_2 \rangle c_1 + \langle \phi_2, \phi_2 \rangle c_2 + \dots + \langle \phi_n, \phi_2 \rangle c_n = \langle f, \phi_2 \rangle \\ \vdots \\ \langle \phi_1, \phi_n \rangle c_1 + \langle \phi_2, \phi_n \rangle c_2 + \dots + \langle \phi_n, \phi_n \rangle c_n = \langle f, \phi_n \rangle. \end{cases} \quad (5.13)$$

Resolvido o sistema normal obtemos os coeficientes de (5.9) que definem a melhor aproximação.

Como $\langle \phi_i, \phi_j \rangle = \langle \phi_j, \phi_i \rangle$, quaisquer que sejam i e j , concluímos que o Sistema Normal é simétrico. A matemática também nos mostra que se as funções $\phi_i(x)$ são linearmente independentes, então a matriz das incógnitas do Sistema Normal é positiva definida, o que nos garante a existência de uma única solução para o sistema.

Pode-se perguntar o porquê da condição de ponto crítico nos levar ao ponto de mínimo. De fato, como estamos minimizando uma função que é quadrática em suas incógnitas, as primeiras derivadas nulas nos levam ao único ponto de mínimo. Se ainda ficaram dúvidas, pode-se pensar na parábola, com coeficiente de c_1^2 positivo, que tem um único mínimo. Ou ainda no parabolóide, que é o caso de duas incógnitas c_1 e c_2 .

No caso discreto, ou também na discretização do caso contínuo, é possível organizar os cálculos do Método dos Quadrados Mínimos, como segue.

Consideremos a matriz $A_{m \times n}$, na qual j -ésima coluna é o vetor u_j , cujas m componentes são os valores da função ϕ_j nos pontos tabelados, isto é,

$$u_j = (\phi_j(x_1), \phi_j(x_2), \dots, \phi_j(x_m))^T, \quad j = 1 : n.$$

Chamemos de f o vetor coluna dos dados:

$$f = (f(x_1), f(x_2), \dots, f(x_m))^T.$$

Nesta notação, pode-se verificar, o sistema normal pode ser escrito na forma matricial por

$$A^T A c = A^T f, \quad (5.14)$$

onde $c = (c_1, c_2, \dots, c_n)^T$ é o vetor coluna dos coeficientes da melhor aproximação.

O epíteto "normal" é justificado pelas equações (5.12), que nos dizem que o resíduo é ortogonal a cada um dos vetores u_j . Neste sentido, a melhor aproximação é a projeção ortogonal dos dados f no espaço gerado por u_j , $j = 1 : n$, colunas da matriz A .

Como já comentamos, o sistema normal (5.14) tem uma única solução se os vetores u_j são linearmente independentes. Entretanto, podem ocorrer colinearidades ou quase-colinearidades entre os vetores u_j , que representam o modelo usado no ajuste. Conseqüentemente, podem surgir matrizes singulares (não inversíveis) ou malcondicionadas no sistema normal. Esta deficiência pode ser constatada pelo posto da matriz ou pela decomposição em valores singulares, a SVD, tratada na seção 2.9. Como foi comentado naquela seção, uma alternativa é desprezar autovalores muito pequenos, a TSVD.

A formulação do Método dos Quadrados Mínimos usando produto escalar de funções permite aumentar a sua potencialidade.

O primeiro proveito será a possibilidade de atribuir diferentes graus de importância no intervalo de aproximação. Isso pode ser feito se introduzimos uma função peso, $w(x)$, no produto escalar. Modificamos (5.11) tomando agora:

$$\langle f, g \rangle = \begin{cases} \sum_{i=1}^m w(x_i) f(x_i) g(x_i), & \text{no caso discreto} \\ \int_a^b w(x) f(x) g(x) dx, & \text{no caso contínuo} \end{cases} \quad (5.15)$$

onde $w(x)$ é uma função contínua e $w(x) > 0$ em (a, b) . Um exemplo de função peso é:

$$w(x) = \frac{1}{\sqrt{1-x^2}}, \quad x \in (-1, 1),$$

que é usada nos polinômios de Chebyshev (Pafnuti Čebyšev, 1821-1894). Neste caso, a função $w(x)$ atribui menos peso à região central do intervalo $(-1, 1)$ e mais ênfase aos pontos próximos aos extremos deste intervalo. De fato, como $w(x)$ é maior nos extremos do intervalo, o resíduo deve ser menor nesta região, de modo a garantir a média baixa.

Ainda tirando proveito da definição de produto escalar entre funções, introduzimos a noção de funções ortogonais na seção seguinte. Conjuntos de funções ortogonais desempenham um papel importante na teoria e aplicação de aproximações de funções. Sem dúvida, o exemplo de destaque fica por conta das séries de Fourier (Joseph Fourier, 1768-1830).

5.4.1 Funções Ortogonais

De modo análogo à definição de vetores ortogonais, caracterizados pelo produto escalar nulo, podemos estabelecer o conceito de ortogonalidade num conjunto de funções. Dizemos que uma família de funções, $\psi_i(x)$, $i = 1, 2, \dots$ é ortogonal com relação ao produto escalar

associado ao peso $w(x)$ se $\langle \psi_i, \psi_j \rangle = 0$ para $i \neq j$. Esta definição vale para o caso contínuo, isto é,

$$\langle \psi_i, \psi_j \rangle = \int_a^b w(x) \psi_i(x) \psi_j(x) dx = 0 \quad \text{se } i \neq j,$$

ou para o caso discreto:

$$\langle \psi_i, \psi_j \rangle = \sum_{k=1}^m w(x_k) \psi_i(x_k) \psi_j(x_k) dx = 0 \quad \text{se } i \neq j.$$

O Método dos Quadrados Mínimos fica mais simples se trabalharmos com aproximações em famílias de funções ortogonais. De fato, se no sistema normal (5.13) temos $\langle \phi_i, \phi_j \rangle = 0$ para $i \neq j$, o sistema será diagonal, o que significa uma grande vantagem. Neste caso, os coeficientes da melhor aproximação podem ser calculados diretamente, ou seja,

$$c_i = \frac{\langle f, \psi_i \rangle}{\langle \psi_i, \psi_i \rangle}, \quad i = 1 : n. \quad (5.16)$$

Existem duas famílias de funções ortogonais muito populares na matemática, por causa de suas importantes aplicações: as funções trigonométricas, que dão origem à série de Fourier, e os polinômios ortogonais que, cada vez mais, aparecem em problemas que envolvem aproximações (o exemplo mais recente são os métodos espectrais; ver [Gottlieb, 1977]).

A série de Fourier está associada às funções trigonométricas:

$$\begin{aligned} \psi_0(x) &= \frac{1}{\sqrt{2\pi}}, \\ \psi_{2k-1}(x) &= \frac{\cos kx}{\sqrt{\pi}}, \quad k = 1, 2, \dots \\ \psi_{2k}(x) &= \frac{\sin kx}{\sqrt{\pi}}, \quad k = 1, 2, \dots, \end{aligned}$$

que são ortonormais ($\langle \psi_i, \psi_i \rangle = 1$ e $\langle \psi_i, \psi_j \rangle = 0$ se $i \neq j$), se tomarmos o produto escalar do caso contínuo no intervalo $(-\pi, \pi)$ com peso $w(x) = 1$.

Assim, a melhor aproximação de uma função $f(x)$ no conjunto gerado pelas funções $\psi_i(x)$ será:

$$g(x) = \sum_{j=0}^m c_j \psi_j(x), \quad \text{onde } c_j = \int_{-\pi}^{\pi} f(x) \psi_j(x) dx.$$

Quando $m \rightarrow \infty$, $g(x)$ é a série de Fourier de $f(x)$, conhecida por suas aplicações na solução de equações diferenciais e nos problemas de aproximação.

No segundo exemplo, estão os Polinômios Ortogonais. Dizemos que a sequência $P_0(x), P_1(x), \dots, P_i(x), \dots$, de polinômios de grau $0, 1, \dots, i, \dots$ respectivamente, é ortogonal se:

$$\langle P_i, P_j \rangle = 0 \quad \text{quando } i \neq j.$$

Várias famílias de polinômios ortogonais podem ser construídas tomando diferentes produtos escalares. A escolha do produto escalar depende do intervalo utilizado, da função peso e de ser contínuo ou discreto.

Os polinômios ortogonais têm propriedades interessantes, dentre as quais citaremos apenas duas, que interessam ao texto. Uma nos fala de suas raízes, o que será útil no capítulo de integração numérica, e a outra se refere a uma relação de recorrência de três termos que auxilia na geração de seqüências de polinômios ortogonais. A demonstração destas propriedades, assim como outras propriedades, pode ser encontrada em [Conte-deBoor, 1981].

Propriedade 1: Seja $P_k(x)$ um membro da família de polinômios ortogonais, com relação a um produto escalar em (a, b) , então $P_k(x)$ tem k raízes reais distintas em (a, b) .

Propriedade 2: Os polinômios ortogonais satisfazem a uma relação de recorrência de três termos. Se chamamos α_k o coeficiente de x_k em P_k ,

$$P_{-1}(x) \equiv 0, \quad P_0 = \alpha_0, \quad A_i = \frac{\alpha_{i+1}}{\alpha_i} \quad \text{e} \quad S_i = \langle P_i, P_i \rangle, \quad \text{para} \quad i = 0 : k-1$$

então

$$P_{i+1}(x) = A_i(x - B_i)P_i(x) - C_iP_{i-1}(x), \quad i = 0 : k-1,$$

onde

$$B_i = \frac{\langle xP_i, P_i \rangle}{S_i} \quad \text{e} \quad C_i = \begin{cases} \frac{A_i S_i}{A_{i-1} S_{i-1}}, & i \neq 0 \\ \text{arbitrário} & \text{se } i = 0. \end{cases}$$

Observe que podemos escolher os coeficientes do polinômio ou assumir valores para A_i de modo que a seqüência seja particularmente simples. Por exemplo, podemos decidir que os polinômios terão os coeficientes da maior potência de x sempre 1. Outra opção seria tomar os coeficientes tais que $\langle P_i, P_i \rangle = 1$. No último caso, temos uma família de polinômios ortonormais.

A seguir, apresentamos duas famílias de polinômios ortogonais.

Polinômios de Legendre são ortogonais em $(-1, 1)$ com a função peso $w(x) \equiv 1$. Os seis primeiros polinômios de Legendre são

$$P_0(x) = 1, \quad P_1(x) = x, \quad P_2(x) = x^2 - 1/3$$

$$P_3(x) = x^3 - (3/5)x, \quad P_4(x) = x^4 - (6/7)x^2 + (3/35)$$

$$P_5(x) = x^5 - (10/9)x^3 + (5/21)x.$$

Polinômios de Chebyshev são ortogonais em $(-1, 1)$ com a função peso $w(x) = (1 - x^2)^{-1/2}$. Os quatro primeiros polinômios de Chebyshev são

$$P_0(x) = 1, \quad P_1(x) = x$$

$$P_2(x) = 2x^2 - 1, \quad P_3(x) = 4x^3 - 3x$$

$$P_4(x) = 8x^4 - 8x^2 + 1.$$

Mais informações sobre polinômios ortogonais poderão ser obtidas nas referências [Conte-deBoor, 1981], [Burden-Faires, 1988] e nos clássicos [Szegő, 1959] ou [Abramowitz-Stegun, 1965].

Exemplo 5.4.3. Usando os polinômios de Legendre, vamos encontrar o polinômio de grau ≤ 3 que melhor aproxima $\sin \frac{\pi}{4}x$ (a função do exemplo 5.4.2) no intervalo $(-1, 1)$. Usando o mesmo argumento anterior, tendo em vista que $\sin \frac{\pi}{4}x$ é uma função ímpar, procuramos c_1 e c_2 tais que

$$\sin \frac{\pi}{4}x \cong c_1x + c_2(x^3 - 0.6x) = c_1 P_1(x) + c_2 P_3(x), \quad \text{em } (-1, 1).$$

Por serem os polinômios de Legendre ortogonais em $(-1, 1)$ com função peso $w(x) = 1$, podemos usar (5.16) para calcular os coeficientes da melhor aproximação. Os cálculos necessários são

$$\langle f, P_1 \rangle = \int_{-1}^1 x \sin \frac{\pi}{4}x \, dx = \left(\frac{4}{\pi}\right)^2 \left\{ \sin \frac{\pi}{4}x - \frac{\pi}{4}x \cos \frac{\pi}{4}x \right\}_{-1}^1 = 0.492004$$

e

$$\langle f, P_3 \rangle = \int_{-1}^1 (x^3 - 0.6x) \sin \frac{\pi}{4}x \, dx = \int_{-1}^1 x^3 \sin \frac{\pi}{4}x \, dx - 0.6 * 0.492004 = -0.003566,$$

$$\langle P_1, P_1 \rangle = \frac{2}{3} \quad \text{e} \quad \langle P_3, P_3 \rangle = \frac{0.32}{7}.$$

Usando (5.16), calculamos os dois coeficientes $c_1 = 0.738006$ e $c_2 = -0.078006$. Portanto

$$\sin \frac{\pi}{4}x \cong 0.738006x - 0.078006(x^3 - 0.6x) = 0.784810x - 0.078006x^3.$$

que, a menos de pequenos erros de arredondamento, coincide com o resultado do exemplo 5.4.2. $\square$

5.5 Quadrados Mínimos Não-Lineares

Na seção 5.4 apresentamos o método dos quadrados mínimos para ajuste de dados por funções que são lineares nos parâmetros, como foi colocado em (5.9). Entretanto, numa grande variedade de aplicações são requisitados ajustes por funções que não são lineares nos parâmetros. Um exemplo é o ajuste de dados obtidos experimentalmente, usando uma soma de Gaussianas:

$$f(x) \approx g(x) = a_1 \exp \left[\frac{(a_2 - x)^2}{a_3} \right] + a_4 \exp \left[\frac{(a_5 - x)^2}{a_6} \right].$$

Neste caso, $g(x)$ é uma função não-linear de seis parâmetros de ajuste, que são $a_1, a_2, \dots, a_6$.

No ajuste não-linear, podemos adotar o princípio básico do Método dos Quadrados Mínimos Lineares. Definimos uma função que representa a soma dos quadrados dos resíduos e procuramos os parâmetros que minimizam esta função. A grande diferença é que, no caso do ajuste não-linear, vamos procurar mínimos de funções não-lineares.

Tendo em vista que o problema de ajuste não-linear aparece com frequência e que a função a ser minimizada tem a particularidade de ser soma de quadrados, justifica-se o uso de métodos específicos de minimização. Vamos abordar alguns deles a seguir.

Para efeito de notação, seja $y(x; a_1, a_2, \dots, a_n)$ a função da variável x e dos parâmetros $a_1, a_2, \dots, a_n$ que será usada no ajuste de um conjunto de dados $y_1, y_2, \dots, y_m$, onde $m \gg n$. Chamamos $f_i(a)$, na qual a é um vetor com os parâmetros de ajuste, o resíduo no ponto x_i , isto é,

$$f_i(a) = y_i - y(x_i, a_1, a_2, \dots, a_n) \quad \text{para } i = 1 : m. \quad (5.17)$$

Usando esta notação, formulamos o problema de minimização da soma dos resíduos, $F(a)$:

Minimizar

$$F(a) = \frac{1}{2} \sum_{i=1}^m f_i^2(a) = \frac{1}{2} \sum_{i=1}^m f_i^2(a_1, \dots, a_n). \quad (5.18)$$

A soma dos resíduos foi dividida por dois para simplificar as contas posteriores.

A seguir, apresentamos alguns métodos que podem ser usados neste processo de minimização. Nestes métodos, a soma dos resíduos ao quadrado, $F(a)$, é substituída por uma aproximação quadrática que será minimizada em cada passo do método.

Genericamente, deveríamos trabalhar com n parâmetros de ajuste, isto é, $a = (a_1, a_2, \dots, a_n)$. Para simplificar, vamos apresentar os cálculos correspondentes ao caso com dois parâmetros de ajuste, isto é, $a = (a_1, a_2)$. Entretanto, estes cálculos podem ser generalizados.

divididos

O desenvolvimento em série de Taylor é, de novo, a ferramenta matemática central. Os três primeiros termos da série de Taylor vão compor a aproximação quadrática que será minimizada.

Os três primeiros termos da série de Taylor em torno de $a = (a_1, a_2)$ e associada ao incremento $p = (p_1, p_2)$ são:

$$F(a_1 + p_1, a_2 + p_2) \cong F(a) + p_1 \frac{\partial F}{\partial a_1}(a) + p_2 \frac{\partial F}{\partial a_2}(a) + \frac{1}{2} \left(p_1^2 \frac{\partial^2 F}{\partial a_1^2}(a) + 2p_1 p_2 \frac{\partial^2 F}{\partial a_1 \partial a_2}(a) + p_2^2 \frac{\partial^2 F}{\partial a_2^2}(a) \right). \quad (5.19)$$

Inicialmente vamos efetuar os cálculos correspondentes aos segundo e terceiro termos de (5.19). A definição de $F(a_1, a_2)$ está em (5.18), com $n = 2$. Assim, as derivadas de primeira ordem desta função são:

$$\frac{\partial F}{\partial a_1}(a) = \sum_{i=1}^m f_i(a) \frac{\partial f_i}{\partial a_1}(a) \quad (5.20)$$

$$\frac{\partial F}{\partial a_2}(a) = \sum_{i=1}^m f_i(a) \frac{\partial f_i}{\partial a_2}(a) \quad (5.21)$$

onde $a = (a_1, a_2)$.

A notação matricial é útil nestes cálculos. Seja f o vetor coluna cujas componentes são os resíduos associados à (a_1, a_2) , definidos em (5.17):

$$f(a) = (f_1(a_1, a_2), f_2(a_1, a_2), \dots, f_m(a_1, a_2))^T.$$

As derivadas das componentes de $f(a)$, calculadas em (a_1, a_2) formarão a matriz Jacobiana:

$$J(a) = \begin{bmatrix} \frac{\partial f_1}{\partial a_1}(a) & \frac{\partial f_1}{\partial a_2}(a) \\ \frac{\partial f_2}{\partial a_1}(a) & \frac{\partial f_2}{\partial a_2}(a) \\ \vdots & \vdots \\ \frac{\partial f_m}{\partial a_1}(a) & \frac{\partial f_m}{\partial a_2}(a) \end{bmatrix}. \quad (5.22)$$

Realizando o produto da matriz $J^T(a)$, a transposta da matriz Jacobiana, pelo vetor f verificamos que o vetor resultante tem como componentes (5.20) e (5.21). Chamando de $p = (p_1, p_2)$ o vetor de incrementos na série de Taylor, verificamos que os termos de primeira ordem da série de Taylor (5.19) podem ser escritos, na notação matricial, como:

$$p_1 \frac{\partial F}{\partial a_1}(a) + p_2 \frac{\partial F}{\partial a_2}(a) = p J^T(a) f(a). \quad (5.23)$$

O resultado destas operações é um número real que será somado a $F(a)$.

Passemos agora aos cálculos relativos aos termos de segunda ordem na série de Taylor, isto é, aos três últimos termos de (5.19). Usando a regra do produto para derivar (5.20) e (5.21) temos

$$\frac{\partial^2 F(a)}{\partial a_1^2} = \sum_{i=1}^m \left(\frac{\partial f_i(a)}{\partial a_1} \right)^2 + \sum_{i=1}^m f_i(a) \frac{\partial^2 f_i}{\partial a_1^2}(a) \quad (5.24)$$

$$\frac{\partial^2 F(a)}{\partial a_1 \partial a_2} = \sum_{i=1}^m \frac{\partial f_i(a)}{\partial a_1} \frac{\partial f_i(a)}{\partial a_2} + \sum_{i=1}^m f_i(a) \frac{\partial^2 f_i}{\partial a_1 \partial a_2}(a) \quad (5.25)$$

$$\frac{\partial^2 F(a)}{\partial a_2^2} = \sum_{i=1}^m \left(\frac{\partial f_i(a)}{\partial a_2} \right)^2 + \sum_{i=1}^m f_i(a) \frac{\partial^2 f_i}{\partial a_2^2}(a) \quad (5.26)$$

Nestas três expressões, as primeiras parcelas englobam produtos de derivadas de primeira ordem de $f_i(a)$. Usando a matriz Jacobiana (5.22), podemos verificar que os somatórios correspondentes às primeiras parcelas em (5.24)-(5.26) são os elementos da matriz simétrica $J^T(a)J(a)$. Os produtos pelos incrementos p_1 e p_2 nos levam a

$$p^T J^T(a)J(a)p. \quad (5.27)$$

Por outro lado, os segundos somatórios de (5.24)-(5.26) incluem as derivadas de segunda ordem de $f_i(a)$, $i = 1 : m$. Não entrando nos detalhes, vamos admitir que estas derivadas vão compor uma matriz $Q(a)$.

Desta maneira, os três últimos termos de (5.19) podem ser reescritos usando (5.27) e a matriz das derivadas de segunda ordem $Q(a)$:

$$p_1^2 \frac{\partial^2 F}{\partial a_1^2}(a) + 2p_1 p_2 \frac{\partial^2 F}{\partial a_1 \partial a_2}(a) + p_2^2 \frac{\partial^2 F}{\partial a_2^2}(a) = p^T (J^T(a)J(a) + Q(a))p.$$

Em resumo, a série de Taylor (5.19) nesta notação matricial é:

$$F(a + p) \cong F(a) + p J^T f(a) + \frac{1}{2} p^T (J^T(a)J(a) + Q(a))p, \quad (5.28)$$

que é uma função quadrática dos incrementos p_1 e p_2 .

Embora tenhamos apresentados a dedução de (5.28) para o caso de ajuste de dois parâmetros, $a = (a_1, a_2)$, todas as passagens são válidas no caso genérico de n parâmetros, $a = (a_1, a_2, \dots, a_n)$. Sendo assim, vamos admitir que a equação (5.28) se refere ao caso genérico.

Assim, no caso de n parâmetros na equação (5.28), $F(a)$ é um número, $J(a)$ é uma matriz com m linhas e n colunas

$$J(a) = \begin{bmatrix} \frac{\partial f_1}{\partial a_1}(a) & \frac{\partial f_1}{\partial a_2}(a) & \dots & \frac{\partial f_1}{\partial a_n}(a) \\ \frac{\partial f_2}{\partial a_1}(a) & \frac{\partial f_2}{\partial a_2}(a) & \dots & \frac{\partial f_2}{\partial a_n}(a) \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial a_1}(a) & \frac{\partial f_m}{\partial a_2}(a) & \dots & \frac{\partial f_m}{\partial a_n}(a) \end{bmatrix} \quad (5.29)$$

e $f(a)$ é um vetor coluna, $f(a) = (f_1(a), f_2(a), \dots, f_m(a))^T$. As matrizes $J^T J$ e Q em (5.28) têm n linhas e n colunas, de modo que o resultado final do lado direito é um número real.

O passo seguinte no processo de minimização será encontrar um incremento que diminua a soma dos resíduos, isto é, encontrar p^k tal que

$$F(a^k + p^k) < F(a^k)$$

em cada passo k . Se isto for possível, $a^k + p^k$ estará mais próximo do mínimo do que a^k .

Como já antecipamos, a substituição de F por uma função quadrática facilita, uma vez que a nova aproximação será obtida minimizando esta função quadrática.

Na minimização da função quadrática, vamos aproveitar alguns cálculos que já foram realizados na seção 3.3, quando introduzimos o método dos gradientes conjugados.

Se compararmos o lado direito de (5.28) com (3.3), observamos a semelhança fazendo

$$A = J^T J + Q \quad \text{e} \quad b = -J^T f(a) \quad (5.30)$$

e adicionamos um termo independente de p , $F(a^k)$. A adição de $F(a^k)$ não altera o cálculo do ponto de mínimo uma vez que ela representa uma translação vertical da função a ser minimizada.

Como vimos na seção 3.3, o mínimo da função quadrática será solução da equação $\text{grad } F(p) = 0$, ou ainda, na notação daquela seção, a solução do sistema $Ap = b$. Assim, com A e b de (5.30), em cada passo k devemos resolver o sistema

$$(J_k^T J_k + Q_k)p_k = -J_k^T f_k. \quad \text{Aqui?} \quad (5.31)$$

Nesta equação o subscrito k denota a respectiva entidade calculada em a^k , valor atual dos parâmetros que estão sendo minimizados.

Resolvido o sistema (5.31), a nova aproximação dos parâmetros é

$$a^{k+1} = a^k + p.$$

Os diversos métodos usados em problemas de quadrados mínimos não-lineares são variações nos procedimentos para resolver o sistema (5.31). Sem a pretensão de esgotar o assunto, comentaremos resumidamente alguns destes métodos.

5.5.1 Método de Gauss-Newton

No Método de Gauss-Newton, admite-se que a^k está próximo do mínimo e, portanto, o resíduo $f(a)$ está próximo de zero. Com esta hipótese é razoável argumentar que a matriz Q_k pode ser desprezada quando comparada com $J^T J$ no sistema (5.31). Nessas condições, o sistema a ser resolvido em cada iteração será:

$$(J_k^T J_k) p_k = -J_k^T f_k. \quad (5.32)$$

Se voltarmos um pouco ao método dos quadrados mínimos, reconheceremos nesta última expressão um sistema normal (5.14) e que resolvê-lo é equivalente ao problema

$$\min_{\text{em } p} |J_k p_k + f_k|.$$

Este problema tem uma única solução, desde que J_k seja inversível.

Embora este método seja relativamente simples, ele pode apresentar problemas de convergência muito lenta, e até mesmo não-convergência, uma vez que o número de condição da matriz $J_k^T J_k$ é o quadrado do número de condição de J_k , o que pode, em algumas situações, ser muito grande.

O resumo do Método de Gauss-Newton, para minimizar a função soma de quadrados dos resíduos,

$$F(a) = \frac{1}{2} \sum_{i=1}^m f_i^2(a_1, a_2, \dots, a_n),$$

onde

$$f_i(a) = y_i - y(x_i; a_1, a_2, \dots, a_n)$$

representa o resíduo no ponto x_i , está apresentado no Algoritmo 5.2.

Algoritmo 5.2 Método de Gauss-Newton

Dados $f_i, \frac{\partial f_i}{\partial a_j}, \text{tol}, a^0$

- 1: Para $k = 0, 1, \dots$, faça
 - 2: Calcular $f(a^k)$
 - 3: Calcular $J = J(a^k)$ (5.29)
 - 4: Calcular $J^T J$ e $J^T f$
 - 5: Resolver o sistema $J^T J p = -J^T f$
 - 6: $a^{k+1} = a^k + p$
 - 7: Se $\max_{1 \leq i \leq n} |p_i| < \text{tol}$ então
 - 8: Saída com $a = a^{k+1}$.
-

5.5.2 Método de Levenberg-Marquardt

A segunda alternativa usada para a minimização da soma de resíduos se baseia na solução do sistema:

$$(\mathbf{J}_k^T \mathbf{J}_k + \lambda_k \mathbf{I}) \mathbf{p}_k = -\mathbf{J}_k^T \mathbf{f}_k, \quad (5.33)$$

onde λ_k é um número real não-negativo e $\mathbf{I}_{n \times n}$ é a matriz identidade.

Se compararmos (5.33) e (5.31), observamos que a proposta do novo sistema é algo mais do que simplesmente desprezar $\mathbf{Q}_k$. Para os que têm alguma experiência no assunto, trata-se da regularização do sistema (5.32). Pode-se mostrar que $F(\mathbf{a}^k + \mathbf{p}^k) < F(\mathbf{a}^k)$ para λ_k convenientemente escolhido.

O algoritmo deste método é análogo ao Algoritmo 5.2, trocando-se apenas a expressão correspondente ao sistema. O programa correspondente ao método de Levenberg-Marquardt pode ser encontrado, por exemplo, em [Press et al., 1990] e em pacotes computacionais como o Matlab.

Exemplo 5.5.1. Vamos usar o Método de Gauss-Newton para ajustar os valores (x_i, y_i) das duas primeiras colunas da tabela 5.2 por uma função do tipo

$$y(x; a_1, a_2, a_3) = a_1 \exp \left(-\frac{(x + a_2)^2}{a_3} \right).$$

Nestes cálculos, tomamos a aproximação inicial $\mathbf{a}^0 = (1, 0, 1)$. Em cada iteração, chamando $\mathbf{a} = \mathbf{a}^k$ calculamos:

i. Vetor resíduo

$$f_i(\mathbf{a}) = y_i - a_1 \exp \left(-\frac{(x_i + a_2)^2}{a_3} \right);$$

ii. matriz Jacobiana. As três colunas desta matriz são

$$\begin{aligned} J_{i1} &= \frac{\partial f_i}{\partial a_1}(\mathbf{a}) = -\exp \left(-\frac{(x_i + a_2)^2}{a_3} \right), \quad i = 1 : m \\ J_{i2} &= \frac{\partial f_i}{\partial a_2}(\mathbf{a}) = 2 \frac{a_1}{a_3} (x_i + a_2) \exp \left(-\frac{(x_i + a_2)^2}{a_3} \right), \quad i = 1 : m \\ J_{i3} &= \frac{\partial f_i}{\partial a_3}(\mathbf{a}) = -a_1 \frac{(x_i + a_2)^2}{a_3^2} \exp \left(-\frac{(x_i + a_2)^2}{a_3} \right), \quad i = 1 : m. \end{aligned}$$

No sistema a ser resolvido, a cada iteração, a matriz $\mathbf{J}^T \mathbf{J}$ é 3×3 . Adotamos como critério de parada soma dos resíduos ao quadrado menor que 10^{-3} .

Os resultados do programa `gaunew.m` cuja listagem é apresentada no Anexo mostram que, na segunda iteração, a soma dos resíduos é 0.0018 e este resultado não é modificado nas próximas iterações. Se nos contentarmos com esta aproximação, os parâmetros ajustados são $a_1 = 0.4914$, $a_2^* = -0.1978$ e $a_3 = 0.7738$. A terceira coluna da tabela 5.2 mostra os valores da função ajustada. Na quarta coluna desta tabela, apresentamos os resíduos, para cada valor de x , associados aos parâmetros ajustados.

x	y	$f(x)$	resíduo
0	0.4500	0.4672	0.0172
0.2000	0.5200	0.4914	-0.0286
0.4000	0.4600	0.4661	0.0061
0.6000	0.4000	0.3987	-0.0013
0.8000	0.2900	0.3075	0.0175
1.0000	0.2100	0.2139	0.0039
1.2000	0.1500	0.1342	-0.0158
1.4000	0.0800	0.0759	-0.0041
1.6000	0.0400	0.0387	-0.0013
1.8000	0.0200	0.0178	-0.0022
2.0000	0.0100	0.0074	-0.0026

Tabela 5.2: Dados a serem ajustados e resultados dos ajustes.

5.5.3 Métodos com Aproximações Quase-Newton

No caso das aproximações iniciais não serem suficientemente próximas da solução, os resíduos são grandes e não é razoável a argumentação usada nos métodos anteriores. Assim sendo, no cálculo da direção de mínimo, a matriz Q_k deve ser levada em conta. Como o cálculo das segundas derivadas pode ser enfadonho (e também muito caro), pode-se optar pela utilização de aproximações para Q_k . No seu cálculo, envolvam apenas as derivadas de primeira ordem. As aproximações deste tipo são chamadas Quase-Newton, que genericamente denotaremos M_k . Neste caso, o sistema que aproxima (5.31) será

$$(J_k^T J_k + M_k)p_k = -J_k f_k$$

e existem várias possibilidades para construção e atualização de M_k . Algumas delas podem ser encontradas em [Gill et al., 1981], por exemplo. De novo, a mudança no algoritmo é só na expressão do sistema a ser resolvido em cada uma das iterações.

Muitas técnicas de derivação automática, por via de manipulação simbólica, têm sido desenvolvidas recentemente. Possivelmente, o problema da derivação será amenizado no futuro, embora continue dispendioso. Entretanto, como os métodos Quase-Newton são econômicos do ponto de vista da Álgebra Linear Computacional, é provável que permaneçam competitivos.

5.6 Interpolação e Ajuste usando o Matlab

A forma mais simples de interpolar a tabela de valores de uma função é usando interpolação linear de pontos adjacentes, isto é fazendo a interpolação linear por partes. A seguir, apresentamos um exemplo interpolando a função $\exp(x)$ usando uma tabela de valores.

```
>> x = linspace(0, 1, 11);           % Criando uma tabela de 11 pontos
                                      % da função exp(x) em [0,1]
>> y = exp(x);
>> y1 = interp1(x, y, [.22 .45 .87]) % Calculando a Interpolação Linear
                                      % por partes nos três pontos dados
y1 =
    1.2471    1.5703    2.3894
```

Usando a função exponencial nos pontos dados, obtemos os erros na interpolação linear: -0.0010 , -0.0020 , e -0.0025 respectivamente. Quando não especificado o tipo de interpolação, o pacote usa interpolação linear por partes. Também estão programadas as interpolações cúbica por partes e usando splines, cuja teoria analisaremos num capítulo posterior. Por exemplo

```
% Interpolação cúbica por partes
>> yc = interp1(x, y, [.22 .45 .87], 'cubic')
yc =
    1.2461    1.5683    2.3869

% Interpolação usando splines
>> ys = interp1(x, y, [.22 .45 .87], 'spline')
ys =
    1.2461    1.5683    2.3869
```

Embora os valores encontrados sejam aparentemente os mesmos, podemos verificar a diferença calculando os erros, tomando como valores "exatos" aqueles da função

AIRS → Box para o TEC
→ copy sobre AIRS.

o cúbica
-yc)

Poi

Primo de outro!

por spline
s)

```
es =
    1.0e-005 *
    0.0052    0.0401    0.2359
```

Para encontrar o polinômio de grau n que interpola $(n+1)$ pontos, podemos usar uma função que é programada para ajuste, mas, que fornece o polinômio interpolador se há compatibilidade entre grau do polinômio e o número de pontos dados. Por exemplo,

```
>> p = polyfit(x, y, 10)    % Coeficientes do polinômio de grau 10
                                % que interpola onze pares (x,y)
```

```
p =
    0.0000    0.0000    0.0000    0.0002    0.0014    0.0083
    0.0417    0.1667    0.5000    1.0000    1.0000
```

Se quisermos comparar os resultados, podemos usar, por exemplo

```
>> xx = linspace(0, 1, 1000); % Gerando 1000 pontos no intervalo
>> yy = polyval(p, xx);      % Calculando os valores interpolados
                                % nestes 1000 pontos
>> plot(xx, yy, 'x', y, '*') % Gráfico do polinômio interpolador e pontos
                                % usados na interpolação (*).
```

Com a seqüência de comandos a seguir, apresentamos uma ilustração do uso do Matlab na interpolação bidimensional e traçado de gráficos de funções de duas variáveis independentes (programa figura2D.m).

Neste exemplo, geramos alguns valores da função $f(x, y) = 4 - 4x^2 + y^2$. Usando uma malha mais fina e o recurso `interp2` com interpolação cúbica, geramos mais pontos para melhorar a resolução do gráfico. No gráfico final, apresentamos os pontos refinados e aqueles que foram usados na interpolação (*).

```
%
% figura2D.m - interpolação bidimensional
%

x = linspace(-1, 1, 10);
y = linspace(-1.5, 2, 10);

[xx, yy] = meshgrid(x, y); % malha dos pontos de interpolação
zz = 4-4*xx.*xx+yy.*yy;    % função que será interpolada

% pontos p/ malha mais fina
```

```

xi = linspace(-1, 1, 50);
yi = linspace(-1.5, 2, 50);

[xxi, yyi] = meshgrid(xi, yi);          % malha mais fina
zzi = interp2(xx, yy, zz, xxi, yyi, 'cubic'); % interpolação malha fina

mesh(xxi, yyi, zzi)    % gráfico superfície
hold on

plot3(xx, yy, zz, '*') % pontos usados na interpolação
hold off

title('Figura 2D')

```

No Matlab, a função `polyfit` resolve o problema do ajuste de uma tabela de dados usando polinômios do grau desejado. Esta função torna fácil o teste do ajuste com vários graus, permitindo uma escolha pela observação gráfica. O exemplo ilustra o uso da função `polyfit`.

```

>> clear          % É sempre bom limpar as variáveis para evitar
                  % que cálculos anteriores interfiram

% Tabela de dados que será usada no ajuste
>> x = [0 .1 .2 .3 .4 .5 .6];
>> y = [-.45 1.9 3.3 6.2 7.1 7.5 7.7];

>> xx = 0:.01:.6;      % Malha mais fina para traçados dos gráficos
>> p1 = polyfit(x, y, 1); % Ajuste por uma reta: polinômio de grau 1
>> y1 = polyval(p1, xx);
>> p3 = polyfit(x, y, 3); % Ajuste por polinômio de grau 3
>> y3 = polyval(p3, xx);
>> p6 = polyfit(x, y, 6);
>> y6 = polyval(p6, xx); % Polinômio Interpolador dos dados

% Traçado dos gráficos para comparação dos resultados

>> plot(x, y, '*', xx, y1, '-.', xx, y3, '.', xx, y6)

```

Detalharemos um pouco mais a utilização das funções splines na interpolação e no ajuste de dados no capítulo sobre Splines. Por outro lado, se quisermos usar outra família de funções, que não os polinômios, no ajuste de dados, um programa em linguagem Matlab torna-se simples, devido às operações matriciais do pacote, inclusive à solução do Sistema Normal, usando o recurso apresentado nos capítulos 2 e 3.

Exercícios

1. Encontre aproximações para $f(2.5)$ usando polinômios interpoladores (fórmula de Lagrange) – de graus 2, 3 e 4 – e os dados abaixo. Compare os resultados. Repita o exercício usando a forma de Newton para o polinômio interpolador.

x	$f(x)$
2.0	0.5103757
2.2	0.5207843
2.4	0.5104147
2.6	0.4813306
2.8	0.4359160

2. Seja $f(x) = (4x - 7)/(x - 2)$ e $x_0 = 1.7$, $x_1 = 1.8$, $x_2 = 1.9$ e $x_3 = 2.1$.
- Aproxime $f(1.75)$ usando polinômio interpolador de $f(x)$ nos pontos x_0 , x_1 e x_2 .
 - Aproxime $f(1.75)$ usando polinômio interpolador de $f(x)$ nos pontos x_0 , x_1 , x_2 e x_3 .
 - Qual é o limitante para o erro nos casos (a) e (b)? Compare esse limitante com o erro no ponto $x = 1.75$.
3. Mostre que o erro máximo da interpolação linear de $f(x)$ nos pontos x_0 e x_1 ocorre no ponto médio $\bar{x} = (x_0 + x_1)/2$ e

$$|f(x) - p_1(x)| \leq (x_0 - x_1)^2 \frac{M}{8}, \quad |f''(x)| \leq M \quad \text{em} \quad [x_0, x_1].$$

4. Determine o espaçamento h para a elaboração de uma tabela de valores igualmente espaçados da função $f(x) = e^x$ em $[0, 1]$, de modo que a interpolação, usando polinômios de grau ≤ 2 , forneça resultados com precisão de 10^{-3} .
5. Use diferenças divididas para aproximar $f(0.3)$ a partir dos dados $f(0.0) = 15.0$, $f(0.2) = 21.0$, $f(0.4) = 30.0$ e $f(0.6) = 51.0$.
6. Encontre os polinômios de grau 1, 2 e 3 que melhor ajustam a tabela abaixo, no sentido dos quadrados mínimos. Qual deles fornece menor resíduo?

x	0	0.15	0.31	0.5	0.6	0.75
$f(x)$	1.0	1.004	1.031	1.117	1.223	1.422

7. Use os polinômios de Legendre para encontrar o polinômio de grau menor ou igual a dois que melhor ajusta $f(x) = x^3 - 1$, no intervalo $[-1, 1]$. Qual é o resíduo desta aproximação? Compare os gráficos.
8. Seja $f_i(a) = e^{a_1 + a_2} - y_i$, $i = 1 : 4$. Suponha que $x_1 = -2$, $x_2 = -1$, $x_3 = 0$, $x_4 = 1$, $y_1 = 0.5$, $y_2 = 1$, $y_3 = 2$ e $y_4 = 4$. Partindo de $a^0 = (1, 1)$, calcule duas iterações usando:

- a. Método de Gauss-Newton;
- b. Método de Levenberg-Maquardt;
- c. aplicando o logaritmo em $f(x)$, podemos linearizar o ajuste. Resolva o problema linear e compare com os resultados de a e b .

Repita o exercício trocando $y_1 = 5$ e $y_4 = -4$.

9. Os dados y_i , $i = 1 : 20$ devem ser ajustados por uma função do tipo

$$y(x) = a_1 + a_2 \exp(-(x + a_3)).$$

Partindo de a_1, a_2 e a_3 conhecidos, perturbe $y(x_i)$, $i = 1 : 20$ estabelecendo os y_i a serem ajustados. Volte agora ao problema de encontrar a_i que melhor ajustam os y_i , usando

- a. Método de Gauss-Newton;
 - b. Método de Levenberg-Maquardt.
10. Compare o esforço computacional para encontrar $p_n(\bar{x})$, $\bar{x}$ fixo, usando a fórmula de Lagrange e a forma Newton para o polinômio interpolador.
11. Verifique que os seguintes problemas não-lineares podem ser linearizados por transformações matemáticas na função:

a. $f(x) \cong \frac{a_1}{a_2 x + a_3};$

b. $f(x) \cong 1 + \frac{a_1}{x^2 + a_2};$

c. $f(x) \cong a_1 e^{a_2 x};$

d. $f(x) \cong \ln(a_1 + a_2 x^2).$

Capítulo 6

SPLINES E APLICAÇÕES

6.1 Introdução

Como tivemos oportunidade de observar anteriormente, um dos pilares do estudo dos métodos numéricos é o desenvolvimento de funções em séries de Taylor. Entretanto, a aproximação por uma soma de potências só é possível quanto a função tem derivadas no ponto ao redor do qual estamos trabalhando. Neste caso, a série de Taylor:

$$f(a+h) = f(a) + f'(a)h + f''(a)\frac{h^2}{2} + \dots + f^{(n)}(a)\frac{h^n}{n!} + \dots$$

nos permite obter uma aproximação de $f(a+h)$ através de polinômios em h .

A aproximação por polinômios é muito conveniente, uma vez que as funções polinomiais têm várias propriedades interessantes. Uma destas propriedades é a analiticidade: podemos calcular as derivadas, de qualquer ordem, dos polinômios.

Entretanto, a necessidade de muitas derivadas da função que está sendo aproximada por polinômios pode ser muito restritiva.

Uma solução de compromisso é usar polinômios por partes: pode-se escapar da analiticidade no intervalo inteiro, permitindo descontinuidades das derivadas de ordem mais elevadas em alguns pontos. Esta característica de "pseudo-analiticidade" confere às funções polinomiais por partes, chamadas *splines*, boas propriedades de aproximação, convergência e estabilidade com respeito aos erros de arredondamento. Para alguns autores, o uso de splines é um estágio intermediário entre os métodos clássicos, que empregam funções analíticas em todo o domínio, e os métodos de diferenças finitas que discretizam o contínuo.

Embora existam indicações de que funções suaves por partes tenham sido usadas por Euler (1707-1783) e outros autores, no início do século XX, devemos a R. Courant (1943) e I. Shoemberg (1946) a formulação de splines que hoje usamos. A teoria dos splines foi desenvolvida a partir de necessidades práticas de aproximação.

A extensão das técnicas que usam funções polinomiais por partes na solução numérica de equações diferenciais deram origem aos métodos conhecidos como elementos finitos, que se tornaram muito populares nas últimas décadas.

Neste capítulo, apresentamos idéias básicas dos splines e alguns exemplos de aplicações destas funções em aproximação.

6.2 Definições

As funções splines estão associadas a uma partição predeterminada do intervalo $[a, b]$, onde pretendemos trabalhar. Uma partição Π será definida pelos pontos $x_0, x_1, \dots, x_m$, tais que,

$$\Pi : a = x_0 < x_1 < x_2 < \dots < x_{m-1} < x_m = b. \quad (6.1)$$

Em cada subintervalo (x_{i-1}, x_i) , $i = 1 : m$, os splines são polinômios (que têm todas as derivadas). Estes “pedaços” de polinômios são colados convenientemente para que algumas derivadas, da ordem ditada pelo problema, existam em todo intervalo $[a, b]$. Existe uma relação de compromisso entre o grau dos “pedaços” de polinômios e a ordem das derivadas exigidas nos pontos da partição. Assim, algumas restrições devem ser impostas na definição geral dos splines.

Definição: Uma função $s(x)$ é chamada *spline* de grau n , associada a uma partição (6.1) de $[a, b]$, se

- i. $s(x)$ é um polinômio de grau n em cada subintervalo (x_{i-1}, x_i) ;
- ii. $s(x)$ tem $(n - 1)$ derivadas contínuas em cada x_i , isto é, $s(x)$ é uma função com $(n - 1)$ derivadas em (a, b) .

O termo “spline” está relacionado com um instrumento de desenho assim denominado. Trata-se de uma régua perfeitamente flexível que, fixada em alguns pontos (os nós da partição), encurva-se na posição de mínima energia. Pode-se mostrar que esta configuração de mínima energia é um spline cúbico ($n = 3$). As figuras 6.1 mostram splines lineares, $n = 1$, e splines cúbicos, $n = 3$.

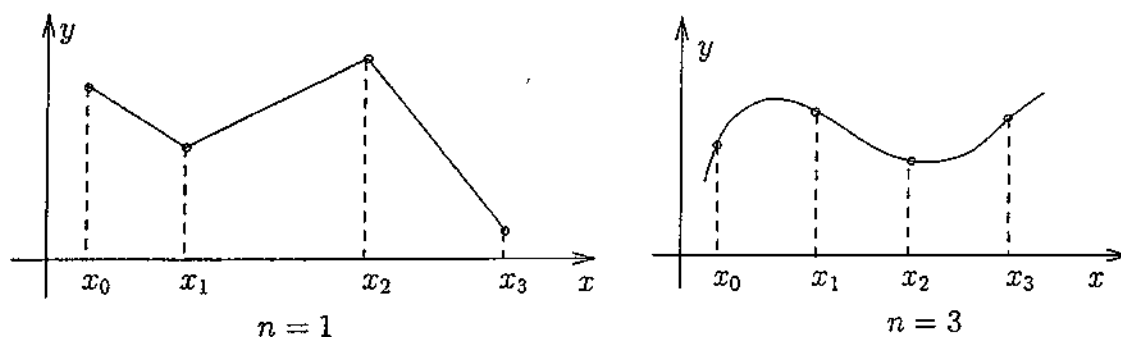


Figura 6.1: Exemplos de splines linear ($n = 1$) e cúbica ($n = 3$).

O conjunto das funções splines de grau n , associado à partição (6.1), forma um espaço vetorial cuja dimensão depende de n e do número de nós da partição. Por exemplo, podemos mostrar que os splines lineares associado à partição (6.1) com m pontos formam

um espaço vetorial de dimensão $m + 1$. No caso dos splines cúbicos associados a esta partição, a dimensão do espaço vetorial é $m + 3$.

A amplitude das aplicações das funções splines na análise numérica deve-se à utilização dessas funções para gerar espaços de dimensão finita que, em algum sentido, aproximam problemas com dimensão infinita. Por exemplo, o conjunto das funções contínuas em $[0, 1]$, que é um espaço de funções com dimensão infinita, pode ser aproximado pelos splines lineares associados à uma partição de $[0, 1]$, como ilustra a figura 6.2. Podemos observar que quanto mais pontos existem na partição melhor será a aproximação por splines lineares.

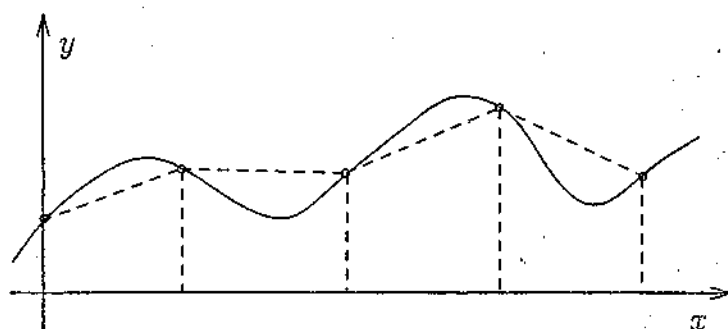


Figura 6.2: Exemplo de spline em $[0, 1]$.

O estudo do erro na aproximação por splines pode ser encontrado, por exemplo, em [Prenter, 1975] e/ou [Schultz, 1973].

6.3 Bases

Uma característica importante das funções splines é a possibilidade de estabelecer bases convenientes para as aplicações. Usando uma das propriedades de espaços vetoriais, se $\{\phi_i(x)\}_{i=1}^k$ é uma base para o espaço vetorial de splines, então qualquer elemento deste espaço vetorial pode ser escrito na forma

$$s(x) = C_1\phi_1(x) + C_2\phi_2(x) + \cdots + C_k\phi_k(x),$$

onde $C_1, C_2, \dots, C_k$ são os coeficientes da combinação linear e k é a dimensão do espaço vetorial.

Dentre as várias possibilidades na escolha das bases, existe uma condição que faz com que uma base seja mais interessante. Como veremos nas aplicações, quanto mais “locais” (se anulam fora de intervalos “pequenos”) forem as bases geradoras dos splines, melhores serão os resultados. Uma base local faz com que os sistemas de equações associados à discretização sejam esparsos e com melhores propriedades de estabilidade na sua solução numérica.

A seguir, apresentamos as bases locais para os splines lineares e cúbicos. Mais detalhes sobre o assunto podem ser obtidos na bibliografia especializada (ver [Schultz, 1973] ou [Prenter, 1975]).

Base para os splines lineares $n = 1$

Consideremos a função $l_i(x)$ definida para cada nó x_i de uma partição do tipo (6.1) por:

$$l_i(x) = \begin{cases} \frac{x - x_{i-1}}{x_i - x_{i-1}} & \text{se } x_{i-1} \leq x \leq x_i \\ \frac{x_{i+1} - x}{x_{i+1} - x_i} & \text{se } x_i \leq x \leq x_{i+1} \\ 0 & \text{se } x < x_{i-1} \text{ ou } x > x_{i+1}. \end{cases} \quad (6.2)$$

O conjunto das $l_i(x)$, $i = 0 : m$, forma uma base para os splines lineares. Observe que cada uma das funções que formam a base é não-nula apenas em dois subintervalos vizinhos.

Restritas ao intervalo $[a, b]$, as funções $l_i(x)$ têm o gráfico ilustrado na figura 6.3. Estas funções são conhecidas como "funções chapéu", por motivos óbvios.

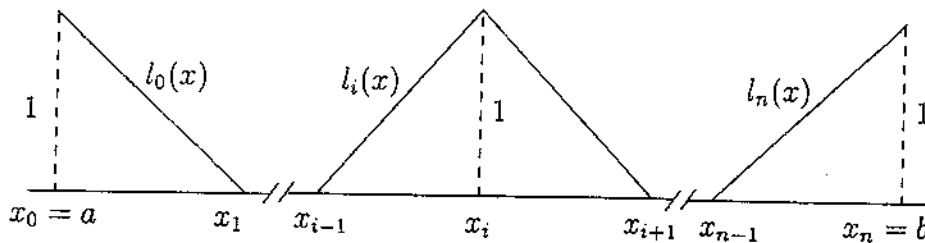


Figura 6.3: Funções chapéu.

Usando esta base, os splines de grau 1 associados à partição definida por x_i , $i = 0 : m$, retas por partes, podem ser escritos na forma:

$$s(x) = s_0 l_0(x) + s_1 l_1(x) + \cdots + s_m l_m(x),$$

onde $s_i = s(x_i)$ são os valores que $s(x)$ assume em x_i .

Se a partição do intervalo é uniforme, isto é, $x_{i+1} - x_i = h$, a base $l_i(x)$ pode ser expressa por meio da mudança de variáveis na função

$$L(x) = \begin{cases} 1 + x & \text{se } x \in [-1, 0] \\ 1 - x & \text{se } x \in [0, 1] \\ 0 & \text{se } x \notin [-1, 1]. \end{cases} \quad (6.3)$$

De fato, por substituição direta em $L(x)$, podemos verificar que

$$l_i(x) = L\left(\frac{x - x_0}{h} - i\right) \quad \text{para } i = 0 : m.$$

Esta representação pode ser útil na implementação computacional.

Base para os splines cúbicos ($n - 3$)

Pela definição (6.1), os splines cúbicos se caracterizam por uma “colagem” de pedaços de polinômios de grau 3, de modo que a função resultante tenha duas derivadas contínuas em $[a, b]$.

As expressões dos elementos da base para splines cúbicos tornam-se complexas se a partição não for uniforme; neste caso sugerimos a referência [Schultz, 1973].

Para as partições uniformes, com comprimento dos subintervalos h , pode-se mostrar que os splines cúbicos são gerados por funções que são nulas fora de intervalos de comprimento $4h$, e não há como melhorar.

Esta base é composta das funções $B_i(x)$, sinos com vértices em x_i , como ilustrado na figura 6.4. Analiticamente, a expressão de $B_i(x)$ é:

$$B_i(x) = \frac{1}{h^3} \begin{cases} (x - x_{i-2})^3 & x_{i-2} \leq x \leq x_{i-1} \\ h^3 + 3h^2(x - x_{i-1}) + 3h(x - x_{i-1})^2 - 3(x - x_{i-1})^3 & x_{i-1} \leq x \leq x_i \\ h^3 + 3h^2(x_{i+1} - x) + 3h(x_{i+1} - x)^2 - 3(x_{i+1} - x)^3 & x_i \leq x \leq x_{i+1} \\ (x_{i+2} - x)^3 & x_{i+1} \leq x \leq x_{i+2} \\ 0 & x \notin [x_{i-2}, x_{i+2}]. \end{cases} \quad (6.4)$$

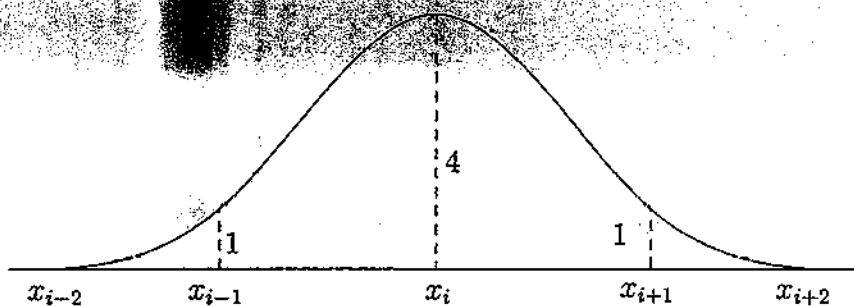


Figura 6.4: Funções da base de splines cúbicas.

Pode-se observar que, para completar a base no intervalo (x_0, x_m) , é necessário introduzir nós adicionais. De fato, se traçarmos o gráfico de $B_{-1}(x)$, veremos que esta função tem

influência no intervalo (x_0, x_m) , uma vez que $B_{-1}(x) \neq 0$ em (x_0, x_1) . De modo análogo, $B_{m+1}(x)$ também contribui no intervalo (x_0, x_m) .

Como $B_i(x)$ é um polinômio de grau 3 em cada subintervalo (x_{i-1}, x_i) , a combinação linear dessas funções,

$$s_3(x) = C_{-1}B_{-1}(x) + C_0B_0(x) + C_1B_1(x) + \cdots + C_mB_m(x) + C_{m+1}B_{m+1}(x), \quad (6.5)$$

também terá esta propriedade.

Fazendo cálculos diretos em (6.4), podemos verificar que as duas primeiras derivadas de $B_i(x)$ são contínuas. Assim sendo, (6.5) define uma função spline de grau 3.

Para facilitar os cálculos das aplicações, apresentamos na tabela 6.1 os valores de $B_i(x)$, bem como de suas derivadas nos pontos $x_{i-2}, x_{i-1}, x_i, x_{i+1}, x_{i+2}$.

	x_{i-2}	x_{i-1}	x_i	x_{i+1}	x_{i+2}
B_i	0	1	4	1	0
B'_i	0	$3/h$	0	$-3/h$	0
B''_i	0	$6/h^2$	$-12/h^2$	$6/h^2$	0

Tabela 6.1: Valores de $B_i(x)$ e derivadas nos pontos da partição.

Como na base dos splines lineares as funções se anulam fora de intervalos de comprimento $2h$, as funções da base dos splines cúbicos são menos locais, posto que elas se anulam fora de intervalos de comprimento $4h$. Isto se reflete na esparsidade das matrizes dos sistemas que surgem nas aplicações. Entretanto, ganhamos mais suavidade ao passar dos splines lineares para os cúbicos.

Chamando

$$B(x) = \begin{cases} (x+2)^3 & -2 \leq x \leq -1 \\ 1 + 3(x+1) + 3(x+1)^2 - 3(x+1)^3 & -1 \leq x \leq 0 \\ 1 + 3(1-x) + 3(1-x)^2 - 3(1-x)^3 & 0 \leq x \leq 1 \\ (2-x)^3 & 1 \leq x \leq 2 \\ 0 & x \notin [-2, 2]. \end{cases} \quad (6.6)$$

Podemos verificar diretamente que

$$B_i(x) = B\left(\frac{x - x_0}{h} - i\right).$$

Bases genéricas

Schoenberg mostrou, em 1946, que existe uma forma genérica de construir bases para os splines de grau n através de convoluções sucessivas. O produto *convolução* entre duas

funções integráveis é definido por

$$(f * g)(x) = \int_{-\infty}^{\infty} f(x-t) g(t) dt.$$

Seja a função $\phi(x)$ definida por

$$\phi(x) = \begin{cases} 1 & \text{se } |x| \leq 0.5 \\ 0 & \text{se } |x| > 0.5. \end{cases}$$

Para um número inteiro l podemos calcular $(l-1)$ convoluções de ϕ :

$$\varphi^l(x) = (\phi * \phi * \dots * \phi)(x).$$

Por exemplo, usando a definição de convolução, por cálculos diretos, podemos mostrar que

$$\varphi^2(x) = (\phi * \phi)(x) = L(x)$$

e que

$$\varphi^4(x) = (\phi * \phi * \phi * \phi)(x) = B(x),$$

onde $L(x)$ e $B(x)$ são as funções definidas em (6.3) e (6.6), respectivamente.

É possível verificar que as funções

$$\varphi_i(x) = \varphi^l\left(\frac{x - x_i}{h}\right)$$

formam uma base para os splines de ordem $(l-1)$, associados a uma partição uniforme na qual $x_{i+1} - x_i = h$.

6.4 Uso de Splines na Interpolação: Interpolação por Partes

No capítulo 5, definimos e encaminhamos o problema de aproximação de funções por via de interpolação polinomial. Podemos agora apresentar outra forma de interpolação usando polinômios por partes. Com o uso das bases introduzidas na seção anterior, este problema torna-se simples e apresenta ótimos resultados de aproximação, fato também demonstrado teoricamente.

Para usar as funções splines em interpolação, tomamos os pontos de interpolação como nós da partição das funções splines. Ilustramos o processo de interpolação por partes usando splines lineares e cúbicas.

Na interpolação linear por partes da função $f(x)$ nos pontos x_i , $i = 0 : n$, procuramos os coeficientes a_i tais que

$$s_1(x) = \sum_{i=0}^n a_i l_i(x) \quad \text{e} \quad s_1(x_j) = f(x_j) = f_j, \quad j = 0 : n.$$

Na primeira expressão, a função spline linear é escrita em termos da base definida em (6.2). As condições de interpolação nos levam às relações à direita.

Como vimos anteriormente, $l_i(x_j) = 0$ se $i \neq j$ e $l_i(x_i) = 1$. Portanto, da aplicação direta da expressão que define $s_1(x)$ nos pontos de interpolação x_i , obtemos

$$s_1(x_i) = a_i = f(x_i) = f_i.$$

Assim, no caso linear o spline que interpola $f_0, f_1, \dots, f_n$ é obtido de imediato

$$s_1(x) = \sum_{i=0}^n f_i l_i(x).$$

No caso da interpolação por splines cúbicos procuramos os coeficientes a_i tais que

$$s_3(x) = \sum_{i=-1}^{n+1} a_i B_i(x) \quad \text{e} \quad s_3(x_j) = f(x_j) = f_j, \quad j = 0 : n.$$

onde $B_i(x)$ é a base dos splines cúbicos, apresentada na seção anterior. A figura 6.4 e a tabela 6.1 nos mostram que, para cada x_j , os únicos valores diferentes de zero são $B_{j-1}(x_j)$, $B_j(x_j)$ e $B_{j+1}(x_j)$. Assim, para cada ponto de interpolação teremos:

$$s_3(x_j) = a_{j-1} B_{j-1}(x_j) + a_j B_j(x_j) + a_{j+1} B_{j+1}(x_j). \quad (6.7)$$

Como os pontos de interpolação foram escolhidos como nós da malha, temos, pela tabela 6.1, $B_{j-1}(x_j) = 1$, $B_j(x_j) = 4$ e $B_{j+1}(x_j) = 1$. Lembrando a condição de interpolação, $s_3(x_j) = f_j$, obtemos um sistema que deve ser satisfeito pelos a_i :

$$a_{j-1} + 4a_j + a_{j+1} = f(x_j) = f_j, \quad \text{para } j = 0 : n.$$

Numa contagem direta, podemos verificar que temos um sistema com $n+1$ equações e $n+3$ incógnitas, que são $a_{-1}, a_0, \dots, a_n, a_{n+1}$. Assim, para resolver o sistema, precisamos de duas condições adicionais. Existem várias possibilidades, mas em geral são usadas condições impostas nos extremos do spline. Podemos tomar, por exemplo:

$$s'_3(x_0) = f'(x_0) \quad \text{e} \quad s'_3(x_n) = f'(x_n) \quad (6.8)$$

ou ainda

$$s''_3(x_0) = 0 \quad \text{e} \quad s''_3(x_n) = 0. \quad (6.9)$$

O primeiro acaso é conhecido como condição de engaste, termo herdado das estruturas que vinculam inclinações nos engastes. Nesta escolha juntamos ao sistema as duas equações correspondentes ao cálculo de $s'_3(x_0)$ e $s'_3(x_n)$, usando (6.8) e a tabela 6.1:

$$-3a_{-1} + 3a_1 = h f'(x_0) \quad \text{e} \quad -3a_{n-1} + 3a_{n+1} = h f'(x_n).$$

A segunda opção, definida pelas equações (6.9) é conhecida como condição de fronteira natural. As equações adicionais para o sistema são obtidas usando a segunda derivada de (6.7) e a tabela 6.1:

$$a_{-1} - 2a_0 + a_1 = 0 \quad \text{e} \quad a_{n-1} - 2a_n + a_{n+1} = 0.$$

Numa terceira opção, as condições adicionais são obtidas supondo mais suavidade na cauda da função. Assim, se exigimos continuidade da terceira derivada em $x = x_1$ e $x = x_{n-1}$, obtemos as duas condições adicionais independentes de condições de contorno.

Alternativamente, podemos encontrar a expressão do spline cúbico que interpola f_i , $i = 0 : n$ aplicando diretamente a definição dos splines, isto é, impondo as condições de continuidade das derivadas. Este procedimento é útil quando os pontos de interpolação não são igualmente espaçados e a expressão geral dos elementos da base é complicada, como comentado anteriormente. O resultado abaixo nos indica como encontrar o polinômio interpolador diretamente da definição de spline.

Teorema 6.1. *Chamemos*

$$x_i - x_{i-1} = h_i, \quad \frac{f_i - f_{i-1}}{h_i} = d_i \quad \text{e} \quad \frac{x - x_{i-1}}{h_i} = t, \quad \text{para } x_{i-1} \leq x \leq x_i.$$

O spline cúbico, com nós em x_i , $i = 0 : n$, tal que $s_3(x_i) = f_i$, $i = 0 : n$, tem como expressão no intervalo $[x_{i-1}, x_i]$, $i = 1 : n$

$$q_i(t(x)) = tf_i + (1-t)f_{i-1} + h_it(1-t)[(k_{i-1} - d_i)(1-t) - (k_i - d_i)t], \quad (6.10)$$

onde $k_0, k_1, \dots, k_n$ satisfazem ao sistema tridiagonal

$$h_{i+1}k_{i-1} + 2(h_i + h_{i+1})k_i + h_ik_{i+1} = 3(h_id_{i+1} + h_{i+1}d_i), \quad i = 1 : n-1. \quad (6.11)$$

Demonstração: Da expressão (6.10) vemos que $q_i(x)$ é um polinômio de grau 3. Observe que $x = x_{i-1}$ corresponde a $t = 0$, ao passo que $x = x_i$ corresponde a $t = 1$. Assim, substituindo $t = 0$ e $t = 1$ em (6.10) teremos:

$$q_i(x_{i-1}) = y_{i-1} \quad \text{e} \quad q_i(x_i) = y_i,$$

portanto, $q_i(x_i) = q_{i+1}(x_i) = y_i$. Dessa maneira, verificamos que a continuidade de $s_3(x)$ e as condições de interpolação são satisfeitas.

Derivando (6.10), usando a regra da cadeia e a definição de d_i temos:

$$q'_i(x(t)) = d_i + (1-2t)[(k_{i-1} - d_i)(1-t) - (k_i - d_i)] + t(1-t)[2d_i - k_{i-1} - k_i].$$

Tomando $t = 0$ e $t = 1$ teremos, respectivamente

$$q'_i(x_{i-1}) = k_{i-1} \quad \text{e} \quad q'_i(x_i) = k_i.$$

Assim sendo, $q'_{i+1}(x_i) = q'_i(x_i)$, e fica demonstrada a condição de continuidade da primeira derivada.

A condição de continuidade da segunda derivada nos pontos $x_1, \dots, x_{n-1}$ nos leva ao sistema (6.11). De fato, usando novamente a regra da cadeia na derivação de $q'_i(x(t))$ obtemos:

$$q''_i(x(t)) = \frac{2}{h_i} \{-(k_{i-1} - d_i)(1 - t) + (k_i - d_i)t + (1 - 2t)(2d_i - k_{i-1} - k_i)\};$$

a condição $q''_i(x_i) = q''_{i+1}(x_i)$ é equivalente ao sistema (6.11).

Assim, os pedaços de polinômios $q_i(x)$ são "colados" de modo que a função resultante tenha a segunda derivada contínua, que é a condição de spline cúbico. O teorema está demonstrado.

Como o sistema (6.10) tem $(n - 1)$ equações e $(n + 1)$ incógnitas, são necessárias duas condições adicionais para determinar univocamente a função interpoladora. Aqui também podemos tomar as equações correspondentes às condições (6.8) e (6.9). No primeiro caso, as equações adicionais são

$$k_0 = f'(x_0) \quad \text{e} \quad k_n = f'(x_n), \quad (6.12)$$

ao passo que, no segundo caso, as condições adicionais são

$$2k_0 + k_1 = 3d_1 \quad \text{e} \quad k_{n-1} + 2k_n = 3d_n. \quad (6.13)$$

Exemplo 6.4.1. Tomando $x_i = 0.2i$, $i = 0 : 5$, calculamos os valores $f_i = \exp(x_i)$, $i = 0 : 5$. Vamos agora calcular as aproximações para $\exp(x)$ usando o spline cúbico que interpola os valores (x_i, y_i) . O sistema tridiagonal (6.11) (como os h_i são constantes podemos cancelá-los), é:

$$k_{i-1} + 4k_i + k_{i+1} = 3(d_{i+1} - d_i), \quad i = 1 : 4.$$

Usando a condição de contorno natural (6.9), a primeira e a última equação do sistema serão:

$$2k_0 + k_1 = 3d_1 \quad \text{e} \quad k_{n-1} + 2k_n = 3d_n.$$

Reescrevendo o sistema na forma matricial e calculando os valores de d_i temos:

$$\begin{bmatrix} 2 & 1 & 0 & 0 & 0 & 0 \\ 1 & 4 & 1 & 0 & 0 & 0 \\ 0 & 1 & 4 & 1 & 0 & 0 \\ 0 & 0 & 1 & 4 & 1 & 0 \\ 0 & 0 & 0 & 1 & 4 & 1 \\ 0 & 0 & 0 & 0 & 1 & 2 \end{bmatrix} \begin{bmatrix} k_0 \\ k_1 \\ k_2 \\ k_3 \\ k_4 \\ k_5 \end{bmatrix} = \begin{bmatrix} 3.32105 \\ 7.37738 \\ 9.01074 \\ 11.00574 \\ 13.44245 \\ 7.39112 \end{bmatrix}$$

cujas solução é $(k_0, k_1, k_2, k_3, k_4, k_5) = (1.05797, 1.20511, 1.49897, 1.80976, 2.26775, 2.56168)$.

Como exemplo, tomando $x = 0.35$, no intervalo (x_1, x_2) temos $i = 2$ e

$$t = \frac{0.35 - 0.2}{0.2} = 0.75.$$

Substituindo os valores de f_1, f_2, d_1, d_2, h e k em (6.10) temos

$$q_2(t) = 1.491825t + 1.221403(1-t) + 0.2t(1-t)[-0.44636(1-t) + 0.15250t].$$

Tomando $t = 0.75$, correspondente à $x = 0.35$, obtemos a aproximação

$$q(0.35) = 1.41871.$$

Comparando com o valor $\exp(0.35) = 1.41906755$, com nove dígitos corretos, vemos que o erro da aproximação obtida na interpolação por partes usando seis pontos é 0.356×10^{-3} . $\square$

6.5 Ajuste de Curvas usando Splines

Vimos, no capítulo anterior, que, se precisarmos ajustar dados que possivelmente estão contaminados com algum tipo de erro, a técnica da interpolação deixa de ser adequada. Uma alternativa é o ajuste por via dos quadrados mínimos. Mas, em muitos casos, não temos informações sobre o tipo de funções que podem ser usadas para gerar o ajuste final, e testar várias escolhas pode ser caro e fadado ao insucesso. O ajuste por uma combinação de splines associados a uma partição predeterminada é, em muitos casos, conveniente. Vamos ilustrar o ajuste por spline, tomando a base dos splines cúbicos.

Sejam (x_i, f_i) , $i = 0 : m$ pontos de uma tabela na qual os pontos x_i estão no intervalo (a, b) . Para ajustar os pontos dessa tabela, escolhemos a função

$$g(x) = c_{-1}B_{-1}(x) + c_0B_0(x) + \dots + c_nB_n(x) + c_{n+1}B_{n+1}(x),$$

onde as funções $B_i(x)$ são definidas por (6.4) tomando uma segunda partição uniforme do intervalo $[a, b]$, que denotaremos por

$$\Pi = a = \underline{x}_0 < \underline{x}_1 < \dots < \underline{x}_{n-1} < \underline{x}_n = b, \quad n \ll m.$$

Observe que em geral $x_i \neq \underline{x}_i$ posto que x_i se refere aos dados observados e $\underline{x}_i$ à partição escolhida.

Como vimos no capítulo anterior, os parâmetros da melhor aproximação no sentido dos quadrados mínimos, isto é, os coeficientes $c_{-1}, c_0, \dots, c_n, c_{n+1}$, são calculados através do sistema normal (5.13):

$$Bc = f.$$

No caso dos splines cúbicos, os elementos da matriz B do sistema normal dependem da escolha do produto escalar. Se escolhemos o produto escalar discreto, estes elementos são:

$$B_{ij} = \sum_{k=0}^m B_i(x_k) B_j(x_k).$$

Por outro lado, no caso contínuo, os elementos da matriz B serão:

$$B_{ij} = \int_a^b B_i(x) B_j(x) dx. \quad (6.14)$$

Quanto ao termo independente do sistema, componentes do vetor f , também serão calculadas de acordo com o produto escalar adotado. Assim tomamos

$$f_i = \sum_{k=0}^m f(x_k) B_i(x_k) \quad \text{e} \quad f_i = \int_a^b f(x) B_i(x) dx \quad (6.15)$$

nos casos discreto e contínuo, respectivamente.

Como $B_i(x) = 0$ fora do intervalo (x_{i-2}, x_{i+2}) (veja figura 6.4 ou (6.4), que define $B_i(x)$), temos:

$$B_i(x) B_j(x) = 0 \quad \text{se } |i - j| > 3;$$

portanto, $B_{ij} = 0$ se $|i - j| > 3$. Dessa maneira, o sistema normal associado ao ajuste por via dos quadrados mínimos, usando splines cúbicos, terá uma matriz de banda com as sete diagonais principais não-nulas, como mostra a figura 6.5.

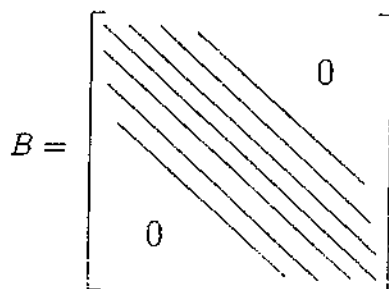


Figura 6.5: Sistema de banda resultante da aproximação por splines cúbicos.

O sistema normal tem $(n + 3)$ incógnitas, uma vez que adotamos $(n + 1)$ pontos na partição que define a base dos splines. Assim, B é uma matriz de dimensão $(n + 3) \times (n + 3)$. Se n for grande, pode-se economizar armazenando esta matriz em 4 vetores de $(n + 3)$ posições, posto que B é uma matriz simétrica.

Como a matriz do sistema normal é simétrica e positiva definida, os métodos de decomposição de Cholesky ou os gradientes conjugados podem ser usados. A estrutura de banda

também deve ser explorada. O Método de Gauss-Seidel também pode ser usado, pois tem a convergência garantida.

O cálculo das integrais (6.14) e (6.15) não é tão simples. Por outro lado, a escolha de uma malha que seja conveniente para os dados que estão sendo ajustados também não é óbvia. De fato, se usamos muitos pontos nesta malha, nos aproximamos da interpolação dos dados, acompanhando os erros que queremos filtrar. No outro extremo, usando poucos pontos, podemos estar suavizando muito os dados e perdendo informações. Estes pontos podem dificultar o uso de splines no ajuste de dados.

A disponibilidade recente de pacotes computacionais com ênfase em algoritmos eficientes pode amenizar estas dificuldades. Nestes pacotes, as funções splines têm sido amplamente usadas, em resposta à crescente necessidade de tratar dados.

Na seção final deste capítulo apresentamos algumas funções do **Matlab** que podem ser úteis nas aplicações práticas de ajuste de dados usando splines.

6.6 Exemplos de Aplicação de Splines em Equações Diferenciais: os Métodos da Colocação e Elementos Finitos

6.6.1 Colocação: um Exemplo

A partir da década de 60, as funções polinomiais por partes receberam uma atenção especial, principalmente por serem convenientes no cálculo de soluções aproximadas para equações diferenciais ordinárias e parciais. Como exemplo, o Método de Galerkin, que toma como base as funções polinomiais por partes, gerou o Método dos Elementos Finitos, popular pela versatilidade de suas aplicações.

Nesta seção, apresentaremos o Método da Colocação como um exemplo da utilização dos splines em equações diferenciais. Para não perdermos a oportunidade, apresentamos também uma aplicação muito simples do Método dos Elementos Finitos à equação de Sturm-Liouville. Particularizaremos tanto a equação diferencial quanto a base escolhida por motivos didáticos. Entretanto, ressaltamos que as idéias se estendem a outras aplicações.

Exemplo 6.6.1. Método da Colocação: Consideremos a equação diferencial de segunda ordem com os dados de contorno:

$$\begin{cases} y''(x) + 2xy'(x) - y(x) = 2(1+x^2) \cos(x), & x \in (1, 2) \\ y(1) = \sin(1) & \text{e} & y(2) = 2\sin(2) \end{cases} \quad (6.16)$$

Inicialmente, definimos uma partição no intervalo $[1, 2]$:

$$\Pi : x_0 = 1.0 < x_1 = 1.1 < x_2 = 1.2 < \dots < x_{10} = 2.0.$$

Assim, tomando $h = 0.1$, definimos os pontos aos quais associaremos a base dos splines:

$$x_i = 1 + ih, \quad i = 0 : 10.$$

Como a solução da equação diferencial é uma função que tem duas derivadas, vamos escolher os B-splines, associados à partição Π , como base. Assim, usando a representação da solução aproximada $\tilde{y}(x)$ através desta base, queremos determinar c_j , $j = -1 : 11$ tal que

$$y(x) \cong \tilde{y}(x) = \sum_{j=-1}^{11} c_j B_j(x). \quad (6.17)$$

Lembre-se que $B_{-1}(x)$ e $B_{11}(x)$ entram nessa combinação linear porque eles contribuem no intervalo $[1, 2]$ onde procuramos a solução.

As derivadas da solução aproximada são obtidas diretamente de (6.17):

$$\tilde{y}'(x) = \sum_{j=-1}^{11} c_j B_j'(x) \quad \text{e} \quad \tilde{y}''(x) = \sum_{j=-1}^{11} c_j B_j''(x).$$

Substituindo as expressões de $\tilde{y}(x)$, $\tilde{y}'(x)$ e $\tilde{y}''(x)$ na equação diferencial, obtemos uma equação que deve ser satisfeita pelos coeficientes c_i para que (6.17) seja uma solução aproximada para (6.16):

$$\sum_{j=-1}^{11} c_j \{B_j''(x) + 2x B_j'(x) - B_j(x)\} = 2(1+x^2) \cos(x). \quad (6.18)$$

Para ser compatível com as treze incógnitas $c_{-1}, c_0, c_1, \dots, c_{11}$, vamos montar um sistema de 13 equações nestas incógnitas. Fazendo com que a equação (6.18) seja satisfeita nos 11 pontos da partição do intervalo $[1, 2]$ e que $\tilde{y}(x)$ respeite as duas condições de contorno definidas em (6.16), definimos 13 condições, que são:

$$\tilde{y}(1) = \sum_{j=-1}^{11} c_j B_j(1) = \text{sen}(1),$$

$$\sum_{j=-1}^{11} c_j \{B_j''(x_i) + 2x_i B_j'(x_i) - B_j(x_i)\} = 2(1+x_i^2) \cos(x_i) \quad 0 \leq i \leq 10 \quad (6.19)$$

e

$$\tilde{y}(2) = \sum_{j=-1}^{11} c_j B_j(2) = 2\text{sen}(2).$$

Como as funções $B_j(x)$ são nulas fora do intervalo (x_{j-2}, x_{j+2}) temos, como ilustrado na figura 6.6,

$$B_j(x_i) = B_j'(x_i) = B_j''(x_i) = 0 \quad \text{quando } j - i > 1.$$

Assim sendo, para cada j , as equações (6.19) têm apenas três termos não-nulos, que são os correspondentes aos c_{j-1} , c_j e c_{j+1} . Consultando a tabela 6.1 obtemos:

$$\begin{aligned} B_{j-1}(x_j) &= 1 & B_j(x_j) &= 4 & B_{j+1}(x_j) &= 1; \\ B'_{j-1}(x_j) &= -\frac{3}{h} & B'_j(x_j) &= 0 & B'_{j+1}(x_j) &= \frac{3}{h}; \\ B''_{j-1}(x_j) &= \frac{6}{h^2} & B''_j(x_j) &= -\frac{12}{h^2} & B''_{j+1}(x_j) &= \frac{6}{h^2}. \end{aligned}$$

Se estes valores são substituídos nas equações intermediárias do sistema (6.19) teremos:

$$\begin{aligned} (6 - 6x_i h - h^2)c_{i-1} - (12 + 4h^2)c_i + (6 + 6x_i h - h^2)c_{i+1} &= \\ = 2h^2(1 + x_i^2) \cos(x_i) &\text{ para } i = 0 : 10. \end{aligned}$$

As equações correspondentes às condições de contorno são:

$$c_{-1} + 4c_0 + c_1 = \sin(1) \quad \text{e} \quad c_9 + 4c_{10} + c_{11} = 2\sin(2).$$

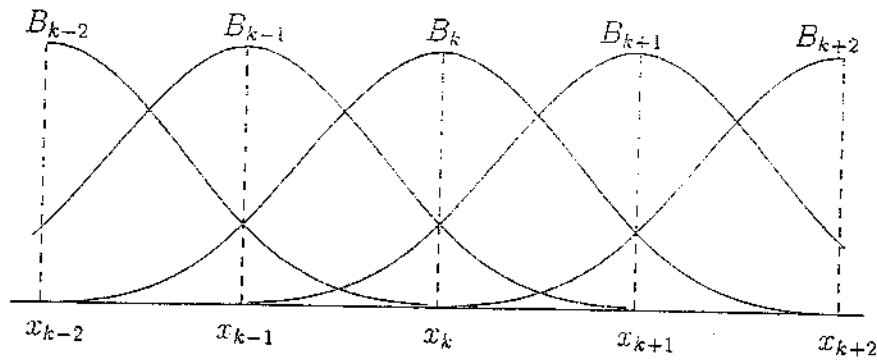
Seguindo esse caminho, os coeficientes de (6.17) serão obtidos pela resolução de um sistema "quase tridiagonal", no qual apenas a primeira e a última equações desrespeitam a condição de tridiagonalidade. A resolução deste sistema nos levou aos seguintes coeficientes:

$$\begin{aligned} c_{-1} &= 0.117353 & c_0 &= 0.140179 & c_1 &= 0.163403 & c_2 &= 0.186507 \\ c_3 &= 0.208956 & c_4 &= 0.230210 & c_5 &= 0.249731 & c_6 &= 0.266995 \\ c_7 &= 0.281495 & c_8 &= 0.292755 & c_9 &= 0.300334 & c_{10} &= 0.303836 \\ c_{11} &= 0.302917 \end{aligned}$$

Para efeito de comparação, na tabela 6.2 apresentamos a solução aproximada obtida pelo método da colocação e a solução analítica da equação diferencial, que neste exemplo é $y(x) = x \sin x$. Nesta tabela, usamos alguns nós da partição e outros valores de x que não pertencem à partição. No último caso, observe que:

$$\tilde{y}(\bar{x}) = \sum_{j=-1}^{11} c_j B_j(\bar{x}).$$

Como $x_k \leq \bar{x} \leq x_{k+1}$ para algum k , apenas quatro $B_j(\bar{x})$ não se anulam, os correspondentes aos $j = k - 1, k, k + 1$ e $k + 2$, como mostra a figura 6.6. □

Figura 6.6: Funções de base relativas a x_k .

x	$y(x) = x \operatorname{sen}(x)$	$\tilde{y}(x)$	$ \tilde{y}(x) - x \operatorname{sen}(x) $
1.25	1.18623	1.16639	0.198×10^{-1}
1.40	1.37963	1.37953	0.103×10^{-3}
1.60	1.59932	1.59920	0.113×10^{-3}
1.75	1.72198	1.69317	0.288×10^{-1}
1.80	1.75293	1.75285	0.786×10^{-4}

Tabela 6.2: Resultados do exemplo.

Embora os resultados da tabela 6.2 possam parecer pobres, não podemos esquecer que a discretização usada também deixa a desejar: $h = 10^{-1}$. Resultados melhores serão obtidos com h menores, e o custo desta melhoria é a resolução de sistemas maiores.

6.6.2 Elementos Finitos: um Exemplo

Como estamos com a bagagem encaminhada, vamos introduzir o *Método dos Elementos Finitos* como mais uma aplicação das funções splines. Por questões didáticas, vamos usar o problema-padrão:

$$\begin{cases} -(p(x)y'(x))' + q(x)y(x) = f(x), & 0 \leq x \leq 1 \\ y(0) = y(1) = 0, \end{cases} \quad (6.20)$$

conhecido como o problema de Sturm-Liouville, estudado, pela primeira vez, em 1836-1838.

Inicialmente, multiplicamos (6.20) por $B_j(x)$ e integramos a equação resultante em $(0, 1)$. Usando a fórmula de integração por partes no primeiro termo da equação resultante, obtemos a *Formulação Fraca* do problema:

$$-p(x)y'(x)B_j(x)\Big|_0^1 + \int_0^1 p(x)y'(x)B_j'(x)dx + \int_0^1 q(x)y(x)B_j(x)dx = \int_0^1 f(x)B_j(x)dx \quad (6.21)$$

No passo seguinte, vamos procurar uma aproximação de $y(x)$ que seja uma combinação dos $B_i(x)$ associada à partição definida pelos nós $x_0, x_1, \dots, x_m$. Assim, a aproximação de $y(x)$ será definida por:

$$\tilde{y}(x) = \sum_{i=-1}^{m+1} c_i B_i(x).$$

Portanto temos $m + 3$ incógnitas $c_{-1}, c_0, c_1, \dots, c_{m+1}$. As condições de contorno do problema nos fornecem duas equações nestas incógnitas: $c_{-1} + 4c_0 + c_1 = 0$ e $c_{m-1} + 4c_m + c_{m+1} = 0$.

Usando $\tilde{y}(x)$ em (6.21), obtemos as $m + 1$ equações que completam o sistema:

$$\begin{aligned} -p(x) B_j(x) \sum_{i=-1}^{m+1} c_i B'_i(x) \Big|_0^1 + \sum_{i=-1}^{m+1} c_i \int_0^1 \{p(x) B'_i(x) B'_j(x) + q(x) B_i(x) B_j(x)\} dx = \\ = \int_0^1 f(x) B_j(x) dx, \quad j = 0 : m. \end{aligned} \quad (6.22)$$

- No primeiro termo desta equação, as funções devem ser calculadas em $x = 0$ e $x = 1$, conforme a notação usual do Cálculo. Observe que $B_i(x) B_j(x) = 0$ para $|j - i| \geq 4$. Por outro lado, o primeiro termo se anula para $2 \leq j \leq m - 2$.

Chamando

$$a = \frac{3p(0)}{h} (c_1 - c_{-1}) \quad \text{e} \quad b = \frac{3p(1)}{h} (c_{m+1} - c_{m-1}),$$

o primeiro termo de (6.22) será $a B_j(0)$ se $j = -1, 0, 1$ e $b B_j(1)$ se $j = m - 1, m, m + 1$. Usando a tabela 6.1 temos

$$B_{-1}(0) = B_1(0) = B_{m-1}(1) = B_{m+1}(1) = 1$$

e

$$B_0(0) = B_n(1) = 4.$$

Assim sendo, as equações (6.22) e as condições de contorno nos fornecem um sistema linear de $m + 3$ equações nas incógnitas $c_{-1}, c_0, c_1, \dots, c_m, c_{m+1}$ que definem a solução aproximada $\tilde{y}(x)$.

6.7 Splines com Ajuda do Matlab

Como vimos neste capítulo, a interpolação por meio de splines cúbicos consiste da junção de pedaços de polinômios de grau três impondo a condição de continuidade da segunda derivada, nos nós que definem a malha usada. Para que o problema de interpolação por partes tenha uma única solução, são necessárias duas condições adicionais. A

função spline do Matlab realiza esta interpolação. Condições que forçam a continuidade da terceira derivada entre primeiro e segundo polinômios cúbicos, assim como entre os dois últimos polinômios, atuam como condições adicionais que garantem a unicidade do polinômio interpolador. Na sua forma mais simples, a função spline usa os dados dos vetores x e y para calcular os valores interpolados nos pontos definidos por um novo vetor xi , como ilustrado a seguir.

```
>> x = 0:10; y = tan(pi*x/25);
>> xi = linspace(0, 10);
>> yi = spline(x, y, xi);
>> plot(x, y, '*', xi, yi)
```

Na função spline, admite-se que a variável independente é monotônica. O exemplo a seguir ilustra como esta função pode ser usada para interpolar dados no plano, no qual não há monotonicidade. Como pode ser verificado, a idéia é usar a parametrização da curva.

```
t = linspace(0, 3*pi, 15);           % Definindo a malha do parâmetro
x = sqrt(t).*cos(t);                 % Abcissa dos pontos numa espiral
y = sqrt(t).*sin(t);                 % Ordenada dos pontos numa espiral
ti = linspace(0, 3*pi, 50);          % Refinamento nos parâmetros
xi = spline(t, x, ti);                % Interpolando as abcissas
yi = spline(t, y, ti);                % Interpolando as ordenadas
plot(x, y, '*', xi, yi, '.')         % Traçado do gráfico
```

A função $yi = \text{interp1}(x, y, xi, 'spline')$ é equivalente à função $yi = \text{spline}(x, y, xi)$.

O *toolbox spline* contém programas básicos do trabalho de Carl de Boor em *A Practical Guide to Splines*, o que facilita a utilização dos polinômios por partes em diversas aplicações. Uma aplicação importante, cuja complexidade da programação inviabilizava, é o ajuste de dados usando splines. Outras aplicações em equações diferenciais e integrais também ficaram mais simples com a disponibilidade deste pacote. Estão também incluídos no *toolbox spline* programas que permitem o trato de funções de mais de uma variável e produto tensorial de splines. Os splines são apresentados em duas formas, a chamada *B-form*, na qual aparecem os coeficientes da base, denotados por c_i neste capítulo, e a *pp-form*, adequada, por exemplo, à interpolação cúbica como apresentada no teorema 6.1 deste capítulo. Em geral, as funções dos pacotes usam ambas as formas.

A seguir, apresentamos um exemplo que ilustra o ajuste de dados usando spline cúbico. Na função $ys = \text{csaps}(x, y, p, xs)$, x e y são vetores que contêm os dados que serão ajustados, p é um parâmetro de suavização (um valor entre 0 e 1), fornecido pelo usuário, e xs é um vetor que define os pontos onde serão calculados os valores ajustados. Se $p = 0$, o resultado é o ajuste por uma reta e, se $p = 1$, o resultado é o polinômio interpolador dos dados x, y . Como aplicação, vamos perturbar a função $\sin(x)$ no intervalo $[0, \pi]$ e ajustar estes dados usando alguns valores de p , na seguinte seqüência de comandos.

```
% Pontos que serão usados no ajuste.
>> x = linspace(0,pi,10); y = sin(x);

% Perturbação aleatória dos valores de y.
>> z = rand(size(y)); y = y+.1*z;

% Malha na qual será calculada a função ajustada.
>> xs = linspace(0, pi);

% Ajuste usando p = .9
>> ys = csaps(x, y, .9, xs);

% Ajuste usando p = .999, próximo da interpolação;
>> ys1 = csaps(x, y, .999, xs);

% Ajuste usando p = .01, próxima de uma reta;
>> ys2 = csaps(x, y, .01, xs);

% Gráfico com valores ajustados e dados.
>> plot(xs, ys, xs, ys1, xs, ys2, x, y, '*')
```

Na prática, podemos verificar que os melhores ajustes são obtidos com $p \approx 1$. Maiores detalhes no ajuste podem ser obtidos com `help csaps`.

Com a ajuda do *toolbox spline*, podemos calcular aproximações de derivadas usando splines, interpolação com várias condições adicionais para os extremos, escolha ótima de nós de interpolação etc. Os demonstrativos de diversas destas funções mostram a potencialidade do pacote. A lista completa das funções disponíveis no *toolbox spline* pode ser obtida usando

```
>> help splines
```

Exercícios

1. Os splines cúbicos podem ser usados na aproximação de funções periódicas. Neste caso, sendo

$$f_0 = f_n, \quad d_0 = d_n \quad \text{e} \quad k_0 = k_n,$$

estabeleça o sistema linear para a determinação de $k_1, k_2, \dots, k_n$.

2. Mostre que

$$f(x) = \begin{cases} x^3 & \text{se } x \geq 0 \\ 0 & \text{se } x < 0 \end{cases}$$

é um spline cúbico. A qual partição ele está associado?

3. Calcule uma aproximação para $f(0.2)$ usando interpolação por spline cúbico nos valores abaixo, com condições de contorno naturais.

x	0.1	0.3	0.4	0.5	0.6
$f(x)$	1.2314	1.91212	2.38554	2.96828	3.68012

4. Repita o exercício 3 usando agora as condições de contorno

$$f'(0.1) = 2.64281 \quad \text{e} \quad f'(0.6) = 7.849023$$

no spline cúbico interpolador. Compare os resultados.

5. Elabore o algoritmo de interpolação por spline cúbico com condições de contorno do tipo engaste (valores predefinidos para as derivadas nos extremos).

6. Explique como os splines cúbicos poderiam ser utilizados no cálculo aproximado da integral de uma função conhecida através de uma tabela.

7. Use a idéia do exercício 6 para aproximar $\int_0^1 e^{-x^2} dx$.

8. Ajuste a tabela abaixo usando splines cúbicos com nós em $x_0 = 0.0, x_1 = 0.3, x_2 = 0.6, x_3 = 0.9, x_4 = 1.2$ e $x_5 = 1.5$.

x	0	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	1.0	1.1	1.2	1.3	1.4	1.5
$f(x)$	0	0.1	0.2	0.4	0.6	0.8	1.1	1.4	1.8	2.2	2.7	3.3	4.0	4.8	5.7	6.7

9. Elabore o algoritmo para o ajuste de curvas usando quadrados mínimos e splines cúbicos. Teste seu algoritmo no exercício 8.

10. Elabore o algoritmo para o método da colocação aplicado ao problema de valor de contorno:

$$y''(x) + p(x)y'(x) + q(x)y(x) = 0 \quad \text{em} \quad (a, b)$$

e

$$y(a) = y_a \quad \text{e} \quad y(b) = y_b.$$

11. Use o método da colocação nos pontos $x_i = ih$ para encontrar uma solução aproximada para o problema:

$$\begin{cases} y''(x) + x y'(x) + y = 2x \\ y(0) = 1 \quad \text{e} \quad y(1) = 0, \end{cases}$$

com $h = 0.2$ e $h = 0.1$. Compare suas respostas.

12. A deflexão de uma longa placa retangular uniformemente carregada e sob tensão axial (para pequenas deflexões) pode ser calculada por uma equação diferencial de segunda ordem:

$$\begin{cases} y''(x) - \frac{S}{D}y(x) = -\frac{9l}{2D} + \frac{9l}{2D}x^2, & 0 \leq x \leq l \\ y(0) = y(l) = 0, \end{cases}$$

onde

l	comprimento da placa	1.0m
D	rigidez	10^7 N m
S	força axial	200 N cm
q	intensidade da carga uniforme	7 N/cm ² .

Calcule aproximações para a deflexão de cada ponto da partição usando o método dos elementos finitos com $h = l/10$ e splines cúbicos como base. Você saberia calcular a solução analítica? Caso afirmativo, compare-a com as aproximações obtidas.

13. Elabore o algoritmo para o métodos dos elementos finitos para problemas do tipo (6.20).

14. Ajuste os dados tabelados no exercício 8 usando a função `csaps` do Matlab, com $p = 0.1$, $p = 0.9$, $p = 0.95$ e $p = 0.99$. Compare graficamente estes ajustes.

Capítulo 7

INTEGRAÇÃO NUMÉRICA

7.1 Introdução

Neste capítulo, apresentaremos algumas técnicas numéricas para o cálculo aproximado da integral de $f(x)$, isto é,

$$\int_a^b f(x) dx.$$

Admitem-se conhecidos os extremos do intervalo de integração, a e b , e a função $f(x)$.

O cálculo de integrais pode aparecer sob várias formas. Por exemplo, a função a ser integrada pode ser apresentada através de uma tabela de valores e esta tabela pode vir, ou não, contaminada de erros. O intervalo de integração pode ser não limitado. Quando apresentada por uma expressão analítica, $f(x)$ pode ser muito complicada ou não existir uma primitiva para $f(x)$.

Lembramos que primitiva para $f(x)$ é uma função $F(x)$ tal que $F'(x) = f(x)$. Se uma primitiva é conhecida, então, pelo Teorema Fundamental do Cálculo (Newton, 1687 e Leibnitz, 1684), o valor da integral definida é:

$$\int_a^b f(x) dx = F(b) - F(a).$$

Na falta da primitiva, os métodos numéricos podem fornecer aproximações tão boas quanto se queira.

A idéia básica da integração numérica reside na aproximação da função integrando por um polinômio. A escolha desse polinômio e dos pontos que serão usados na sua determinação vai definir os diversos métodos de integração. Uma vez estabelecida a aproximação polinomial conveniente, sua integração é fácil.

As fórmulas de integração são somatórios cujas parcelas são valores da função $f(x)$ calculados em pontos e multiplicados por pesos convenientemente escolhidos. Assim, vamos procurar desenvolver fórmulas de integração do tipo:

$$\int_a^b f(x) dx \cong \sum_{i=0}^n w_i f(x_i), \quad (7.1)$$

onde $a \leq x_0 < x_1 < \dots < x_n \leq b$ são chamados de *pontos de integração* e w_i são os pesos da fórmula de integração. Para cada fórmula de integração, serão estabelecidos os pesos e os pontos de integração correspondentes ao critério de aproximação adotado.

7.2 Fórmulas de Newton-Cotes

As fórmulas de Newton-Cotes (1711) para a integração numérica são caracterizadas por pontos de integração igualmente espaçados no intervalo de integração (a, b) . Seja n um número inteiro, e $h = \frac{b-a}{n}$. Os pontos de integração das fórmulas de Newton-Cotes serão definidos por:

$$x_j = a + jh, \quad j = 0 : n.$$

Consideremos agora o polinômio de grau n que interpola os $(n+1)$ pontos $(x_i, f(x_i))$, $i = 0 : n$. Como vimos no capítulo 5, a fórmula de Lagrange pode ser usada para escrever este polinômio interpolador. No mesmo capítulo, apresentamos a expressão do erro do polinômio interpolador. Assim,

$$p_n(x) = \sum_{i=0}^n f(x_i) l_i(x)$$

$$f(x) - p_n(x) = \frac{f^{(n+1)}(\alpha)}{(n+1)!} (x - x_0) \dots (x - x_n),$$

onde $l_i(x)$ são as funções de Lagrange (5.2). Integrando esta última expressão no intervalo (a, b) e usando a expressão de $l_i(x)$ temos:

$$\int_a^b f(x) dx = \sum_{i=0}^n f(x_i) \int_a^b l_i(x) dx + \int_a^b \frac{f^{(n+1)}(\alpha)}{(n+1)!} (x - x_0) \dots (x - x_n) dx$$

onde $\alpha = \alpha(x)$ é um ponto do intervalo (a, b) .

Desta maneira, temos uma expressão do tipo (7.1), onde os pesos da fórmula de integração são obtidos pela integração de $l_i(x)$, isto é,

$$w_i = \int_a^b l_i(x) dx = \int_a^b \frac{(x - x_0) \dots (x - x_{i-1})(x - x_{i+1}) \dots (x - x_n)}{(x_i - x_0) \dots (x_i - x_{i-1})(x_i - x_{i+1}) \dots (x_i - x_n)} dx. \quad (7.2)$$

O erro correspondente à fórmula de integração associada a estes pesos será

$$\frac{1}{(n+1)!} \int_a^b f^{(n+1)}(\alpha(x)) (x - x_0) \dots (x - x_n) dx. \quad (7.3)$$

Assim apresentadas, podem-se obter fórmulas de tipo Newton-Cotes para polinômios de qualquer grau. Historicamente, as primeiras fórmulas foram estabelecidas para polinômios

de graus baixos. Por exemplo, no caso $n = 2$, as bem conhecidas fórmulas de Simpson já tinham sido desenvolvidas por Cavalieri, um dos pioneiros na integração, em 1639.

A seguir, examinaremos detalhes de alguns casos particulares de fórmulas de integração do tipo Newton-Cotes, nas quais os pesos w_i são calculados por (7.2) com diferentes valores de n . As expressões dos erros associados a estas fórmulas serão obtidas de (7.3) com os respectivos valores de n . As fórmulas citadas a seguir são muito usadas na integração numérica porque, além de simples, oferecem bons resultados.

7.2.1 Fórmula dos Trapézios: $n = 1$

A fórmula dos trapézios corresponde à interpolação da função a ser integrada por um polinômio de grau $n = 1$. Como a interpolação linear pede 2 pontos, usaremos os extremos do intervalo de integração, isto é, $a = x_0$ e $b = x_1$.

A expressão (7.2) nos permite encontrar os pesos da regra dos trapézios:

$$w_0 = \int_{x_0}^{x_1} \frac{(x - x_1)}{(x_0 - x_1)} dx = \frac{-(x - x_1)^2}{2h} \Big|_{x=x_0}^{x=x_1} = \frac{h}{2}$$

$$w_1 = \int_{x_0}^{x_1} \frac{(x - x_0)}{(x_1 - x_0)} dx = \frac{(x - x_0)^2}{2h} \Big|_{x=x_0}^{x=x_1} = \frac{h}{2}.$$

Para calcular o erro da fórmula dos trapézios, tomamos $n = 1$, $a = x_0$ e $b = x_1$ em (7.3). Assim,

$$E_T = \frac{1}{2} \int_{x_0}^{x_1} f''(\alpha(x))(x - x_0)(x - x_1) dx.$$

Para simplificar a expressão acima, observamos que a função $(x - x_0)(x - x_1)$ não muda de sinal no intervalo (x_0, x_1) , o que nos permite usar o Teorema do Valor Médio para integrais. Assim, existe $\beta \in (x_0, x_1)$, tal que

$$E_T = \frac{f''(\beta)}{2} \int_{x_0}^{x_1} (x - x_0)(x - x_1) dx = -f''(\beta) \frac{h^3}{12},$$

sendo a última passagem calculada pela integração de um polinômio de segundo grau.

Com estas argumentações, estabelecemos a *fórmula dos trapézios* para a integração no intervalo (x_0, x_1) :

$$\int_{x_0}^{x_1} f(x) dx = \frac{h}{2} [f(x_0) + f(x_1)] - \frac{h^3}{12} f''(\beta), \quad (7.4)$$

onde β é um ponto, não conhecido, do intervalo (x_0, x_1) .

A justificativa da escolha do nome fórmula dos trapézios é apresentada na figura 7.1 (a): área sob a curva foi aproximada pela área do trapézio.

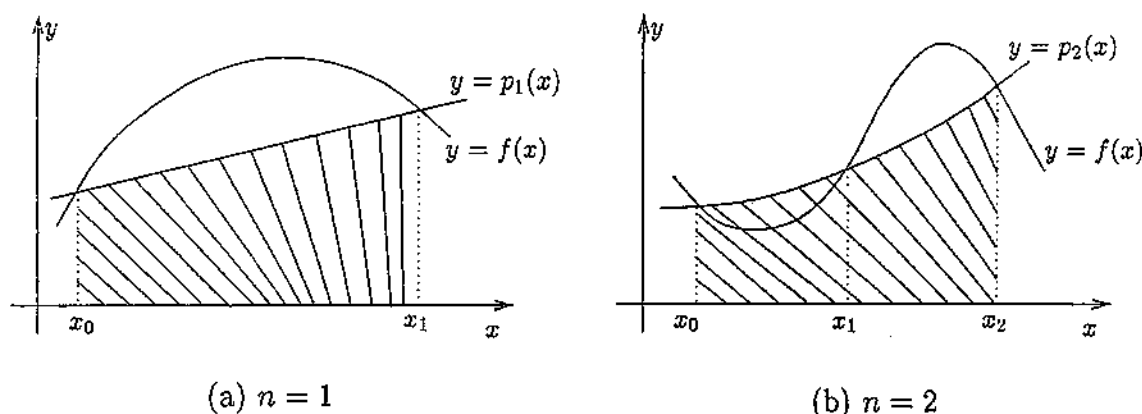


Figura 7.1: Aproximação de integral por trapézio e Simpson.

7.2.2 Fórmula de Simpson: $n = 2$

Para estabelecer a fórmula de Simpson, vamos interpolar $f(x)$ usando um polinômio de grau 2 que coincide com essa função nos pontos x_0 , x_1 e x_2 . Assim, tomamos $n = 2$, $x_0 = a$, $x_1 = (a + b)/2$ e $x_2 = b$ em (7.2). Integrando os polinômios de grau 2, estabelecemos os pesos da *fórmula de Simpson*:

$$w_0 = \int_{x_0}^{x_2} \frac{(x - x_1)(x - x_2)}{(x_0 - x_1)(x_0 - x_2)} dx = \frac{h}{3}$$

$$w_1 = \int_{x_0}^{x_2} \frac{(x - x_0)(x - x_2)}{(x_1 - x_0)(x_1 - x_2)} dx = \frac{4h}{3}$$

$$w_2 = \int_{x_0}^{x_2} \frac{(x - x_0)(x - x_1)}{(x_2 - x_0)(x_2 - x_1)} dx = \frac{h}{3}$$

O cálculo das integrais acima pode ser simplificado lembrando que

$$x_1 - x_0 = h, \quad x_2 - x_1 = h, \quad x_2 - x_0 = 2h$$

e usando a substituição de variáveis $t = x - x_1$ e portanto $x - x_0 = t + h$, $x - x_2 = t - h$. Por exemplo, fazendo estas substituições no cálculo do peso w_0 temos

$$w_0 = \int_{x_0}^{x_1} \frac{(x - x_1)(x - x_2)}{(x_0 - x_1)(x_0 - x_2)} dx = \frac{1}{2h^2} \int_{-h}^h t(t - h) dt = \frac{h}{3}.$$

A expressão do erro na fórmula de Simpson pode ser obtida tomando $n = 2$, $x_0 = a$, $x_1 = (a + b)/2$ e $x_2 = b$ em (7.3). Com alguns argumentos matemáticos, não tão simples como aqueles usados na fórmula dos trapézios, mostra-se que existe um $x_0 < \beta < x_2$ tal

que

$$E_S = - \left(\frac{h^5}{90} \right) f^{(4)}(\beta). \quad (7.5)$$

Dessa maneira, usando polinômios interpoladores de grau 2, estabelecemos a fórmula de Simpson e seu respectivo erro:

$$\int_{x_0}^{x_2} f(x) dx = \left(\frac{h}{3} \right) [f(x_0) + 4f(x_1) + f(x_2)] - \left(\frac{h^5}{90} \right) f^{(4)}(\beta), \quad (7.6)$$

onde β é um ponto não conhecido no intervalo (x_0, x_2) .

A interpretação gráfica da fórmula de Simpson é apresentada na figura 7.1 (b). A área sob a curva $y = f(x)$, entre $x = x_0$ e $x = x_2$, é aproximada pela área sob a parábola que passa pelos pontos $(x_0, f(x_0))$, $(x_1, f(x_1))$ e $(x_2, f(x_2))$, a área hachurada na figura 7.1 (b).

Das expressões obtidas para os respectivos erros, podemos observar que a fórmula dos trapézios é exata para polinômios de grau menor ou igual a 1, ao passo que a fórmula de Simpson é exata para polinômios de grau menor ou igual a 3.

De fato, se $f(x)$ é um polinômio de grau ≤ 1 , então $f''(x) \equiv 0$ e o termo do erro em (7.4) será nulo. Por outro lado, se $f(x)$ é um polinômio de grau ≤ 3 , então $f^{(4)}(x) \equiv 0$, o termo do erro em (7.6), será nulo e a fórmula de Simpson será exata.

A última afirmação pode parecer incoerente, uma vez que estamos interpolando polinômios de grau 3 usando polinômios de grau 2 e o resultado é exato. Como ilustra a figura 7.1, existe uma compensação de áreas e, podemos calcular a integral de um polinômio de grau 3 usando apenas uma parábola que interpole três pontos deste polinômio, e o valor será exato qualquer que seja o tamanho do intervalo de integração.

7.2.3 Fórmulas para $n = 3$ e $n = 4$

A título de informação, apresentamos a seguir fórmulas que são obtidas por meio da interpolação, da função a ser integrada, usando polinômios de graus 3 e 4 respectivamente. Pode-se observar, pelas expressões do erro, que elas são mais precisas que as anteriores. Em contrapartida, elas exigem cálculos de valores de $f(x)$ em um número maior de pontos de integração. A relação de compromisso entre a precisão obtida e o esforço computacional necessário vai depender do que exige a aplicação.

Fórmulas de Newton-Cotes para $n = 3$ e $n = 4$

$$\int_{x_0}^{x_3} f(x) dx = \left(\frac{3h}{8} \right) [f(x_0) + 3f(x_1) + 3f(x_2) + f(x_3)] - \left(\frac{3h^5}{80} \right) f^{(4)}(\beta), \quad x_0 \leq \beta \leq x_3; \quad (7.7)$$

$$\int_{x_0}^{x_4} f(x) dx = \left(\frac{2h}{45}\right) [7f(x_0) + 32f(x_1) + 12f(x_2) + 32f(x_3) + 7f(x_4)] - \left(\frac{8h^7}{945}\right) f^{(6)}(\beta), \quad (7.8)$$

para $x_0 \leq \beta \leq x_4$.

Exemplo 7.2.1. Sabemos que

$$\ln 2 = \int_1^2 \frac{1}{x} dx \cong 0.69314718,$$

a última, uma aproximação com oito dígitos corretos.

Usando as fórmulas de Newton-Cotes apresentadas, obtemos as seguintes aproximações para $\ln 2$: à direita de cada aproximação calculada, apresentamos seu erro relativo à aproximação apresentada acima.

- Para $n = 1$, com $h = 1$:

$$\int_1^2 \frac{1}{x} dx \cong \frac{1}{2} \left[1 + \frac{1}{2} \right] = 0.75, \quad \text{com erro} = 8.2\%.$$

- Para $n = 2$, com $h = 1/2$:

$$\int_1^2 \frac{1}{x} dx \cong \frac{1}{6} \left[1 + \frac{8}{3} + \frac{1}{2} \right] = 0.69444444, \quad \text{com erro} = 0.19\%.$$

- Para $n = 3$, com $h = 1/3$:

$$\int_1^2 \frac{1}{x} dx \cong \frac{3}{24} \left[1 + \frac{9}{4} + \frac{9}{5} + \frac{1}{2} \right] = 0.6937500, \quad \text{com erro} = 0.09\%.$$

- Para $n = 4$, com $h = 1/4$:

$$\int_1^2 \frac{1}{x} dx \cong \frac{2}{45} \frac{1}{4} \left[7 + 32\frac{4}{5} + 12\frac{2}{3} + 32\frac{4}{7} + 7\frac{1}{2} \right] = 0.69317460,$$

com erro = 0.004%.

□

7.2.4 Fórmulas Repetidas

Quando o intervalo de integração é grande, pode não ser conveniente aumentar o grau do polinômio interpolador para estabelecer fórmulas de integração mais precisas, uma vez que estas fórmulas tornam-se cada vez mais complicadas, como foi visto no exemplo 7.2.1, ao passar de $n = 1$ para $n = 4$. Pode-se imaginar o que seria uma fórmula quando $n = 20$ (sem falar nas complicações para estabelecer fórmulas de integração com n grande).

A alternativa mais usada neste caso é subdividir o intervalo de integração e aplicar fórmulas mais simples repetidas vezes. Assim, são obtidas as *fórmulas compostas* ou *fórmulas repetidas*.

Uma vez que já estabelecemos as fórmulas simples, não teremos dificuldades em compô-las. Vejamos como proceder na composição das fórmulas dos trapézios e Simpson, as mais populares nas aplicações.

Vamos dividir o intervalo de integração $[a, b]$ em n subintervalos de igual comprimento $h = (b - a)/n$. Chamando $x_0 = a$, $x_i = x_{i-1} + h$ e $x_n = b$, podemos aplicar a regra dos trapézios para cada um dos subintervalos. Assim, lembrando que $x_i - x_{i-1} = h$, e as propriedades de integrais, temos

$$\begin{aligned} \int_a^b f(x) dx &= \int_{x_0}^{x_1} f(x) dx + \int_{x_1}^{x_2} f(x) dx + \int_{x_2}^{x_3} f(x) dx + \cdots + \int_{x_{n-1}}^{x_n} f(x) dx \cong \\ &\cong \left(\frac{h}{2}\right) [f(x_0) + 2f(x_1) + 2f(x_2) + \cdots + 2f(x_{n-1}) + f(x_n)]. \end{aligned} \quad (7.9)$$

A figura 7.2 nos dá uma idéia gráfica da aproximação de áreas correspondentes à regra dos trapézios repetida. Os cálculos podem ser executados seguindo o Algoritmo 7.1.

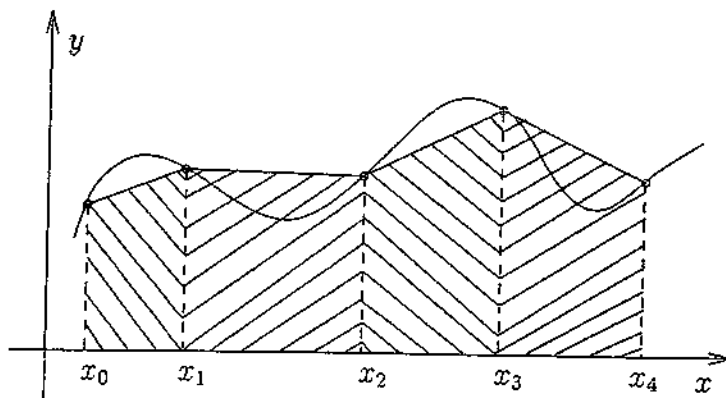


Figura 7.2: Aproximação de integral por trapézio repetido.

Se optarmos por aplicar a fórmula de Simpson repetida, devemos repartir o intervalo num número par de subintervalos, uma vez que cada parábola requer três pontos de

Algoritmo 7.1 Integração por trapézios repetidosDados $h, f(x_i)$ para $i = 0 : n$

- 1: soma = 0
- 2: Para $i = 1 : n - 1$, faça
- 3: soma = soma + $f(x_i)$
- 4: $I_{TR} = \frac{h}{2}[2\text{soma} + f(x_0) + f(x_n)]$

interpolação. Assim, se n é um número par,

$$\begin{aligned}
 \int_a^b f(x) dx &= \int_{x_0}^{x_2} f(x) dx + \int_{x_2}^{x_4} f(x) dx + \cdots + \int_{x_{n-2}}^{x_n} f(x) dx \\
 &= \left(\frac{h}{3}\right) [f(x_0) + 4f(x_1) + 2f(x_2) + 4f(x_3) + \cdots + 2f(x_{n-2}) + 4f(x_{n-1}) + f(x_n)] \\
 &= \frac{h}{3} \{f(x_0) + 4[f(x_1) + f(x_3) + \cdots + f(x_{n-1})] + \\
 &\quad + 2[f(x_2) + f(x_4) + \cdots + f(x_{n-2})] + f(x_n)\}.
 \end{aligned}$$

Para os cálculos, podemos seguir o Algoritmo 7.2.

Algoritmo 7.2 Integração por Simpson repetidaDados $h, f(x_i)$ para $i = 0 : n$, com n par

- 1: soma = 0
- 2: Para $i = 1, 3, 5, \dots, n - 1$, faça
- 3: soma = soma + $2f(x_i) + f(x_{i+1})$
- 4: $I_{SIM} = \frac{h}{3}[2\text{soma} + f(x_0) + f(x_n)]$

Exemplo 7.2.2. Ainda calculando aproximações para o $\ln 2$, podemos aplicar fórmulas repetidas. Se tomarmos quatro trapézios no intervalo $[1, 2]$, portanto $h = 0.25$, teremos a aproximação

$$\int_1^2 \frac{1}{x} dx \cong \frac{0.25}{2} \left[1 + \frac{8}{5} + \frac{4}{3} + \frac{8}{7} + \frac{1}{2} \right] = 0.69702381,$$

cujo erro relativo à aproximação usada no exemplo 7.2.1 é 0.56%.

Se, por outro lado, tomarmos duas parábolas na fórmula de Simpson repetida, obtaremos a aproximação

$$\int_1^2 \frac{1}{x} dx \cong \frac{0.25}{3} \left[1 + \frac{16}{5} + \frac{4}{3} + \frac{16}{7} + \frac{1}{2} \right] = 0.69325397,$$

cujo erro relativo é 0.01%.

□

Comparando este exemplo com o anterior, podemos verificar que a aproximação da fórmula de Simpson repetida é inferior à obtida pelo polinômio de grau 4 ($n = 4$ no exemplo 7.2.1). Como o número de cálculos de valores da função é o mesmo nos dois casos, neste exemplo a fórmula de ordem superior é mais eficiente que a repetição de fórmulas de ordem inferior. Esta observação pode ser comprovada pela análise do erro na fórmula repetida, que faremos a seguir.

O erro final de uma fórmula repetida pode ser obtido pela soma dos erros parciais. Por exemplo, se estamos usando n trapézios, cada um deles vai contribuir com o erro $\frac{h^3}{12} f''(\beta_i)$. Somando as parcelas, estimamos o erro final, que será

$$E_{TR} = - \left(\frac{h^3}{12} \right) [f''(\beta_1) + f''(\beta_2) + \dots + f''(\beta_n)], \quad x_{i-1} \leq \beta_i \leq x_i.$$

Admitindo que $f''(x)$ é contínua no intervalo $[a, b]$, chamando $m = \min_{[a,b]} f''(x)$ e $M = \max_{[a,b]} f''(x)$, teremos:

$$n m \leq \sum_{i=1}^n f''(\beta_i) \leq n M, \quad \text{onde } n \text{ é o número de trapézios.}$$

Essa desigualdade e o teorema do valor intermediário do Cálculo Integral nos dizem que existe um valor γ tal que:

$$\sum_{i=1}^n f''(\beta_i) = n f''(\gamma).$$

Portanto, o erro na regra dos trapézios repetida, com n trapézios, pode ser estimado usando uma das expressões:

$$E_{TR} = -n \frac{h^3}{12} f''(\gamma) = -(b-a) \frac{h^2}{12} f''(\gamma), \quad (7.10)$$

onde, na segunda igualdade fizemos a substituição $nh = b - a$.

As mesmas idéias nos darão um resultado análogo para o caso da fórmula de Simpson repetida aplicada nos $(n+1)$ pontos igualmente espaçados $x_0, x_1, \dots, x_n$:

$$E_{SR} = -n \frac{h^5}{180} f^{(4)}(\gamma) = -(b-a) \frac{h^4}{180} f^{(4)}(\gamma). \quad (7.11)$$

Observe que, neste caso, foram usadas $n/2$ parábolas, e portanto $n/2$ parcelas de erro.

Comparando (7.11) com (7.8) verificamos o porquê do comentário anterior, sobre erros associados às fórmulas com o mesmo número de cálculo de funções. Estas expressões nos

dizem que o erro da fórmula correspondente a $n = 4$ é de ordem de h^7 , ao passo que a fórmula de Simpson repetida introduz erro da ordem de h^4 .

Embora as expressões apresentadas neste capítulo nos forneçam uma relação entre erro e espaçamento entre pontos de integração, h , o valor exato deste erro não pode ser calculado, uma vez que não sabemos o valor de γ . Entretanto, as expressões podem ser utilizadas na delimitação do erro da fórmula de integração usada. De fato, lembrando a definição de máximo de uma função (cuja existência é garantida, desde que o intervalo considerado seja fechado e as derivadas contínuas) temos, para trapézios e Simpson, respectivamente,

$$|E_{TR}| = \frac{(b-a)}{12} h^2 |f''(\gamma)| \leq \frac{(b-a)}{12} h^2 \max_{a \leq x \leq b} |f''(x)|,$$

e

$$|E_{SR}| = \frac{(b-a)}{180} h^4 |f^{(4)}(\gamma)| \leq \frac{(b-a)}{180} h^4 \max_{a \leq x \leq b} |f^{(4)}(x)|.$$

As estimativas acima podem ser usadas para calcular o número de subintervalos das fórmulas repetidas com precisão pré-fixada. O exemplo abaixo ilustra esta aplicação das estimativas de erro das fórmulas repetidas.

Exemplo 7.2.3. Quantos intervalos devem ser usados no cálculo de $\int_0^1 e^{-x^2} dx$ para que a aproximação tenha erro menor que 10^{-4} nos casos (a) fórmula dos trapézios e (b) fórmula de Simpson?

As derivadas de $f(x) = e^{-x^2}$ podem ser calculadas pela regra da cadeia:

$$\begin{aligned} f'(x) &= -2xe^{-x^2} & f''(x) &= 2e^{-x^2}(2x^2 - 1) \\ f'''(x) &= 4e^{-x^2}(3x - 2x^3) & f^{(4)}(x) &= 4e^{-x^2}(3 - 12x^2 + 4x^4). \end{aligned}$$

Encontrar máximos de funções pode ser difícil. Em alguns casos, podemos usar resultados do Cálculo para resolver a questão. Na expressão do erro na fórmula dos trapézios repetida precisamos estimar $\max_{[0,1]} |f''(x)|$.

Podemos verificar graficamente, ou pelo sinal de $f'''(x)$, que

$$\max_{[0,1]} |f''(x)| = 2.$$

Para estimar o erro da fórmula de Simpson repetida, a situação muda, porque a expressão da derivada de quarta ordem é mais complicada. Podemos usar o seguinte recurso:

$$\max_{[0,1]} |f^{(4)}(x)| = \max_{[0,1]} 4e^{-x^2} |3 - 12x^2 + 4x^4| \leq \max_{[0,1]} 4e^{-x^2} \max_{[0,1]} |3 - 12x^2 + 4x^4| \leq 4 \times 3 = 12.$$

O passo final é encontrar h_T , para trapézios, satisfazendo à:

$$\frac{(b-a)}{12} h_T^2 \max_{[0,1]} |f''(x)| = \frac{2h_T^2}{12} \leq 10^{-4}$$

e h_S , para Simpson, satisfazendo à:

$$\frac{(b-a)}{180} h_S^4 \max_{[a,b]} |f^{(4)}(x)| = \frac{12h_S^4}{180} \leq 10^{-4}.$$

Concluimos que $h_T \leq 2.44 \times 10^{-2}$, que corresponde ao número de subintervalos $n_T \geq 1/h_T = 40.8$, isto é, $n_T = 41$. No caso da fórmula de Simpson repetida, temos $h_S \leq 1.96 \times 10^{-1}$, que corresponderia ao número de subintervalos $n_S > 1/h_S = 5.08$. Como n_S tem de ser um número inteiro e par, podemos tomar $n_S = 6$. $\square$

7.3 Integração de Romberg

As fórmulas de Newton-Cotes podem ser combinadas de modo a melhorar a aproximação da integração numérica. Uma técnica eficiente para estabelecer esta combinação é através da *integração de Romberg*, que é baseada numa ideia útil também em outros ramos da análise numérica: a *extrapolação de Richardson*. Aproveitamos a oportunidade para descrevê-la.

Seja $g(0)$ um valor a ser calculado, e $g(h)$ sua aproximação, calculada em função do parâmetro h . Se fizermos a expansão em série de Taylor, em torno de zero, de $g(h)$ e $g(h/2)$ teremos, respectivamente:

$$g(h) = g(0) + g'(0)h + g''(0)\frac{h^2}{2} + g'''(0)\frac{h^3}{3!} + \dots$$

$$g\left(\frac{h}{2}\right) = g(0) + g'(0)\frac{h}{2} + g''(0)\frac{h^2}{8} + g'''(0)\frac{h^3}{48} + \dots$$

Tomando os dois primeiros termos do lado direito destas duas expressões, vemos que $g(h)$ e $g(h/2)$ são aproximações para $g(0)$, ambas com ordem de aproximação $O(h)$. Vamos combinar convenientemente estes dois somatórios para definir uma aproximação mais precisa. Seja

$$g_1(h) = 2g\left(\frac{h}{2}\right) - g(h) = g(0) - g''(0)\frac{h^2}{4} - 3g'''(0)\frac{h^3}{24} + \dots$$

Concluimos que, se $g_1(h)$ for usada para calcular $g(0)$, então a aproximação é $O(h^2)$, isto é, ganhamos na ordem de aproximação.

Repetindo a ideia, tomamos agora $g_1(h)$ e $g_1(h/2)$ para definir outra aproximação $g_2(h)$. Sejam

$$g_1(h) = g(0) - g''(0)\frac{h^2}{4} - 3g'''(0)\frac{h^3}{24} + \dots$$

e

$$g_1\left(\frac{h}{2}\right) = g(0) - g''(0)\frac{h^2}{16} - 3g'''(0)\frac{h^3}{192} + \dots$$

Subtraindo a primeira equação da segunda, multiplicada por 4, teremos:

$$4g_1\left(\frac{h}{2}\right) - g_1(h) = 3g(0) + 3g'''(0)\frac{h^3}{48} + \dots$$

Dessa maneira,

$$g_2(h) = \frac{1}{3} \left\{ 4g_1\left(\frac{h}{2}\right) - g_1(h) \right\} = g(0) + g'''(0)\frac{h^3}{48} + \dots \quad (7.12)$$

Desta maneira, usamos uma combinação conveniente de $g_1(h)$ e $g_1(h/2)$, que são aproximações de $g(0)$ da ordem de $O(h^2)$, e estabelecemos uma aproximação $g_2(h)$ que é de ordem de $O(h^3)$.

Em resumo, a idéia é aplicar sucessivamente combinações de aproximações para melhorar a aproximação final. Pode-se verificar que a expressão genérica do n -ésimo estágio deste procedimento é

$$g_n(h) = \frac{(2^n g_{n-1}(h/2) - g_{n-1}(h))}{2^n - 1} = g(0) + O(h^{n+1}). \quad (7.13)$$

Como a aplicação de nosso interesse no momento é a integração numérica, vejamos como o processo de extrapolação de Richardson pode ser aplicado em combinações de fórmulas de integração de trapézios.

Chamemos $I_1(h)$ e $I_1(h/2)$ os resultados obtidos pela regra dos trapézios tomando $2h$ e h , respectivamente, em (x_0, x_2) , como ilustra a figura 7.3.

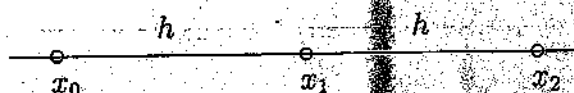


Figura 7.3: Intervalo (x_0, x_2)

Aplicando a fórmula dos trapézios simples e repetida temos:

$$I_1(h) = \frac{2h}{2} [f(x_0) + f(x_2)] \quad \text{e} \quad I_1\left(\frac{h}{2}\right) = \frac{h}{2} [f(x_0) + 2f(x_1) + f(x_2)].$$

Usando a fórmula de extrapolação (7.12) temos:

$$\begin{aligned} I_2(h) &= \frac{1}{3} \left[4I_1\left(\frac{h}{2}\right) - I_1(h) \right] \\ &= \frac{1}{3} \left[4 \cdot \frac{h}{2} [f(x_0) + 2f(x_1) + f(x_2)] - h[f(x_0) + f(x_2)] \right] \\ &= \frac{h}{3} [f(x_0) + 4f(x_1) + f(x_2)]. \end{aligned}$$

Desta expressão, observamos que $I_2(h)$ corresponde à fórmula de Simpson aplicada no intervalo (x_0, x_2) . Como vimos anteriormente, a fórmula de Simpson fornece uma aproximação da ordem de $O(h^4)$. Se quisermos avançar para obter uma expressão que forneça melhores resultados, podemos usar a expressão genérica (7.13) com $n = 4$, como ilustrado no exemplo a seguir.

Exemplo 7.3.1. Queremos calcular $\int_0^1 e^{-x^2} dx$ usando extrapolação de Richardson com $n = 4$.

Usando $n = 4$ na expressão genérica da extrapolação, (7.13), temos

$$I_4(h) = \frac{1}{2^4 - 1} \left\{ 2^4 I_3\left(\frac{h}{2}\right) - I_3(h) \right\}. \quad (7.14)$$

Por outro lado,

$$I_3(h) = \frac{1}{2^3 - 1} \left\{ 2^3 I_2\left(\frac{h}{2}\right) - I_2(h) \right\}.$$

e

$$I_2(h) = \frac{1}{2^2 - 1} \left\{ 2^2 I_1\left(\frac{h}{2}\right) - I_1(h) \right\}.$$

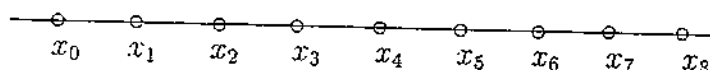


Figura 7.4: Intervalo $[0, 1]$ particionado em x_i , $i = 0 : 8$.

Para entender melhor os cálculos, tomemos $h = x_8 - x_0 = 1$, como ilustra a figura 7.4. Substituindo h por $h/2$ na expressão de $I_2(h)$ temos

$$I_2\left(\frac{h}{2}\right) = \frac{1}{2^2 - 1} \left\{ 2^2 I_1\left(\frac{h}{4}\right) - I_1\left(\frac{h}{2}\right) \right\}.$$

Assim,

$$I_3(h) = \frac{1}{2^3 - 1} \left\{ 2^3 \left[\frac{1}{2^2 - 1} \left(4I_1\left(\frac{h}{4}\right) - I_1\left(\frac{h}{2}\right) \right) \right] - \frac{1}{2^2 - 1} \left(4I_1\left(\frac{h}{2}\right) - I_1(h) \right) \right\}.$$

Calculando $I_3(h/2)$ e procedendo à extrapolação (7.14), obtemos $I_4(h)$, que será uma aproximação da ordem de h^5 para a integral. Podemos ver que o cálculo de $I_3(h/2)$ envolve os valores de $I_1(h/2)$, $I_1(h/4)$ e $I_1(h/8)$. Dessa maneira, inicialmente aplicamos a regra dos trapézios com h , $h/2 = x_4 - x_0$, $h/4 = x_2 - x_0$ e $h/8 = x_1 - x_0$, construindo a primeira coluna da tabela 7.1.

Os valores de I_2 são calculados fazendo

$$I_2\left(\frac{h}{2^i}\right) = \frac{1}{3} \left\{ 4I_1\left(\frac{h}{2^{i+1}}\right) - I_1\left(\frac{h}{2^i}\right) \right\}, \quad i = 0, 1, 2.$$

Para o cálculo de I_3 , fazemos

$$I_3\left(\frac{h}{2^i}\right) = \frac{1}{7} \left\{ 8I_2\left(\frac{h}{2^{i+1}}\right) - I_2\left(\frac{h}{2^i}\right) \right\}, \quad i = 0, 1.$$

Finalmente, obtemos a aproximação desejada:

$$\int_0^1 e^{-x^2} dx \cong I_4(h) = \frac{1}{15} \left\{ 16I_3\left(\frac{h}{2}\right) - I_3(h) \right\} = 0.746824.$$

Os resultados destes cálculos são apresentados na tabela 7.1.

i	$I_1\left(\frac{h}{2^i}\right)$	$I_2\left(\frac{h}{2^i}\right)$	$I_3\left(\frac{h}{2^i}\right)$	$I_4\left(\frac{h}{2^i}\right)$
0	0.683940	0.747180	0.746809	0.746824
1	0.731370	0.746855	0.746823	
2	0.742984	0.746826		
3	0.745866			

Tabela 7.1: Resultados do exemplo 7.3.1.

□

7.3.1 Fórmulas Gaussianas

Nesta seção, estabeleceremos fórmulas de integração, ainda do tipo (7.1), nas quais os pontos de integração $x_0, x_1, \dots, x_n$ não são igualmente espaçados. A idéia agora é escolher os pontos de integração de modo que, para um mesmo número de cálculos de valores da função a ser integrada, consigam-se melhores resultados que os obtidos com as fórmulas de Newton-Cotes.

Esta idéia foi usada inicialmente por Gauss, em 1844, portanto um século depois das fórmulas estabelecidas por Newton. Por isto, elas são conhecidas como *fórmulas gaussianas*, ou *quadraturas gaussianas*, para integração numérica.

Estaremos medindo a eficiência de uma fórmula de integração através do maior grau do polinômio para o qual a fórmula é exata. De acordo com este critério, se uma fórmula é exata para polinômios de grau $\leq k$, $p_k(x)$, isto é, se vale a igualdade de (7.1) para $f(x) = p_k(x)$:

$$\int_a^b p_k(x) dx = \sum_{i=1}^n w_i p_k(x_i), \quad (7.15)$$

então, ela será considerada melhor que aquelas que são exatas para polinômios de grau $\leq k-1$.

Como uma das aplicações dos polinômios ortogonais que estudamos na seção 5.4 vamos justificar teoricamente a eficiência das fórmulas gaussianas através do resultado que apresentamos a seguir. Veremos que há vantagem quando usamos raízes de polinômios como pontos de integração.

Teorema 7.1. *Sejam*

- i. $\psi_k(x)$, $k = 0 : n$, polinômios de grau k , ortogonais relativamente ao produto escalar $\langle f, g \rangle = \int_a^b f(x)g(x) dx$;
- ii. $x_0, x_1, \dots, x_n$ as raízes de $\psi_{n+1}(x)$.

Se a fórmula de integração

$$\int_a^b f(x) dx \cong \sum_{i=1}^n w_i p_k(x_i),$$

é exata para polinômios de grau $\leq n$, então ela também será exata para polinômios de grau $\leq 2n+1$.

Demonstração: Se $p(x)$ é um polinômio qualquer de grau $\leq 2n+1$, ele pode ser escrito na forma:

$$p(x) = \psi_{n+1}(x) q(x) + r(x), \quad (7.16)$$

onde $q(x)$ e $r(x)$ são polinômios de grau $\leq n$, e $\psi_{n+1}(x)$ é o polinômio da família ortogonal, de grau $n+1$. Integramos esta expressão no intervalo usado na definição do produto escalar das funções ortogonais, aqui designado (a, b) :

$$\int_a^b p(x) dx = \int_a^b \psi_{n+1}(x) q(x) dx + \int_a^b r(x) dx. \quad (7.17)$$

Como $q(x)$ pode ser escrito em termos dos $\psi_i(x)$, $i \leq n$ (propriedade dos polinômios ortogonais) e sendo a família $\psi_k(x)$ ortogonal, concluímos que:

$$\int_a^b \psi_{n+1}(x) q(x) dx = 0. \quad (7.18)$$

Por hipótese, a fórmula de integração é exata para polinômios de grau $\leq n$, que é o caso de $r(x)$, isto é,

$$\int_a^b r(x) dx = \sum_{i=0}^n w_i r(x_i). \quad (7.19)$$

Usando (7.18) e (7.19) em (7.17) e lembrando que $r(x) = p(x) - \psi_{n+1}(x)q(x)$, temos

$$\int_a^b p(x) dx = \int_a^b r(x) dx = \sum_{i=0}^n w_i r(x_i) = \sum_{i=0}^n w_i [p(x_i) - \psi_{n+1}(x_i) q(x_i)].$$

Finalmente, observamos que os pontos de integração foram escolhidos como as raízes de ψ_{n+1} e, portanto, $\psi_{n+1}(x_i) = 0$, anulando o termo final da última expressão. Obtemos desta maneira a expressão desejada,

$$\int_a^b p(x) dx = \sum_{i=0}^n w_i p(x_i),$$

o que demonstra ser a fórmula de integração exata para $p(x)$, que é um polinômio de grau menor ou igual a $2n + 1$.

O teorema anterior nos diz que, para cada produto escalar, existe um conjunto de fórmulas de integração satisfazendo ao critério de eficiência citado no início desta seção.

As fórmulas gaussianas mais populares são chamadas Gauss-Legendre por estarem associadas aos polinômios de Legendre. Como vimos anteriormente, seção 5.4, os polinômios de Legendre são ortogonais com relação ao produto escalar definido por

$$\langle f, g \rangle = \int_{-1}^1 f(x) g(x) dx.$$

Os quatro primeiros polinômios de Legendre são:

$$p_0(x) = 1, \quad p_1(x) = x, \quad p_2(x) = x^2 - \frac{1}{3}, \quad p_3(x) = x^3 - \frac{3}{5}x.$$

Para a fórmula gaussiana de dois pontos, tomamos as raízes de $p_2(x)$, que são $x_1 = -\sqrt{1/3}$ e $x_2 = \sqrt{1/3}$. Os pesos desta fórmula são w_0 e w_1 , que fazem de

$$\int_{-1}^1 f(x) dx \cong w_0 f\left(-\sqrt{\frac{1}{3}}\right) + w_1 f\left(\sqrt{\frac{1}{3}}\right)$$

uma fórmula exata para polinômios de grau ≤ 1 . A igualdade das fórmulas nos casos $f(x) = 1$ e $f(x) = x$, polinômios de grau 0 e 1, nos leva a um sistema com duas equações nas incógnitas w_0 e w_1 ,

$$\int_{-1}^1 1 dx = w_0 + w_1 \implies w_0 + w_1 = 2$$

e

$$\int_{-1}^1 x dx = -w_0 \frac{1}{\sqrt{3}} + w_1 \frac{1}{\sqrt{3}} \implies w_0 - w_1 = 0.$$

A solução deste sistema é $w_0 = w_1 = 1$. Concluindo, a fórmula Gauss-Legendre de dois pontos é

$$\int_{-1}^1 f(x) dx = f\left(-\frac{1}{\sqrt{3}}\right) + f\left(\frac{1}{\sqrt{3}}\right), \quad (7.20)$$

e é exata para polinômios de grau ≤ 3 ($n = 1$ no teorema 7.1).

Observe que, por ser exata para polinômios de grau ≤ 3 , a precisão desta fórmula é equivalente à da fórmula de Simpson. Entretanto, ela requer o cálculo de dois valores da função, ao passo que na fórmula de Simpson é necessário calcular os valores da função em três pontos. Assim, no sentido de eficiência definido no início desta seção, a fórmula gaussiana de dois pontos seria melhor. Entretanto, para fazer justiça à fórmula de Simpson, no uso das fórmulas repetidas o terceiro ponto não precisa ser recalculado em subintervalos vizinhos. Neste caso, fica cancelada a eficiência que as fórmulas gaussianas de dois pontos poderiam ter sobre Simpson.

Podemos também estabelecer a fórmula (7.20) sem recorrer aos polinômios ortogonais usando o procedimento que descreveremos a seguir.

Podemos encontrar x_0, x_1, w_0, w_1 para a expressão

$$\int_{-1}^1 f(x) dx \cong w_0 f(x_0) + w_1 f(x_1)$$

fazendo com que ela seja exata para $f(x) = 1$, $f(x) = x$, $f(x) = x^2$ e $f(x) = x^3$, polinômios de graus ≤ 3 . Os valores exatos da integral, para estas quatro funções, são, respectivamente, 2, 0, $2/3$ e 0. Assim, estabelecemos um sistema de quatro equações, não-lineares, nas incógnitas x_0, x_1, w_0, w_1 :

$$\begin{aligned} w_0 + w_1 &= 2 \\ w_0 x_0 + w_1 x_1 &= 0 \\ w_0 x_0^2 + w_1 x_1^2 &= 2/3 \\ w_0 x_0^3 + w_1 x_1^3 &= 0. \end{aligned}$$

Da segunda equação verificamos que $w_0 x_0 = -w_1 x_1$. Substituindo $w_0 x_0$ na quarta equação, por $-w_1 x_1$, temos

$$-w_1 x_1 x_0^2 + w_1 x_1^3 = w_1 x_1 (-x_0^2 + x_1^2) = 0$$

e, portanto, $x_0^2 = x_1^2$. Como os pontos de integração são distintos, a única solução é $x_0 = -x_1$.

Usando este resultado na segunda equação, obtemos $w_0 = w_1$ e, portanto, pela primeira equação, $w_0 = w_1 = 1$. Usando agora a terceira equação do sistema, temos $2x_0^2 = 2/3$. Assim obtemos os pontos de integração $x_0 = -1/\sqrt{3}$ e $x_1 = 1/\sqrt{3}$ e os pesos $w_0 = w_1 = 1$, como em (7.20). A desvantagem desta segunda abordagem é que temos de resolver um sistema não-linear.

Para estabelecer a fórmula de Gauss-Legendre de três pontos, usamos as raízes do polinômio de grau 3 que são $x_0 = -\sqrt{0.6}$, $x_1 = 0$ e $x_2 = \sqrt{0.6}$. Os pesos satisfazem ao sistema de três equações lineares:

$$\begin{aligned} \text{Fórmula exata para } p(x) = 1 & \Rightarrow w_0 + w_1 + w_2 = 2 \\ \text{Fórmula exata para } p(x) = x & \Rightarrow -w_0 + w_2 = 0 \\ \text{Fórmula exata para } p(x) = x^2 & \Rightarrow 0.6 w_0 + 0.6 w_2 = 2/3. \end{aligned}$$

Resolvido o sistema linear, teremos $w_0 = 5/9$, $w_1 = 8/9$ e $w_2 = 5/9$. Desta maneira, a Fórmula Gauss-Legendre de três pontos é:

$$\int_{-1}^1 f(x) dx = \frac{1}{9} (5f(-\sqrt{0.6}) + 8f(0) + 5f(\sqrt{0.6})). \quad (7.21)$$

As abscissas e os pesos de fórmulas gaussianas com mais de três pontos podem ser encontrados em vários livros de métodos numéricos e tabelas matemáticas como, por exemplo, em [Abramowitz-Stegun, 1965].

Embora as fórmulas Gauss-Legendre tenham sido deduzidas para a integração no intervalo $[-1, 1]$, elas podem ser utilizadas no intervalo genérico $[a, b]$, desde que façamos uma mudança de variáveis. Seja

$$x = \frac{2t - (a + b)}{b - a}, \quad \text{ou seja, } t = \frac{1}{2}[(b - a)x + (b + a)]. \quad (7.22)$$

A integral de $f(t)$ no intervalo $[a, b]$, pode ser transformada em uma integral em $[-1, 1]$, usando esta mudança de variáveis:

$$\int_a^b f(t) dt = \frac{b - a}{2} \int_{-1}^1 f\left(\frac{b - a}{2}x + \frac{b + a}{2}\right) dx \cong \frac{b - a}{2} \sum_{i=0}^n w_i f\left(\frac{b - a}{2}x_i + \frac{b + a}{2}\right). \quad (7.23)$$

Na última passagem, usamos pesos, w_i , e pontos de integração, x_i , correspondentes à fórmula gaussiana que será usada na aplicação.

Exemplo 7.3.2. Vamos aproximar $\int_1^2 \frac{1}{t} dt$ usando as fórmulas gaussianas de dois e três pontos.

Tomando $a = 1$ e $b = 2$ na transformação de variáveis (7.22) teremos:

$$t = \frac{b - a}{2}x + \frac{b + a}{2} = 0.5x + 1.5.$$

Usando a mudança de variáveis na integral temos

$$\int_1^2 \frac{1}{t} dt = 0.5 \int_{-1}^1 \frac{1}{0.5x + 1.5} dx \cong 0.5 \sum_{i=0}^n w_i \frac{1}{0.5x_i + 1.5}.$$

Na fórmula gaussiana com dois pontos, $w_0 = w_1 = 1$, $x_0 = -1/\sqrt{3}$ e $x_1 = 1/\sqrt{3}$, teremos

$$\int_1^2 \frac{1}{t} dt \cong 0.69230769,$$

cujo erro relativo é 0.12%.

Por outro lado, se aproximarmos a integral usando a fórmula de três pontos, com os pesos e pontos de integração definidos em (7.21), teremos

$$\int_1^2 \frac{1}{t} dt \cong 0.69312169,$$

cujo erro relativo é 0.0037%. Os erros destas aproximações podem ser comparados com os erros obtidos no exemplo 7.2.1. $\square$

7.3.2 Integrais Múltiplas

As fórmulas de integração apresentadas nas seções anteriores podem ser usadas para estabelecer fórmulas de integração de funções com duas ou mais variáveis. Para fazer a devida adaptação, recorreremos à definição de integrais múltiplas.

Para simplificar, detalharemos aqui apenas o caso da integral dupla

$$\iint_R f(x, y) dx dy,$$

onde R é uma região do plano, como ilustrado na figura 7.5.

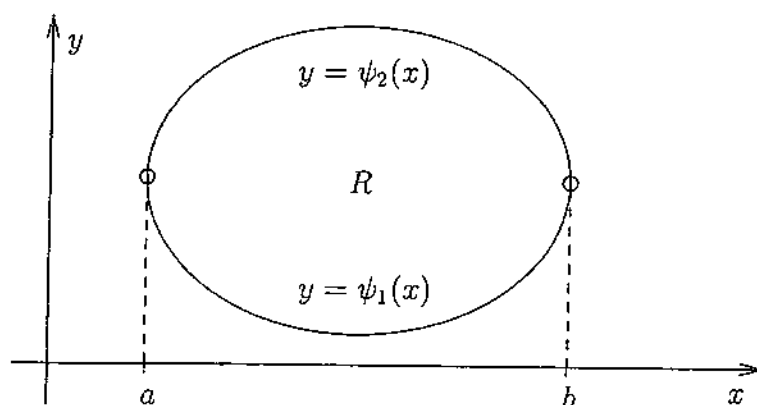


Figura 7.5: Região de integração R .

Do Cálculo Integral para funções de duas variáveis, usamos as funções $\psi_1(x)$ e $\psi_2(x)$ desenhadas na figura 7.5. Chamando

$$F(x) = \int_{\psi_1(x)}^{\psi_2(x)} f(x, y) dy, \quad (7.24)$$

temos

$$\int_R \left(\int f(x, y) dy \right) dx = \int_a^b F(x) dx.$$

Dessa maneira, a integral dupla pode ser calculada pela integração de funções de uma variável, das quais (7.24) tem extremos de integração também dependendo de x .

Para exemplificar, tomaremos o caso mais simples, onde R é um retângulo, e usaremos a fórmula de Simpson na integração, como ilustra a figura 7.6.

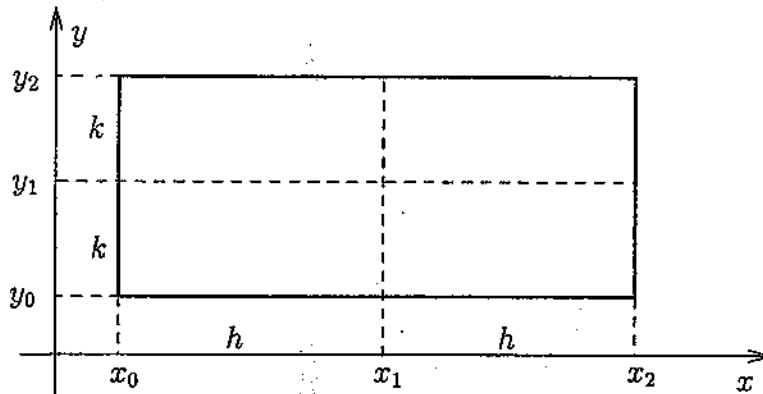


Figura 7.6. Região R retangular.

Para calcular a integral dupla, vamos integrar primeiro na variável y , obtendo uma função de x , neste caso $F(x) = f(x, y_0) + 4f(x, y_1) + f(x, y_2)$. Em seguida, podemos aplicar novamente a fórmula de Simpson, agora na variável x . Assim, chamando $a = x_0$, $b = x_2$, $c = y_0$ e $d = y_2$,

$$\begin{aligned} \iint_R f(x, y) dx dy &= \int_a^b \left\{ \int_c^d f(x, y) dy \right\} dx \cong \frac{k}{3} \int_a^b (f(x, y_0) + 4f(x, y_1) + f(x, y_2)) dx \cong \\ &\cong \frac{hk}{3} \{ f(x_0, y_0) + 4f(x_1, y_0) + f(x_2, y_0) + 4[f(x_0, y_1) + 4f(x_1, y_1) + f(x_2, y_1)] + \\ &\quad + f(x_0, y_2) + 4f(x_1, y_2) + f(x_2, y_2) \}. \end{aligned}$$

As fórmulas repetidas de integração de funções de duas variáveis seguem a idéia de justaposição usada nas fórmulas repetidas de funções de uma variável. Na figura 7.7, apresentamos um exemplo com os coeficientes nos respectivos pontos de integração. Quando usamos a fórmula de Simpson na região retangular discretizada, os pesos são obtidos multiplicando estes coeficientes por $\frac{kh}{9}$. Neste exemplo, o intervalo (a, b) foi dividido em $m = 6$ subintervalos e (c, d) em $n = 4$ subintervalos.

O procedimento ilustrado acima pode ser adotado qualquer que seja a fórmula de integração de uma variável. Entretanto, no caso da integral dupla, as regiões de integração nem sempre são retângulos ou podem ser aproximadas por retângulos.

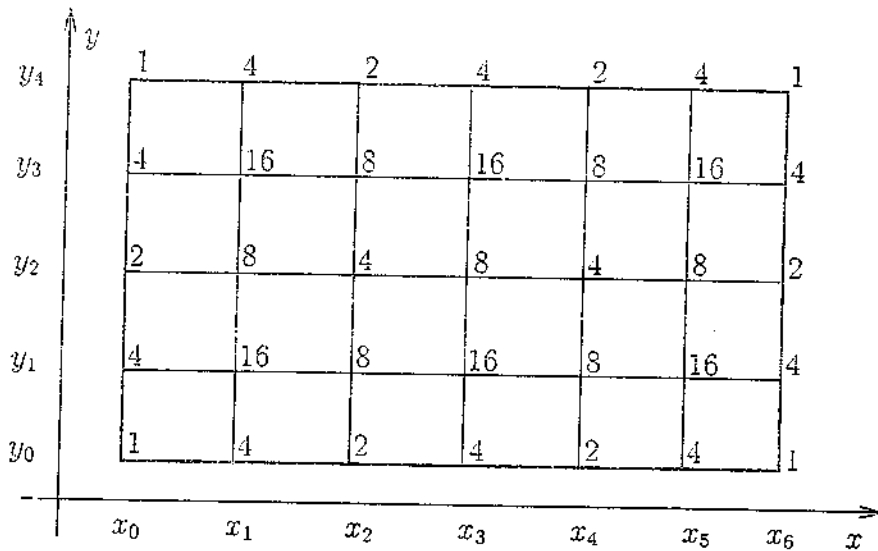


Figura 7.7: Pesos para Simpson repetido em região R retangular.

Uma alternativa bastante usada é subdividir a região de integração e transportar cada uma destas sub-regiões para um elemento-padrão, no qual uma fórmula de integração é aplicada. O transporte para o elemento-padrão é de fato uma mudança de variáveis na integração.

Este procedimento é atrativo do ponto de vista computacional e tem sido amplamente usado no Método de Elementos Finitos, por exemplo. A possibilidade de ser usado em regiões mais gerais deu ao Método de Elementos Finitos versatilidade e popularidade. Livros especializados em Elementos Finitos descrevem detalhadamente esta técnica de integração de funções de duas ou mais variáveis.

7.4 Uso do Matlab na Integração Numérica

Nesta seção, apresentamos algumas funções do Matlab que podem ser úteis na integração numérica de funções. No caso da função ser fornecida através de uma tabela, a integração pela regra dos trapézios repetida pode ser realizada diretamente, como exemplificamos a seguir.

```
>> x=0:.01:1; y=x.*exp(-x.^2); % Tabela da função f(x)=x exp(-x2)
                                % no intervalo [0 1];
>> ITR = trapz(x,y)           % Integração Numérica
ITR =
    0.3160                    % Valor aproximado usando trapézios com h=0.01
```

Neste exemplo, o valor exato da integral pode ser calculado por substituição de variáveis. No intervalo $[0, 1]$, a integral da função será:

```
>> IE = .5*(1-exp(-1))
IE =
    0.31606027941428
```

e, portanto, o erro absoluto da aproximação pela regra dos trapézios será:

```
>> abs(IE-ITR)
ans =
    1.1399e-005
```

Calculando a derivada de segunda ordem da função, podemos verificar que ela é, em módulo, menor do que 1.63 e, portanto, o limitante de erro da regra dos trapézios, dado em (7.10), para a aproximação ITR é 1.36×10^{-5} .

A fórmula de Simpson poderá ser aplicada se criarmos uma M-function. Tomemos como exemplo a mesma função usada acima, que agora será definida pela

```
function y = funcao1(x)
y = x.*exp(-x.^2);
```

Para usar a regra de Simpson com escolha adaptativa de subintervalos ([Conte-deBoor, 1981], por exemplo):

```
>> ITS = quad('funcao1',0,1)
ITS =
```

```
    0.31606633042777
```

Podemos também calcular a integral pela fórmula de Newton-Cotes com $n = 8$ adaptativa usando

```
>> ITS8 = quad8('funcao1',0,1)
ITS8 =
```

```
    0.31606027941427
```

Observamos que, nas funções `quad` e `quad8`, não é necessário fornecer o valor de h , pois há escolha automática de subintervalos de tamanhos variados, dependendo da variação de $f(x)$. Na falta da tolerância do erro, como fizemos acima, será usado erro relativo menor do que 10^{-3} . Entretanto podem-se escolher outros valores para o erro relativo, veja o help se for o caso.

A integral de uma função de duas variáveis em retângulos também pode ser realizada no Matlab, definindo-a numa function, integrando no exemplo, e usando o recurso

```
dblquad('integrando',a, b, c, d) % Integral Dupla em [a,b]x[c,d]
```

Exercícios

1. Use as fórmulas repetidas dos trapézios e de Simpson para aproximar as integrais abaixo, tomando como número de pontos de integração os valores indicados ao lado. Compare seus resultados, se possível, com os valores exatos.

a. $\int_0^1 x^2 \sqrt{1+x^2} dx, \quad n = 6$

b. $\int_0^{2\pi} x \sin x dx, \quad n = 8$

c. $\int_0^1 x^2 e^x dx, \quad n = 6$

d. $\int_0^2 x^2 e^{-x^2} dx, \quad n = 8$

2. Encontre um delimitante (o menor de que você for capaz) para o erro das aproximações obtidas no exercício 1.

3. Encontre a área da região limitada pela curva normal

$$y(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{1}{2}\left(\frac{\sigma}{x}\right)^2\right)$$

e pelo eixo dos x no intervalo $[-\sigma, \sigma]$, usando Simpson repetida e $n = 8$. Encontre também um limitante para o erro desta aproximação.

4. Determine n de modo que a aproximação de $\int_0^1 e^{-x^2} dx$ usando Simpson tenha erro menor que 10^{-4} .

5. Aproxime as integrais propostas no exercício 1 usando a integração de Romberg $I^4(h)$. Compare seus resultados.

6. Elabore o algoritmo correspondente à integração de Romberg, $I^n(h)$, de $f(x)$, num intervalo (a, b) . Escreva o programa correspondente. Teste seu programa em

$$I = \int_1^2 x \ln(x) dx.$$

7. Aproxime as integrais do exercício 1 usando fórmulas gaussianas com $n = 2$. Compare com os resultados anteriores.

8. Verifique que (7.23) pode ser usada para estabelecer as fórmulas gaussianas repetidas. Estabeleça as fórmulas repetidas, com m subintervalos e

a. $n = 2$ (fórmulas de dois pontos repetidas),

b. $n = 3$ (fórmulas de três pontos repetidas).

9. Elabore o algoritmo e um programa para a integração numérica usando a fórmula gaussiana de 3 pontos aplicada a m subintervalos do intervalo de integração.

10. Usando as fórmulas repetidas encontradas no exercício 8, aproxime as integrais (a) e (c) do exercício 1, usando $m = 4$ e $n = 2$ pontos de integração por subintervalo. Idem para os exercícios (b) e (d) com $m = 3$ e $n = 3$. Compare com os resultados anteriores.

11. Levando em consideração o número de operações realizadas, podemos concluir que existe alguma vantagem da fórmula gaussiana repetida sobre a fórmula de Simpson repetida? Comente estas duas opções.

12. Os polinômios de Laguerre

$$L_0(x) = 1$$

$$L_1(x) = 1 - x$$

$$L_2(x) = x^2 - 4x + 2$$

$$L_3(x) = -x^3 + 9x^2 - 18x + 6$$

são ortogonais com relação ao produto escalar

$$\langle f, g \rangle = \int_0^{\infty} e^{-x} f(x) g(x) dx.$$

Encontre uma fórmula de integração para $\int_0^{\infty} e^{-x} f(x) dx$ exata para polinômios de grau 3.

13. Aplique a fórmula encontrada no exercício 12 para aproximar

$$\int_0^{\infty} e^{-x} \sin x dx.$$

14. Encontre os pesos correspondentes à aplicação da regra dos trapézios repetida na integração de uma função de duas variáveis, num retângulo $a \leq x \leq b$ e $c \leq y \leq d$. Repita

a figura 7.7 para este caso.

15. Elabore o algoritmo para o esquema de integração estabelecido no exercício anterior.

16. Use as fórmulas repetidas de trapézios e Simpson para aproximar as integrais abaixo. Neste exercício, m é o número de subdivisões no eixo x e n representa o número de subdivisões no eixo y .

a. $\int_2^3 \int_1^2 xy^2 dx dy$, $m = n = 3$;

b. $\int_0^1 \int_1^3 e^{-y-x} dx dy$, $m = 4$ e $n = 2$;

c. $\int_0^1 \int_0^\pi y \sin x dx dy$, $m = 3$ e $n = 2$.

17. Comente como as idéias apresentadas no fim deste capítulo podem ser utilizadas para estabelecer algoritmos de integração de uma função de três variáveis, isto é,

$$\iiint f(x, y, z) dx dy dz.$$

18. Encontre os valores das integrais propostas no exercício 16 usando o Matlab.

Capítulo 8

SOLUÇÕES APROXIMADAS PARA EQUAÇÕES DIFERENCIAIS ORDINÁRIAS

8.1 Introdução

Muitos modelos matemáticos usados em simulações unidimensionais envolvem equações cujas incógnitas são funções de uma variável e suas derivadas. Por exemplo, para encontrar equações matemáticas que descrevam o movimento de um pêndulo circular aplicamos leis da Física — a Lei de Newton com o balanço de forças — e obtemos um modelo matemático representado por uma equação diferencial ordinária do tipo

$$\theta''(t) + b \sin(\theta) = 0,$$

onde $\theta''(t) = d^2\theta/dt^2$ é a derivada de segunda ordem da função $\theta(t)$ com relação à variável independente t . Na equação diferencial acima, b é uma constante conhecida que depende do comprimento do pêndulo, da aceleração da gravidade e da massa. $\theta(t)$ representa o ângulo de desvio da posição de equilíbrio. Se apresentamos a posição inicial e a velocidade de partida do pêndulo, o problema tem uma única solução $\theta(t)$.

Neste capítulo, apresentaremos alguns métodos numéricos que nos conduzam a aproximações da função de uma única variável $y(x)$, solução de equações diferenciais ordinárias dos tipos

$$y'(x) = f(x, y(x)) \quad \text{e} \quad y''(x) = f(x, y(x), y'(x)), \quad (8.1)$$

onde $f(x, y)$ e $f(x, y, y')$ são funções conhecidas.

Para definir uma única função $y(x)$, temos de fornecer dados adicionais à equação diferencial. Os dados adicionais podem definir dois tipos de problemas. Tomemos, para exemplificar, a segunda equação de (8.1), uma equação diferencial de segunda ordem, para a qual são necessárias duas condições adicionais.

No problema de valor inicial (PVI), procuramos $y(x)$ que, além de satisfazer a equação diferencial para $x > a$, atenda às condições preestabelecidas no início do intervalo onde vamos resolvê-la:

$$y(a) = y_0 \quad \text{e} \quad y'(a) = v_0,$$

onde y_0 e y_n são valores conhecidos.

Nos problemas de valor de contorno (PVC), a equação diferencial deverá ser satisfeita no intervalo (a, b) e são preestabelecidos valores para $y(x)$ nos extremos deste intervalo:

$$y(a) = y_0 \quad \text{e} \quad y(b) = y_n,$$

com y_0 e y_n conhecidos.

Infelizmente, conhecemos poucas técnicas matemáticas gerais para encontrar soluções analíticas para as equações diferenciais ordinárias. Uma das exceções é a equação diferencial linear com coeficientes constantes. Neste caso, as raízes do polinômio característico nos levarão a um conjunto de soluções linearmente independentes que geram as soluções particulares com condições de contorno.

O problema se agrava quando a equação é não-linear, isto é, $f(x, y, y')$ é uma função não-linear nas variáveis y e y' . Algumas técnicas específicas são descritas em livros de equações diferenciais ordinárias como por exemplo [Boyce-DiPrima, 1969] e catalogadas por [Kambe, 1959].

Na falta de soluções analíticas para equações diferenciais, os métodos numéricos têm atuado como importante ferramenta. O crescente desenvolvimento das calculadoras eletrônicas tem favorecido a maior especialização e utilização destes métodos.

Neste capítulo, estaremos usando $y(x)$ para denotar soluções de equações diferenciais ordinárias com valores iniciais ou valores de contorno. Apresentamos algumas técnicas úteis no cálculo de aproximações numéricas para $y(x)$.

8.2 Método das Diferenças Finitas

A essência dos métodos numéricos está na discretização do contínuo. É esta discretização que torna "finito" o problema e, portanto, viabiliza sua "solução" através de computadores.

A incógnita de uma equação diferencial ordinária é uma função $y(x)$ definida em todos os pontos do intervalo no qual a equação está sendo resolvida. O primeiro passo de qualquer método destinado à solução numérica de equações diferenciais é discretizar a região onde procuramos a solução. Neste passo, vamos definir uma *malha*, que é um conjunto finito de pontos, freqüentemente chamados nós da malha.

Seja x_0 um ponto de referência e h um número positivo. A *malha* de passo h associada a x_0 é o conjunto de pontos

$$x_i = x_0 \pm ih, \quad i = 1, 2, \dots$$

As aproximações de $y(x)$ serão calculadas nos pontos desta malha.

No Método de Diferenças Finitas, o segundo passo consiste na discretização das derivadas que estão na equação diferencial. Neste passo, as derivadas são aproximadas por diferenças entre valores da solução discretizada.

A ferramenta matemática básica na definição de aproximações para as derivadas é a série de Taylor. O mérito da série de Taylor é o de nos dizer como várias informações sobre a função, no ponto x , podem ser usadas na avaliação desta função numa vizinhança de x , isto é, no ponto $x + h$. Se assumirmos que $y(x)$ têm derivadas até ordem $n + 1$ em x , sua expansão em série de Taylor é:

$$y(x + h) = y(x) + h y'(x) + \frac{h^2}{2!} y''(x) + \cdots + \frac{h^n}{n!} y^{(n)}(x) + \frac{h^{n+1}}{(n+1)!} y^{(n+1)}(\xi), \quad (8.2)$$

onde ξ está entre x e $x + h$. O último termo da expressão acima representa o erro da aproximação de $y(x + h)$ pelos $n + 1$ primeiros termos da série de Taylor.

Se tomarmos $n = 1$ em (8.2), teremos a fórmula avançada para a discretização da derivada e seu erro:

$$y'(x) = \frac{y(x + h) - y(x)}{h} - \frac{h}{2} y''(\xi). \quad (8.3)$$

De modo semelhante, tomando $-h$ em (8.2), ainda com $n = 1$, obtemos a fórmula atrasada e seu erro:

$$y'(x) = \frac{y(x) - y(x - h)}{h} + \frac{h}{2} y''(\xi). \quad (8.4)$$

Tomemos agora $n = 2$ em (8.2) para escrever

$$y(x + h) = y(x) + h y'(x) + \frac{h^2}{2} y''(x) + \frac{h^3}{3!} y'''(\xi_1)$$

e

$$y(x - h) = y(x) - h y'(x) + \frac{h^2}{2} y''(x) - \frac{h^3}{3!} y'''(\xi_2).$$

Subtraindo a última expressão da penúltima, obtemos a fórmula centrada para discretização da derivada e o seu erro:

$$y'(x) = \frac{y(x + h) - y(x - h)}{2h} - \frac{h^2}{3!} y'''(\xi). \quad (8.5)$$

Observe que a fórmula centrada fornece uma aproximação da derivada na qual o erro é da ordem de h^2 , enquanto nas fórmulas avançada e atrasada o erro é da ordem de h .

Outras expressões com erro da ordem de h^2 podem ser obtidas a partir da manipulação da expansão em Taylor. Por exemplo,

$$y'(x) = \frac{-3y(x) + 4y(x + h) - y(x + 2h)}{2h} + \mathcal{O}(h^2)$$

e

$$y'(x) = \frac{3y(x) - 4y(x - h) + y(x - 2h)}{2h} + \mathcal{O}(h^2)$$

são aproximações que usam três pontos unilaterais.

Seguindo as mesmas idéias, podemos estabelecer uma expressão para o cálculo de aproximações para a segunda derivada. Tomando $n = 3$ em (8.2), temos

$$y(x+h) = y(x) + hy'(x) + \frac{h^2}{2}y''(x) + \frac{h^3}{3!}y'''(x) + \frac{h^4}{4!}y^{(iv)}(\xi_1)$$

e

$$y(x-h) = y(x) - hy'(x) + \frac{h^2}{2}y''(x) - \frac{h^3}{3!}y'''(x) + \frac{h^4}{4!}y^{(iv)}(\xi_1)$$

Somando estas duas últimas fórmulas e explicitando $y''(x)$, obtemos uma fórmula para discretizar a derivada de segunda ordem, bem como o erro associado a esta discretização:

$$y''(x) = \frac{y(x+h) - 2y(x) + y(x-h)}{h^2} - \frac{h^2}{12}y^{(iv)}(\xi) \quad (8.6)$$

para algum ξ no intervalo $(x-h, x+h)$.

No *Método de Diferenças Finitas*, as derivadas presentes na equação diferencial são substituídas por aproximações descritas acima.

Embora as fórmulas apresentadas sejam as discretizações usuais, outros tipos de aproximações das derivadas podem ser usadas no método de diferenças finitas. Por exemplo, podemos obter discretizações associadas à uma malha irregular na qual $x_i - x_{i-1} = h_i$.

Resumindo, para cada ponto da malha no interior do intervalo onde a equação diferencial está definida, denotamos por y_i a aproximação de $y(x_i)$, usamos

$$y'(x_i) \cong \frac{y_{i+1} - y_i}{h}, \quad y'(x_i) \cong \frac{y_i - y_{i-1}}{h} \quad \text{ou} \quad y'(x_i) \cong \frac{y_{i+1} - y_{i-1}}{2h},$$

isto é, uma das fórmulas avançada, atrasada ou centrada, respectivamente, e

$$y''(x_i) \cong \frac{y_{i+1} - 2y_i + y_{i-1}}{h^2}.$$

como discretizações das derivadas de primeira e segunda ordens que aparecem na equação diferencial.

Exemplo 8.2.1. Neste exemplo, vamos usar o método de diferenças finitas para encontrar uma aproximação para a solução da equação diferencial ordinária

$$y'' - y' + xy = e^x(x^2 + 1), \quad x \in (0, 1), \quad (8.7)$$

com condições de contorno

$$y(0) = 0 \quad y(1) = e.$$

Discretizamos o intervalo $[0, 1]$, tomando a malha associada a $h = 0.1$, e usamos a fórmula centrada (8.5) para aproximar $y'(x_i)$ e (8.6) para discretizar $y''(x_i)$. Teremos

$$\frac{y_{i+1} - 2y_i + y_{i-1}}{h^2} - \frac{y_{i+1} - y_{i-1}}{2h} + x_i y_i = e^{x_i}(x_i^2 + 1), \quad (8.8)$$

onde, como dito anteriormente, $y_i \cong y(x_i)$.

Observamos que (8.8) é uma equação linear nas incógnitas y_{i-1} , y_i e y_{i+1} que são aproximações de $y(x)$ nos pontos vizinhos x_{i-1} , x_i e x_{i+1} respectivamente. Ela é uma forma discretizada da equação diferencial (8.7), no ponto $x = x_i$.

Como esta forma discretizada pode ser escrita em todos os pontos internos da malha tomando $x = x_i$, $i = 1 : 9$, temos um sistema com 9 equações nas nove incógnitas y_i , $i = 1 : 9$. Como $x_i = ih$, este sistema tridiagonal pode ser escrito na forma

$$(2 - h)y_{i+1} + (2ih^3 - 4)y_i + (2 + h)y_{i-1} = 2h^2e^{ih}(i^2h^2 + 1).$$

As condições de contorno aparecerão nas equações correspondentes a $i = 1$ e $i = 9$. Substituindo, nestas duas equações, $y_0 = 0$ e $y_{10} = e$, temos para $i = 1$

$$(2 - h)y_2 + 2(2h^3 - 2)y_1 = 2h^2e^h(h^2 + 1).$$

e para $i = 9$

$$2(9h^3 - 2)y_9 + (2 + h)y_8 = 2h^2e^{9h}(81h^2 + 1) - (2 - h)e.$$

Resolvendo o sistema linear tridiagonal encontramos $y_1, \dots, y_9$ cujos valores estão na terceira coluna da tabela 8.1.

Pode-se verificar, por substituição direta na equação diferencial, que $y(x) = xe^x$ é a solução analítica para o problema de contorno proposto neste exemplo. Os valores desta solução estão representados na segunda coluna da tabela 8.1. Os resultados correspondentes a uma malha com $h = 0.05$ estão apresentados na quarta coluna da tabela 8.1. Mais tarde, comentaremos sobre a quinta coluna da tabela 8.1. Para resolver o sistema linear, usamos o algoritmo 2.4 para solução numérica de sistemas tridiagonais.

x_i	$y(x_i)$	y_i com $h = 0.1$	y_i com $h = 0.05$	Extrapolação
0.1	0.110517	0.110401	0.110488	0.110517
0.2	0.244281	0.244056	0.244224	0.244280
0.3	0.404958	0.404638	0.404878	0.404958
0.4	0.596730	0.596333	0.596630	0.596730
0.5	0.823461	0.823911	0.824248	0.824360
0.6	1.09327	1.09280	1.09315	1.09327
0.7	1.40963	1.40918	1.40952	1.40963
0.8	1.78043	1.78007	1.78034	1.78043
0.9	2.21364	2.21342	2.21359	2.21364

Tabela 8.1: Comparação entre soluções da edo (8.7).

□

Os passos seguidos no exemplo anterior podem ser generalizados em um algoritmo para problemas de contorno de equações diferenciais de segunda ordem.

Sejam $p(x)$, $q(x)$ e $r(x)$ funções contínuas definidas em $[a, b]$. O problema de encontrar $y(x)$ tal que:

$$y'' + p(x)y' + q(x)y = r(x), \quad a < x < b, \quad (8.9)$$

com valores de contorno

$$y(a) = y_0, \quad y(b) = y_n, \quad (8.10)$$

é conhecido como problema de Sturm-Liouville e tem várias aplicações práticas. A hipótese

$$q(x) \leq 0, \quad a \leq x \leq b$$

é suficiente para garantir a existência de uma única solução, mas não é necessária, como pode ser visto em [Courant-Hilbert, 1962].

Dividindo o intervalo $[a, b]$ em n partes iguais de comprimento h , introduzimos a malha $a = x_0 < x_1 < \dots < x_n = b$. Se em cada ponto interior x_i usamos as aproximações:

$$y'(x_i) \cong \frac{y_{i+1} - y_{i-1}}{2h} \quad \text{e} \quad y''(x_i) \cong \frac{y_{i+1} - 2y_i + y_{i-1}}{h^2},$$

obtemos a discretização da equação diferencial:

$$\frac{y_{i+1} - 2y_i + y_{i-1}}{h^2} + p(x_i) \frac{y_{i+1} - y_{i-1}}{2h} + q(x_i)y_i = r(x_i), \quad i = 1 : n-1,$$

ou ainda, usando a notação $p_i = p(x_i)$, $q_i = q(x_i)$ e $r_i = r(x_i)$,

$$\left(1 - \frac{h}{2}p_i\right)y_{i-1} + (-2 + h^2q_i)y_i + \left(1 + \frac{h}{2}p_i\right)y_{i+1} = h^2r_i,$$

para $i = 1 : n-1$.

Como fizemos no exemplo anterior, a primeira e a última equações devem ser modificadas pela utilização das condições de contorno. O algoritmo 8.1 corresponde ao método das diferenças finitas aplicado a (8.9). Neste algoritmo d , d_i e d_s são os vetores que contêm as diagonal, diagonal inferior e diagonal superior da matriz tridiagonal, respectivamente. O vetor t_i contém o termo independente do sistema tridiagonal. O primeiro e último elementos deste vetor são corrigidos no final do algoritmo, para incluir as condições de contorno (8.10).

A solução do sistema será $y_i \cong y(x_i)$, $i = 1 : n-1$, calculada pelo algoritmo 2.4.

A condição $q(x) \leq 0$ e a escolha de h tal que

$$h \max_{[a,b]} |p(x)| < 2 \quad (8.11)$$

Algoritmo 8.1 Método de Diferenças Finitas para a equação de Sturm-Liouville

Dados $a, b, p(x), q(x), r(x), y_0, y_n, n$

- 1: $h = (b-a)/n$
- 2: Para $j = 1 : n - 1$, faça
- 3: $x = a + jh$
- 4: $di(j) = 1 - \frac{h}{2}p(x)$
- 5: $d(j) = -2 + h^2q(x)$
- 6: $ds(j) = 1 + \frac{h}{2}p(x)$
- 7: $ti(j) = h^2r(x)$
- 8: $ti(1) = ti(1) - y_0(1 - \frac{h}{2}p(a+h))$
- 9: $ti(n-1) = ti(n-1) - y_n(1 + \frac{h}{2}p(b-h))$
- 10: Usar o algoritmo 2.4 para resolver o sistema tridiagonal

tornam o sistema diagonalmente dominante, com todas as desigualdades estritas, o que garante a existência de uma única solução do sistema linear e, portanto, uma única solução associada à malha escolhida.

Os valores aproximados da solução calculados para as malhas com passos h e $h/2$ podem ser usados para gerar novos valores com melhores resultados de aproximação. Para isto, vamos recorrer à *extrapolação de Richardson*, introduzida na seção 7.3.

Como usamos a fórmula centrada, $y_i(h)$ e $y_i(h/2)$ são aproximações (calculadas com h e $h/2$ respectivamente) de ordem h^2 . Se usarmos (7.12), isto é,

$$\bar{y}_i = \frac{1}{3} \left\{ 4y_i\left(\frac{h}{2}\right) - y_i(h) \right\},$$

obteremos uma nova aproximação para $y(x_i)$, solução de (8.9), com erro da ordem de h^3 . Os resultados correspondentes à extrapolação de Richardson são apresentados na última coluna da tabela 8.1.

A escolha da fórmula centrada para a discretização de $y'(x)$ se justifica por que ela é uma aproximação de ordem h^2 , ao passo que as fórmulas avançada e atrasada fornecem aproximações de ordem h . Se tivéssemos escolhido a fórmula avançada

$$y(x_i) \cong \frac{y_{i+1} - y_i}{h},$$

teríamos:

$$\frac{y_{i+1} - 2y_i + y_{i-1}}{h^2} + p(x_i)\frac{y_{i+1} - y_i}{h} + q(x_i)y_i = r(x_i),$$

e o correspondente sistema tridiagonal

$$y_{i-1} + (-2 - hp_i + h^2 q_i) y_i + (1 + hp_i) y_{i+1} = h^2 r_i.$$

Para efeito de comparação, resolvendo o sistema acima com $p(x) = -1$, $q(x) = x$, $r(x) = e^x(x^2 + 1)$ e $h = 0.1$, como no exemplo 8.2.1, teremos a quarta coluna da tabela 8.2. Na tabela, apresentamos também os valores de $y(x_i)$ e as aproximações obtidas usando diferenças atrasadas.

x_i	$y(x_i)$	y_i (atrasada)	y_i (avançada)
0.2	0.244281	0.256954	0.229982
0.4	0.596730	0.619059	0.571347
0.6	1.09327	1.11950	1.06324
0.8	1.78043	1.80092	1.75680

Tabela 8.2: Exemplo 8.2.1 com diferenças avançada e atrasada.

Observe que, embora a fórmula centrada forneça resultados mais precisos, a condição (8.11) que nos garante a existência de uma única solução para o sistema tridiagonal pode ser restritiva. Por exemplo, quando aplicamos as diferenças centradas na equação diferencial $y'' - 50y' = 0$, com dados de contorno $y(0) = 0$ e $y(1) = 1$, e $h = 0.1$, surgem oscilações na vizinhança de $x = 1$. Estas oscilações desaparecem quando usamos $h = 1/50$ ou então diferenças atrasadas na discretização de $y'(x)$. A solução do problema é

$$y(x) = \frac{1 - \exp(50x)}{1 - \exp(50)}.$$

No capítulo de métodos numéricos para equações diferenciais parciais, tratamos dos métodos *upwind*, ou *upstream*, nos quais as derivadas de primeira ordem são discretizadas usando fórmulas avançadas ou atrasadas, dependendo do tipo de equação diferencial.

8.3 Método da Série de Taylor

O desenvolvimento de funções em série de Taylor também é a base de um dos mais antigos métodos de aproximação da solução para o problema de valor inicial, que descrevemos nesta seção.

Na tentativa de simplificar nossa apresentação, usaremos equações de primeira ordem com dados iniciais. Desta maneira, vamos apresentar o *Método da Série de Taylor* usando o problema padrão: encontrar $y(x)$ tal que

$$\begin{aligned} y'(x) &= f(x, y), \quad x > a, \\ y(a) &= y_0. \end{aligned} \tag{8.12}$$

Entretanto, as mesmas idéias podem ser usadas em equações diferenciais de segunda ordem, ou ainda de ordem superior, desde que apliquemos um truque matemático para transformar uma equação diferencial de ordem maior que 1 num sistema de equações diferenciais de primeira ordem.

Tomemos, como exemplo, o problema de valor inicial de segunda ordem:

$$\begin{aligned}y''(x) &= f(x, y, y'), \quad x > a \\y(a) &= y_0 \\y'(a) &= v_0.\end{aligned}\tag{8.13}$$

Criamos uma nova variável $v(x) = y'(x)$ cuja derivada é $v'(x) = y''(x)$. Substituindo $y'(x)$ e $y''(x)$ na equação diferencial (8.13), formamos o sistema de equações diferenciais de primeira ordem:

$$\begin{aligned}y' &= v \\v' &= f(x, y, v),\end{aligned}$$

cujos dados iniciais são $y(a) = y_0$ e $v(a) = v_0$.

Usando a notação vetorial definimos dois vetores, u e F :

$$u = \begin{pmatrix} y \\ v \end{pmatrix} \quad \text{e} \quad F(x, u) = \begin{pmatrix} v \\ f(x, u) \end{pmatrix} = \begin{pmatrix} v \\ f(x, y, v) \end{pmatrix}$$

Nesta notação, a equação (8.13) será reescrita na forma de sistema de equações de primeira ordem,

$$\begin{aligned}u' &= F(x, u) \\u(0) &= u_0,\end{aligned}\tag{8.14}$$

onde $u_0 = (y_0, v_0)^T$.

Assim, os métodos numéricos desenhados para (8.12) podem ser generalizados para o caso de equações diferenciais de ordem maior que 1, reescritas na forma vetorial (8.14).

Voltemos a examinar o problema (8.12).

Usando repetidas vezes a regra da cadeia, vamos obter as derivadas de $y(x)$. Derivando a equação diferencial

$$y'(x) = f(x, y),$$

obtemos

$$y''(x) = f_x(x, y) + f_y(x, y)y'(x) = f_x + f_y f.$$

Derivando esta última expressão e usando a equação diferencial original, $y' = f$, temos

$$\begin{aligned}y'''(x) &= f_{xx} + f_{xy}y' + f_{yx}f + f_{yy}f y' + f_y f_x + f_y f_y y' \\&= f_{xx} + 2f_{xy}f + f_{yy}f^2 + f_y f_x + f_y^2 f.\end{aligned}\tag{8.15}$$

Observe que nestas expressões estamos empregando a notação usual de derivadas parciais:

$$f_x = \frac{\partial f}{\partial x}, \quad f_{xy} = \frac{\partial^2 f}{\partial x \partial y}, \dots$$

Também omitimos as variáveis da função para simplificar; por exemplo f_{xx} representa $f_{xx}(x, y)$.

De posse dessas derivadas podemos voltar à expansão em série de Taylor (8.2) para calcular y_{i+1} a partir de informações em x_i . Como $y_{i+1} = y(x_{i+1}) = y(x_i + h)$ temos, pela série de Taylor,

$$y_{i+1} = y_i + h y'(x_i) + \frac{h^2}{2} y''(x_i) + \frac{h^3}{3!} y'''(x_i) + \mathcal{O}(h^4).$$

Usando as derivadas calculadas em (8.15) obtemos uma aproximação com erro de quarta ordem:

$$\begin{aligned} y_{i+1} = & y_i + h f(x_i, y_i) + \frac{h^2}{2} [f_x(x_i, y_i) + f_y(x_i, y_i) f(x_i, y_i)] + \\ & + \frac{h^3}{3!} [f_{xx}(x_i, y_i) + 2f_{xy}(x_i, y_i) f(x_i, y_i) + f_{yy}(x_i, y_i) f^2(x_i, y_i) + \\ & + f_y(x_i, y_i) f_x(x_i, y_i) + f_y^2(x_i, y_i) f(x_i, y_i)] + \mathcal{O}(h^4). \end{aligned} \quad (8.16)$$

Temos então um método de solução do problema de valor inicial (8.12) que consiste em usar os valores (x_i, y_i) em (8.16) para calcular $y_{i+1} = y(x_{i+1})$.

Exemplo 8.3.1. Usaremos o Método da Série de Taylor para calcular uma aproximação da solução do problema de valor inicial

$$\begin{aligned} y'(x) &= \frac{1}{1+x^2} - 2y^2, \quad x > 0 \\ y(0) &= 0. \end{aligned}$$

Calcularemos y_{i+1} usando os três primeiros termos de (8.16),

$$y_{i+1} = y_i + h f(x_i, y_i) + \frac{h^2}{2} [f_x(x_i, y_i) + f_y(x_i, y_i) f(x_i, y_i)] + \mathcal{O}(h^3), \quad (8.17)$$

onde $\mathcal{O}(h^3)$ é a ordem do erro desta aproximação. Calculando as derivadas de $f(x, y) = \frac{1}{1+x^2} - 2y^2$ obtemos

$$f_x = \frac{-2x}{(1+x^2)^2} \quad \text{e} \quad f_y = -4y.$$

Tomando a condição inicial, $x_0 = 0$ e $y_0 = 0$, temos

$$f(x_0, y_0) = 1, \quad f_x(x_0, y_0) = 0 \quad \text{e} \quad f_y(x_0, y_0) = 0.$$

Vamos escolher o passo $h = 0.1$ na evolução de x . Aplicando estes valores em (8.17) temos

$$y(0.1) \cong y_1 = 0.1.$$

No segundo passo usamos $x_1 = 0.1$ e $y_1 = 0.1$ nas expressões de $f(x, y)$ e suas derivadas para obter

$$f(x_1, y_1) = 0.9701, \quad f_x(x_1, y_1) = -0.19606 \quad \text{e} \quad f_y(x_1, y_1) = -0.4.$$

Tomando $i = 1$ em (8.17) obtemos

$$y(0.2) \cong y_2 = 0.194090.$$

Na tabela 8.3, apresentamos valores calculados usando $h = 0.1$ e $h = 0.01$. Apresentamos, na segunda coluna desta tabela, os valores da solução analítica, que na equação diferencial deste exemplo é

$$y(x) = \frac{x}{1+x^2}.$$

x_i	$y(x_i)$	y_i com $h = 0.1$	y_i com $h = 0.01$
0.2	0.192308	0.194089	0.192738
0.4	0.344828	0.347009	0.345338
0.6	0.441176	0.442519	0.441481
0.8	0.487805	0.488119	0.487868

Tabela 8.3: Resultados com $h = 0.1$ e $h = 0.01$.

□

Observando (8.15), vemos que as expressões das derivadas tornam-se cada vez mais complicadas. Essa é uma das desvantagens da utilização do método apresentado nesta seção. Entretanto, os recentes avanços no cálculo simbólico têm contribuído para que o Método da Série de Taylor volte a uma posição de destaque entre os métodos para equações diferenciais ordinárias. Assim, este método ilustra a forte interligação que existe entre os equipamentos computacionais à disposição e a aplicabilidade de um método numérico.

As soluções das equações diferenciais de segunda ordem (8.13) podem ser calculadas recorrendo-se à notação vetorial (8.14).

Pode-se usar o método descrito nesta seção em sistemas, mais gerais, de equações diferenciais de primeira ordem. Neste caso, o vetor u em (8.14), assim como a função vetorial F , tem n componentes, $u = (u_1(x), u_2(x), \dots, u_n(x))$ e $F = (F_1(x, u), F_2(x, u), \dots, F_n(x, u))$. Por exemplo, no caso de um sistema com duas equações diferenciais de primeira ordem, o vetor solução $u = (u_1(x), u_2(x))$ pode ser aproximado por

$$u_{i+1} = u_i + h F(x_i, u_i) + \frac{h^2}{2} \frac{d}{dx} F(x, u)|_{(x_i, u_i)} + \frac{h^3}{3!} \frac{d^2}{dx^2} F(x, u)|_{(x_i, u_i)} + \dots, \quad (8.18)$$

onde $F = (F_1(x, u), F_2(x, u))$ e

$$\frac{d}{dx}F(x, u) = \frac{d}{dx} \begin{bmatrix} F_1(x, u) \\ F_2(x, u) \end{bmatrix} = \begin{bmatrix} \frac{d}{dx}F_1(x, u) \\ \frac{d}{dx}F_2(x, u) \end{bmatrix}$$

As derivadas são calculadas usando a regra da cadeia como fizemos no caso escalar, isto é, em (8.15).

Exemplo 8.3.2. Consideremos o problema de valor inicial associado a uma equação diferencial de segunda ordem

$$y'' = y^2 - x^2$$

$$y(0) = 0, \quad y'(0) = 1.$$

Usando a notação vetorial, chamamos $y'(x) = v$ e podemos reescrever a equação na variável vetorial $u(x) = (y(x), v(x))^T$:

$$u'(x) = F(x, u) = F(x, y, v) = \begin{pmatrix} v \\ y^2 - x^2 \end{pmatrix},$$

com condição inicial $u'(0) = (0, 1)^T$.

Para calcular as derivadas que aparecem em (8.18), observamos que a equação diferencial nos diz que $v'(x) = y'' = y^2 - x^2$. Assim temos:

$$\frac{d}{dx}F(x, y, v) = \begin{pmatrix} v'(x) \\ 2y(x)y'(x) - 2x \end{pmatrix} = \begin{pmatrix} y^2 - x^2 \\ 2yy' - 2x \end{pmatrix}$$

$$\frac{d^2}{dx^2}F(x, y, v) = \begin{pmatrix} 2yy' - 2x \\ 2(y')^2 + 2yy'' - 2 \end{pmatrix} = 2 \begin{pmatrix} yy' - x \\ (y')^2 + yy'' - 1 \end{pmatrix}.$$

Substituindo estas derivadas em (8.18), podemos calcular u_{i+1} usando os valores de y_i e y'_i :

$$u_{i+1} = \begin{pmatrix} y_{i+1} \\ y'_{i+1} \end{pmatrix} = \begin{pmatrix} y_i \\ y'_i \end{pmatrix} + h \begin{pmatrix} v'_i \\ y_i^2 - x_i^2 \end{pmatrix} + \frac{h^2}{2} \begin{pmatrix} y_i^2 - x_i^2 \\ 2y_i y'_i - 2x_i \end{pmatrix} + \frac{2h^3}{3!} \begin{pmatrix} y_i y'_i - x_i \\ (y'_i)^2 + y_i(y_i^2 - x_i^2) - 1 \end{pmatrix}.$$

Tomando, por exemplo, $h = 0.3$ temos:

$$u_1 = \begin{pmatrix} y_1 \\ y'_1 \end{pmatrix} = \begin{pmatrix} 0.3 \\ 1.0 \end{pmatrix} \quad \text{e} \quad u_2 = \begin{pmatrix} y_2 \\ y'_2 \end{pmatrix} = \begin{pmatrix} 0.6 \\ 1.0 \end{pmatrix}.$$

Por substituição direta, podemos verificar que $y(x) = x$ é a solução analítica da equação diferencial e, portanto, os resultados numéricos são exatos. De fato, como neste exemplo $y''(x) = 0$, as aproximações que usam os três primeiros termos da série de Taylor são exatas. Neste caso particular, nem era necessário usar os termos correspondentes a h^3 . $\square$

8.4 Métodos de Runge-Kutta

Os Métodos de Runge-Kutta são os mais usados dentre aqueles apropriados para os problemas de valor inicial. Os seus atrativos são simplicidade, alta precisão e versatilidade nas aplicações.

Tomemos a equação diferencial de primeira ordem, com dado inicial

$$\begin{aligned} y'(x) &= f(x, y) \\ y(x_0) &= y_0. \end{aligned} \quad (8.19)$$

Considerando a malha definida pelo passo h , podemos usar a fórmula de diferenças finitas avançada para discretizar a derivada de $y(x)$ no ponto x_k . Assim, obtemos uma versão discretizada de (8.19):

$$\frac{y_{k+1} - y_k}{h} = f(x_k, y_k). \quad (8.20)$$

Esta equação permite que calculemos y_{k+1} a partir de y_k , e define o *Método de Euler* (1707-1783), estabelecido em 1768. Sua aplicação é muito simples:

$$\begin{aligned} y_0 &= y(x_0) \\ y_{k+1} &= y_k + h f(x_k, y_k), \quad \text{para } k = 0, 1, \dots \end{aligned}$$

O método numérico de Euler foi usado por Cauchy (1789-1857) em um procedimento matemático para demonstrar a existência de solução para (8.19). O argumento de Cauchy foi generalizado por Lipschitz (1832-1903), exigindo menos da função $f(x, y)$.

Como vimos anteriormente, a fórmula avançada introduz erro da ordem de h em cada passo.

A idéia de Euler tem versões de maior precisão nos trabalhos de Runge em 1895, para o caso de uma equação, e Kutta em 1901, para o caso de sistemas de equações diferenciais de primeira ordem. Consideremos inicialmente os Métodos Runge-Kutta de segunda ordem definidos pela expressão

$$\frac{y_{k+1} - y_k}{h} = \beta_0 f(x_k, y_k) + \beta_1 f(x_k + \gamma h, y_k + \delta h),$$

onde os parâmetros β_0 , β_1 , γ e δ serão convenientemente calculados de modo a aumentar a precisão do resultado.

Explicitando y_{k+1} nesta relação, temos

$$y_{k+1} = y_k + h \beta_0 f(x_k, y_k) + h \beta_1 f(x_k + \gamma h, y_k + \delta h). \quad (8.21)$$

O último termo do lado direito desta equação pode ser desenvolvido através da série de Taylor para funções de duas variáveis, isto é,

$$f(x_k + \gamma h, y_k + \delta h) = f(x_k, y_k) + h\gamma f_x(x_k, y_k) +$$

$$+ \delta h f_y(x_k, y_k) + \frac{1}{2} [h^2 \gamma^2 f_{xx}(x_k, y_k) + 2h^2 \gamma \delta f_{xy}(x_k, y_k) + h^2 \delta^2 f_{yy}(x_k, y_k)] + O(h^3).$$

Levando esta expansão para (8.21) e agrupando os termos de iguais potências de h teremos:

$$\begin{aligned} y_{k+1} = & y_k + h[(\beta_0 + \beta_1) f(x_k, y_k)] + \frac{h^2}{2} [2\beta_1 \gamma f_x(x_k, y_k) + 2\beta_1 \delta f_y(x_k, y_k)] + \\ & + \frac{h^3}{6} [3\beta_1 \gamma^2 f_{xx}(x_k, y_k) + 6\beta_1 \gamma \delta f_{xy}(x_k, y_k) + 3\beta_1 \delta^2 f_{yy}(x_k, y_k)] + O(h^4). \end{aligned} \quad (8.22)$$

Nos Métodos de Runge-Kutta adota-se, como critério de escolha dos quatro parâmetros β_0 , β_1 , γ e δ , a ordem de aproximação dos valores de y_{k+1} obtidos por (8.16) e (8.22). Detalhamos a seguir apenas o caso simples, que é chamado Runge-Kutta de segunda ordem.

Tomando os desenvolvimentos (8.16) e (8.22), vamos escolher β_0 , β_1 , γ e δ de modo que os multiplicadores de h e h^2 sejam iguais nas duas expressões. Desta maneira, os parâmetros devem satisfazer às três relações

$$\beta_0 + \beta_1 = 1 \quad 2\beta_1 \gamma = 1 \quad 2\beta_1 \delta = f(x_k, y_k).$$

Como são quatro os parâmetros e três as condições que eles devem satisfazer, existem várias escolhas de β_0 , β_1 , γ e δ de modo que o cálculo de y_{k+1} usando (8.21) coincida com o da série de Taylor, (8.16), até o termo de segunda ordem. Por exemplo poderíamos tomar

$$\beta_0 = \beta_1 = \frac{1}{2}, \quad \gamma = 1 \quad \text{e} \quad \delta = f(x_k, y_k)$$

ou

$$\beta_0 = 0, \beta_1 = 1, \quad \gamma = \frac{1}{2} \quad \text{e} \quad \delta = \frac{1}{2} f(x_k, y_k).$$

Estes são parâmetros que definem Métodos de Runge-Kutta de segunda ordem.

O Algoritmo 8.2 mostra como é simples a implementação computacional do método apresentado.

Para estabelecer outras fórmulas mais gerais de Runge-Kutta, que coincidam com a série de Taylor até os termos de ordem h^3 , h^4 etc., generalizamos (8.21) de modo que

$$\begin{aligned} y_{k+1} = & y_k + h[\beta_0 f(x_k, y_k) + \beta_1 f(x_k + \gamma_1 h, y_k + \delta_1 h) + \\ & + \beta_2 f(x_k + \gamma_2 h, y_k + \delta_2 h) + \beta_3 f(x_k + \gamma_3 h, y_k + \delta_3 h) + \dots] \end{aligned}$$

e procedemos de maneira análoga para estabelecer relações entre os parâmetros.

Nos Algoritmos 8.3 e 8.4, apresentamos versões dos Métodos de Runge-Kutta de terceira e quarta ordem respectivamente.

Algoritmo 8.2 Runge-Kutta de segunda ordem

Dados y_0, h e $f(x, y)$

- 1: Para $k = 0, 1, 2, \dots$, faça
 - 2: $m_0 = h f(x_k, y_k)$
 - 3: $m_1 = h f(x_k + h, y_k + m_0)$
 - 4: $y_{k+1} = y_k + \frac{1}{2}(m_0 + m_1)$
-

Algoritmo 8.3 Runge-Kutta de terceira ordem

Dados y_0, h e $f(x, y)$

- 1: Para $k = 0, 1, 2, \dots$, faça
 - 2: $m_0 = h f(x_k, y_k)$
 - 3: $m_1 = h f\left(x_k + \frac{h}{2}, y_k + \frac{m_0}{2}\right)$
 - 4: $m_2 = h f(x_{k+1}, y_k - m_0 + 2m_1)$
 - 5: $y_{k+1} = y_k + \frac{1}{6}(m_0 + 4m_1 + m_2)$
-

Algoritmo 8.4 Runge-Kutta de quarta ordem

Dados y_0, h e $f(x, y)$

- 1: Para $k = 0, 1, 2, \dots$, faça
 - 2: $m_0 = h f(x_k, y_k)$
 - 3: $m_1 = h f\left(x_k + \frac{h}{2}, y_k + \frac{m_0}{2}\right)$
 - 4: $m_2 = h f\left(x_k + \frac{h}{2}, y_k + \frac{m_1}{2}\right)$
 - 5: $m_3 = h f(x_{k+1}, y_k + m_2)$
 - 6: $y_{k+1} = y_k + \frac{1}{6}(m_0 + 2m_1 + 2m_2 + m_3)$
-

O Método de Runge-Kutta de quarta ordem é mais usado por ser uma combinação de simplicidade, alta precisão e economia.

Exemplo 8.4.1. Na equação diferencial:

$$y'(x) = \frac{1}{1+x^2} - 2y^2$$

$$y(0) = 0$$

aplicamos os métodos de Euler e Runge-Kutta de quarta ordem. Os resultados correspondentes a $h = 0.1$ e $h = 0.01$ estão apresentados na tabelas 8.4 e 8.5. Nestas tabelas, também apresentamos comparações com a solução analítica para a equação diferencial:

$$y(x) = \frac{x}{1+x^2}.$$

Estes resultados podem também ser comparados com a aproximação do exemplo (8.19).

x	sol. analítica	Euler	erro	RK 4a. ordem	erro
0.2	0.192308	0.197010	0.47×10^{-2}	0.192307	0.95×10^{-6}
0.4	0.344828	0.360854	0.16×10^{-1}	0.344825	0.26×10^{-5}
0.6	0.441176	0.465566	0.24×10^{-1}	0.441172	0.42×10^{-5}
0.8	0.487805	0.513707	0.26×10^{-1}	0.487800	0.47×10^{-5}

Tabela 8.4: Resultados com $h = 0.1$.

x	Euler	erro	RK 4a. ordem	erro
0.2	0.192842	0.53×10^{-3}	0.192308	0.15×10^{-7}
0.4	0.346439	0.16×10^{-2}	0.344828	0.0
0.6	0.443536	0.23×10^{-2}	0.441176	0.298×10^{-7}
0.8	0.490303	0.25×10^{-2}	0.487805	0.298×10^{-7}

Tabela 8.5: Resultados com $h = 0.01$.

□

Nesta seção apresentamos os Métodos de Runge-Kutta com tamanho do passo fixo, h , dado. A tendência nas últimas décadas tem sido explorar os métodos com passo variável, nos quais os passos são diminuídos nas regiões de variações bruscas da solução. Estimativas de erro de truncamento dos métodos são usadas para elaborar os programas com tamanho de passo variável. Existem duas maneiras de elaborar estes métodos (por exemplo, ver [Buchanan-Turner, 1992]).

Na primeira, comparam-se os candidatos à soluções associados a h_0 e $h_0/2$ para estabelecer um passo cuja aproximação tenha erro de truncamento menor que a tolerância pré-fixada.

A outra alternativa, que é mais econômica, são os Métodos Runge-Kutta-Felberg, desenvolvidos na década de 60. Nestes métodos, consideram-se os erros de truncamento local associados aos Métodos Runge-Kutta de ordem n e $n+1$ para calcular o tamanho do passo que garante a precisão desejada. Os Métodos Runge-Kutta-Felberg de ordem 2(3) e 4(5) são as funções `ode23` e `ode45` do pacote **Matlab**.

As fórmulas de Runge-Kutta podem ser aplicadas às equações diferenciais de segunda ordem, bem como àsquelas de ordem superior. Para isso, recorreremos à notação vetorial para transformar uma equação de ordem maior do que 1 num sistema de equações de primeira ordem, como fizemos na seção anterior.

Por exemplo, vamos detalhar o caso das equações de segunda ordem. Tomando a notação (8.14), podemos reescrever o algoritmo de Runge-Kutta de quarta ordem componente a componente.

Para simplificar a notação, tomemos inicialmente m_0, m_1, m_2 e m_3 como vetores de duas componentes. Estas componentes podem ser calculadas por:

$$\begin{aligned} m_0(1) &= h v_k \\ m_0(2) &= h f(x_k, y_k, v_k) \\ m_1(1) &= h \left(v_k + \frac{m_0(2)}{2} \right) \\ m_1(2) &= h f \left(x_k + \frac{h}{2}, y_k + \frac{m_0(1)}{2}, v_k + \frac{m_0(2)}{2} \right) \\ m_2(1) &= h \left(v_k + \frac{m_1(2)}{2} \right) \\ m_2(2) &= h f \left(x_k + \frac{h}{2}, y_k + \frac{m_1(1)}{2}, v_k + \frac{m_1(2)}{2} \right) \\ m_3(1) &= h (v_k + m_2(2)) \\ m_3(2) &= h f(x_k + h, y_k + m_2(1), v_k + m_2(2)). \end{aligned}$$

Os valores da solução e sua derivada no ponto $x_k + h$ são calculados por

$$y_{k+1} = y_k + \frac{1}{6} \{m_0(1) + 2m_1(1) + 2m_2(1) + m_3(1)\}$$

$$v_{k+1} = v_k + \frac{1}{6} \{m_0(2) + 2m_1(2) + 2m_2(2) + m_3(2)\}.$$

No Algoritmo 8.5 apresentamos a forma final simplificada deste método, na qual não é necessário usar duas componentes para m_0, m_1, m_2 e m_3 .

8.5 Conceito de Estabilidade

No vasto uso de computadores para resolver equações diferenciais, observa-se que alguns algoritmos apresentam soluções com erros que são muito maiores que os simples erros

Algoritmo 8.5 Runge-Kutta de quarta ordem para $y'' = f(x, y, y')$

Dados y_0, v_0 e h

- 1: Para $k = 0, 1, 2, \dots$, faça
 - 2: $m_0 = h f(x_k, y_k, v_k)$
 - 3: $m_1 = h f\left(x_k + \frac{h}{2}, y_k + \frac{h v_k}{2}, v_k + \frac{m_0}{2}\right)$
 - 4: $m_2 = h f\left(x_k + \frac{h}{2}, y_k + \frac{h v_k}{2} + \frac{h m_0}{4}, v_k + \frac{m_1}{2}\right)$
 - 5: $m_3 = h f\left(x_{k+1}, y_k + h v_k + \frac{h m_1}{2}, v_k + m_2\right)$
 - 6: $y_{k+1} = y_k + h v_k + \frac{h}{6}(m_0 + m_1 + m_2)$
 - 7: $v_{k+1} = v_k + \frac{1}{6}(m_0 + 2m_1 + 2m_2 + m_3)$
-

associados à discretização. Além disso, também se pode verificar experimentalmente que, em alguns casos, as soluções numéricas se afastam cada vez mais da solução analítica à medida que diminuimos o passo de discretização. Estas constatações contradizem o pressuposto de que a discretização se aproxima do contínuo quando o passo tende a zero e caracterizam instabilidade do esquema numérico.

Embora existam várias maneiras de definir e analisar a estabilidade de métodos numéricos, comentaremos dois procedimentos mais comuns.

No primeiro caso, a discretização é testada através do problema-padrão:

$$\begin{aligned} y'(x) &= qy(x) \\ y(0) &= 1, \end{aligned} \quad (8.23)$$

onde q é uma constante negativa. O problema-padrão é simples, e sua solução analítica – pode-se verificar diretamente – é $y(x) = \exp(qx)$. A existência de uma expressão para a solução analítica é conveniente nos testes numéricos. Observamos que $\lim_{x \rightarrow \infty} y(x) = 0$, pois $q < 0$. Portanto as aproximações numéricas devem satisfazer esta condição no limite.

Como ilustração, tomemos duas discretizações para o problema-padrão: o Método de Euler e uma modificação deste método.

1ª Discretização: Aplicando o Método de Euler em (8.23) teremos:

$$\frac{y_{n+1} - y_n}{h} = q y_n, \quad (8.24)$$

ou ainda,

$$y_{n+1} = y_n(1 + qh). \quad (8.25)$$

Assim, como $y_0 = 1$, então:

$$y_1 = (1 + qh), \quad y_2 = (1 + qh)^2, \quad \dots, \quad y_n = (1 + qh)^n.$$

Vemos então que, se $|1 + qh| > 1$, os valores de y_n , que seriam as aproximações para $y(x_n)$, crescem exponencialmente, isto é, $y_n \rightarrow \infty$ quando $n \rightarrow \infty$.

Temos então uma condição de estabilidade $|1 + qh| < 1$ que deve ser respeitada se quisermos que as aproximações acompanhem, pelo menos qualitativamente, a solução analítica.

No caso particular, $q = -10$ os valores de h devem satisfazer à condição de estabilidade $h < 0.2$. Na tabela 8.6, apresentamos os valores obtidos com $h = 0.25$, que não satisfaz à condição de estabilidade.

n	e^{-10x_n}	y_n
1	0.08208	-1.5
2	0.00674	2.25
3	0.00055	-3.375
4	0.00005	5.0625

Tabela 8.6: Exemplo de instabilidade no Método de Euler.

2ª Discretização: Consideremos a modificação do Método de Euler na qual o lado direito de (8.24) será a média entre y_n e y_{n+1} . Assim,

$$\frac{y_{n+1} - y_n}{h} = \frac{q}{2}(y_n + y_{n+1}). \quad (8.26)$$

Explicitando y_{n+1} teremos uma relação de recorrência entre y_n e y_{n+1} dada por:

$$y_{n+1} = \frac{1 + \frac{hq}{2}}{1 - \frac{hq}{2}} y_n. \quad (8.27)$$

Como $y_0 = 1$, usando a relação de recorrência entre y_{n+1} e y_n temos

$$y_1 = \left(\frac{1 + \frac{hq}{2}}{1 - \frac{hq}{2}} \right), \quad y_2 = \left(\frac{1 + \frac{hq}{2}}{1 - \frac{hq}{2}} \right)^2, \quad \dots \quad y_n = \left(\frac{1 + \frac{hq}{2}}{1 - \frac{hq}{2}} \right)^n.$$

Como $q < 0$, observamos que o denominador será maior que o numerador, qualquer que seja h . Assim, $y_n \rightarrow 0$ independentemente da escolha de h . Dizemos que o método correspondente à discretização definida por (8.26) é *incondicionalmente estável*. Por exemplo, se tomamos os mesmos valores de h e q que foram usados na primeira discretização, isto é, $h = 0.25$ e $q = -10$ temos $hq = -2.5$. Neste caso, a relação de recorrência será:

$$y_{n+1} = \frac{1 - 1.25}{1 + 1.25} y_n = -\frac{0.25}{2.25} y_n = \left(-\frac{1}{9} \right)^{n+1} y_0.$$

Como $y_0 = 1$, temos $y_1 = -0.1111$, $y_2 = 0.0123$, $y_3 = -0.0014$ etc. De fato, vemos que $y_n \rightarrow 0$ quando $n \rightarrow \infty$, uma vez que os y_n são potências de um número que é, em valor absoluto, menor que 1.

Os dois exemplos anteriores são casos particulares de um procedimento genérico, que usa as raízes dos polinômios associados às equações de diferenças na análise de estabilidade. Não tendo a pretensão de esgotar o assunto, gostaríamos de deixar idéias.

Equações do tipo (8.25) e (8.27) são chamadas equações de diferenças associadas aos métodos (8.24) e (8.26), respectivamente. De um modo geral, suponha que a um método numérico destinado ao problema

$$\begin{aligned} y'(x) &= f(x, y), \quad x > a \\ y(a) &= y_0 \end{aligned}$$

associamos a equação de diferenças, discretização da equação diferencial:

$$\alpha_k y_{n+k} + \alpha_{k-1} y_{n+k-1} + \cdots + \alpha_0 y_n = h (\beta_k f_{n+k} + \beta_{k-1} f_{n+k-1} + \cdots + \beta_0 f_n). \quad (8.28)$$

Os coeficientes $\alpha_k, \dots, \alpha_0, \beta_k, \dots, \beta_0$ da equação de diferenças (8.28) dependem do método escolhido, isto é, do número de pontos usados e da combinação adequada destes pontos. Por exemplo, em (8.25) $k = 1$, $\alpha_1 = 1$, $\alpha_0 = -(1 + qh)$, $\beta_0 = \beta_1 = 0$. Na equação de diferenças (8.27) $k = 1$, $\alpha_1 = 1 - \frac{qh}{2}$, $\alpha_0 = -(1 + \frac{qh}{2})$, $\beta_0 = \beta_1 = 0$.

A estabilidade do método é analisada através das raízes do polinômio característico,

$$p(\lambda) = \alpha_k \lambda^k + \alpha_{k-1} \lambda^{k-1} + \cdots + \alpha_1 \lambda + \alpha_0,$$

formado pelos coeficientes α_i usados no lado esquerdo da equação de diferenças (8.28).

Sejam λ_i , $i = 1 : k$ as raízes da equação $p(\lambda) = 0$. Se $|\lambda_i| \leq 1$, $i = 1 : k$ e todas as raízes com valor absoluto igual a 1 – se existirem – são raízes simples (isto é, $|\lambda_i| = 1$ então $p'(\lambda_i) \neq 0$), então, dizemos que o método satisfaz a *condição da raiz*.

Num contexto mais amplo, a estabilidade de um método de discretização para equações diferenciais é caracterizada por meio da seguinte definição:

Definição:

- i. Se um método satisfaz a condição da raiz e $\lambda = 1$ é a única possibilidade para as raízes de módulo 1, então ele é chamado *fortemente estável*;
- ii. os métodos que satisfazem a condição da raiz e têm mais de uma raiz com módulo igual a 1 são chamados *fracamente estáveis*;
- iii. os métodos que não satisfazem a condição da raiz são chamados *instáveis*.

Como aplicação, voltemos às duas discretizações propostas anteriormente para a equação diferencial (8.23).

No primeiro caso, a equação de diferenças é (8.24) e o polinômio característico correspondente é $p(\lambda) = \lambda - (1 + qh)$, que tem raiz simples $\lambda = 1 + qh$. Assim, o método é fortemente estável se $|1 + qh| < 1$. Esta é a condição de estabilidade que tínhamos encontrado anteriormente. Caso esta condição não seja satisfeita, o método é instável.

Na segunda discretização, a equação de diferenças é dada por (8.27) e o polinômio característico correspondente será:

$$p(\lambda) = \lambda \left(1 - \frac{qh}{2} \right) - \left(1 + \frac{qh}{2} \right),$$

cujas únicas raízes são

$$\lambda = \frac{1 + \frac{qh}{2}}{1 - \frac{qh}{2}}.$$

Se $q < 0$, então $|\lambda| < 1$, qualquer que seja o h escolhido e, conseqüentemente, o método é fortemente estável.

Dessa maneira, dizemos que a primeira discretização é condicionalmente estável (em determinadas condições sobre h podemos ter estabilidade), ao passo que a segunda é incondicionalmente estável (a estabilidade independe de condições em h).

Como exemplo final, tomemos um método de passo múltiplo. Um método é chamado passo múltiplo se, para calcular y_{i+1} , aproximação da solução em x_{i+1} , usamos vários valores de y : $y_i, y_{i-1}, \dots, y_{i-m}$. Por exemplo, as aproximações do método de Adams-Bashforth de quarta ordem são definidas por

$$y_{i+1} = y_i + \frac{h}{24}(55f_i - 59f_{i-1} + 37f_{i-2} - 9f_{i-3}),$$

onde, para simplificar, denotamos $f_j = f(x_j, y_j)$, $j = i-3 : i$. Comparando a equação de diferenças do método de Adams-Bashforth com a equação de diferenças geral (8.28), vemos que $k = 4$, $\alpha_4 = 1$, $\alpha_3 = -1$, $\alpha_2 = 0$, $\alpha_1 = 0$ e $\alpha_0 = 0$. Assim sendo, o polinômio característico associado à equação de diferenças é:

$$p(\lambda) = \lambda^4 - \lambda^3 = \lambda^3(\lambda - 1),$$

cujas raízes são $\lambda_1 = \lambda_2 = \lambda_3 = 0$ e $\lambda_4 = 1$. Como o método satisfaz a condição (i) da caracterização de estabilidade, ele é fortemente estável, independente do valor de h .

8.6 Uso do Matlab em Equações Diferenciais Ordinárias

Na versão de 1995 do Matlab, aparece uma coleção de programas destinada à solução de problemas de valor inicial. Estes programas substituem os ode23 e ode45 das versões anteriores do Matlab, com a incorporação de cinco novos programas e uma série de outras

utilidades, o que ampliou a potencialidade de uso deste pacote na solução de equações diferenciais ordinárias. Podem-se conferir as facilidades introduzidas usando a função `odedemo`. Seguindo a notação usada neste capítulo, vamos considerar o problema de valor inicial

$$\begin{aligned} y'(x) &= f(x, y), \quad x > x_0, \\ y(x_0) &= y_0 \end{aligned}$$

onde y e f podem também ser vetores, no caso de sistemas.

Antes de acessar um dos programas para resolver problemas de valor inicial, precisamos definir a função associada à equação diferencial ordinária, $f(x, y)$. Como exemplo, vamos resolver o PVI

$$\begin{aligned} y'(x) &= -y^3, \quad 0 < x < 0.49, \\ y(0) &= 1 \end{aligned}$$

Definimos $f(x, y)$ do PVI numa function:

```
function f = funode1(x, y)      % Definindo a Equação Diferencial
f = -y.^3;
```

A seguir, apresentamos uma sequência de comandos que resolve esta equação diferencial no intervalo $[0, 0.49]$ usando os recursos `ode23` e `ode45`:

```
>> xx = [0 .49];              % Definindo intervalo de cálculo da solução

% Solução usando Runge-Kutta de segunda ordem
>> [x23, y23] = ode23('funode1', xx, 1);

% Solução usando Runge-Kutta de quarta ordem
>> [x45, y45] = ode45('funode1', xx, 1);

>> plot(x23, y23, '* ', x45, y45, '.')
```

No recurso `ode23` é aplicado o Método Explícito Runge-Kutta-Fehlberg de segunda ordem, o qual é uma modificação do Método Runge-Kutta de segunda ordem que fornece aproximações de terceira ordem; ver, por exemplo, [Burden-Faires, 1988] ou [Conte-deBoor, 1981]. Na modificação proposta por Fehlberg em 1970, o cálculo de mais valores da função permite estimar automaticamente o erro e fornece melhores resultados que aqueles obtidos pelo método-padrão de segunda ordem. Neste método, usa-se também o controle automático do tamanho do passo, permitindo maior refinamento nas regiões de variações bruscas da solução. A idéia é repetida no método `ode45` que é um Runge-Kutta de quarta ordem, modificado para fornecer aproximações de quinta ordem. Como nos gráficos usamos símbolos diferentes, podemos observar que a discretização é diferente quando aplicamos os dois métodos, sendo que o refinamento fornecido pelo `ode45` é maior, como era de se esperar por ser este método mais preciso.

Os outros recursos para os PVI disponíveis na versão 5 do Matlab são descritos resumidamente a seguir. Mais detalhes podem ser obtidos usando o help de cada uma destas funções.

- ode113: Usa métodos de passo múltiplo de ordens variadas (1 a 13). É recomendável para problemas que requerem moderada precisão, nos quais $f(x, y)$ é suave.
- ode23s: Usa método implícito de um passo e é recomendado para problemas com variações bruscas na solução, os chamados problemas *stiff*, quando se pode trabalhar com precisão moderada.
- ode15s: Método implícito de passo múltiplo de ordem variável (1 a 5). Recomendável para problemas de variações bruscas.

A seguir, vamos utilizar o Matlab para resolver um sistema de equações diferenciais de primeira ordem com valores iniciais. De fato, vamos transformar uma equação diferencial de segunda ordem num sistema de duas equações diferenciais de primeira ordem, para então usar Matlab.

Consideremos a equação de Van der Pol:

$$y''(x) - m(1 - y^2)y'(x) + y = 0$$

onde m é um parâmetro maior do que zero. Para transformar a equação num sistema de equações diferenciais de primeira ordem, vamos usar a mudança de variáveis $y_1(x) = y(x)$ e $y_2(x) = y'(x)$. Teremos então o sistema

$$\begin{aligned} y_1' &= y_2 \\ y_2' &= m(1 - y_1^2)y_2 + y_1. \end{aligned}$$

Inicialmente, definimos a função vetorial associada à equação diferencial, na qual vamos tomar $m = 2$ e condições iniciais $y(0) = 2$ e $y'(0) = 0$.

```
function f = funode2(x, y)
mu = 2;
f = [y(2); mu*(1-y(1)^2)*y(2)-y(1)];
```

Realizamos a seguinte sequência para calcular a solução da equação Van der Pol no intervalo $[0, 20]$:

```
>> xx = [0 20];           % Definindo o intervalo da solução
>> y0 = [2;0];           % Condições Iniciais

% Solução usando Runge-Kutta de quarta ordem
>> [x,y] = ode45('funode2', xx, y0);
```

```
% Traçado dos Gráficos da solução e sua derivada (...)
>> plot(x, y(:,1), x, y(:,2), 'r')
>> xlabel('Tempo, x')
>> title('Solução da Equação de Van der Pol')
```

Exercícios

1. Use o método de diferenças finitas para encontrar aproximações para os seguintes problemas com dados de contorno. O passo h é dado em cada caso. Em alguns itens, fornecemos a expressão da solução analítica, que poderá ser usada para conferir os resultados obtidos.

a. $y'' + y = 0$, $x \in (0, 1)$ com condições $y(0) = 0$ e $y(1) = 1$.

Passo $h = 0.1$. Solução analítica $y(x) = \sin x / \sin 1$.

b. $y'' + y = 0$, $x \in (0, 1)$ com condições $y'(0) + y(0) = 0$ e $y(1) = 1$.

Passo $h = 0.1$.

c. $y'' + (x+1)y' - 2y = (1-x^2)e^{-x}$, $x \in (0, 1)$ com condições $y(0) = -1$ e $y(1) = 0$.

Passo $h = 0.1$. Solução analítica $y(x) = (x-1)e^{-x}$.

d. $y'' = 100y$, $x \in (0, 1)$ com condições $y(0) = 1$ e $y(1) = e^{-10}$.

Passos $h = 0.1$ e $h = 0.05$. Solução analítica $y(x) = e^{-10x}$.

2. Escreva uma discretização com a ordem de aproximação $\mathcal{O}(h^2)$ para a equação diferencial

$$y'' + xy' + y = 2x, \quad y(0) = 1 \quad \text{e} \quad y(1) = 0.$$

Prepare o sistema de equações correspondentes a $h = 0.1$, elabore e execute um programa correspondente.

3. Encaminhe um procedimento que, acoplando o método de Newton à discretização, por via de diferenças finitas, forneça soluções aproximadas para os problemas não-lineares com dados de contorno:

a. $y'' = 0.5y^3$, $x \in (1, 2)$ com condições $y(1) = -2/3$ e $y(2) = -1$.

Passo $h = 0.05$. Solução analítica $y(x) = 2/(x-4)$.

b. $y'' = y^3 - yy'$, $x \in (1, 2)$ com condições $y(1) = 1/2$ e $y(2) = 1/3$.

Passo $h = 0.1$.

4. Usando o desenvolvimento em série de Taylor até os termos de segunda ordem, encontre aproximações para a solução dos problemas de valor inicial abaixo. Compare, quando possível, seus resultados com a solução analítica.

a. $y' = 1/x^2 - y/x - y^2$, $x > 1$ com $y(1) = -1$.

Passos $h = 1/16$ e $h = 1/32$. Solução analítica $y(x) = -1/x$.

b. $y' = -xy + 1/y^2$, $x > 1$ com $y(1) = 1$.

Passo $h = 0.05$.

5. Encontre uma aproximação para $y(0.1)$, com quatro dígitos corretos, sabendo que $y' = xy + 1$ e $y(0) = 1$.

6. Resolva por diferenças finitas a equação diferencial com dados de contorno $y'' + 2y' + y = x$, $y(0) = y(1) = 0$ usando $h = 0.1$ e $h = 0.05$. Use estes resultados para melhorar suas aproximações com extrapolação de Richardson.

7. Considere o problema 4a.

a. Usando um Método de Runge-Kutta de segunda ordem, encontre uma solução no intervalo $(1, 2)$ tomando $h = 0.1$.

b. Use interpolação, linear e quadrática, para encontrar valores aproximados para $y(1.053)$ e $y(1.555)$.

c. Repita (a) e (b) usando agora o Método de Runge-Kutta de quarta ordem. Compare seus resultados.

8. Um foguete de massa $m = 0.11\text{kg}$ é lançado verticalmente com velocidade inicial $v(0) = 8\text{m/seg}$; o movimento é amortecido pela ação da gravidade ($f = mg$, com $g = -9.8\text{m/seg}^2$) e da resistência do ar ($f = -kv|v|$, com $k = 0.002\text{kg/m}$). Pela Lei de Newton, a equação diferencial na velocidade $v(t)$ é $mv' = mg - kv|v|$.

a. Use o Método de Runge-Kutta com $h = 0.1$ para encontrar uma aproximação para a velocidade decorridos 5s;

b. encontre o tempo para o qual o foguete atinge a altura máxima. Qual é esta altura?

9. No Método de Milne de quarta ordem, as aproximações para o PVI $y' = f(x, y)$, $y(0) = y_0$ são calculadas por:

$$y_{i+1} = y_{i-3} + \frac{4h}{3}(2f_i - f_{i-1} + 2f_{i-2}).$$

Observe que se trata de um método de passo múltiplo. Que dados são necessários para iniciarmos o processo de cálculo? Verifique que este método satisfaz a condição da raiz, mas é fracamente estável, uma vez que as raízes do polinômio característico são 1 , -1 , i , e $-i$ ($i^2 = -1$).

10. O PVI $y' = -6y + 6$, $t > 0$ com $y(0) = 2$ tem como solução analítica $y(t) = 1 + e^{-6t}$. Usando como condição inicial os valores exatos de $y(0.1)$, $y(0.2)$ e $y(0.3)$, encontre aproximações para a solução através de:

- a. Método de Adams-Bashforth, apresentado no fim da seção 8.5;
- b. Método de Milne, apresentado no exercício 9.

Embora os dois métodos usados em (a) e (b) sejam de quarta ordem, você vai verificar que os erros são bem diferentes. A que você atribui este fato?

Capítulo 9

EQUAÇÕES DIFERENCIAIS PARCIAIS: MÉTODOS PARA SOLUÇÕES SUAVES

9.1 Introdução

A maioria dos fenômenos físicos é descrita matematicamente através de equações, ou sistemas de equações, que envolvem derivadas parciais da função incógnita. Isto ocorre porque as entidades físicas que procuramos (como, por exemplo, distribuição de pressão ou de temperatura em um meio), são funções de mais de uma variável. Por exemplo, a pressão pode variar ponto a ponto, no meio, e depender do tempo. Neste exemplo, no caso tridimensional teríamos como incógnita uma função $u(x, y, z, t)$.

As taxas de variação destas entidades são representadas por suas derivadas parciais. Se elas estão presentes na modelagem matemática do fenômeno, o resultado poderá ser um conjunto de equações diferenciais parciais.

No vasto campo das equações diferenciais parciais, nos restringiremos às equações diferenciais de primeira e segunda ordens que envolvem funções de duas variáveis independentes, aqui denotadas por x e y ou x e t .

Para efeito de classificação das equações diferenciais parciais lineares de segunda ordem podemos usar a representação geral

$$a u_{xx} + 2b u_{xy} + c u_{yy} + d u_x + e u_y + f u = g$$

onde a, b, c, d, e, f e g são constantes ou funções conhecidas das variáveis independentes x e y .

As funções a, b e c são tais que

$$a^2 + b^2 + c^2 \neq 0,$$

e $u(x, y)$ representa a função procurada.

Se os coeficientes a, b e c são constantes, é útil usá-los numa classificação básica das equações diferenciais parciais lineares de segunda ordem. Assim, em analogia às cônicas, dizemos que a equação é

Elítica	se $b^2 - ac < 0$
Parabólica	se $b^2 - ac = 0$
Hiperbólica	se $b^2 - ac > 0$

Se os coeficientes variam com (x, y) , esta classificação é feita ponto a ponto. Em alguns casos, podemos ter uma das desigualdades verificada em todo o domínio, como veremos nos exemplos.

Classificar equações diferenciais parciais em três grandes grupos tem importância nas aplicações, na análise teórica do problema e na elaboração de algoritmos numéricos convenientes para cada grupo, como veremos neste capítulo.

Esta classificação tem também um respaldo físico, uma vez que as equações diferenciais modelam fenômenos distintos. Por exemplo, os três tipos acima são modelos lineares no estudo do potencial associado ao escoamento estacionário de um fluido incompressível, da difusão do calor num corpo e da propagação de ondas em um meio elástico, respectivamente. Por outro lado, os procedimentos utilizados na análise, tanto matemática quanto numérica, são também distintos nos três tipos catalogados nesta classificação.

Em 1902, o matemático Hadamard introduziu o conceito de *problema bem-posto*: aquele que apresenta exatamente uma solução (existência e unicidade), a qual varia continuamente com os dados (estabilidade). Num certo sentido, este conceito fornece uma indicação de adequação do modelo matemático ao sistema real que se pretende estudar. O fenômeno efetivamente ocorre – assegura-se a existência; a hipótese do determinismo garante a unicidade e a estabilidade exclui a possibilidade de saltos bruscos entre dados e resultados.

A seguir, apresentaremos alguns exemplos de equações diferenciais parciais. Os três primeiros são protótipos das equações catalogadas como elíticas, parabólicas e hiperbólicas. Mais tarde, detalharemos as condições de contorno que auxiliarão na definição do problema bem-posto associado à equação diferencial.

Exemplo 9.1.1. Consideremos a equação diferencial, definida numa região $\mathcal{R}$ do plano, $u(x, y)$ tal que

$$u_{xx} + u_{yy} = f(x, y), \quad (x, y) \in \mathcal{R}. \quad (9.1)$$

Comparando com a representação geral, vemos que $a = c = 1$ e $b = 0$. Assim, (9.1) é uma equação diferencial elítica. Ela é chamada equação do *potencial*, ou equação de Poisson, (Siméon-Denis Poisson, 1781-1840). No caso de $f(x, y) = 0$, é mais comum chamá-la de equação de *Laplace* (Pierre-Simon Laplace, 1749-1827). Funções que satisfazem à equação de Laplace são chamadas *harmônicas*.

A equação de Poisson é o modelo matemático de diversas aplicações: distribuição de tensões em placas carregadas, distribuição de temperatura no caso estacionário, potencial gerado por cargas elétricas etc.

A equação (9.1) será o protótipo das equações diferenciais elíticas. As notações usuais para equação de Poisson são:

$$\Delta u = f(x, y)$$

ou

$$\nabla^2 u = f(x, y),$$

e o operador $\Delta u = u_{xx} + u_{yy}$ é chamado Laplaciano.

No caso tridimensional, o operador Laplaciano será

$$\Delta u = u_{xx} + u_{yy} + u_{zz} = \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} + \frac{\partial^2 u}{\partial z^2}.$$

Para definir um problema bem-posto para (9.1), devemos impor condições para $u(x, y)$ no contorno da região $\mathcal{R}$. Podem-se preestabelecer os valores de u na fronteira, ou em parte da fronteira completando-a com valores da derivada de u . Voltaremos ao assunto quando detalharmos a discretização da equação de Poisson. $\square$

Exemplo 9.1.2. Seja $I = (\alpha, \beta)$ um intervalo na reta e $a(x, t) > 0$, se $x \in I$ e $t > 0$. Consideremos a equação diferencial

$$a(x, y) u_{xx} - u_t = f(x, t), \quad x \in I, t > 0. \quad (9.2)$$

Trocamos a variável y por t , motivados pelos problemas de evolução, nos quais uma das variáveis é o tempo, t . Na simulação numérica destes problemas, a variável tempo merece, em geral, tratamento diferente do dispensado às variáveis espaciais, neste exemplo, x . Isto se deve ao fato da equação ser, na variável tempo, um problema de valor inicial e, na variável (ou variáveis) espacial, um problema de valor de contorno. Novamente a motivação desta diferença está nas aplicações práticas.

Na equação (9.2) $a(x, t) > 0$, $b = c = 0$, portanto como $b^2 - ac = 0$, podemos classificá-la como parabólica. Ela é chamada de equação do calor, ou da difusão, e será o protótipo das equações diferenciais parabólicas. De fato, a equação (9.2) simula uma vasta gama de processos com difusão, inclusive distribuição de temperatura.

As equações do calor para problemas com duas ou três variáveis espaciais são, respectivamente

$$u_t - a(u_{xx} + u_{yy}) = f(x, y, t), \quad (x, y) \in \mathcal{R}, t > 0,$$

e

$$u_t - a(u_{xx} + u_{yy} + u_{zz}) = f(x, y, z, t), \quad (x, y, z) \in \mathcal{R}, t > 0.$$

Nestas equações, $a(\cdot)$ são funções estritamente positivas conhecidas no domínio da equação diferencial, aqui representada por $\mathcal{R}$.

Nos problemas de evolução, as condições de contorno na variável temporal não são adequadas, pois isto seria prefixar o estado do sistema num instante futuro. Neste caso, o que torna o problema bem-posto é estabelecer condições iniciais, isto é, fixamos com antecipação a solução em $t = 0$ na região espacial onde a equação é satisfeita. Entretanto, nos pontos da fronteira de $\mathcal{R}$, para $t > 0$, são impostas condições de contorno semelhantes às do exemplo 9.1.1. Estas condições adicionais, condições iniciais no tempo e condições de contorno nas variáveis espaciais, são ditadas pelo processo físico que queremos descrever. São elas que garantem unicidade de solução. $\square$

Exemplo 9.1.3. Ainda com $a(x, t) > 0$ e $I = (\alpha, \beta)$, tomemos outro problema de evolução, definido pela equação diferencial

$$a(x, t) u_{xx} - u_{tt} = f(x, t), \quad x \in I, t > 0. \quad (9.3)$$

Como $a(x, t) > 0$, $b = 0$ e $c = -1$, $b^2 - ac > 0$ e, portanto, temos o protótipo das equações hiperbólicas.

A equação (9.3) é o modelo matemático para o fenômeno de propagação de ondas e por isso ela é chamada de *equação da onda*. Por exemplo, no caso de uma variável espacial, a solução $u(x, t)$ representaria a posição de uma partícula da corda vibrante, no instante t .

Substituindo-se u_{xx} da equação (9.3) pelo operador Laplaciano, teremos as equações de ondas no plano e no espaço tridimensional.

Como nas equações hiperbólicas a função incógnita aparece sob duas derivadas com relação à variável tempo, para torná-las um problema bem-posto são necessárias duas condições iniciais, $u(x, 0)$ e $u_t(x, 0)$. Também para que o problema seja bem-posto, são necessárias condições de contorno nas variáveis espaciais (que podem envolver valores da função incógnita ou suas derivadas no contorno), além das condições iniciais. $\square$

A classificação nos três tipos acima é importante porque ela respeita a diferença estrutural das equações diferenciais parciais. Entretanto ela nem de longe esgota o assunto. Os modelos matemáticos que simulam fenômenos reais se tornam mais complexos à medida que o desenvolvimento científico e tecnológico caminha a passos rápidos. Os dois exemplos que seguem ilustram este fato. No primeiro, a caracterização do tipo de equação diferencial depende dos valores assumidos por parâmetros presentes na equação. No segundo exemplo não-linearidades estão presentes, e o modelo é descrito por um sistema de diferentes tipos de equações.

Exemplo 9.1.4. O exemplo 9.1.2 é um modelo matemático para o efeito da difusão em processos físicos. Além do fluxo provocado pela difusão pode-se também considerar o transporte de substâncias devido à velocidade do meio. Por exemplo, a temperatura de um meio é influenciada pela velocidade do ar, o vento. Neste caso, um termo convectivo, u_x , também aparece na equação diferencial. Uma equação diferencial que considere os efeitos difusivo e convectivo se escreve, na forma adimensionalizada:

$$u_t = \frac{1}{P_e} u_{xx} - u_x. \quad (9.4)$$

Nesta equação, P_e é o número de Péclet e representa a razão entre o transporte convectivo e o difusivo.

A equação acima é conhecida como *convecção-difusão*. Quando P_e é pequeno, a equação se comporta como parabólica, na qual a difusão predomina. Por outro lado, se $P_e \rightarrow \infty$, ela pode ser olhada como uma perturbação de uma equação hiperbólica de primeira ordem. Como os métodos numéricos convenientes para cada um destes casos são distintos, a solução

numérica da equação de convecção-difusão é mais elaborada do que a solução numérica da equação de difusão.

No caso de coeficientes constantes, como neste exemplo, pode-se encontrar solução analítica aplicando a transformada de Laplace ou uma mudança de variáveis que transforma a equação convecção-difusão numa equação puramente difusiva, a equação do calor [Lake, 1989, Farlow, 1982]. $\square$

Exemplo 9.1.5. As simulações em reservatórios petrolíferos são descritas por equações diferenciais com fortes não-linearidades. Por exemplo, as equações que governam o fluxo de água-óleo incompressível num meio poroso formam um sistema de equações diferenciais parciais

$$\begin{aligned}\phi s_t + \nabla \cdot \vec{v} f(s) &= 0 \\ \nabla \cdot \vec{v} &= 0 \\ \vec{v} &= -K\lambda \nabla p,\end{aligned}\tag{9.5}$$

onde $s = s(x, t)$ é a fração de volume de água, $\phi(x)$ é a porosidade, $\vec{v}(x, t)$ é a velocidade total de fluido (água e óleo), $p(x, t)$ é a pressão, $K(x)$ é o tensor permeabilidade da rocha, $\lambda = \lambda(s)$ é a mobilidade do fluido e $f(s)$, chamado fluxo fracionário do fluido, relaciona a velocidade da fase água à velocidade total do fluido. Neste sistema, denotamos por $\nabla \cdot \vec{v}$ o divergente do vetor velocidade e por ∇p o gradiente da pressão, o que permite a simulação em uma, duas ou três dimensões.

Temos então um exemplo de um modelo matemático descrito por três equações diferenciais. A primeira representa a conservação de fluido, a segunda a incompressibilidade e a terceira é a Lei de Darcy. As incógnitas do sistema são as funções $s(x, t)$, $v(x, t)$ e $p(x, t)$, e as não-linearidades estão presentes nas primeira e terceira equações. $\square$

Como exemplo final, apresentaremos as leis de conservação.

Exemplo 9.1.6. A utilização dos princípios físicos de conservação (de massa, energia ou momentum), na modelagem de processos físicos, dão origem a uma classe especial de equações diferenciais parciais chamadas *leis de conservação*. No caso de uma variável espacial, a forma geral das leis de conservação é:

$$u_t + f(u)_x = 0,\tag{9.6}$$

onde $u = u(x, t)$ é a quantidade conservada, ou variável de estado e $f(u)$ representa o fluxo desta variável no ponto (x, t) . Por exemplo, se chamamos $v(x, t)$ a velocidade um fluido, no ponto x e no instante t , aplicando o princípio da conservação de massa teremos:

$$\rho_t + (\rho v)_x = 0,$$

onde $\rho(x, t)$ representa a densidade deste fluido, no ponto x e instante t . Em geral, não é possível obter soluções analíticas para leis de conservação e os métodos para resolvê-las formam um capítulo à parte dos métodos numéricos.

Usando a regra da cadeia na equação (9.6), podemos reescrevê-la na forma não conservativa

$$u_t + f'(u)u_x = 0.$$

□

Neste capítulo, apresentamos alguns métodos numéricos para equações diferenciais cujas soluções são suaves. No próximo capítulo, abordaremos alguns métodos introdutórios para o caso de soluções com descontinuidades, como é o caso das leis de conservação não-lineares.

9.2 Definição da Malha e Discretização do Laplaciano

Na construção de soluções aproximadas para equações diferenciais parciais, seguiremos o mesmo caminho percorrido quando trabalhamos com o método de diferenças finitas nas equações diferenciais ordinárias. Assim, em primeiro lugar discretizamos o domínio onde a equação diferencial é definida. Com isso, definimos uma malha sobre a qual será calculada a solução aproximada.

Para simplificar, apresentaremos apenas o caso bidimensional. Entretanto, a generalização para o caso tridimensional é imediata, exigindo apenas um pouco de cuidado na notação, uma vez que o número de variáveis é maior.

Seja $\mathcal{R}$ o conjunto dos pontos (x, y) onde a equação diferencial está definida. Chamamos S o contorno deste conjunto.

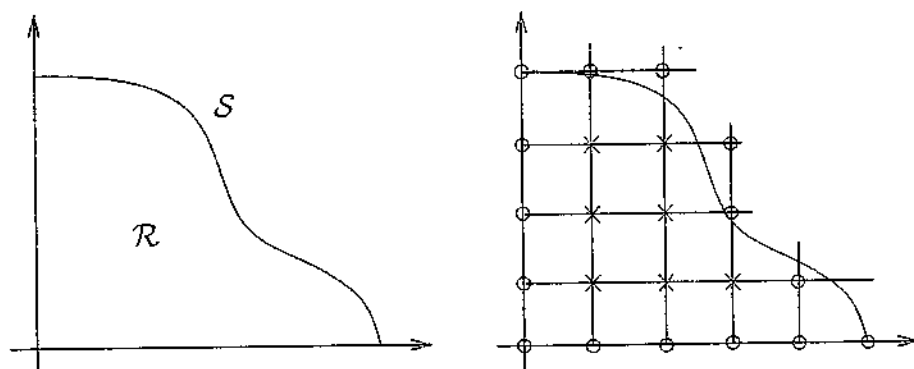
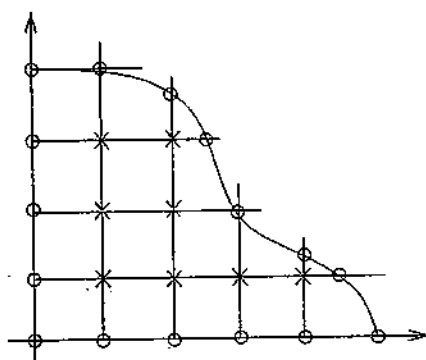
Tomemos inicialmente uma malha de pontos igualmente espaçados descritos por:

$$\mathcal{R}_h = \{(x_i, y_i) = (\underline{x} + ih, \underline{y} + jh), \quad i, j = \pm 1, \pm 2, \dots\},$$

onde $(\underline{x}, \underline{y})$ é um ponto de referência arbitrário, mas fixo, do plano, e h é uma constante positiva chamada passo da malha. A figura 9.1 ilustra o que estamos chamando de malha: os pontos representados por xis ou bolinhas na figura à direita.

De saída, como ilustra a figura, temos um problema de adaptação da malha ao contorno S . Dependendo da região, a malha $\mathcal{R}_h$ pode não conter os elementos de S . Assim, torna-se necessário definir o contorno de $\mathcal{R}_h$. Uma das alternativas é aproximar $\mathcal{R}$ de boa maneira por $\mathcal{R}_h$ anexando ou desprezando pedaços de $\mathcal{R}$. Por exemplo, podemos considerar S_h como o conjunto dos pontos representados por bolinhas na Figura 9.1 à direita e $\mathcal{R}_h$ o conjunto dos pontos representados por xis, nesta figura.

Outra alternativa seria considerar uma malha de tamanho variável e incorporar as condições de contorno nas interseções das retas paralelas aos eixos coordenados que passam por $\underline{x} + ih$ e $\underline{y} + jh$ com a fronteira S . Assim, S_h seria composto pelos pontos representados

Figura 9.1: Discretização da região $\mathcal{R}$.Figura 9.2: Discretização variável da região $\mathcal{R}$.

por bolinhas na Figura 9.2. Mais tarde, definiremos a discretização com malha variável associada a esta alternativa.

Na seção anterior, observamos que os três protótipos apresentados, equações de potencial, do calor e da onda, têm em comum o operador laplaciano

$$\Delta u = u_{xx} + u_{yy}.$$

Por este motivo, apresentamos inicialmente formas de discretização do operador laplaciano.

O método de diferenças finitas apresentado no capítulo anterior pode ser usado na aproximação de derivadas parciais. Por exemplo, usando a série de Taylor temos:

$$u(x+h, y) = u(x, y) + u_x(x, y)h + u_{xx}(x, y)\frac{h^2}{2} + u_{xxx}(\alpha, y)\frac{h^3}{3!}$$

$$u(x-h, y) = u(x, y) - u_x(x, y)h + u_{xx}(x, y)\frac{h^2}{2} - u_{xxx}(\beta, y)\frac{h^3}{3!},$$

combinando estas expressões, obtemos

$$\begin{aligned} u_x(x, y) &\simeq \frac{u(x+h, y) - u(x, y)}{h} && \text{com erro } \mathcal{O}(h) \\ u_x(x, y) &\simeq \frac{u(x, y) - u(x-h, y)}{h} && \text{com erro } \mathcal{O}(h) \\ u_{xx}(x, y) &\simeq \frac{u(x+h, y) - 2u(x, y) + u(x-h, y)}{h^2} && \text{com erro } \mathcal{O}(h^2). \end{aligned}$$

Usando mais um termo na série de Taylor, temos também a fórmula centrada para a primeira derivada,

$$u_x(x, y) \simeq \frac{u(x+h, y) - u(x-h, y)}{2h} \quad \text{com erro } \mathcal{O}(h^2).$$

As derivadas parciais com relação à variável y são obtidas analogamente tomando agora um acréscimo $\Delta y = k$. Por conveniência, vamos usar a seguinte notação para os pontos da malha:

$$\begin{aligned} u(x_i, y_j) &\cong u_{ij} && u(x_i + h, y_j) \cong u_{i+1,j} \\ u(x_i - h, y_j) &\cong u_{i-1,j} && u(x_i, y_j + k) \cong u_{i,j+1} \\ u(x_i, y_j - k) &\cong u_{i,j-1} \end{aligned}$$

Nesta notação

$$u_{xx}(x_i, y_j) \simeq \frac{u_{i+1,j} - 2u_{ij} + u_{i-1,j}}{h^2}$$

e

$$u_{yy}(x_i, y_j) \simeq \frac{u_{i,j+1} - 2u_{ij} + u_{i,j-1}}{k^2}.$$

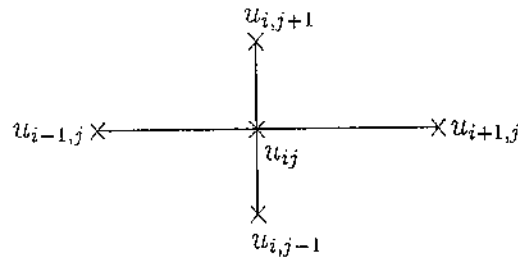
A maneira mais simples de discretizar o laplaciano é usar $\Delta x = \Delta y = h$, isto é, uma malha uniforme e igualmente espaçada nas duas direções. Neste caso, teremos:

$$\Delta u|_{(x_i, y_j)} = \frac{u_{i+1,j} - 2u_{ij} + u_{i-1,j}}{h^2} + \frac{u_{i,j+1} - 2u_{ij} + u_{i,j-1}}{h^2} + \mathcal{O}(h^2)$$

ou

$$\Delta u|_{(x_i, y_j)} \cong \frac{1}{h^2} \{u_{i+1,j} + u_{i-1,j} + u_{i,j+1} + u_{i,j-1} - 4u_{ij}\}. \quad (9.7)$$

Esta discretização envolve, em cada ponto da malha, além do valor da solução neste ponto, os valores que a solução assume nos quatro pontos cardeais, como ilustra o desenho a seguir.



Observando (9.7), verificamos que, se $\Delta u(x_i, y_i) = 0$ então u_{ij} é a média dos seus quatro vizinhos. De fato, se $u(x, y)$ é a solução da equação de Laplace $\Delta u = 0$, então $u(x, y)$ é a média de seus vizinhos, qualquer que seja (x, y) no domínio de $u(x, y)$.

Uma segunda opção para discretização é a escolha de uma malha com pontos igualmente espaçados em cada direção. Assim, tomando $\Delta x = h_1$ e $\Delta y = h_2$, nas aproximações da segunda derivada, temos:

$$\Delta u|_{x_i, y_j} = \frac{1}{h_1^2}(u_{i+1,j} - 2u_{ij} + u_{i-1,j}) + \frac{1}{h_2^2}(u_{i,j+1} - 2u_{ij} + u_{i,j-1}) + \mathcal{O}(h_1^2 + h_2^2). \quad (9.8)$$

Uma opção mais geral é usar uma malha irregular. Neste caso, tomaríamos pontos (x_i, y_j) de modo que $x_{i+1} - x_i = h_i$ e $y_{j+1} - y_j = h_j$. Observe que, para obtermos a nova expressão para a derivada de segunda ordem, usamos a expansão em Taylor nas duas direções, isto é, $u(x + h_1)$ e $u(x - h_2)$ (ver figura 9.3) que, convenientemente adicionadas, nos darão:

$$u_{xx}(x_1) = 2 \frac{h_2 u_2 - (h_1 + h_2) u_1 + h_1 u_0}{h_2 h_1 (h_2 + h_1)} + (h_2 - h_1) u_{xxx}(x_1) + \frac{h_1^3 \alpha + h_2^3 \beta}{h_1 + h_2},$$

onde α e β são valores que dependem de x_1 , h_1 e h_2 .

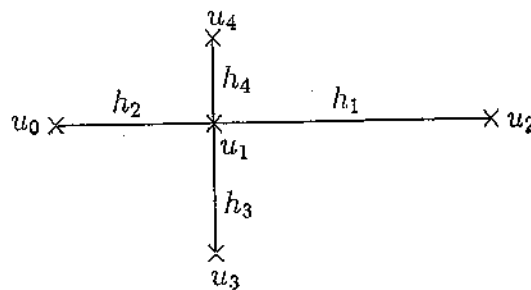


Figura 9.3: Discretização variável em detalhe.

Usando a expressão análoga para aproximar u_{yy} , teremos finalmente a discretização do

laplaciano numa malha irregular:

$$u_{xx} + u_{yy} \cong 2 \left\{ \frac{u_0}{h_2(h_1 + h_2)} + \frac{u_2}{h_1(h_1 + h_2)} + \frac{u_3}{h_4(h_3 + h_4)} + \frac{u_4}{h_3(h_3 + h_4)} - \left(\frac{1}{h_1 h_2} + \frac{1}{h_3 h_4} \right) u_1 \right\}. \quad (9.9)$$

Uma discretização deste tipo pode ser usada nos pontos próximos à fronteira, para um melhor ajuste da malha ao domínio. Observe também que, além desta fórmula ser bem mais complicada que aquela obtida quando a malha é uniforme nas duas direções, houve outra perda: enquanto (9.7) fornece uma aproximação com erro da ordem de h^2 , em (9.9) o erro será da ordem de $h = \min\{h_1, h_2, h_3, h_4\}$. Observe também que, no caso $h_1 = h_2 = h_3 = h_4$, as expressões (9.9) e (9.8) se reduzem a (9.7), como não poderia deixar de ser.

9.3 Equação de Poisson com Dados de Contorno

Como foi comentado na primeira seção deste capítulo, problemas envolvendo equações diferenciais e dados adicionais são bem-postos se podemos garantir (i) existência de solução, (ii) unicidade desta solução, (iii) que pequenas perturbações nos dados de entrada acarretem pequenas variações na solução (estabilidade). Para que tenhamos uma formulação matemática de um processo físico, é importante que as condições (i)–(iii) sejam satisfeitas.

Nas aplicações de métodos numéricos que apresentamos nesta notas, supomos que o problema satisfaça a estes critérios de bom comportamento. Entretanto, estudos mais recentes têm mostrado que existem muitas situações das ciências naturais que não satisfazem a uma (ou mais) destas condições. Em geral, este são os problemas malpostos ou problemas inversos.

Consideremos a equação de Poisson com dados no contorno. Queremos encontrar $u(x, y)$ tal que

$$\begin{aligned} \Delta u &= u_{xx} + u_{yy} = f(x, y), & \text{ numa região } \mathcal{R} \text{ do plano} \\ u &= g(x, y), & \text{ em } S, \text{ fronteira de } \mathcal{R}. \end{aligned} \quad (9.10)$$

Em outras palavras, procuramos uma solução para a equação de Poisson, preestabelecendo seus valores no contorno da região considerada.

Vários matemáticos demonstraram, por diferentes métodos analíticos, que este problema é bem-posto (ver, por exemplo [Courant-Hilbert, 1962]). Entretanto a determinação de $u(x, y)$ para uma região genérica é complicada. Para algumas regiões particulares (retângulo e círculos, por exemplo), é possível obter expressões analíticas (por via da série de Fourier ou integral de Poisson) para a solução. Mesmo nestes casos, nem sempre as soluções apresentadas são calculáveis, porque as séries podem convergir muito lentamente;

em outros formatos da solução, podem ser necessárias integrações impraticáveis. Diante destas dificuldades, os algoritmos numéricos se tornaram importantes na solução efetiva do problema.

Na seção anterior, encaminhamos o que é a essência da solução aproximada da equação de Poisson: a discretização do laplaciano. Para sistematizar o procedimento geral, usamos os seguintes passos:

- Estabelecer a malha, definindo quais são os pontos da fronteira.
- Escolher a discretização do laplaciano, para pontos que não sejam da fronteira. Aqui usamos uma das expressões da seção anterior.
- Montar o sistema de equações lineares usando a discretização nos pontos interiores, $f(x_i, y_j)$, e as condições de contorno nos pontos da fronteira.
- Resolver o sistema obtido no passo anterior, usando algum método numérico para obter $u_{ij} \cong u(x_i, y_j)$, que são as aproximações nos pontos interiores.

A seguir, apresentaremos um exemplo que deve ajudar na compreensão deste procedimento.

Exemplo 9.3.1. Vamos considerar (9.10), na qual $\mathcal{R}$ é um retângulo com um de seus vértices na origem do sistema de coordenadas cartesianas. As arestas dos eixos coordenadas têm comprimentos L_x e L_y , nos eixos x e y respectivamente. Assim,

$$\mathcal{R} = \{(x, y) / 0 < x < L_x \text{ e } 0 < y < L_y\}$$

e a fronteira é formada pelas arestas deste retângulo.

A figura 9.6 ilustra a malha que vamos usar na discretização de $\mathcal{R}$. Vamos usar $N + 2$ pontos na direção x e $M + 2$ pontos na direção y . Assim,

$$h_x = \Delta x = \frac{L_x}{N + 1} \quad \text{e} \quad h_y = \Delta y = \frac{L_y}{M + 1}.$$

Valores diferentes para h_x e h_y nos permitem refinamentos adequados à situação que pretendemos simular. Em alguns casos, uma direção deve ser olhada com mais detalhes e, portanto devemos escolher valores menores de h naquela direção.

Como estamos considerando a condição de contorno definida em (9.10), os valores da solução nas arestas do retângulo são conhecidas. Procuramos os valores da solução nos pontos $1, 2, \dots, MN$, ordenados como ilustra a figura 9.6. Nesta ordenação das incógnitas definimos o vetor solução, com barras verticais separando blocos de N elementos:

$$u = (u_{11}, u_{21}, \dots, u_{N1} \mid u_{12}, u_{22}, \dots, u_{N2} \mid \dots \mid u_{1M}, u_{2M}, \dots, u_{NM}).$$

Na discretização do laplaciano, vamos usar a expressão (9.8). Assim, a equação discretizada num ponto arbitrário (x_i, y_j) é

$$\frac{u_{i-1,j} - 2u_{ij} + u_{i+1,j}}{h_x^2} + \frac{u_{i,j-1} - 2u_{ij} + u_{i,j+1}}{h_y^2} = f_{ij}, \quad (9.11)$$

ou ainda,

$$h_x^2 u_{i,j-1} + h_y^2 u_{i-1,j} - 2(h_x^2 + h_y^2) u_{ij} + h_y^2 u_{i+1,j} + h_x^2 u_{i,j+1} = h_x^2 h_y^2 f_{ij}. \quad (9.12)$$

Para escrever o sistema de equações lineares associado à discretização, é conveniente escrever o vetor solução usando só um índice. Na transformação do índice duplo, u_{ij} , num único índice, v_k , observamos a ordenação dos cinco pontos usados num ponto genérico, ilustrado na figura 9.4.

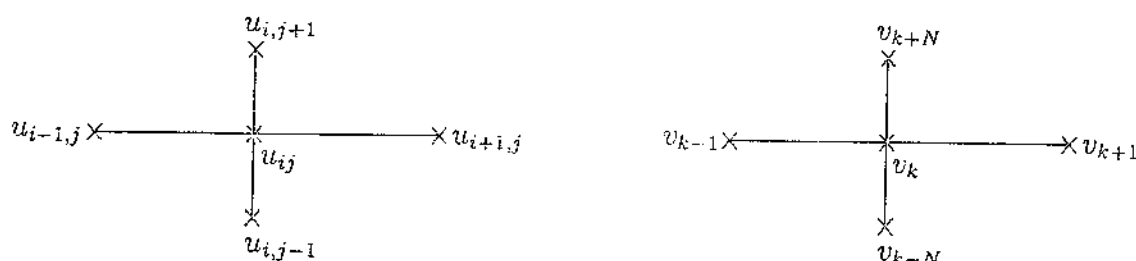


Figura 9.4: Mudança de índices.

Assim, a equação (9.12) é reescrita na variável v_k como:

$$h_x^2 v_{k-N} + h_y^2 v_{k-1} - 2(h_x^2 + h_y^2) v_k + h_y^2 v_{k+1} + h_x^2 v_{k+N} = h_x^2 h_y^2 f_{ij}, \quad (9.13)$$

para $1 \leq k \leq NM$.

Chamando $A = [a_{ij}]$ a matriz do sistema linear, sua linha genérica, não consideradas as condições de contorno, será:

$$\begin{aligned} a_{ii} &= -2(h_x^2 + h_y^2) \\ a_{i,i-1} &= a_{i,i+1} = h_y^2 \\ a_{i,i-N} &= a_{i,i+N} = h_x^2. \end{aligned}$$

A matriz A tem NM linhas e NM colunas, e sua estrutura é pentadiagonal, como ilustra a figura 9.5.

Até aqui, as condições de contorno do problema não foram incorporadas ao sistema de equações lineares. Vamos modificar algumas das equações (9.13) de modo a incorporar a condição de contorno definida em (9.10).

[illegible]

Figura 9.5: Matriz pentadiagonal.

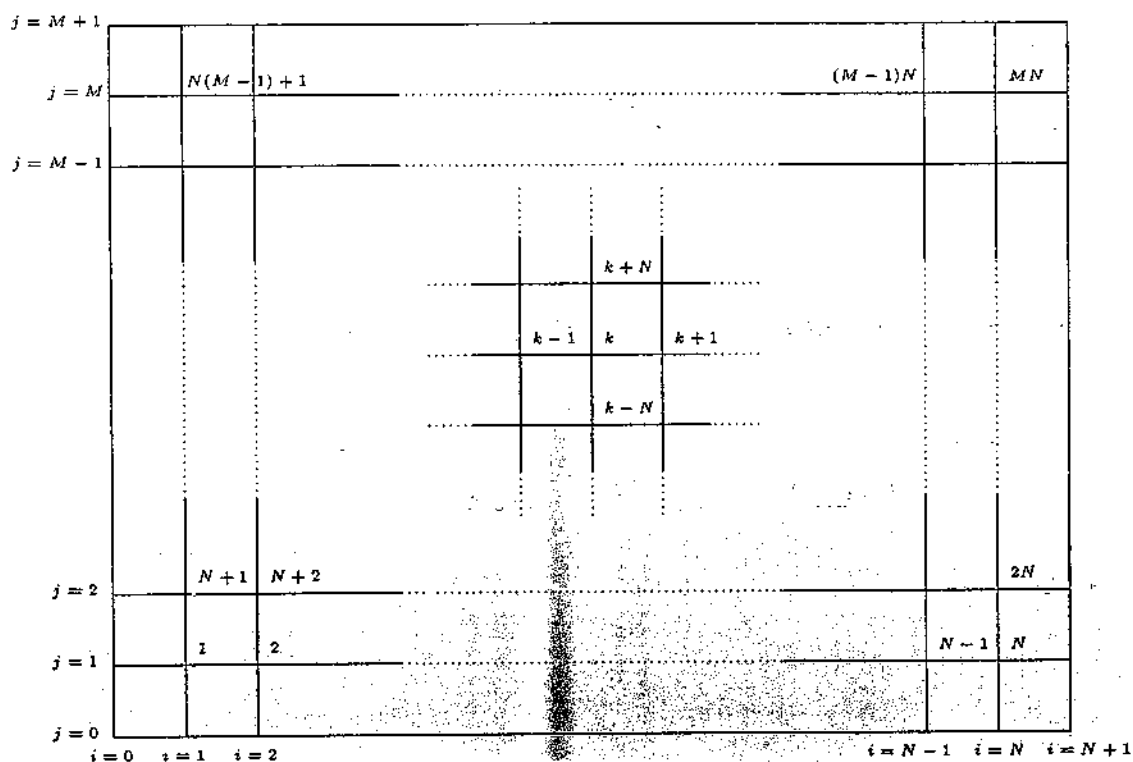


Figura 9.6: Discretização do domínio.

Observe que a condição de contorno deve aparecer nas equações correspondentes aos nós da malha que estão vizinhos à fronteira da região onde está definida a equação diferencial. Sendo assim, vamos incorporar valores de contorno na equação (9.13), quando

$$\begin{aligned} k &= 1:N \\ k &= jN \text{ e } k = (j+1)N, \quad \text{para } j = 2:M-2 \\ k &= (M-1)N+1:MN \end{aligned}$$

onde, pela convenção adotada, $k = 1 : N$ significa $k = 1, 2, \dots, N$.

A condição de contorno relativa às arestas $y = 0$ e $y = L_y$ podem ser incorporadas usando um vetor cujos elementos não-nulos estão nas N primeiras posições e nas N últimas posições:

$$\mathbf{b}_y = h_x^2 (g_{1,0}, g_{2,0}, \dots, g_{N,0} \mid \underbrace{0, \dots, 0}_{N \text{ elementos}} \mid \dots \mid g_{1,M+1}, g_{2,M+1}, \dots, g_{N,M+1})^T.$$

Neste vetor, na notação usual,

$$g_{ij} = g(x_i, y_j),$$

e a barra vertical separa blocos com N elementos do vetor.

Por outro lado, as condições de contorno correspondentes às arestas $x = 0$ e $x = L_x$ podem ser incorporadas ao sistema por meio do vetor

$$\mathbf{b}_x = h_y^2 (g_{0,1}, 0, \dots, 0, g_{N+1,1} \mid g_{0,2}, 0, \dots, 0, g_{N+1,2} \mid \dots \mid g_{0,M}, 0, \dots, 0, g_{N+1,M})^T.$$

Sendo assim, devemos suprimir os termos das equações (9.13) que correspondam às condições de contorno. Por exemplo, na equação correspondente a $k = 1$, os dois primeiros termos devem ser suprimidos e ficamos com

$$-2(h_x^2 + h_y^2) v_1 + h_y^2 v_2 + h_x^2 v_{N+1} = h_x^2 h_y^2 f_{11} - h_x^2 g_{10} - h_y^2 g_{01}.$$

De fato, observamos pela figura 9.6 que a discretização correspondente ao nó 1 deve ter como incógnitas v_1, v_2 e v_{N+1} já que os valores ao sul e oeste deste nó são fornecidos pela condição de contorno. Correção análoga deve ser feita na equação NM .

Para completar o sistema de equações lineares, tomemos o vetor que representa a função $f(x, y)$ de (9.10):

$$\mathbf{f} = h_x^2 h_y^2 (f_{11}, f_{21}, \dots, f_{N1} \mid f_{12}, f_{22}, \dots, f_{N2} \mid \dots \mid f_{1M}, f_{2M}, \dots, f_{NM})^T.$$

Assim, o sistema linear se escreve como

$$\mathbf{A}\mathbf{v} = \mathbf{f} - \mathbf{b}_x - \mathbf{b}_y,$$

onde $\mathbf{A}$ é uma matriz pentadiagonal com alguns elementos nulos nas diagonais secundárias. A figura 9.7 ilustra a estrutura da matriz $\mathbf{A}$. Vemos que $\mathbf{A}$ é uma matriz esparsa, com menos de $5MN$ elementos não-nulos num total de $(MN)^2$ elementos. Esta matriz pode ser dividida em M^2 blocos, sendo cada qual uma submatriz com N^2 elementos. Os blocos que estão nas diagonais, bloco-diagonais, são matrizes tridiagonais. Seus vizinhos são matrizes diagonais e todos os outros são nulos.

O algoritmo 9.1 indica como são os cálculos para compor o sistema linear associado à discretização da equação de Poisson (9.10) em um retângulo. $\square$

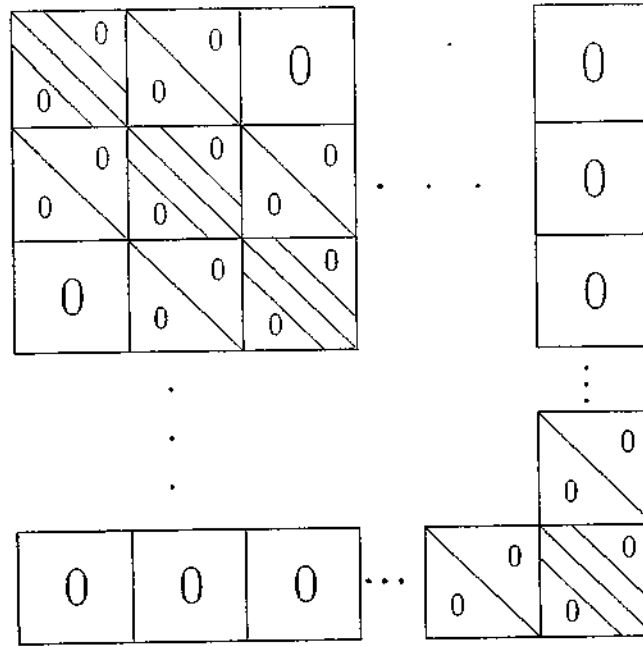


Figura 9.7: Estrutura da matriz A.

Algoritmo 9.1 Discretização da equação de Poisson

Dados $N, M, L_x, L_y, f(x, y)$ e $g(x, y)$

1: $h_x = \frac{L_x}{N+1}$ e $h_y = \frac{L_y}{M+1}$

2: Para $k = 1 : MN$, faça $a_{kk} = -2(h_x^2 + h_y^2)$ {Diagonal principal}

3: Para $k = 1 : MN - 1$, faça $a_{k,k+1} = h_y^2$ {Diagonal superior}

4: Para $k = 2 : MN$, faça $a_{k,k-1} = h_y^2$ {Diagonal inferior}

5: Para $k = 1 : (M-1)N$, faça $a_{k,k+N} = h_x^2$ {Diagonal superior afastada}

6: Para $k = N+1 : MN$, faça $a_{k,k-N} = h_x^2$ {Diagonal inferior afastada}

7: $k = 0$ {condição de contorno e fonte}

8: Para $j = 1 : M$, faça

9: Para $i = 1 : N$, faça

10: $k = k + 1$; $b_k = h_x^2 h_y^2 f(x_i, y_j)$

11: Se $i = 1$, faça $b_k = b_k - h_y^2 g(0, y_j)$, se $j > 1$, $a(k, k-1) = 0$

12: Se $i = N$, faça $b_k = b_k - h_y^2 g(L_x, y_j)$, se $j < M$, $a(k, k+1) = 0$

13: Se $j = 1$, faça $b_k = b_k - h_x^2 g(x_i, 0)$

14: Se $j = M$, faça $b_k = b_k - h_x^2 g(x_i, L_y)$

Exemplo 9.3.2. Para ilustrar o procedimento que descrevemos no exemplo 9.3.1, vamos considerar a solução numérica, calculada pelo método de diferenças finitas, da equação de Poisson com dados de contorno. Seja

$$\mathcal{R} = \{(x, y) / 0 < x < 1 \text{ e } 0 < y < 1/2\}$$

e a equação de Poisson com condições de contorno do tipo Dirichlet e condição inicial:

$$\Delta u = u_{xx} + u_{yy} = -2 \operatorname{sen} \pi x \cos \pi y$$

$$u(0, y) = u(1, y) = u(x, 1/2) = 0$$

$$u(x, 0) = \frac{\operatorname{sen} \pi x}{\pi^2}$$

O programa `poisson.m` foi elaborado para resolver este problema no Matlab, usando uma malha com N pontos na direção x e M pontos na direção y . O programa segue o Algoritmo 9.1 e está listado no Anexo deste livro. A solução aproximada associada à malha preestabelecida pode ser comparada com a solução analítica que neste exemplo é conhecida:

$$u_e(x, y) = \frac{\operatorname{sen}(\pi x) \cos(\pi y)}{\pi^2}.$$

Rodando o programa para $M = 9$ e $N = 9$, o erro da aproximação obtida nos pontos da malha, $u(x_i, y_j)$ é

$$\left\{ \sum_{i,j} [u_e(x_i, y_j) - u(x_i, y_j)]^2 \right\}^{\frac{1}{2}} = 0.89 \times 10^{-3}$$

ou

$$\max_{i,j} |u_e(x_i, y_j) - u(x_i, y_j)| \leq 0.001.$$

Os gráficos das soluções analítica e aproximada também são fornecidos pelo programa `poisson.m`, apresentado no Anexo. $\square$

A distribuição dos elementos não-nulos da matriz associada à discretização do laplaciano depende da enumeração adotada na malha.

Por exemplo, tomemos as duas enumerações da região ilustrada na figura 9.8.

	6	7	8	9	10
	1	2	3	4	5

(a)

	2	4	6	8	10
	1	3	5	7	9

(b)

Figura 9.8: Numerações para $\mathcal{R}$.

Os sistemas lineares associados à discretização do laplaciano, nas malhas da figura 9.8, envolvem 10 incógnitas, isto é as matrizes A são 10×10 . Entretanto a distribuição dos elementos não-nulos nestas matrizes é diferente. De fato, na enumeração da figura 9.8 (a), a distribuição dos elementos não-nulos na matriz está representada na figura 9.9 (a), onde as retas significam diagonais onde se localizam os elementos não-nulos. Na figura 9.9 (b), representamos a distribuição dos elementos não-nulos da matriz no caso da enumeração adotada em 9.8 (b).

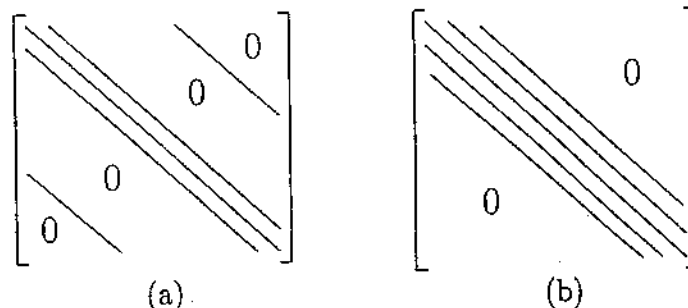


Figura 9.9: Resultado das diferentes numerações.

Como vimos no capítulo 2, as matrizes nas quais os elementos não-nulos concentram-se numa faixa em torno da diagonal principal são chamadas *matrizes de banda*. Estas matrizes oferecem vantagens computacionais em relação àquelas que, tendo a mesma quantidade de elementos nulos, estes aparecem de forma desordenada. De fato, podemos observar que, aplicado o método de eliminação na matriz da figura 9.9 (a), teríamos um “enchimento” (*fill-in* no inglês) da faixa que inicialmente só tinha elementos nulos, o que acarretará mais operações em cada passo subsequente da eliminação. Por outro lado, a matriz da figura 9.9 (b) poderá ser armazenada em uma matriz de 5×10 posições e no processo de eliminação não haverá “enchimento” com elementos não-nulos.

Em resumo, nosso comentário visa alertar para a escolha, na enumeração dos pontos da malha, de modo que os “vizinhos” do domínio correspondam aos “vizinhos” na enumeração. Esta escolha nos levará a um sistema com maior concentração em torno da diagonal principal, evitando o “enchimento”.

Comentários sobre Condições de Contorno

A condição de contorno adotada em (9.10) é chamada *condição de Dirichlet* (Gustav Lejeune Dirichlet, 1805-1859). A equação de Poisson pode ser olhada como o modelo matemático para o estado estacionário de distribuição de temperatura numa região plana. Neste caso, a função $f(x,y)$ representa uma fonte interna de geração de calor. Com a condição de contorno do tipo Dirichlet, estamos admitindo que a temperatura é conhecida na fronteira da região em estudo. Assim, a equação (9.10) amplia para o caso bidimensional o cálculo da distribuição estacionária de temperatura numa haste longa (na qual a

dimensão comprimento é muito maior que as dimensões da seção reta da haste), quando são conhecidas as temperaturas nos extremos da haste.

Mudando as condições de contorno, a equação de Poisson pode simular outras situações. Por exemplo, podemos modelar o caso em que parte da fronteira está imersa em um líquido com temperatura conhecida. Neste caso, a lei experimental de Fourier (Joseph Fourier, 1768-1830) nos diz que o fluxo de calor através da fronteira é proporcional à derivada da temperatura na direção normal à fronteira.

Outra possibilidade é preestabelecer o fluxo de calor na fronteira, que corresponde a substituir a segunda condição de (9.10) pela condição de contorno envolvendo a derivada normal

$$\frac{\partial u}{\partial \eta}(x, y) = g(x, y). \quad (9.14)$$

Observe que, se a região estiver termicamente isolada então, $g(x, y) = 0$. Condições de contorno que envolvem a derivada normal à fronteira são chamadas condições de Neumann (John von Neumann, 1903-1957). Podemos englobar os dois tipos de condições de contorno em uma condição mista

$$\alpha u(x, y) + \beta \frac{\partial u}{\partial \eta}(x, y) = g(x, y). \quad (9.15)$$

onde α e β , que podem também ser funções de (x, y) , devem satisfazer à

$$\alpha^2 + \beta^2 \neq 0$$

para garantir que alguma condição de contorno seja fornecida.

Para exemplificar, denotando por $S = \Gamma_1 \cup \Gamma_2$ o contorno da região $\mathcal{R}$, ilustrada na figura 9.10, procuramos $u(x, y)$ tal que

$$\begin{cases} \Delta u = f(x, y) & \text{em } \mathcal{R} \\ u = g_1(x, y) & \text{em } \Gamma_1 \\ \frac{\partial u}{\partial \eta} = g_2(x, y) & \text{em } \Gamma_2 \end{cases}$$

Neste caso, na discretização do laplaciano, a condição em Γ_1 é introduzida de maneira análoga ao que fizemos no início desta seção. Mas precisamos também simular a derivada normal de $u(x, y)$ na região Γ_2 . Se trabalharmos em uma região retangular, isto não oferece dificuldades, posto que a derivada normal será na direção x , ou na direção y , e sabemos como discretizá-la. Em situações mais gerais, precisamos escolher os elementos da malha que podem ser usados na discretização adequada da derivada normal.

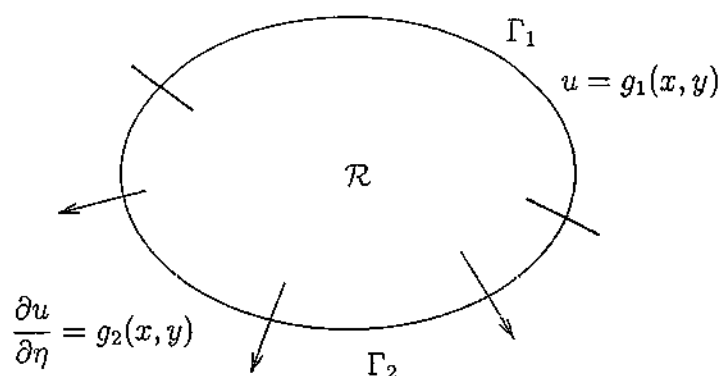


Figura 9.10: Condição de contorno mista.

Finalizamos esta seção com algumas palavras sobre equações diferenciais elíticas não-lineares. Existem modelos matemáticos que envolvem equações diferenciais elíticas não-lineares na solução $u(x, y)$. Como exemplo, as três equações diferenciais parciais não-lineares apresentadas a seguir simulam matematicamente os fenômenos físicos designados à direita:

$u_{xx} + u_{yy} = e^u$	equação de radiação
$u_{xx} + u_{yy} = u^2$	equação da interação molecular
$(1 + u_y^2) u_{xx} - 2u_x u_y u_{xy} + (1 + u_x^2) u_{yy} = e^u$	equação da película de sabão.

Nestes exemplos, as duas primeiras são equações diferenciais do tipo

$$u_{xx} + u_{yy} = F(x, y, u), \quad (9.16)$$

que são consideradas não-linearidades moderadas. Equações diferenciais que são lineares nos termos das derivadas de maior ordem são chamadas semilineares. Neste caso particular, se em (9.16)

$$\frac{\partial F}{\partial u} \geq 0,$$

pode-se mostrar que, sob condições de contorno adequadas, existe uma única solução para o problema; veja [Courant-Hilbert, 1962] para maiores detalhes.

Para obter soluções aproximadas de equações do tipo (9.16), podemos aplicar uma das discretizações propostas na seção anterior para o laplaciano. Por exemplo, na equação da radiação teríamos, para cada (x_i, y_j) da malha,

$$\frac{u_{i-1,j} - 2u_{ij} + u_{i+1,j}}{\Delta x^2} + \frac{u_{i,j-1} - 2u_{ij} + u_{i,j+1}}{\Delta y^2} = e^{u_{ij}}, \quad i, j = 1, 2, \dots$$

onde, como anteriormente, $u_{ij} \cong u(x_i, y_j)$. Para cada par (i, j) temos uma equação não-linear em u_{ij} . Aqui teríamos de recorrer a algum método numérico para resolver sistemas não-lineares. A indicação usual é o Método de Newton, suas variações, ou um método do tipo preditor-corretor.

9.4 A Equação do Calor

Para introduzir alguns métodos numéricos destinados a problemas transientes, usaremos inicialmente a equação do calor que, como dissemos anteriormente, representa o protótipo das equações parabólicas. Para simplificar, tomaremos a equação do calor unidimensional na variável espacial. Como veremos mais tarde, estas idéias podem ser generalizadas de um modo natural, para problemas com duas, ou mais, variáveis espaciais.

Consideremos a região $\mathcal{R} = \{(x, t), 0 < x < l \text{ e } t > 0\}$. Estamos agora interessados em métodos para encontrar aproximações de $u(x, t)$ definidas pela equação parabólica

$$u_t - a(x, t) u_{xx} = f(x, t), \quad (x, t) \in \mathcal{R}, a(x, t) > 0. \quad (9.17)$$

Para este problema ser bem-posto, devemos impor condições iniciais e de contorno à solução.

A condição inicial deve contemplar a distribuição da função u no início do processo, que convencionamos chamar $t = 0$. Assim, a condição inicial será fornecida por uma função conhecida $u^0(x)$:

$$u(x, 0) = u^0(x), \quad 0 < x < l. \quad (9.18)$$

Por outro lado, as condições de contorno devem refletir a situação física que pretendemos simular. Podemos aproveitar os comentários sobre condições de contorno apresentados no final da seção anterior para considerar as seguintes possibilidades:

a. Condição de Dirichlet

$$u(0, t) = g_1(t) \quad \text{e} \quad u(l, t) = g_2(t); \quad (9.19)$$

b. Condição de Neumann (fluxo na fronteira)

$$\frac{du}{dx}(0, t) = g_1(t) \quad \text{e} \quad \frac{du}{dx}(l, t) = g_2(t); \quad (9.20)$$

c. Condição mista

$$\frac{du}{dx}(0, t) + \lambda u(0, t) = g_1(t) \quad \text{e} \quad \frac{du}{dx}(l, t) + \lambda u(l, t) = g_2(t). \quad (9.21)$$

Seguindo a linha das seções anteriores, procuramos soluções aproximadas usando o método de diferenças finitas.

Tomamos uma malha com espaçamento constante, h , na discretização da variável espacial, $x_i = ih$, $i = 0 : m$. Na variável tempo usamos $t_j = jk$, $j = 0, 1, 2, \dots$, onde k é o passo associado à variável tempo.

Para distinguir tempo e espaço denotaremos por u_i^j o valor da aproximação da solução no nível de tempo t_j e na posição x_i , isto é

$$u_i^j \cong u(x_i, t_j).$$

Assim, seguimos a convenção usual dos problemas de evolução: o índice superior se refere à malha usada no tempo ao passo que os subíndices se referem à malha usada nas variáveis espaciais.

Na discretização da derivada de segunda ordem, usamos a fórmula:

$$u_{xx}(x_i, t_j) \cong \frac{u_{i+1}^j - 2u_i^j + u_{i-1}^j}{h^2}. \quad (9.22)$$

Quanto à derivada com relação ao tempo, existem as três possibilidades apresentadas no capítulo anterior:

a. Fórmula avançada

$$u_t(x_i, t_j) \cong \frac{u_i^{j+1} - u_i^j}{k}. \quad (9.23)$$

b. Fórmula atrasada

$$u_t(x_i, t_j) \cong \frac{u_i^j - u_i^{j-1}}{k}. \quad (9.24)$$

c. Fórmula centrada

$$u_t(x_i, t_j) \cong \frac{u_i^{j+1} - u_i^{j-1}}{2k}. \quad (9.25)$$

Apresentamos os três métodos de discretização associados a cada uma dessas escolhas que são os métodos Explícito (9.23), Implícito (9.24) e Crank-Nicolson (9.25).

O Método Explícito

O primeiro esquema proposto é obtido usando diferenças avançadas no tempo (9.23) e a discretização usual para a segunda derivada em x (9.22), calculada no nível t_j . Substituindo estas discretizações em (9.17), temos:

$$\frac{u_i^{j+1} - u_i^j}{k} - a_i^j \frac{u_{i+1}^j - 2u_i^j + u_{i-1}^j}{h^2} = f_i^j, \quad (9.26)$$

onde $a_i^j = a(x_i, t_j)$ e $f_i^j = f(x_i, t_j)$. Levando em conta as expressões dos erros, apresentadas no início da seção 9.3, verificamos que o erro de truncamento da aproximação definida por (9.26) é:

$$e_i^j = \frac{k}{2} u_{tt}(x_i, t_j) - a_i^j \frac{h^2}{12} u_{xxx}(\beta_i, t_j) = \mathcal{O}(k + h^2),$$

onde $t_j < \alpha_j < t_{j+1}$ e $x_{i-1} < \beta_i < x_{i+1}$. Explicitando u_i^{j+1} em função das demais variáveis teremos o esquema explícito, associado a (9.26):

$$u_i^{j+1} = \left(1 - 2 \frac{k}{h^2} a_i^j\right) u_i^j + a_i^j \frac{k}{h^2} (u_{i+1}^j + u_{i-1}^j) + k f_i^j, \quad i = 1 : m-1, \quad j > 0. \quad (9.27)$$

As duas condições de contorno entram na primeira e na última equações, isto é, quando $i = 1$ e $i = m-1$.

Como a solução é conhecida no tempo inicial, tomando $j = 0$ em (9.27) calculamos os valores u_i^1 . Calculados estes valores, para todo i , poderemos passar ao nível de tempo $j = 2$, e assim sucessivamente. Temos então um esquema de evolução na variável temporal no qual a cada nível de tempo a solução é calculada de modo explícito. Na figura 9.11, assinalamos os pontos envolvidos em cada passo do processo. Designamos com "x" os pontos nos quais u é conhecida e usada para calcular as aproximações nos pontos assinalados por "o".

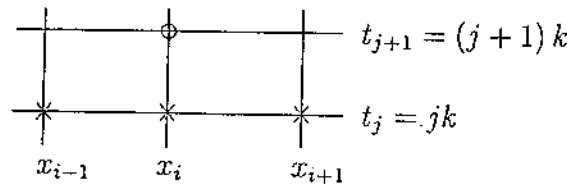


Figura 9.11: Esquema para o Método Explícito.

No Algoritmo 9.2 esquematizamos os cálculos do método explícito para a equação do calor (9.17) com condições de contorno do tipo Dirichlet (9.19). Neste algoritmo, a solução é calculada para $t \leq T$.

Algoritmo 9.2 Método explícito para a equação do calor com condição de Dirichlet

Dados $a(x, t)$, $f(x, t)$, $g_1(t)$, $g_2(t)$, $u^0(x)$, T , L , k e h

- 1: Calcule $N_1 = T/k$, $N_2 = L/h$, $\lambda = k/h^2$ e $u_i^0 = u^0(ih)$
 - 2: Para $j = 0 : N_1$, faça
 - 3: $u_0^j = g_1(jk)$
 - 4: $u_{N_2}^j = g_2(jk)$
 - 5: Para $i = 1 : N_2 - 1$, faça
 - 6: $u_i^{j+1} = (1 - 2\lambda a_i^j) u_i^j + \lambda a_i^j (u_{i+1}^j + u_{i-1}^j) + k f_i^j$
-

Pode-se analisar a estabilidade do método explícito através dos autovalores de uma matriz associada à discretização usada. Para simplificar, estudaremos a estabilidade do método explícito para a equação do calor com $a(x, t) = a$, uma constante.

Chamando $\lambda = ak/h^2$ e $u^j = (u_1^j, u_2^j, \dots, u_{m-1}^j)$, o vetor da solução em t_j , podemos verificar que as equações (9.23) podem ser agrupadas numa equação matricial,

$$u^{j+1} = Au^j + k f^j, \quad (9.28)$$

onde A é a matriz

$$A = \begin{bmatrix} (1-2\lambda) & \lambda & 0 & \dots & \dots & 0 \\ \lambda & (1-2\lambda) & \lambda & 0 & \dots & 0 \\ 0 & \lambda & (1-2\lambda) & \lambda & \dots & 0 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ 0 & 0 & \dots & 0 & \lambda & (1-2\lambda) \end{bmatrix}.$$

Na análise de estabilidade do método tomemos o caso mais simples onde $f(x, t) = 0$. Podemos verificar diretamente que

$$u(x, t) = e^{-\beta at} e^{i\sqrt{\beta}x}$$

com $i = \sqrt{-1}$, é solução analítica da equação do calor, qualquer que seja $\beta > 0$ (neste caso particular, no qual $a(x, t) = a$ e $f(x, t) = 0$). Vemos também que $u \rightarrow 0$ é o comportamento assintótico da solução analítica quando $t \rightarrow \infty$.

Por outro lado, tomando $f = 0$ em (9.28), temos

$$u^1 = Au^0, \quad u^2 = AAu^0 = A^2u^0, \quad \dots \quad u^j = A^ju^0, \quad \forall j > 0.$$

Por um resultado clássico da matemática ([Blum, 1972], por exemplo), mostra-se que $\lim_{r \rightarrow \infty} A^r = 0$ se e somente se todos os autovalores de A são, em módulo, menores que 1. Por outro lado, pode-se verificar que os autovalores de A são

$$\mu_i = 1 - 4\lambda \left(\sin \frac{i\pi}{2m} \right)^2, \quad i = 1 : m-1,$$

onde $(m-1)$ é a dimensão da matriz A .

Juntando estes dois resultados verificamos que $u^j \rightarrow 0$ (quando $j \rightarrow \infty$, isto é, quando $t \rightarrow \infty$), se e somente se a condição de estabilidade $|\mu_i| < 1$, $i = 1 : m-1$ for satisfeita.

Usando a expressão de μ_i , a condição de estabilidade pode ser escrita na forma:

$$-1 < 1 - 4\lambda \left(\sin \frac{i\pi}{2m} \right)^2 < 1,$$

ou ainda, pela simplificação desta desigualdade,

$$0 < \lambda \left(\sin \frac{i\pi}{2m} \right)^2 < \frac{1}{2}.$$

Esta desigualdade será respeitada se $0 < \lambda < \frac{1}{2}$, ou ainda, como $\lambda = ak/h^2$, h e k positivos, se

$$a \frac{k}{h^2} < \frac{1}{2}. \quad (9.29)$$

Em resumo, (9.29) estabelece a relação entre o passo na discretização do tempo, k , e o espaçamento na discretização em x , h , de modo que o esquema numérico associado à discretização seja estável.

Como vimos no capítulo 8, quando a estabilidade de um método depende de uma relação entre os tamanhos dos passos utilizados na discretização das variáveis independentes da equação, dizemos que ele é *condicionalmente estável*. Assim a condição (9.29) está nos dizendo que o Método Explícito para a equação do calor é condicionalmente estável. Observe que esta condição pode ser restritiva. Por exemplo, se $a = 5$ e $h = 10^{-2}$, a estabilidade estará garantida se $k < 10^{-5}$.

O Método Implícito

Para definir um método incondicionalmente estável, tomaremos diferenças finitas atrasadas no tempo, para discretizar o termo u_t da equação do calor. O *Método Implícito*, como é conhecido o método que usa (9.24) e (9.22) para discretizar a equação (9.17), é definido por:

$$\frac{u_i^j - u_i^{j-1}}{k} - a_i^j \frac{u_{i+1}^j - 2u_i^j + u_{i-1}^j}{h^2} = f_i^j; \quad (9.30)$$

como o erro de truncamento é a soma dos erros de truncamento das duas discretizações utilizadas, o erro do método implícito será:

$$e_i^j = -\frac{k}{2} u_{tt}(x_i, \alpha_j) - a_i^j \frac{h^2}{12} u_{xxxx}(\beta_i, t_j) = \mathcal{O}(k + h^2).$$

Na discretização do Método Implícito, aparecem aproximações da solução nos pontos da malha (x_i, t_{j-1}) , (x_i, t_j) , (x_{i-1}, t_j) , (x_{i+1}, t_j) , conforme a figura 9.12.

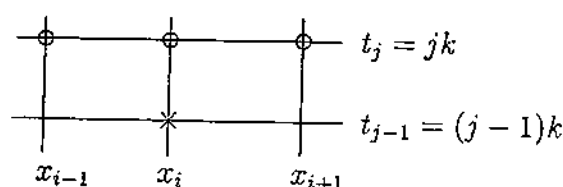


Figura 9.12: Esquema para o Método Implícito.

Separando as aproximações conhecidas, as do nível de tempo t_{j-1} , das não conhecidas, as do nível t_j , e denotando $\lambda_{ij} = a_i^j \frac{k}{h^2}$, teremos a equação genérica que define o Método Implícito:

$$-\lambda_{ij} u_{i-1}^j + (1 + 2\lambda_{ij}) u_i^j - \lambda_{ij} u_{i+1}^j = u_i^{j-1} + k f_i^j$$

para $i = 1 : m - 1$ e $j = 1, 2, \dots$

Observe que estas equações reunidas formam um sistema tridiagonal de equações lineares. Ao resolver o sistema, evoluímos do nível $j - 1$ para o nível j .

Na forma matricial, usadas as condições de contorno, o sistema a ser resolvido, a cada nível de tempo, é:

$$Au^j = u^{j-1} + k f^j, \quad (9.31)$$

onde,

$$A = \begin{bmatrix} (1 + 2\lambda_{1j}) & -\lambda_{1j} & 0 & \dots & \dots & 0 \\ -\lambda_{2j} & (1 + 2\lambda_{2j}) & -\lambda_{2j} & 0 & \dots & 0 \\ 0 & -\lambda_{3j} & (1 + 2\lambda_{3j}) & -\lambda_{3j} & \dots & 0 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ 0 & 0 & \dots & 0 & \lambda_{m-1,j} & (1 + 2\lambda_{m-1,j}) \end{bmatrix}$$

$$u^j = \begin{bmatrix} u_1^j \\ u_2^j \\ u_3^j \\ \vdots \\ u_{m-1}^j \end{bmatrix} \quad f^j = \begin{bmatrix} f_1^j + \lambda_{1j}g_1^j \\ f_2^j \\ f_3^j \\ \vdots \\ f_{m-1}^j + \lambda_{m-1,j}g_2^j \end{bmatrix}$$

Como $\lambda_{ij} > 0$, vemos que a matriz do sistema, A , é diagonalmente dominante. Assim, sendo A simétrica e tridiagonal, além do método de eliminação pode-se optar por resolver o sistema (9.31) usando métodos iterativos, pois a convergência estará garantida.

Com a solução do sistema, encontramos as aproximações no nível t_j , a partir de valores conhecidos, o nível t_{j-1} . Dessa maneira, partindo da condição inicial $u^0(x)$, caminhamos passo a passo para a solução em um tempo $t_j = jk$.

Para analisar a estabilidade do Método Implícito, vamos tomar o caso particular $a(x, t) = a$, constante, e $f(x, t) = 0$. Neste caso, usando (9.31) temos

$$u^1 = A^{-1}u^0, \quad u^2 = (A^{-1})^2u^0, \quad \dots \quad u^j = (A^{-1})^ju^0,$$

onde, como anteriormente, $u = (u_1, \dots, u_{m-1})$ é o vetor solução no nível de tempo considerado. Como a análise de estabilidade será feita para o caso $\lambda = \text{constante}$, pode-se verificar que neste caso os autovalores de A são:

$$\mu_i = 1 + 4\lambda \left(\sin \frac{i\pi}{2m} \right)^2, \quad i = 1 : m-1.$$

Assim, $\mu_i > 1$, $i = 1 : m-1$. Como os autovalores de A são não-nulos, a inversa A^{-1} existe e seus autovalores são, respectivamente, o recíproco dos autovalores de A . Portanto, todos os autovalores de A^{-1} são menores, em módulo, que 1. Desta maneira, $\lim_{r \rightarrow \infty} (A^{-1})^r = 0$ e $u^n \rightarrow 0$, quando $n \rightarrow \infty$, independentemente da escolha de λ , e conseqüentemente o Método Implícito é incondicionalmente estável.

Nos dois métodos numéricos para equação do calor já apresentados, as discretizações da derivada com relação ao tempo são aproximações de $\mathcal{O}(k)$, ao passo que na derivada com relação a x usamos aproximações da ordem de $\mathcal{O}(h^2)$. Seria desejável estabelecer um método cujo erro de truncamento fosse da ordem $\mathcal{O}(k^2 + h^2)$, o que será feito no método seguinte.

Método de Crank-Nicolson

A idéia do Método de Crank-Nicolson é aplicar a discretização da equação diferencial num nível intermediário entre t_j e t_{j+1} , ou seja, no ponto, que não é malha, $(x_i, t_j + \frac{k}{2})$. Usamos a fórmula centrada na variável t . A derivada em relação a x em $t_j + \frac{1}{2}$ será aproximada pela média das discretizações de segunda derivada nos níveis de tempo t_j e t_{j+1} .

Assim, no Método de Crank-Nicolson, as aproximações são definidas por:

$$\frac{u_i^{j+1} - u_i^j}{k} - \frac{a_i^{j+1/2}}{2} \left[\frac{u_{i+1}^j - 2u_i^j + u_{i-1}^j}{h^2} + \frac{u_{i+1}^{j+1} - 2u_i^{j+1} + u_{i-1}^{j+1}}{h^2} \right] - f_i^{j+1/2} = 0, \quad (9.32)$$

onde

$$a_i^{j+1/2} = a(x_i, t_j + \frac{k}{2}) \quad \text{e} \quad f_i^{j+1/2} = a(x_i, t_j + \frac{k}{2}).$$

Como observamos anteriormente, as aproximações definidas por (9.32) têm erro de truncamento da ordem de $\mathcal{O}(k^2 + h^2)$ que é a soma dos erros das discretizações (9.22) e (9.25).

Na equação geral do Método de Crank-Nicolson (9.32) aparecem valores da solução em seis pontos das malha, como ilustrado na figura 9.13. Três destes valores são conhecidos, pois correspondem à solução no nível de tempo t_j . Os outros três valores serão calculados ao se resolver um sistema tridiagonal, como mostramos a seguir. Por isto o Método de Crank-Nicolson também é implícito.

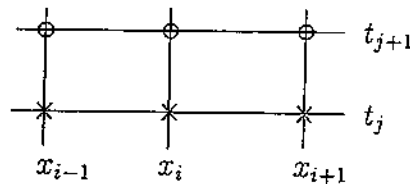


Figura 9.13: Esquema para o Método de Crank-Nicolson.

Para estabelecer a forma matricial do Método de Crank-Nicolson, também tomamos o caso particular $a(x, t) = a = \text{constante}$. Usando, como antes, $\lambda = ak/h^2$ e $u^j = (u_1^j, u_2^j, \dots, u_{m-1}^j)$, teremos

$$Au^{j+1} = Bu^j + kf, \quad (9.33)$$

onde, no caso de condições de contorno do tipo Dirichlet temos

$$A = \begin{bmatrix} (1+\lambda) & -\lambda/2 & 0 & \dots & \dots & 0 \\ -\lambda/2 & (1+\lambda) & -\lambda/2 & 0 & \dots & 0 \\ 0 & -\lambda/2 & (1+\lambda) & -\lambda/2 & \dots & 0 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ 0 & 0 & \dots & 0 & -\lambda/2 & (1+\lambda) \end{bmatrix},$$

$$B = \begin{bmatrix} (1-\lambda) & \lambda/2 & 0 & \dots & \dots & 0 \\ \lambda/2 & (1-\lambda) & \lambda/2 & 0 & \dots & 0 \\ 0 & \lambda/2 & (1-\lambda) & \lambda/2 & \dots & 0 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ 0 & 0 & \dots & 0 & \lambda/2 & (1-\lambda) \end{bmatrix}, \quad f = \begin{bmatrix} f_1^{j+1/2} + \frac{\lambda}{k} g_1^{j+1/2} \\ \vdots \\ \vdots \\ f_{m-1}^{j+1/2} + \frac{\lambda}{k} g_2^{j+1/2} \end{bmatrix},$$

onde $g_1(t)$ e $g_2(t)$ são as funções que definem as condições de contorno na formulação do problema. Como A é diagonalmente dominante, não-singular, o sistema (9.33) tem uma única solução. Pode-se mostrar que o Método de Crank-Nicolson é incondicionalmente estável.

O Algoritmo 9.3 esquematiza os cálculos do Método de Crank-Nicolson.

Algoritmo 9.3 Método de Crank-Nicolson

Dados $a(x, t)$, $f(x, t)$, $g_1(t)$, $g_2(t)$, $u^0(x)$, M , L e k

- 1: Calcule $h = L/M$, $\lambda = ak/h^2$ e $u_i^0 = u^0(ih)$
- 2: Para $i = 1 : M - 1$, faça
- 3: $u^0(i) = u^0(ih)$
- 4: Para $j = 0, 1, \dots$ faça
- 5: Usar o Algoritmo 2.4 para resolver o sistema (9.33)

$$Au^{j+1} = Bu^j + kf$$

Exemplo 9.4.1. Neste exemplo vamos aplicar os métodos Explícito, Implícito e de Crank-Nicolson para encontrar aproximações para a equação do calor

$$\begin{aligned} u_t - u_{xx} &= 0, & 0 < x < 1, \quad t > 0 \\ u(0, t) &= u(1, t) = 0 & \text{condições de contorno} \\ u(x, 0) &= \sin(\pi x) & \text{condição inicial} \end{aligned} \quad (9.34)$$

Por derivação, nas variáveis t e x pode-se verificar diretamente que $u_e(x, t) = e^{-\pi^2 t} \sin(\pi x)$ satisfaz à equação diferencial e as condições adicionais do problema. Elaboramos o programa calor.m para os métodos Explícito, Implícito e de Crank-Nicolson, aplicados neste caso em particular. No Anexo, apresentamos a listagem deste programa.

Na tabela 9.1, apresentamos os erros correspondentes a cada um dos métodos, para dois níveis de tempo. Para obter estes resultados usamos $h = 0.01$, $\lambda = 0.5$, o que corresponde ao passo no tempo $\Delta t = 0.5 \times 10^{-4}$. Observamos que o Método de Crank-Nicolson fornece os melhores resultados, como prevê a teoria. Ele é seguido do Método Explícito que, apesar de ter ordem de aproximação igual ao método Implícito, acumula menos erros de arredondamento por envolver cálculos mais simples.

Usamos o erro relativo,

$$\text{erro}(u) = h \frac{\left(\sum_i (u_i - u_{e_i})^2 \right)^{\frac{1}{2}}}{\max(u_e)}$$

como medida de erro apresentada na tabela 9.4.1. A título de ilustração, os seguintes valores são obtidos, no ponto médio do intervalo, usando os três métodos; para comparação apresentamos também o valor da solução analítica neste ponto.

$$\begin{aligned} u_{\text{imp}}(0.5, 0.01) &= 0.9089 \\ u_{\text{exp}}(0.5, 0.01) &= 0.9045 \\ u_{\text{cn}}(0.5, 0.01) &= 0.9067 \\ u_e(0.5, 0.01) &= 0.9060. \end{aligned}$$

	Explícito	Implícito	Crank-Nicolson
$t = 0.01$	0.03%	0.06%	0.02%
$t = 1$	3.12%	7.47%	1.67%

Tabela 9.1: Erros de aproximação do Exemplo 9.4.1.

□

Equação do Calor com duas variáveis espaciais

Os métodos numéricos para a equação do calor com duas variáveis espaciais são generalizações dos métodos já apresentados nesta seção.

Na formulação da equação do calor bidimensional, estaremos interessados na função $u(x, y, t)$ que satisfaça a equação diferencial

$$u_t - a(x, y, t) (u_{xx} + u_{yy}) = f(x, y, t) \quad (9.35)$$

onde $(x, y) \in \mathcal{R}$, uma região do plano (x, y) , $a(x, y, t) > 0$ e $t > 0$.

A primeira derivada parcial com relação ao tempo pede uma condição adicional que é a condição inicial

$$u(x, y, 0) = u^0(x, y), \quad (x, y) \in \mathcal{R}. \quad (9.36)$$

Por outro lado, as derivadas parciais com relação às variáveis espaciais formam o operador laplaciano. Como vimos anteriormente, as condições de contorno podem ser do tipo Dirichlet como em (9.10), na qual o valor $u(x, y, t)$ é conhecido na fronteira de $\mathcal{R}$, agora para todo $t > 0$. Podemos também preestabelecer o fluxo normal à fronteira, isto é, usar a condição do tipo Neumann:

$$\frac{\partial u}{\partial \eta} = g(x, y, t) \quad \text{para} \quad (x, y) \in \mathcal{R}, t > 0. \quad (9.37)$$

Outra possibilidade é usar condições mistas, como ilustrado na figura 9.10.

A seguir, vamos detalhar discretizações no caso em que $\mathcal{R}$ é um retângulo

$$\mathcal{R} = \{(x, y) \text{ tais que } 0 < x < l_1 \text{ e } 0 < y < l_2\}.$$

Neste caso, a malha usada na discretização espacial será:

$$\begin{aligned} x_i &= i \Delta x & i &= 0 : m_1, \text{ onde } m_1 \Delta x = l_1 \\ y_j &= j \Delta y & j &= 0 : m_2, \text{ onde } m_2 \Delta y = l_2. \end{aligned}$$

Na variável tempo, tomaremos $t_k = k \Delta t$, $k = 0, 1, \dots$. Seguindo a linha anterior, denotaremos por $u_{i,j}^k$ a aproximação de $u(x, y, t)$ no ponto da malha (x_i, y_j, t_k) , isto é,

$$u_{i,j}^k \cong u(x_i, y_j, t_k).$$

Agora podemos lançar mão da discretização do laplaciano definida em (9.7) ou (9.8). Usando as fórmulas avançada, atrasada e centrada, respectivamente, na discretização do tempo, são estabelecidos os esquemas numéricos correspondentes aos métodos: Explícito, Implícito e de Crank-Nicolson.

As equações gerais destes três métodos são apresentadas a seguir.

Método Explícito

$$\frac{u_{i,j}^{k+1} - u_{i,j}^k}{\Delta t} - a_{i,j}^k \left\{ \frac{u_{i+1,j}^k - 2u_{i,j}^k + u_{i-1,j}^k}{\Delta x^2} + \frac{u_{i,j+1}^k - 2u_{i,j}^k + u_{i,j-1}^k}{\Delta y^2} \right\} = f(x_i, y_j, t_k),$$

com $i = 1 : m_1 - 1$, $j = 1 : m_2 - 1$ e $k = 0, 1, \dots$

Método Implícito

$$\frac{u_{i,j}^{k+1} - u_{i,j}^k}{\Delta t} - a_{i,j}^{k+1} \left\{ \frac{u_{i+1,j}^{k+1} - 2u_{i,j}^{k+1} + u_{i-1,j}^{k+1}}{\Delta x^2} + \frac{u_{i,j+1}^{k+1} - 2u_{i,j}^{k+1} + u_{i,j-1}^{k+1}}{\Delta y^2} \right\} = f(x_i, y_j, t_{k+1}),$$

com $i = 1 : m_1 - 1, j = 1 : m_2 - 1$ e $k = 0, 1, \dots$

Crank-Nicolson

$$\begin{aligned} \frac{u_{i,j}^{k+1} - u_{i,j}^k}{\Delta t} - \frac{a_{i,j}^{k+1}}{2} \left\{ \frac{u_{i+1,j}^{k+1} - 2u_{i,j}^{k+1} + u_{i-1,j}^{k+1}}{\Delta x^2} + \frac{u_{i,j+1}^{k+1} - 2u_{i,j}^{k+1} + u_{i,j-1}^{k+1}}{\Delta y^2} \right\} - \\ - \frac{a_{i,j}^k}{2} \left\{ \frac{u_{i+1,j}^k - 2u_{i,j}^k + u_{i-1,j}^k}{\Delta x^2} + \frac{u_{i,j+1}^k - 2u_{i,j}^k + u_{i,j-1}^k}{\Delta y^2} \right\} = f(x_i, y_j, t_{k+1/2}), \end{aligned}$$

com $i = 1 : m_1 - 1, j = 1 : m_2 - 1$ e $k = 0, 1, \dots$

As condições de contorno do problema devem ser incorporadas às equações que correspondem aos pontos da malha para os quais pelo menos um dos pontos cardeais, da discretização do laplaciano, esteja na fronteira de $\mathcal{R}$. Juntando todas as equações, com as correções devidas às condições de contorno, obtemos equações matriciais que generalizam (9.28), para o Método Explícito, (9.31), para o Método Implícito e (9.33) para o Método de Crank-Nicolson.

A resolução do problema pelo esquema explícito é relativamente simples. Entretanto, assim como no caso unidimensional, pode-se mostrar que deve existir uma relação entre os incrementos no tempo e no espaço para garantir a estabilidade. Assim, o Método Explícito também é condicionalmente estável no caso bidimensional com condição de estabilidade – no caso $a(x, y, t) = a$, constante

$$a \Delta t \leq \frac{1}{2[(\Delta x)^{-2} + (\Delta y)^{-2}]}.$$

Esta condição é mais restritiva do que o caso unidimensional. Para comparar, tomamos $a = 5, \Delta x = \Delta y = 10^{-2}$ e teremos $\Delta t < \frac{1}{2} 10^{-5}$ que é metade do calculado com (9.29).

Por outro lado, se a opção for pelo esquema implícito, temos de resolver o sistema de equações lineares em cada nível do tempo. Por exemplo, se tomamos $\Delta x = \Delta y$ e $a(x, y, t)$ constante, e lembrando que $\lambda = a \Delta t / \Delta x^2$, a equação genérica do Método Implícito seria

$$-\lambda u_{i-1,j}^{k+1} - \lambda u_{i,j-1}^{k+1} + (1 + 4\lambda) u_{i,j}^{k+1} - \lambda u_{i,j+1}^{k+1} - \lambda u_{i+1,j}^{k+1} = u_{i,j}^k + f(x_i, y_j, t_k) \Delta t.$$

Assim, teremos de resolver um sistema pentadiagonal a cada nível de tempo. A estrutura da matriz deste sistema é como a da matriz obtida na discretização do laplaciano ilustrada na figura 9.5. Esta “desvantagem” de resolver sistemas é compensada pela estabilidade incondicional do Método Implícito.

O Método ADI

Os métodos do tipo Implícito com direções alternadas – ADI, na literatura inglesa – tentam amenizar o preço do sistema pentadiagonal substituindo-o por dois sistemas tridiagonais. O princípio do Método ADI é “partir” o problema em duas equações de diferenças que são usadas consecutivamente em intervalos de tempo de comprimento $\Delta t/2$. A primeira equação é implícita na variável x e explícita na variável y ; e na segunda equação, fazemos o inverso, isto é, tomamos uma discretização explícita em x e implícita em y . O uso de passos fracionários no tempo foi proposto, pela primeira vez, por Douglas e Rachford em 1956.

Assim, a cada nível de tempo, chamando $\bar{u}_{i,j}$ os valores no tempo intermediário, temos

Primeiro passo

$$\frac{\bar{u}_{i,j} - u_{i,j}^k}{\Delta t/2} - a \left(\frac{\bar{u}_{i+1,j} - 2\bar{u}_{i,j} + \bar{u}_{i-1,j}}{\Delta x^2} + \frac{u_{i,j+1}^k - 2u_{i,j}^k + u_{i,j-1}^k}{\Delta y^2} \right) = f(x_i, y_j, t_{k+1/2})$$

Segundo passo

$$\frac{u_{i,j}^{k+1} - \bar{u}_{i,j}}{\Delta t/2} - a \left(\frac{\bar{u}_{i+1,j} - 2\bar{u}_{i,j} + \bar{u}_{i-1,j}}{\Delta x^2} + \frac{u_{i,j+1}^{k+1} - 2u_{i,j}^{k+1} + u_{i,j-1}^{k+1}}{\Delta y^2} \right) = f(x_i, y_j, t_{k+1})$$

Agrupando convenientemente os termos, tomando por simplicidade $\Delta x = \Delta y$, e usando $\lambda = a\Delta t/\Delta x^2$, as equações que definem o método podem ser reescritas na forma:

Primeiro passo

$$-\frac{\lambda}{2}\bar{u}_{i-1,j} + (1+\lambda)\bar{u}_{i,j} - \frac{\lambda}{2}\bar{u}_{i+1,j} = \frac{\lambda}{2}u_{i,j-1}^k + (1-\lambda)u_{i,j}^k + \frac{\lambda}{2}u_{i,j+1}^k + k f_{i,j}^{k+1/2}.$$

Segundo passo

$$-\frac{\lambda}{2}u_{i,j-1}^{k+1} + (1+\lambda)u_{i,j}^{k+1} - \frac{\lambda}{2}u_{i,j+1}^{k+1} = \frac{\lambda}{2}\bar{u}_{i-1,j} + (1-\lambda)\bar{u}_{i,j} + \frac{\lambda}{2}\bar{u}_{i+1,j} + k f_{i,j}^{k+1}.$$

Do primeiro passo, resolvendo um sistema tridiagonal, obtemos os valores intermediários, que alguns autores chamam preditores. Usando estes valores intermediários e novamente resolvendo um sistema tridiagonal, cuja matriz de coeficientes é a mesma do sistema anterior, teremos os valores corrigidos no final do subintervalo de comprimento Δt .

Pode-se mostrar que o Método ADI é incondicionalmente estável. A análise de estabilidade deste método pode ser encontrada, por exemplo em [Sod, 1985].

Exemplo 9.4.2. A distribuição da pressão em reservatórios petrolíferos de baixa compressibilidade pode ser simulada por equação diferencial parabólica [Peaceman, 1977]

$$p_t = a(p_{xx} + p_{yy}), \quad (x, y) \in \mathcal{R}, t > 0 \quad (9.38)$$

onde $p(x, y, t)$ representa a pressão no modelo bidimensional, e

$$a = \frac{k}{\phi \mu c},$$

com k = permeabilidade, ϕ = porosidade, μ = viscosidade e c = compressibilidade, são propriedades do meio poroso e do fluido. Uma escolha adequada na distribuição dos poços no reservatório é conhecida como *five-spot*. Neste caso, pode-se considerar $\mathcal{R}$ como o quadrado $0 < x < 1$ e $0 < y < 1$, com fluxo de fluido nulo nas arestas devido à simetria na distribuição dos poços.

Como, pela Lei de Darcy,

$$v_x = -\frac{k}{\mu} \frac{\partial p}{\partial x} \quad \text{e} \quad v_y = -\frac{k}{\mu} \frac{\partial p}{\partial y},$$

a condição de contorno será derivada normal da pressão nula nas arestas:

$$\frac{\partial p}{\partial \eta}(x, y, t) = 0 \quad \text{para } (x, y) \in \partial \mathcal{R}. \quad (9.39)$$

Existem também no quadrado $\mathcal{R}$ dois poços diagonalmente opostos. Aqui estaremos considerando que o poço injetor está localizado em $x = y = 0$ e o poço produtor em $x = y = 1$. A pressão no poço injetor é conhecida:

$$p(0, 0, t) = p_i, \quad t > 0 \quad (9.40)$$

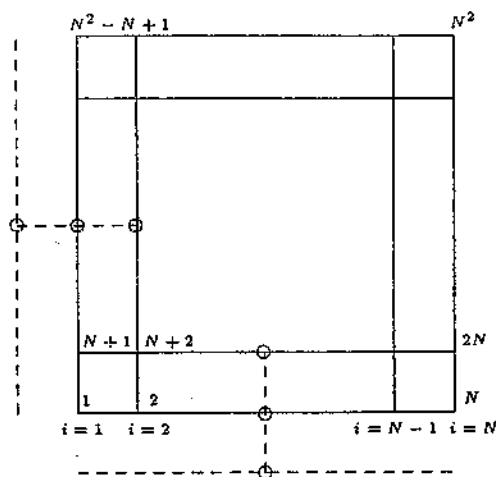
Em resumo, a distribuição de pressão em um reservatório tipo *five-spot* pode ser calculada usando a equação (9.38) com as condições de contorno (9.39) e (9.40) e uma condição inicial conhecida, que aqui será $p(x, y, 0) = 0$, como nível de referência de pressão.

Vamos tomar uma malha uniforme em $\mathcal{R}$, isto é, $\Delta x = \Delta y = 1/(N - 1)$. Neste exemplo, vamos discretizar a condição de contorno (9.39) usando diferenças centradas. Desta maneira o erro introduzido é $\mathcal{O}(\Delta x^2)$, e portanto compatível com o erro da discretização do laplaciano (9.7).

Para um nó da fronteira, usamos

$$\left. \frac{\partial p}{\partial \eta} \right|_{i=1,j} = \frac{p_{2,j}^k - p_{-1,j}^k}{2\Delta x} \quad \text{ou} \quad \left. \frac{\partial p}{\partial \eta} \right|_{i,j=1} = \frac{p_{i,2}^k - p_{i,-1}^k}{2\Delta y}$$

e fórmulas análogas para pontos sobre as arestas $x = 1$ e $y = 1$. Assim, estamos usando pontos fictícios correspondentes a $i = -1, j = -1, i = N + 1$ e $j = N + 1$, que estão fora

Figura 9.14: Discretização de $\mathcal{R}$.

do domínio da solução. Entretanto os pontos $p_{2,j}^k$ e $p_{-1,j}^k$ estão relacionados pela condição de contorno da derivada. No nosso caso (9.39) implica

$$p_{-1,j}^k = p_{2,j}^k, \quad \forall j,$$

e relações análogas para as outras arestas. Assim, substituímos os pontos fictícios nas equações discretizadas correspondentes aos pontos da fronteira.

A seguir, apresentamos algumas equações do sistema linear $\mathbf{Ap} = \mathbf{b}$ associado à discretização da equação (9.38) usando (9.7) e diferenças centradas na condição de contorno (9.39).

Neste exemplo o esquema implícito é

$$p_{i,j}^k - p_{i,j}^{k-1} - a \frac{\Delta t}{\Delta x^2} (p_{i,j-1}^k + p_{i,j+1}^k - 4p_{i,j}^k + p_{i+1,j}^k + p_{i-1,j}^k) = 0$$

Chamando $\lambda = a \Delta t / \Delta x^2$ e transformando o duplo índice, $p_{i,j}$ em índice simples p_i , como foi feito na seção 9.2, temos a equação geral da discretização:

$$-\lambda p_{i-N}^k - \lambda p_{i-1}^k + (1 + 4\lambda) p_i^k - \lambda p_{i+1}^k - \lambda p_{i+N}^k = p_i^{k-1}.$$

A condição no poço injetor, (9.40), é usada escrevendo a primeira equação do sistema na forma

$$p_1 = p_i.$$

Os outros elementos da matriz $\mathbf{A}_{N^2 \times N^2}$ podem ser calculados fazendo correções devidas às condições de contorno na equação geral. Este procedimento está esquematizado no Algoritmo 9.4.

Algoritmo 9.4 Discretização da equação para pressão no *five-spot*

Dados $N, \Delta x, \Delta t$ e a

- 1: $\lambda = a \Delta t / \Delta x^2$
 - 2: $a_{11} = 1$ {pressão de injeção: $p_1 = p_i$ }
 - 3: Para $l = 2 : N^2$, faça
 - 4: $a_{ll} = 1 + 4\lambda$ {diagonal principal}
 - 5: Para $l = 2 : N^2$, faça
 - 6: $a_{l,l-1} = -\lambda$ {diagonal inferior}
 - 7: Para $l = 2 : N^2 - 1$, faça
 - 8: $a_{l,l+1} = -\lambda$ {diagonal superior}
 - 9: Para $l = N + 1 : N^2$, faça
 - 10: $a_{l,l-N} = -\lambda$ {diagonal inferior afastada}
 - 11: Para $l = 2 : N^2 - N$, faça
 - 12: $a_{l,l+N} = -\lambda$ {diagonal superior afastada}
 - {Na correção para incorporar as condições de contorno}
 - 13: Para $l = 2 : N$, faça
 - 14: $a_{l,l+N} = -2\lambda$ {fronteira fictícia inferior}
 - 15: Para $l = N : N : N^2$, faça
 - 16: $a_{l,l-1} = -2\lambda$ {fronteira fictícia à direita}
 - 17: $a_{l,l+1} = 0$
 - 18: Para $l = N + 1 : N : N^2 - N + 1$, faça
 - 19: $a_{l,l-1} = 0$ {fronteira fictícia à esquerda}
 - 20: $a_{l,l+1} = -2\lambda$
 - 21: Para $l = N^2 - N + 1 : N^2$, faça
 - 22: $a_{l,l-N} = -2\lambda$ {fronteira fictícia superior}
 - 23: Para $l = N^2$, faça
 - 24: $a_{ll} = 1, a_{l,l-1} = 0$ e $a_{l,l-N} = 0$ {condição de contorno no poço produtor}
-

Neste algoritmo estamos usando a notação usual de programação, isto é, $i = 1 : N$ significa passo 1, ou seja, $i = 1, 2, 3, \dots, N$ e $i = 1 : N : N^2$ significa passo $N, 2N, 3N, \dots, N^2$.

Desta maneira, encontramos o sistema linear associado ao Método Implícito,

$$L_{\Delta P}^k = p^{k-1},$$

onde p é o vetor solução, bloqueado como no caso do laplaciano, isto é,

$$p = [p_1, p_2, \dots, p_N \mid p_{N+1}, p_{N+2} \dots p_{2N} \mid p_{N^2-N}, p_{N^2-N+1} \dots p_{N^2}]^T.$$

A condição de contorno do poço injetor está na primeira equação, $p_1 = p_i$; e a condição de contorno no poço produtor, apresentada no passo 24 do algoritmo, é $p_{N^2} = 0$ (pressão de referência). $\square$

9.5 A Equação da Onda

O matemático francês Jean R. D'Alembert (1717-1783) encontrou uma expressão para a solução da equação que descreve o movimento de uma corda de comprimento infinito com condições iniciais,

$$u_{tt} = c^2 u_{xx}, \quad -\infty < x < \infty, t > 0$$

$$u(x, 0) = f(x)$$

$$u_t(x, 0) = g(x) \quad -\infty < x < \infty.$$

A solução de D'Alembert, apresentada em 1747, é [Farlow, 1982]

$$u(x, t) = \frac{1}{2}[f(x - ct) + f(x + ct)] + \frac{1}{2c} \int_{x-ct}^{x+ct} g(\xi) d\xi.$$

Tomemos um caso particular no qual $f(x)$ tem duas descontinuidades,

$$f(x) = \begin{cases} 1 & \text{se } -1 < x < 1 \\ 0 & \text{para outros valores de } x \end{cases}$$

e $g(x) = 0$. A expressão de D'Alembert nos diz que $u(x, t)$ é uma soma de duas "ondas quadradas" que caminham com velocidade c , em direções opostas, como ilustrado na figura 9.15.

Este exemplo nos mostra uma propriedade marcante da equação de onda que é o transporte do formato da condição inicial, inclusive descontinuidades. Vemos assim que a equação da onda é intrinsecamente diferente da equação do calor, que tem caráter suavizador, como são os fenômenos difusivos.

Lembramos que os métodos de diferenças finitas têm como base expansões em série de Taylor que pressupõem suavidade da função. Chegamos então a um impasse: usar métodos destinados às funções suaves em problemas cujas soluções podem não ser suaves.

No próximo capítulo, apresentamos alguns métodos para cálculo de aproximações quando a solução da equação diferencial apresenta variações abruptas ou descontinuidades.

Nesta seção, introduzimos métodos numéricos para equação da onda de segunda ordem que podem ser úteis no caso de soluções suaves.

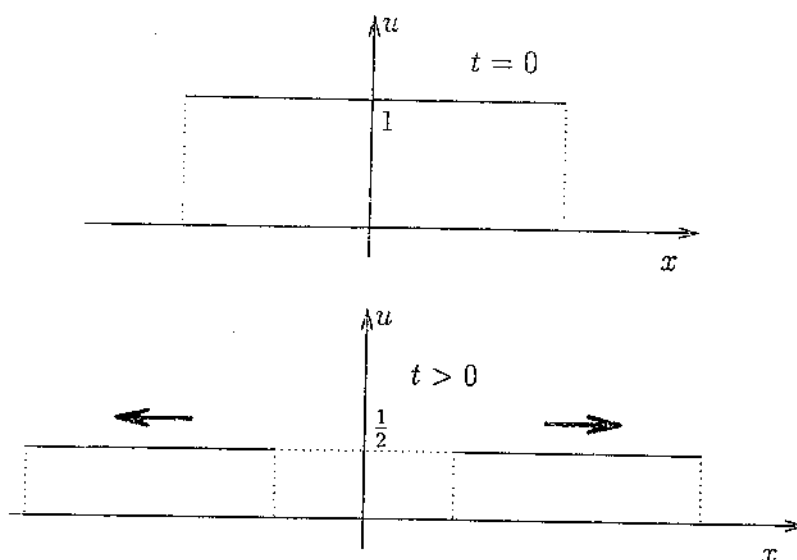


Figura 9.15: Solução de D'Alembert para equação da onda.

Para simplificar, analisamos a equação da onda unidimensional na variável espacial. A exemplo da equação do calor, a passagem para o caso bidimensional é natural.

Estamos interessados na função $u(x, t)$ que satisfaça a equação diferencial:

$$u_{tt} - a(x, t) u_{xx} = f(x, t), \quad 0 < x < l, t > 0 \quad (9.41)$$

onde $a(x, t) > 0$ e $f(x, t)$ são funções conhecidas. Como no caso das equações de Poisson e do calor, são necessárias condições adicionais para completar o problema, isto é, para torná-lo bem-posto no sentido de Hadamard. No caso da equação da onda, devemos tomar duas condições iniciais porque temos derivada de segunda ordem no tempo, u_{tt} . Sejam as condições iniciais:

$$\begin{cases} u(x, 0) = u^0(x), & 0 < x < l \\ u_t(x, 0) = v^0(x). \end{cases}$$

Como também temos duas derivadas na variável espacial devemos fornecer duas condições de contorno,

$$\begin{cases} u(0, t) = g_1(t), & t > 0 \\ u(l, t) = g_2(t). \end{cases}$$

A figura 9.16 é ilustrativa: preestabelecidas condições no contorno enegrecido, procuramos a solução nos pontos internos da região.

Para criar a discretização de (9.41), vamos seguir os mesmos passos anteriores.

- i. Estabelecer a malha em $\mathcal{R} = \{(x, t), 0 < x < l, t > 0\}$;
- ii. escolher expressões que aproximem as derivadas que aparecem na equação diferencial e nas condições adicionais;

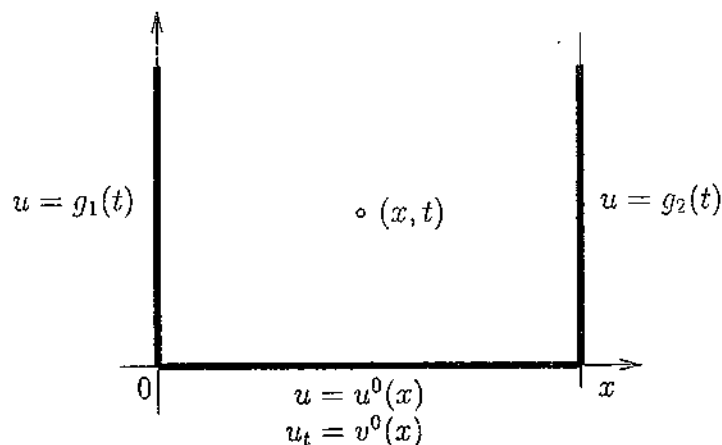


Figura 9.16: Condições para a equação da onda.

- iii. montar o sistema, nas incógnitas u_i^j , associado à discretização; neste sistema, estão também embutidas as condições de contorno e iniciais;
- iv. resolver o sistema.

Estabelecida a malha, a expressão (8.6) será usada para aproximar as derivadas de segunda ordem. Os dois métodos apresentados a seguir têm diferenças pequenas. No Método Explícito, u_{xx} é aproximada em t_k . No Método Implícito, u_{xx} é aproximada pela média das discretizações correspondentes aos níveis t_{k-1} e t_{k+1} ; neste caso, a solução no nível t_{k+1} é calculada pela solução de um sistema de equações lineares.

Método Explícito

$$\frac{u_i^{k+1} - 2u_i^k + u_i^{k-1}}{\Delta t^2} - a_i^k \left(\frac{u_{i+1}^k - 2u_i^k + u_{i-1}^k}{\Delta x^2} \right) = f_i^k \quad (9.42)$$

onde, como anteriormente, $a_i^k = a(x_i, t_k)$ e $g_i^k = f(x_i, t_k)$.

Chamando $\lambda = \Delta t / \Delta x$ e explicitando u_i^{k+1} , a equação (9.42) nos leva ao Algoritmo 9.5, que será apresentado depois que detalharmos uma maneira conveniente de discretizar a segunda condição inicial.

Na equação (9.42), são usados 5 pontos conforme ilustra a figura 9.17, na qual assinalamos com "x" os pontos nos quais u é conhecida e usada para calcular os valores de $u(x, t)$ designados por "o".

Método Implícito

$$\frac{u_i^{k+1} - 2u_i^k + u_i^{k-1}}{\Delta t^2} - \frac{1}{2} \left\{ a_i^{k+1} \frac{u_{i+1}^{k+1} - 2u_i^{k+1} + u_{i-1}^{k+1}}{\Delta x^2} + a_i^{k-1} \frac{u_{i+1}^{k-1} - 2u_i^{k-1} + u_{i-1}^{k-1}}{\Delta x^2} \right\} =$$

$$= \frac{1}{2}(f_i^{k+1} + f_i^{k-1}). \quad (9.43)$$

No Método Implícito, 7 pontos serão usados, conforme esquematiza a figura 9.17. Nesta figura, estamos, de novo, designando com "x" os pontos nos quais u é conhecida e usada para calcular os valores nos pontos assinalados por "o". Observamos que a equação (9.43) envolve três incógnitas u_{i+1}^{k+1} , u_i^{k+1} e u_{i-1}^{k+1} . Agrupando as equações, temos um sistema linear que, resolvido, fornece-nos o vetor solução no nível de tempo t_{k+1} .

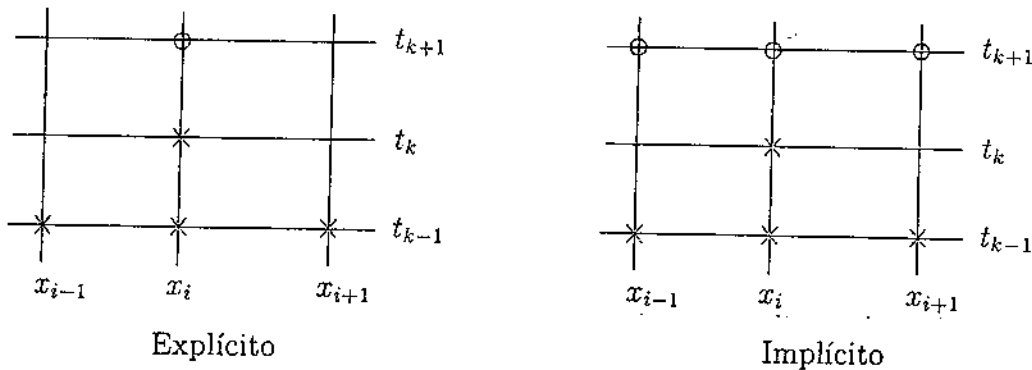


Figura 9.17: Métodos Explícito e Implícito para equação da onda.

Olhando para o erro de truncamento das discretizações utilizadas, verificamos que os dois métodos apresentados têm erro de truncamento $\mathcal{O}(k^2 + h^2)$.

Nas equações (9.42) e (9.43) admite-se, como condição prévia para o início do processo, o conhecimento de u_i^0 e u_i^1 , $i = 1, 2, \dots$. Os valores u_i^0 são facilmente calculados usando a função $u^0(x)$ apresentada na primeira condição inicial:

$$u_i^0 = u^0(i \Delta x).$$

Para calcular u_i^1 com um erro compatível com a discretização, precisamos de alguns argumentos adicionais. Usando o desenvolvimento em série de Taylor na variável t temos, para o nível de tempo t_1 :

$$u(x_i, t_1) = u(x_i, \Delta t) = u(x_i, 0) + \Delta t u_t(x_i, 0) + \left(\frac{\Delta t}{2}\right)^2 u_{tt}(x_i, 0) + \mathcal{O}(\Delta t^3).$$

As condições iniciais são usadas para calcular os dois primeiros termos deste desenvolvimento, pois $u(x, 0) = u^0(x)$ e $u_t(x_i, 0) = v^0(x)$. Para calcular o terceiro termo usamos a equação da onda:

$$u_{tt}(x, 0) = f(x, 0) + a u_{xx}(x, 0) = f(x, 0) + a \frac{d^2}{dx^2} u^0(x).$$

Na segunda passagem, usamos a derivada da condição inicial. Dessa maneira, estabelecemos aproximações para u_i^1 que são $\mathcal{O}(\Delta t^3)$:

$$u_i^1 = u_i^0 + \Delta t v_i^0 + \left(\frac{\Delta t}{2}\right)^2 \left(f(x_i, 0) + a^2 \frac{d^2}{dx^2} u^0(x) \Big|_{x_i} \right). \quad (9.44)$$

Reunimos os argumentos no Algoritmo 9.5, que é uma esquematização do Método Explícito.

O algoritmo do Método Implícito tem as alterações correspondentes ao sistema de equações lineares que deve ser montado e resolvido.

Quanto à estabilidade dos métodos apresentados nesta seção, observamos que o Método Explícito é condicionalmente estável (ver [Mitchell-Griffiths, 1980] ou [Ames, 1977], para maiores detalhes). A estabilidade do Método Implícito está sujeita à condição

$$\sqrt{a} \frac{\Delta t}{\Delta x} < 1.$$

Por outro lado o Método Implícito é incondicionalmente estável.

Algoritmo 9.5 Método Explícito para equação da onda

Dados $\Delta t, \Delta x, N, M, u^0(x), v^0(x), g_1(t), g_2(t), f(x, t), \lambda = \Delta t/\Delta x$ e $w(x) = a(x, 0) \frac{d^2 u^0(x)}{dx^2}$

{Condições iniciais}

1: Para $i = 1 : M - 1$, faça

2: $u_i^0 = u^0(i \Delta x)$

3: $u_i^1 = u_i^0 + \Delta t v_i^0(i \Delta x) + \left(\frac{\Delta t}{2}\right) [f(i \Delta x, 0) + w(i \Delta x)]$

{Condições de contorno}

4: Para $k = 1 : N$, faça

5: $u_0^k = g_1(k \Delta t)$

6: $u_M^k = g_2(k \Delta t)$

7: Para $k = 1 : N - 1$, faça

8: Para $i = 1 : M - 1$, faça

9: $u_i^{k+1} = 2(1 - \lambda^2 a_i^k) u_i^k + \lambda^2 a_i^k (u_{i+1}^k + u_{i-1}^k) - u_i^{k-1} + \Delta t^2 f_i^k$

Exercícios

1. Verifique que a mudança de variáveis $\xi = x - Vt$ (onde V é uma constante) $\tau = t$, numa função $w(x, t)$, faz com que

$$w_t = w_\xi V + w_\tau, \quad w_x = w_\xi \quad \text{e} \quad w_{xx} = w_{\xi\xi}.$$

Usando este fato, mostre que a equação convecção-difusão nas variáveis (x, t) ,

$$w_t = \frac{1}{P_e} w_{xx} - V w_x,$$

transforma-se numa equação parabólica nas variáveis (ξ, τ)

$$w_\tau = \frac{1}{P_e} w_{\xi\xi}.$$

Você vê alguma vantagem nesta transformação?

2. Use o Método de Diferenças Finitas para encontrar aproximações para as equações de Poisson abaixo. As equações são definidas em retângulos e os dados são fornecidos no contorno destes retângulos. Quando possível, compare com a solução analítica. Use vários valores para Δx e Δy , de acordo com suas possibilidades.

a. $u_{xx} + u_{yy} = x$, $0 < x < 1$ e $0 < y < 1$ com condições

$$u(x, 0) = u(x, 1) = \frac{x^3}{6}, \quad 0 \leq x \leq 1$$

$$u(0, y) = 0, \quad u(1, y) = \frac{1}{6}, \quad 0 \leq y \leq 1;$$

b. $u_{xx} + u_{yy} = -2 - \pi^2 \sinh(\pi) \sin(\pi y)$, $0 < x < 1$ e $0 < y < 1$ com condições

$$u(x, 0) = u(x, 1) = x(1 - x), \quad 0 \leq x \leq 1$$

$$u(0, y) = u(1, y) = \sinh(\pi) \sin(\pi y), \quad 0 \leq y \leq 1,$$

com solução analítica $u(x, y) = \sinh(\pi) \sin(\pi y) + x(1 - x)$;

c. $u_{xx} + u_{yy} = -[\cos(x + y) + \cos(x - y)]$, $0 < x < \pi$ e $0 < y < \frac{\pi}{2}$ com condições

$$u(x, 0) = \cos(x), \quad u(x, \pi/2) = 0, \quad 0 \leq x \leq \pi$$

$$u(0, y) = \cos(y), \quad u(\pi, y) = -\cos(y), \quad 0 \leq y \leq \pi/2,$$

com solução analítica $u(x, y) = \cos(x) \cos(y)$.

3. Descreva como o Método de Diferenças Finitas pode ser usado para encontrar aproximações de $u(x, y)$ que resolvem o seguinte problema:

$$u_{xx} + u_{yy} = 32u \quad \text{se } -1 < x < 1 \text{ e } -1 < y < 1$$

$$u(x, -1) = 1, \quad u(x, 1) = 0, \quad 0 \leq x \leq 1$$

$$u_x(-1, y) = \frac{u}{2}, \quad u(1, y) = -\frac{u}{2}, \quad -1 < y < 1.$$

4. Encontre as soluções aproximadas para os problemas parabólicos apresentados abaixo, usando os métodos Explícito, Implícito e de Crank-Nicolson. Compare os resultados obtidos pelos três métodos e também com a solução analítica, quando for possível.

a. $u_t - u_{xx} = 0$, $0 < x < 2$ e $t > 0$ com condições

$$u(0, t) = u(2, t) = 0$$

$$u(x, 0) = x(2 - x), \quad 0 \leq x \leq 2;$$

b. $\pi^2 u_t - u_{xx} = 0$, $0 < x < 1$ e $t > 0$ com condições

$$u(0, t) = u(1, t) = 0, \quad t > 0$$

$$u(x, 0) = \cos \pi(x - 0.5), \quad 0 \leq x \leq 1,$$

com solução analítica $u(x, t) = e^{-t} \cos \pi(x - 0.5)$;

c. $u_t - u_{xx} = 2$, $0 < x < 1$ e $t > 0$ com condições

$$u(0, t) = u(1, t) = 0, \quad t > 0$$

$$u(x, 0) = \sin \pi x + x(1 - x), \quad 0 \leq x \leq 1,$$

com solução analítica $u(x, t) = e^{-\pi^2 t} \sin \pi x + x(1 - x)$.

5. Seja u solução da equação $u_t - x u_{xx} = 0$, $0 < x < 0.5$, $t > 0$, com condições de contorno

$$u(0, t) = 0 \quad \text{e} \quad u_x(0.5, t) = -0.5 u, \quad t > 0$$

e condição inicial $u(x, 0) = x(1 - x)$, $0 < x \leq 0.5$.

a. Verifique que as equações abaixo definem uma discretização (explícita) para o problema

$$u_i^{j+1} = i \lambda h u_{i-1}^j + (1 - 2i \lambda h) u_i^j + i \lambda h u_{i+1}^j, \quad i = 1 : N - 1$$

e

$$u_N^{j+1} = 2N \lambda h u_{N-1}^j + (1 - N \lambda h(h + 2)) u_N^j$$

onde

$$\lambda = k/h^2, \quad Nh = 0.5 \quad \text{e} \quad u_0 = 0.$$

- b. Tome $h = 0.1$, $\lambda = 0.5$ e calcule aproximações, no primeiro nível de tempo, correspondentes a $i = 3$ e $i = 5$. As aproximações serão boas nos pontos próximos de $(0.5, 0)$? Explique.

6. Verifique as discretizações possíveis para o problema não-linear:

$$u_t - a(u) u_{xx} = f(x, t) \quad 0 < x < 1, \quad t > 0$$

$$u(0, t) = u(1, t) = 0, \quad t > 0$$

$$u(x, 0) = g(x), \quad 0 \leq x \leq 1,$$

onde $a(u) > 0$, $f(x, t)$ e $g(x)$ são funções conhecidas. Estabeleça as vantagens e desvantagens de cada um dos métodos propostos.

7. Encontre soluções aproximadas para os problemas associados às equações da onda

- a. $u_{tt} - 0.25 u_{xx} = 0$, $0 < x < 0.5$ e $t > 0$, com condições

$$u(0, t) = u(0.5, t) = 0, \quad t > 0$$

$$u(x, 0) = 0, \quad u_t(x, 0) = \sin 4\pi x \quad 0 \leq x \leq 0.5;$$

- b. $u_{tt} - u_{xx} = 0$, $0 < x < 1$ e $t > 0$, com condições

$$u(0, t) = u(1, t) = 0, \quad t > 0$$

$$u(x, 0) = \sin \pi x, \quad u_t(x, 0) = 0 \quad 0 \leq x \leq 1,$$

com solução analítica $u(x, t) = \sin \pi x \sin \pi t$;

- c. $u_{tt} - u_{xx} = 0$, $0 < x < 1$ e $t > 0$, com condições

$$u(0, t) = u(1, t) = 0, \quad t > 0$$

$$u(x, 0) = \begin{cases} 1 & \text{se } 0 \leq x \leq 0.5, \\ -1 & \text{se } 0.5 \leq x \leq 1 \end{cases}$$

$$u_t(x, 0) = 0, \quad 0 \leq x \leq 1.$$

Verifique como as soluções se comportam em $t = 0.5$ quando usamos $h = 0.1$ e $h = 0.01$.

Capítulo 10

MÉTODOS PARA EQUAÇÕES DIFERENCIAIS PARCIAIS COM PREDOMINÂNCIA CONVECTIVA

10.1 Introdução

O objetivo deste capítulo é apresentar alguns métodos numéricos aplicáveis às equações diferenciais parciais nas quais o termo convectivo predomina. Dentro do escopo deste livro, apresentaremos métodos baseados em diferenças finitas e nos restringiremos aos problemas com apenas uma variável espacial.

Métodos numéricos que reproduzam soluções que possuam frentes abruptas têm desafiado engenheiros e analistas numéricos nas últimas décadas. Muito tem sido publicado sobre diferenças finitas, elementos finitos, métodos adaptativos, malhas que se movem etc. Entretanto, não há consenso de métodos numéricos completamente satisfatórios. Ocorre que os métodos de discretização usuais foram desenvolvidos para funções suaves, como foi visto na dedução das fórmulas que aproximam derivadas por diferenças finitas. Quando estas discretizações são usadas na equação diferencial, introduzem junto de si o erro de truncamento da fórmula utilizada. Desta maneira, a equação diferencial está sendo modificada pela introdução de um termo de derivada superior.

Vimos, no capítulo anterior, que o erro de truncamento não danifica as aproximações de soluções suaves. Entretanto, no caso de soluções descontínuas, o erro de truncamento pode suavizar excessivamente frentes abruptas ou introduzir oscilações espúrias nas aproximações.

10.2 Método das Características

Nesta seção, apresentamos uma técnica importante na análise e solução de equações diferenciais de primeira ordem.

Vamos considerar problemas de valor inicial para equações de primeira ordem. Dadas

as funções $a(x, t)$, $b(x, t)$ e $d(x, t)$, queremos encontrar $u(x, t)$, tal que

$$\begin{cases} a(x, t) u_x + b(x, t) u_t + d(x, t) u = 0, & t > 0, -\infty < x < \infty \\ u(x, 0) = u_0(x). \end{cases} \quad (10.1)$$

Vamos escrever a equação diferencial linear de primeira ordem (10.1), definida no sistema de coordenadas (x, t) , em um outro sistema de coordenadas (s, t) definido convenientemente. Consideremos as curvas $x(s)$ e $t(s)$ definidas por:

$$\begin{aligned} \frac{dx}{ds} &= a(x(s), t(s)), & x(0) &= x_i \\ \frac{dt}{ds} &= b(x(s), t(s)), & t(0) &= t_i \end{aligned} \quad (10.2)$$

As curvas definidas por (10.2) são chamadas características do problema. Usando a regra da cadeia para derivar $u(x(s), t(s))$ temos, ao longo das características,

$$\frac{du}{ds} = u_x \frac{dx}{ds} + u_t \frac{dt}{ds} = a(x, t) u_x + b(x, t) u_t.$$

Como o lado direito desta equação é parte da equação diferencial (10.1), temos

$$\begin{cases} \frac{du}{ds} = -d(x(s), t(s)) u \\ u(0) = u_0(x_i) \end{cases} \quad (10.3)$$

isto é, definimos uma equação diferencial ordinária no novo sistema de coordenadas (s, t) .

Comparando as duas formulações do problema, (10.1) e (10.3), observamos que passamos de uma equação diferencial parcial para uma equação diferencial ordinária. É como se, através de um truque matemático, tivéssemos reduzido a dimensão do problema: duas variáveis independentes, x e t , são usadas na primeira formulação, ao passo que uma única variável, s , define a segunda formulação.

Como todo ganho tem seu custo, a solução será encontrada em pontos que estão sobre as características, uma vez que a formulação (10.3) vale ao longo das características. Um processo de interpolação pode ser usado se estas características se afastam em algumas regiões.

Podemos sistematizar o Método das Características nos seguintes passos:

Passo 1 Resolvemos o sistema (10.2) usando algum método analítico ou numérico destinado a equações diferenciais ordinárias com dados iniciais. Neste passo encontramos a característica que passa por $(x_i, 0)$.

Passo 2 Ao longo dessa característica, resolvemos o outro problema de valor inicial (10.3).

Aqui também podemos usar algum método numérico conveniente, por exemplo, Métodos de Runge-Kutta.

Nem sempre é necessário recorrer a métodos numéricos para resolver as equações diferenciais ordinárias, como mostram os exemplos seguintes. Por esta razão o Método das Características também pode ser considerado um método para solução analítica das equações diferenciais de primeira ordem.

Exemplo 10.2.1. Vamos usar o Método das Características para resolver o problema de valor inicial

$$2u_x + u_t = 0, \quad -\infty < x < \infty, \quad t > 0$$

$$u(x, 0) = \begin{cases} 1 & \text{se } x < 0 \\ 2 & \text{se } x > 0. \end{cases}$$

No primeiro passo, vamos encontrar as características definidas por

$$\frac{dx}{ds} = 2 \quad \text{e} \quad \frac{dt}{ds} = 1.$$

As soluções são retas $x = 2s + c_1$ e $t = s + c_2$, e as constantes de integração são encontradas pelas condições iniciais, isto é, valores de $x(0)$ e $t(0)$ para $s = 0$.

Como as condições iniciais são fornecidas em $t = 0$, tomemos $t(0) = 0$, isto é, $c_2 = 0$. Vamos calcular as características que, no plano (x, t) , partem do eixo dos x , isto é, $x(0) = x_i$. Assim, as características desse problema são

$$x(s) = 2s + x_i \quad \text{e} \quad t(s) = s.$$

Substituindo s na curva $x(s)$ temos a expressão das características no plano (x, t) , as retas

$$x = 2t + x_i$$

representadas na figura 10.1.

Encontradas as características, resolvemos a equação diferencial correspondente a (10.3) que, neste caso, é

$$\frac{du}{ds} = 0$$

ou seja, u é uma constante ao longo das características. Como a condição inicial é descontínua em $x = 0$: $u = 1$, ao longo das características correspondentes a $x_i < 0$, e $u = 2$, ao longo das características correspondentes a $x_i > 0$. Traçando a reta $x = 2t$ ($x_i = 0$), vemos que $u(x, t) = 1$ à esquerda desta reta e $u(x, t) = 2$ à sua direita. Finalmente, podemos escrever a solução:

$$u(x, t) = \begin{cases} 1 & \text{se } x < 2t \\ 2 & \text{se } x > 2t \end{cases}$$

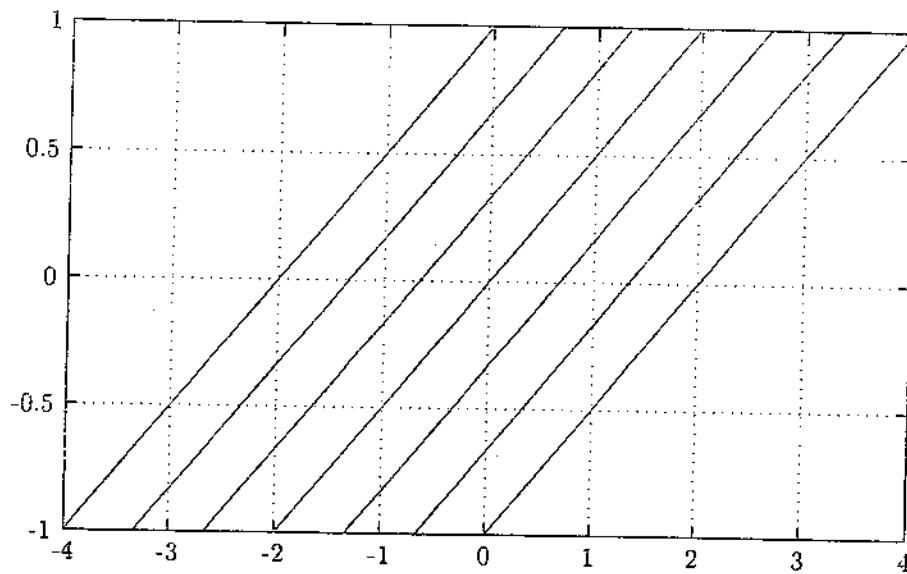


Figura 10.1: Características $x = 2t + x_i$.

que é uma função constante por partes, qualquer que seja $T > 0$, com descontinuidade em $x = 2T$. Este exemplo ilustra uma importante peculiaridade das características: informações iniciais são transportadas ao longo destas curvas. $\square$

Exemplo 10.2.2. Neste exemplo, aplicamos o Método das Características para resolver a equação diferencial com coeficientes variáveis

$$\begin{aligned} \frac{1}{\sqrt{t}} u_x + 2u_t &= 1, & -\infty < x < \infty, t > 0 \\ u(x, 0) &= x, & -\infty < x < \infty. \end{aligned}$$

As características são definidas por

$$\frac{dx}{ds} = \frac{1}{\sqrt{t}} \quad \text{e} \quad \frac{dt}{ds} = 2.$$

Pela segunda equação temos $t = 2s$. Substituindo t na primeira equação temos

$$\frac{dx}{ds} = \frac{1}{\sqrt{2s}}$$

cujas solução é obtida por integração $x(s) = \sqrt{2s} + x_i$, onde x_i é o ponto de partida da característica. No plano (x, t) , as características são $x = \sqrt{t} + x_i$, que são as parábolas $t = (x - x_i)^2$.

No segundo passo resolvemos a equação diferencial

$$\frac{du}{ds} = 1.$$

Assim, $u(s) = s + c = t/2 + c$, ao longo das características. A condição inicial do problema nos permite calcular a constante de integração: $u(x_i, 0) = x_i = c$. Sendo assim, a expressão da solução será

$$u(x, t) = \frac{t}{2} + x_i,$$

ao longo das curvas $t = (x - x_i)^2$, para qualquer x_i .

Neste caso particular poderíamos substituir o valor $x_i = x - \sqrt{t}$ na expressão da solução e obter a forma geral

$$u(x, t) = x + \frac{t}{2} - \sqrt{t}.$$

□

Observação 1 Um esquema análogo ao descrito nesta seção pode ser usado na resolução das equações hiperbólicas semilineares.

Para ilustrar tomemos o problema não linear,

$$\begin{cases} a(x, y, u) u_x + b(x, y, u) u_y + d(x, y, u) = 0 \\ u(x, y) \text{ conhecida em uma curva (curva inicial).} \end{cases}$$

Neste caso, as características são definidas por

$$\begin{cases} \frac{dx}{ds} = a(x, y, u), & x(0) = x_i \\ \frac{dy}{ds} = b(x, y, u), & y(0) = y_i \end{cases}$$

e a equação diferencial ao longo das características é

$$\frac{du}{ds} = -d(x, y, u), \quad u(0) = u(x_i, y_i).$$

Para resolver estas equações diferenciais ordinárias não-lineares, podemos usar o Método de Runge-Kutta.

Observação 2 Os problemas não-lineares, como a equação apresentada acima, trazem algumas complicações do ponto de vista matemático. Por exemplo, nem sempre é garantida a unicidade da solução. O exemplo clássico, e talvez o mais estudado, é a equação de Burger

$$u_t + u u_x = 0.$$

Pode-se mostrar que, com o mesmo dado inicial, a equação de Burger apresenta mais de uma solução.

Observação 3 A idéia central do método da características pode ser generalizada para sistemas de equações diferenciais de primeira ordem e para equações de ordem superior, que podem ser transformadas em sistemas de primeira ordem. Através de mudanças de variáveis convenientes, escreve-se o sistema em uma forma mais simples (chamada canônica) e as direções características ficam definidas no próprio procedimento. O detalhamento deste processo exigiria um formalismo matemático que foge ao escopo deste livro. Os leitores interessados podem consultar, por exemplo, [LeVeque, 1990].

10.3 Métodos de Discretização para Equações Lineares

Nesta seção vamos considerar o problema de valor inicial

$$\begin{cases} u_t + a(x, t) u_x = 0, & t > 0 \\ u(x, 0) = u_0(x). \end{cases} \quad (10.4)$$

Nos métodos numéricos, a variável x deve estar em um domínio finito, $\alpha \leq x \leq \beta$. São, portanto, necessárias condições de contorno apropriadas em $x = \alpha$ e/ou $x = \beta$. Em alguns casos, é possível supor periodicidade da solução, isto é,

$$u(\alpha, t) = u(\beta, t), \quad t > 0,$$

o que é equivalente ao problema com condição inicial periódica.

A região discretizada é formada pelos pontos (x_i, t_n) , do plano (x, t) , construída com espaçamentos constantes $h = \Delta x$ e $k = \Delta t$,

$$x_i = \alpha + ih \quad \text{e} \quad t_n = jk.$$

Com a função $u_0(x)$ definimos a condição inicial da discretização, $U_i^0 = u_0(x_i, 0)$.

Os métodos de discretização de (10.4) podem ser estabelecidos pela substituição das derivadas por aproximações convenientes. Raramente são usados métodos implícitos neste tipo de equação diferencial. Por outro lado, alguns métodos explícitos são impraticáveis por problemas de instabilidade. Apresentamos a seguir alguns métodos explícitos que são usualmente aplicados na discretização de (10.4).

Euler: Fórmula avançada no tempo e centrada no espaço:

$$U_i^{n+1} = U_i^n - \frac{k}{2h} a_i^n (U_{i+1}^n - U_{i-1}^n). \quad (10.5)$$

Erro de truncamento: $\mathcal{O}(k) + \mathcal{O}(h^2)$.

Upwind: Fórmula avançada no tempo e avançada ou atrasada no espaço:

$$\begin{aligned} U_i^{n+1} &= U_i^n - \frac{k}{h} a_i^n (U_i^n - U_{i-1}^n) \quad \text{se } a_i^n > 0 \\ &= U_i^n - \frac{k}{h} a_i^n (U_{i+1}^n - U_i^n) \quad \text{se } a_i^n < 0. \end{aligned} \quad (10.6)$$

Erro de truncamento: $\mathcal{O}(k) + \mathcal{O}(h)$.

Lax-Friedrichs: Fórmula avançada no tempo, usando média para calcular U_i^n , e centrada no espaço:

$$U_i^{n+1} = \frac{1}{2} (U_{i-1}^n + U_{i+1}^n) - \frac{k}{2h} a_i^n (U_{i+1}^n - U_{i-1}^n). \quad (10.7)$$

Erro de truncamento: $\mathcal{O}(k) + \mathcal{O}(h^2)$.

Leapfrog: [Roache-Dietrich, 1988] Fórmula centrada no tempo e no espaço:

$$U_i^{n+1} = U_i^{n-1} - \frac{k}{h} a_i^n (U_{i+1}^n - U_{i-1}^n). \quad (10.8)$$

Erro de truncamento: $\mathcal{O}(k^2) + \mathcal{O}(h^2)$.

Para o caso coeficiente constante, $a(x, t) \equiv a$ em (10.4), pode-se também usar o Método Lax-Wendroff e suas variações. O Método de Lax-Wendroff foi desenvolvido em 1960-1964 e é baseado na expansão em série de Taylor

$$u(x, t+k) = u(x, t) + k u_t(x, t) + \frac{k^2}{2} u_{tt}(x, t) + \dots$$

Pela equação diferencial, $u_t = -a u_x$ e, portanto, a derivada de segunda ordem no tempo, será

$$u_{tt} = -a u_{xt} = -a u_{tx} = -a (-a u_x)_x = a^2 u_{xx}.$$

Assim, a expansão em série de Taylor fica

$$u(x, t+k) = u(x, t) - ka u_x(x, t) + \frac{k^2}{2} a^2 u_{xx}(x, t) + \dots$$

Usando a fórmula centrada para aproximar u_x e a fórmula convencional para aproximar u_{xx} , temos o

Lax-Wendroff:

$$U_i^{n+1} = U_i^n - \frac{k}{2h} a_i^n (U_{i+1}^n - U_{i-1}^n) + \frac{k^2 (a_i^n)^2}{2h^2} (U_{i+1}^n - 2U_i^n + U_{i-1}^n). \quad (10.9)$$

Erro de truncamento: $\mathcal{O}(k^2) + \mathcal{O}(h^2)$.

Uma versão do tipo *upwind* para o Método Lax-Wendroff é o

Método Beam-Warming: ($a = a_i^n > 0$)

$$U_i^{n+1} = U_i^n - \frac{k}{2h} a (3U_i^n - 4U_{i-1}^n + U_{i-2}^n) + \frac{k^2}{2h^2} a^2 (U_i^n - 2U_{i-1}^n + U_{i-2}^n). \quad (10.10)$$

Erro de truncamento: $\mathcal{O}(k^2) + \mathcal{O}(h^2)$.

Dizemos que um esquema de discretização é *consistente* com a equação diferencial se o erro de truncamento tende a zero quando $h \rightarrow 0$ e $k \rightarrow 0$. Nesta definição, os métodos apresentados acima são consistentes.

Um dos primeiros trabalhos sobre métodos de diferenças finitas para equações diferenciais parciais foi escrito por Courant, Friedrichs e Lewy em 1928, traduzido para o inglês em 1967. O método foi então usado como uma forma de provar a existência de soluções. Na demonstração de convergência, eles observam que uma condição necessária de estabilidade de um método numérico é que o domínio de dependência do meio inclua o domínio de dependência da equação diferencial [LeVeque, 1990].

Como vimos na seção anterior, nas equações diferenciais do tipo (10.4), as informações são transportadas pelas características $\frac{dx}{dt} = a$ (nas equações de segunda ordem temos também a característica $\frac{dx}{dt} = -a$).

Assim, os espaçamentos, no tempo e no espaço, devem satisfazer à condição

$$|a| \frac{k}{h} \leq 1, \quad (10.11)$$

conhecida como condição CFL, iniciais dos autores Courant, Friedrich e Lewy.

Embora a condição CFL nem sempre seja suficiente para garantir estabilidade (aqui entendida como limitação das aproximações no processo de evolução), ela é suficiente para a estabilidade dos esquemas apresentados nesta seção, (10.5)–(10.10).

Com argumentos de análise funcional Lax mostrou, em 1954, o teorema da equivalência, cuja demonstração pode ser encontrada, por exemplo, em [Richtmyer-Morton, 1967]. Este teorema fundamental para métodos de diferenças finitas consistentes com equações diferenciais parciais lineares nos diz que *a estabilidade é necessária e suficiente para a convergência das aproximações*.

Exemplo 10.3.1. Neste exemplo, vamos aplicar os métodos *upwind*, Lax-Friedrichs, Lax-Wendroff e Beam-Warming na equação diferencial do exemplo 10.2.1:

$$u_t + 2u_x = 0, \quad -\infty < x < \infty, \quad t > 0$$

$$u(x, t) = \begin{cases} 1 & \text{se } x < 0 \\ 2 & \text{se } x > 0 \end{cases}$$

Como vimos naquele exemplo a solução tem uma descontinuidade que se propaga ao longo da característica que parte da origem. No programa `uplax.m` (Anexo), implementamos os quatro métodos usando $h = 0.01$ e $\lambda = \Delta t/\Delta x = 0.1$. Os resultados estão apresentados na figura 10.2. Como podemos observar, as aproximações do método *upwind* reproduzem a descontinuidade com suavização provocada pelo erro da discretização. Esta difusão aumenta no Método Lax-Friedrichs. No Método de Lax-Wendroff aparecem fortes oscilações atrás da descontinuidade. Também no Método Beam-Warming, as oscilações se propagam agora na frente da descontinuidade.

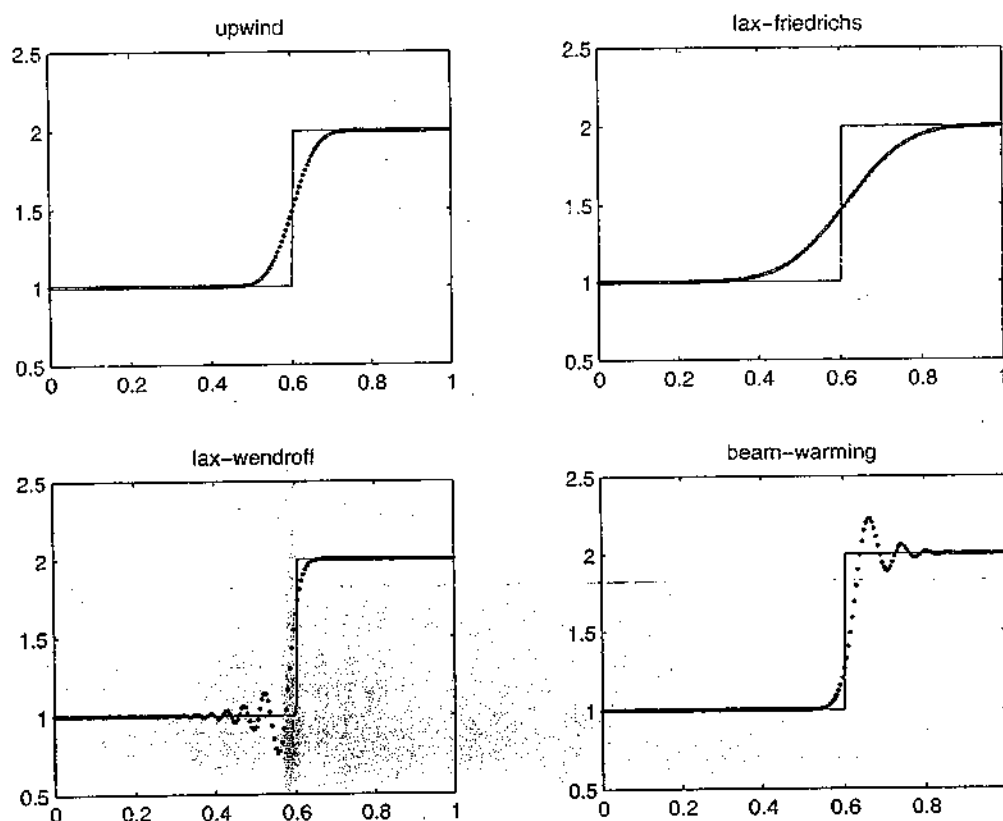


Figura 10.2: Resultado da aplicação de vários métodos.

Entretanto, se aplicarmos os quatro métodos em problemas com soluções suaves, os resultados dos dois últimos serão melhores devido aos erros da discretização. $\square$

10.4 Soluções Descontínuas: Equações Modificadas

O exemplo 10.3.1 mostra que, apesar de os métodos Lax-Wendroff e Beam-Warming terem erro de truncamento de segunda ordem, eles fornecem aproximações qualitativamente

piores do que os métodos *upwind* e Lax-Friedrichs, que têm erro de truncamento de primeira ordem. Observamos também que estes métodos de primeira ordem suavizam a região onde ocorre a descontinuidade da solução, não reproduzindo a descontinuidade que sabíamos existir pelo Exemplo 10.2.1.

Estes fatos podem ser explicados recorrendo às equações modificadas, [LeVeque, 1990]. A idéia é lançar mão de equações diferenciais que tenham soluções suaves e que sejam fiéis às equações discretizadas, nas quais incorporamos os erros de arredondamento. Vamos ilustrar esta idéia através do método *upwind*, aplicado em (10.4) com $a > 0$. Neste caso, usamos as fórmulas avançada no tempo e atrasada no espaço.

Desenvolvendo $u(x, t)$ em série de Taylor temos

$$u(x, t+k) = u(x, t) + k u_t(x, t) + \frac{k^2}{2} u_{tt}(x, t) + \frac{k^3}{6} u_{ttt}(x, \xi_1)$$

e

$$u(x-h, t) = u(x, t) - h u_x(x, t) + \frac{h^2}{2} u_{xx}(x, t) - \frac{h^3}{6} u_{xxx}(\xi_2, t).$$

Assim, podemos acrescentar mais termos nas aproximações de derivadas pelas diferenças avançadas e atrasadas:

$$\frac{U_i^{n+1} - U_i^n}{k} = u_t(x, t) + \frac{k}{2} u_{tt}(x, t) + \frac{k^2}{6} u_{ttt}(x, \xi_1)$$

e

$$\frac{U_i^n - U_{i-1}^n}{h} = u_x(x, t) - \frac{h}{2} u_{xx}(x, t) + \frac{h^2}{6} u_{xxx}(\xi_2, t).$$

Lembrando que, no método *upwind*, (10.6), usamos diferenças avançadas no tempo e atrasadas no espaço (supondo $a > 0$), vemos agora que este método fornece aproximações de segunda ordem para a equação diferencial

$$u_t + a u_x + \frac{1}{2} (k u_{tt} - ah u_{xx}) = 0. \quad (10.12)$$

Vamos trabalhar um pouco mais nos dois últimos termos desta equação. Derivando a equação com relação a t e x temos, respectivamente

$$u_{tt} = -a u_{xt} - \frac{1}{2} (k u_{ttt} - ah u_{xtt})$$

e

$$u_{xt} = u_{tx} = -a u_{xx} - \frac{1}{2} (k u_{ttx} - ah u_{xxx}).$$

Portanto, juntando estas duas equações, teremos:

$$u_{tt} = -a(-a u_{xx}) + \mathcal{O}(k) + \mathcal{O}(h).$$

Substituindo u_{tt} na equação (10.12) e supondo $k/h = \text{constante}$, temos

$$u_t + a u_x + \frac{a}{2} (ka - h) u_{xx} + \mathcal{O}(k^2) + \mathcal{O}(h^2) = 0.$$

Esta equação nos diz que a discretização do método *upwind* fornece aproximações de segunda ordem para a equação de convecção-difusão,

$$u_t + a u_x = \frac{a}{2} h \left(1 - \frac{k}{h} \right) u_{xx}.$$

Desta maneira, os resultados computacionais fornecerão melhores aproximações para uma equação que tem um termo difusivo, do que para a equação sem o termo difusivo,

$$u_t + a u_x = 0,$$

para a qual ele foi desenhado.

Estes argumentos justificam qualitativamente os resultados apresentados na figura do exemplo 10.3.1, para os métodos *upwind* e Lax-Friedrichs. O coeficiente de difusão introduzido pela discretização Lax-Friedrichs é [LeVeque, 1990]

$$D = \frac{h^2}{2k} \left(1 - \frac{k^2}{h^2} a^2 \right).$$

Por outro lado, usando algumas manipulações análogas às do método *upwind*, pode-se mostrar que a equação modificada mais apropriada para o Método Lax-Wendroff é uma equação dispersiva:

$$u_t + a u_x = \frac{h^2}{6} a \left(\frac{k^2}{h^2} a^2 - 1 \right) u_{xxx}.$$

Os componentes oscilatórios da solução deste tipo de equação [Trefethen, 1982] justificam as oscilações que caracterizam as aproximações fornecidas pelos métodos com erro de truncamento de segunda ordem, como é o caso das discretizações de Lax-Wendroff e Beam-Warming, ilustradas na figura 10.2.

10.5 Problemas Não-Lineares: Leis de Conservação

Nesta seção, fazemos uma breve introdução sobre cálculo de aproximações para as equações diferenciais parciais do tipo

$$u_t + f(u)_x = 0, \quad (10.13)$$

chamadas leis de conservação não-lineares. As leis de conservação formam um capítulo à parte no estudo das equações diferenciais parciais e têm sido objeto de muitos estudos analíticos e numéricos nas últimas décadas.

Vários são os motivos de tanto interesse no assunto. Como não pretendemos nos aprofundar no assunto, aqui apresentamos apenas algumas idéias com o ambicioso intento de colaborar na solução numérica de problemas práticos desta natureza.

Grande parte do estudo de leis de conservação foi motivado pelas equações de Euler para a dinâmica de gases. Este sistema de equações é modelo matemático para a conservação de massa, de energia e da quantidade de movimento para fluxo de gases, cujo efeito suavizador da viscosidade pode ser desprezado:

$$\frac{\partial}{\partial t} \begin{bmatrix} \rho \\ \rho v \\ E \end{bmatrix} + \frac{\partial}{\partial x} \begin{bmatrix} \rho v \\ \rho v^2 + p \\ v(E + p) \end{bmatrix} = 0. \quad (10.14)$$

Neste sistema de leis de conservação $\rho = \rho(x, t)$ representa a densidade, v a velocidade, ρv a quantidade de movimento, E a energia e p a pressão. Todas estas funções dependem do tempo e da variável espacial.

Estas equações exemplificam a importância prática do problema. Com elas, pode-se verificar a propagação de ondas de choques, descontinuidades na densidade, pressão e velocidade, e regiões onde estas funções permanecem constantes.

Nesta aparente contradição entre soluções descontínuas para equações que envolvem suas derivadas, destacamos os dois pontos que comentamos a seguir.

Solução Fraca

As soluções descontínuas não satisfazem às equações diferenciais parciais no sentido clássico. Para contornar esta dificuldade, usam-se formulações integrais que permitam soluções descontínuas. Esta formulação integral define soluções num sentido mais amplo: as chamadas soluções fracas.

Não-Unicidade

Ao ampliar o conceito de solução, enfrentamos outra dificuldade que é a existência de mais de uma solução fraca. Se estamos modelando fenômenos físicos, é natural que apenas uma das soluções fracas tenha significado físico. O fato de existir mais de uma solução é consequência de alguma hipótese simplificadora. No caso das equações de Euler, por exemplo, desprezamos a viscosidade. Modelos que incorporam a viscosidade, por pequena que seja, são as equações de Navier-Stokes, que, como é sabido, têm uma única solução. As aparentes frentes de descontinuidade podem ser, na realidade, regiões de variações bruscas na solução.

Existindo mais de uma solução, é necessário estabelecer o critério de escolha da solução fisicamente relevante. A sugestão é introduzir um termo difusivo (como o termo da viscosidade nas equações de Euler), para obter unicidade de solução suave e, então, fazer com que este termo tenda para zero. Algumas relações matemáticas viabilizam a utilização prática deste critério de escolha da única solução. Estas relações são chamadas *condições de entropia*, termo herdado da dinâmica dos gases.

Por exemplo, como vimos na equação modificada do método *upwind* (10.12) e manipulações posteriores, o termo difusivo é responsável pela aproximação apresentada na figura 10.2.

Como vimos na seção anterior, só nos métodos de discretização com erro de truncamento de primeira ordem não surgiram oscilações indesejáveis nas proximidades das descontinuidades. Os métodos numéricos para leis de conservação devem, além de ser de primeira ordem, também ser discretizações da equação diferencial na forma conservativa, como ilustra o exemplo a seguir.

Exemplo 10.5.1. Uma das leis de conservação mais conhecidas é a equação de Burger, na qual

$$f(u) = \frac{u^2}{2},$$

isto é, a equação diferencial, escrita na forma conservativa,

$$u_t + \frac{1}{2} (u^2)_x = 0. \quad (10.15)$$

Como $\frac{\partial}{\partial x}(u^2) = 2u u_x$, a equação de Burger pode ser reescrita na forma não conservativa

$$u_t + u u_x = 0. \quad (10.16)$$

Na discretização da equação de Burger, vamos aplicar a fórmula avançada no tempo e atrasada no espaço em (10.16). Assim, obtemos o

Método 1

$$U_i^{n+1} = U_i^n - \frac{k}{h} U_i^n (U_i^n - U_{i-1}^n).$$

Por outro lado, poderíamos usar a forma conservativa (10.15) aplicando a discretização no segundo termo, sem "desmanchar" a derivada parcial com relação a x :

$$\frac{U_i^{n+1} - U_i^n}{k} + \frac{f(U_i^n) - f(U_{i-1}^n)}{h} = 0.$$

Temos então um novo método numérico para o problema:

Método 2

$$U_i^{n+1} = U_i^n - \frac{k}{2h} [(U_i^n)^2 - (U_{i-1}^n)^2].$$

Para um exemplo numérico, tomemos a seguinte condição inicial

$$u(x, 0) = \begin{cases} u_e = 2 & \text{se } x < 0 \\ u_d = 0.5 & \text{se } x > 0. \end{cases}$$

No programa `burger.m`, listagem no Anexo, aplicamos os métodos 1 e 2, com $h = 0.01$ e $\lambda = \Delta t / \Delta x = 0.1$. Os resultados estão na figura 10.3, na qual a solução obtida pelo método 2, que é conservativo, acompanha a solução analítica, apresentando a suavização associada à difusão numérica. A solução analítica do problema é uma função com o degrau definido pela condição inicial que se desloca com velocidade

$$\frac{dx}{dt} = \frac{1}{2}(u_e + u_d).$$

Os detalhes da obtenção desta solução exata podem ser obtidos, por exemplo, no capítulo 3 de [LeVeque, 1990].

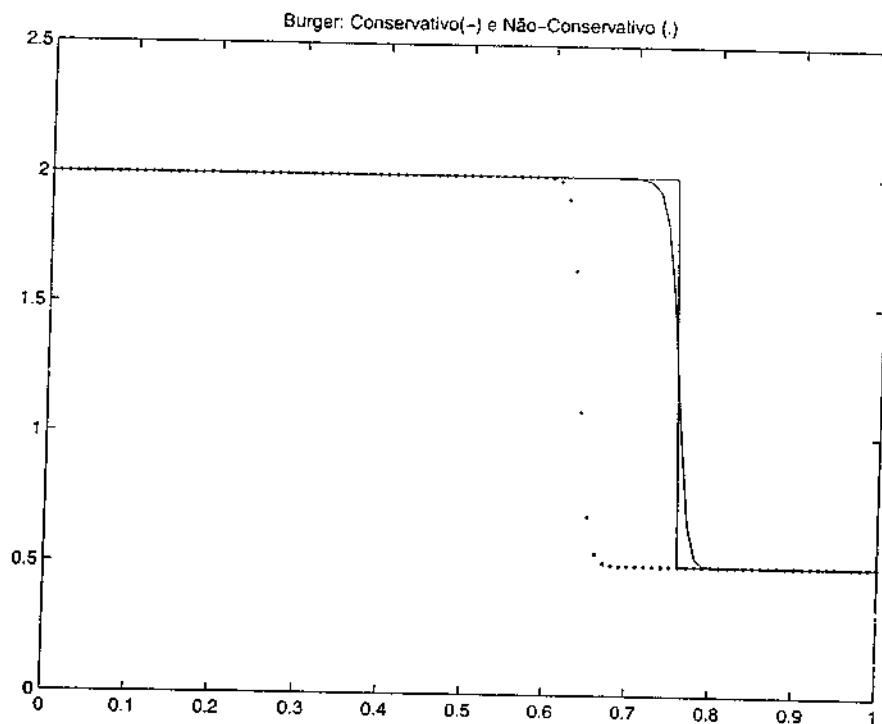


Figura 10.3: Solução da equação de Burger.

Por outro lado, vemos pela figura 10.3 que a solução apresentada pelo método 1, que é não conservativo, não acompanha a velocidade da descontinuidade. Esta “solução errada” é o gráfico pontilhado na figura 10.3. □

Em resumo, os argumentos apresentados nesta seção recomendam que os métodos de discretização das leis de conservação sejam métodos com erro de truncamento de primeira ordem e consistentes com a equação diferencial na forma conservativa.

10.6 Método TVD: um Exemplo de Métodos de Alta Resolução

Como vimos nas seções anteriores, ao resolver numericamente leis de conservação devemos usar aproximações calculadas com métodos de primeira ordem. Isto nos obriga a usar malhas refinadas para conseguir resultados com precisão.

Como as descontinuidades ocorrem em algumas regiões isoladas no domínio da solução, ao usar um método de primeira ordem em todo o domínio, estamos deixando de usar métodos que poderiam ser mais econômicos nas regiões de suavidade da solução.

A idéia dos métodos de alta resolução é aproveitar as regiões nas quais a solução é suave usando nelas algum método de alta ordem. Para atingir o objetivo (usando métodos de ordem acima de 1), introduzimos modificações na vizinhança da descontinuidade, de modo a aumentar a dissipação numérica nestas regiões. Os outros itens, comentados anteriormente, devem ser respeitados: discretização consistente com a forma conservativa, ausência de oscilações falsas, boa reprodução de frentes abruptas, estabilidade, forma discretizada da condição de entropia para escolha da solução fisicamente correta.

Denotaremos por F o fluxo do esquema numérico associado à equação (10.13):

$$U_j^{n+1} = U_j^n - \frac{k}{h} [F(U_j^n, U_{j+1}^n) - F(U_{j-1}^n, U_j^n)]. \quad (10.17)$$

Por exemplo, no método *upwind* para a equação (L de linear, isto é, ordem 1)

$$F_L(U_j, U_{j+1}) = f(U_j), \quad \text{se } f' > 0 \quad (10.18)$$

ao passo que, no Método Lax-Wendroff (H de *high order*)

$$F_H(U_j, U_{j+1}) = \frac{1}{2} [F(U_{j+1}) + F(U_j)] - \frac{k}{2h} [F(U_{j+1}) - F(U_j)] \quad (10.19)$$

Nos métodos com limitadores de fluxo, combinamos métodos de primeira ordem com métodos de ordem superior e usamos uma função cuja finalidade é corrigir o fluxo, quando necessário.

Assim, usamos o fluxo

$$F(U; j) = F_L(U; j) + \phi(U; j) [F_H(U; j) - F_L(U; j)] \quad (10.20)$$

no qual $\phi(U; j)$ é o limitador. Observe que se $\phi \equiv 0$, $F = F_L$ e se $\phi \equiv 1$, $F = F_H$.

Analisemos inicialmente o caso linear, $u_t + a u_x = 0$, $a > 0$. Tomando o fluxo do Método de Lax-Wendroff (10.19), que é de segunda ordem, como F_H , e o fluxo do método *upwind*

(10.18), como F_L temos

$$\begin{aligned} F_H - F_L &= \frac{a}{2} (U_{j+1} + U_j) - \frac{k}{2h} a (U_{j+1} - U_j) - a U_j = \\ &= \frac{a}{2} (U_{j+1} - U_j) - \frac{k}{2h} a (U_{j+1} - U_j) = \\ &= \frac{a}{2} \left(1 - \frac{k}{h}\right) (U_{j+1} - U_j). \end{aligned}$$

O fluxo do método com limitador de fluxo, (10.20), no caso linear $u_t + a u_x = 0$, $a > 0$, será

$$F(U; j) = a U_j + \frac{a}{2} \left(1 - \frac{k}{h}\right) (U_{j+1} - U_j) \phi_j \quad (10.21)$$

Uma das maneiras de testar a suavidade de $u(x, t_n)$ é através do quociente entre os valores da derivada em pontos consecutivos, acrescentando ϵ no denominador para evitar divisão por zero. Assim, para testar suavidade em x_j usamos o coeficiente

$$\theta_j = \frac{U_j - U_{j-1}}{U_{j+1} - U_j + \epsilon}. \quad (10.22)$$

Desta expressão, observamos que $\theta_j \approx 1$ indica que a solução deve ser suave em x_j , ao passo que $|\theta_j| \gg 1$ indica alguma singularidade de $u(x, t_n)$ em $x = x_j$.

Podemos então definir o limitador de fluxo como uma função de θ_j :

$$\phi_j = \phi(\theta_j).$$

Levando em conta o objetivo de ϕ como corretor de fluxo ($\phi = 0 \Rightarrow F = F_L$ e $\phi = 1 \Rightarrow F = F_H$) e a definição dos indicadores de choque (10.22), devemos ter

$$\phi(0) = 0 \quad \text{e} \quad \phi(1) = 1.$$

Vários autores estudaram expressões para os limitadores de fluxo, como pode ser verificado na referências de [LeVeque, 1990]. Por exemplo, [Roe, 1985] sugere

$$\phi(\theta) = \max\{0, \min(1, 2\theta), \min(\theta, 2)\} \quad (10.23)$$

e [VanLeer, 1974] usa

$$\phi(\theta) = \frac{|\theta| + \theta}{1 + |\theta|}. \quad (10.24)$$

Na versão *upwind* do método com limitadores de fluxo, tomamos uma fórmula válida para qualquer que seja o sinal de a , em $u_t + a u_x = 0$:

$$F_L(U; j) = \frac{1}{2} a (U_j + U_{j+1}) - \frac{1}{2} |a| (U_{j+1} - U_j). \quad (10.25)$$

Usando a adaptação de (10.19) para este caso e recalculando $F_H - F_L$, obtemos a generalização de (10.21). Assim, a versão *upwind* com limitador de fluxo para $u_t + a u_x = 0$ será

$$F(U; j) = F_L + \frac{a}{2} \left(\operatorname{sgn}(a) - \frac{k}{h} \right) (U_{j+1} - U_j) \phi_j \quad (10.26)$$

onde usamos a função $\operatorname{sgn}(a)$:

$$\operatorname{sgn}(a) = \begin{cases} 1 & \text{se } a > 0 \\ -1 & \text{se } a < 0. \end{cases}$$

Nesta versão, o limitador $\phi_j = \phi(\theta_j)$ é calculado em θ_j , o quociente das inclinações na direção de *upwind*, que dependem do sinal de a . Tomando $j' = j - \operatorname{sgn}(a) = j \pm 1$, tomamos

$$\theta_j = \frac{U_{j'+1} - U_{j'}}{U_{j+1} - U_j}.$$

A versão deste método para leis de conservação não-lineares pode ser obtida substituindo a por um valor local, para cada ponto da malha:

$$a_j = \frac{f(U_{j+1}) - f(U_j)}{U_{j+1} - U_j}.$$

Adaptando a expressão (10.26) para o caso não-linear, temos, fazendo $\lambda = k/h$, o seguinte caso de discretização:

$$\text{Se } a_j > 0: \quad \theta_j = \frac{U_j - U_{j-1}}{U_{j+1} - U_j + \epsilon}; \quad \phi_j = \phi(\theta_j),$$

$$U_j^{n+1} = U_j^n - \lambda [f(U_j) - f(U_{j-1})] + \phi_j \frac{\lambda}{2} (1 - \lambda) [f(U_{j+1}) - 2f(U_j) + f(U_{j-1})];$$

$$\text{se } a_j < 0: \quad \theta_j = \frac{U_{j+2} - U_{j+1}}{U_{j+1} - U_j + \epsilon}; \quad \phi_j = \phi(\theta_j),$$

$$U_j^{n+1} = U_j^n - \lambda [f(U_{j+1}) - f(U_j)] - \phi_j \frac{\lambda}{2} (1 + \lambda) [f(U_{j+1}) - 2f(U_j) + f(U_{j-1})];$$

onde $\phi(\theta)$ pode ser (10.23), (10.24) ou outra expressão para os limitadores de fluxo.

[Harten, 1983] introduziu limitadores de fluxo relacionados ao não-aumento da variação total da solução no processo de evolução. Estes limitadores exigem que

$$TV(U^{n+1}) \leq TV(U^n)$$

onde

$$TV(U^n) = \sum_i |U_{i+1}^n - U_i^n|.$$

Muitos trabalhos têm sido realizados neste assunto nas últimas décadas; estes métodos são chamados TVD; ver por exemplo [LeVeque, 1990].

Para concluir esta seção, apresentamos um exemplo no qual aplicamos uma versão do método TVD apresentada por [Chen et al., 1991] e que pode também ser encontrada em [Finlayson, 1992].

Exemplo 10.6.1. A equação Buckley-Leverett é usada para simular o fluxo bifásico em meios porosos, quando a aproximação unidimensional é válida. A função fluxo, que neste caso é chamada *fluxo fracionário*, é uma função crescente em forma de S, isto é, com um ponto de inflexão. Uma expressão usada é

$$f(s) = \frac{s^3}{s^3 + a(1-s)^3},$$

onde a representa a razão de viscosidade dos fluidos [Lake, 1989].

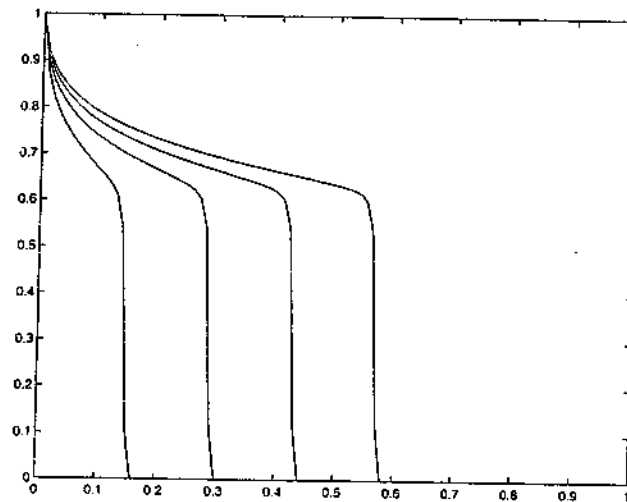


Figura 10.4: Resultado do Método TVD.

No programa `bltvd.m`, Anexo, aplicamos o Método TVD na equação

$$\frac{\partial s}{\partial t} + \frac{\partial f(s)}{\partial x} = 0, \quad x > 0 \quad \text{e} \quad t > 0,$$

com condição inicial $s(x, 0) = 0$ e condição de contorno $s(0, t) = 1$.

Na figura 10.4, apresentamos a evolução no perfil de saturação: da esquerda para direita a distribuição de $s(x, T)$ com $T = 0.1, 0.2, 0.3$ e 0.4 . Nesta discretização usamos $\Delta t/\Delta x = 0.05$ e $\Delta x = 0.01$. $\square$

Exercícios

1. Considere o sistema hiperbólico, com duas equações nas incógnitas $u(x, t)$ e $v(x, t)$:

$$\begin{aligned}v_t + 2v_x + u_x &= 0 \\u_t + 4v_x - u_x &= 0.\end{aligned}$$

Aplique os métodos numéricos deste capítulo para resolver o sistema.

2. Use o método das características para resolver as equações hiperbólicas de primeira ordem, com dados iniciais:

a. $u_t + u_x + 2u = 0$, $-\infty < x < \infty$ e $t > 0$ com condição

$$u(x, 0) = \sin x, \quad -\infty < x < \infty;$$

b. $u_t + u_x + tu = 0$, $-\infty < x < \infty$ e $t > 0$ com condição

$$u(x, 0) = \sin x, \quad -\infty < x < \infty;$$

c. $u_t + \frac{t}{\sqrt{u}}u_x = 2t$, $x > 0$ e $t > 0$ com condições

$$u(x, 0) = 0, \quad x > 0$$

$$u(t, 0) = 0, \quad t > 0.$$

3. Resolva as equações diferenciais do exercício anterior usando os métodos numéricos apresentados neste capítulo.

4. Resolva a equação de Buckley-Leverett (exemplo 10.6.1) com um método conservativo e um não conservativo.

5. Compare a solução obtida no exemplo 10.6.1 com a solução obtida pelo método *upwind*, com o mesmo valor de h .

REFERÊNCIAS BIBLIOGRÁFICAS

- [Abramowitz-Stegun, 1965] ABRAMOWITZ, M. e STEGUN, I. A. (1965): *Handbook of Mathematical Functions*, National Bureau of Standards, Washington D.C.
- [Acton, 1970] ACTON, F. S. (1970): *Numerical Methods that Work*, Harper and Row.
- [Ames, 1977] AMES, W. (1977): *Numerical Methods for Differential Equations*, Academic Press.
- [Atkinson, 1988] ATKINSON, K. E. (1988): *An Introduction to Numerical Analysis*, 2nd edition, John Wiley.
- [Aziz-Settari, 1979] AZIZ, K. e SETTARI, A. (1979): *Petroleum Reservoir Simulation*, Applied Science Publishers.
- [Björk, 1996] BJÖRCK, A. (1996): *Numerical Methods for Least Square Problems*, SIAM.
- [Blum, 1972] BLUM, E. K. (1972): *Numerical Analysis and Computation: Theory and Practice*, Adison Wesley.
- [Boyce-DiPrima, 1969] BOYCE, W. e DIPRIMA, R. C. (1969): *Elementary Differential Equations and Boundary Value Problems*, Wiley.
- [Boyer, 1974] BOYER, C. B. (1974): *História da Matemática*, Edgard Blucher Ltda e EDUSP.
- [Buchanan-Turner, 1992] BUCHANAN, J. e TURNER, P. R. (1992): *Numerical Methods and Analysis*, McGraw-Hill Book Co.
- [Burden-Faires, 1988] BURDEN, R. L. e FAIRES, J. D. (1988): *Numerical Analysis*, 4th edition, PWS-Kent Publishing Co.
- [Carnahan et al., 1969] CARNAHAN, B., LUTHER, H. A. e WILKES, J. O. (1969): *Applied Numerical Methods*, John Wiley.
- [Chen et al., 1991] CHEN, W. H., DURLOFSKY, B. J., ENGQUIST, B. e OSHER, S. (1991): *Minimization of grid orientation effects through use of high order finite difference methods*, Soc. Pet. Eng., paper 22887.

- [Conte-deBoor, 1981] CONTE, S. e DEBOOR, C. (1981): *Elementary Numerical Analysis: an Algorithmic Approach*, 3rd edition, McGraw-Hill Book Co.
- [Courant, 1943] COURANT, R. (1943): Variational methods for the solution of problems and vibrations, *Bull. Amer. Math. Soc.*, 49, 1-23.
- [Courant et al., 1967] COURANT, R., FRIEDRICHS, K. O. e LEWY, H. (1967): On the partial difference equations of mathematical physics, *IBM Journal*, 11, 215-234.
- [Courant-Hilbert, 1962] COURANT, R. e HILBERT, D. (1962): *Methods of Mathematical Physics*, Interscience.
- [Dahlquist-Björck, 1974] DAHLQUIST, G. e BJÖRCK, A. (1974): *Numerical Methods*, Prentice Hall Inc.
- [Davis-Rabinowitz, 1975] DAVIS, P. J. e RABINOWITZ, P. (1975): *Methods of Numerical Integration*, Academic Press.
- [DeBoor, 1978] DEBOOR, C. (1978): *A Practical Guide to Splines*, Springer Verlag.
- [Dembo et al., 1982] DEMBO, R. S., EISENSTAT, S. C. e STEIHAUG, T. (1982): Inexact Newton Methods, *SIAM J. Num. Anal.*, 14, 400-408.
- [Dennis-Schnabel, 1996] DENNIS, J. E. e SCHNABEL (1996): *Numerical Methods for Unconstrained Optimization and Nonlinear Equation*, SIAM Classics in Applied Mathematics, 16.
- [Douglas-Rachford, 1956] DOUGLAS, J. Jr e RACHFORD, H. H. (1956): On the numerical solution of heat conduction problems in two and three space variables, *Trans. Amer. Math. Soc.*, 82, 421.
- [Duff et al., 1986] DUFF, I. S., ERISMAN, A. M. e REID, J. K. (1986): *Direct Methods for Sparse Matrices*, Clarendon Press.
- [Enander et al., 1996] ENANDER, E. P., SJÖBERG, A., MELIN, B. e ISAACSON, P. (1996): *The MATLAB Handbook*, Addison-Wesley.
- [Ewing, 1983] EWING, R. E. (1983): *The Mathematics of Reservoir Simulation*, SIAM.
- [Farlow, 1982] FARLOW, S. J. (1982): *Partial Differential Equations for Scientists and Engineers*, John Wiley.
- [Finlayson, 1992] FINLAYSON, B. A. (1992): *Numerical Methods for Problems with Moving Fronts*, Ravenna Park Pub. Inc.

- [Gear, 1971] GEAR, C. W. (1971): *Numerical Initial Value Problems in Ordinary Differential Equations*, Prentice Hall.
- [Gill et al., 1981] GILL, P., MURRAY, W. e WRIGHT, M. (1981): *Practical Optimization*, Academic Press.
- [Goldstine, 1977] GOLDSTINE, H. H. (1977): *A History of Numerical Analysis from the 16th through the 19th Century*, Springer Verlag.
- [Golub-van Loan, 1989] GOLUB, G. H. e VAN LOAN, C. F. (1989): *Matrix Computations*, John Hopkins University Press.
- [Gottlieb, 1977] GOTTLIEB, D. e ORSZAG, S. (1977): *Numerical Analysis of Spectral Methods: Theory and Applications*, SIAM.
- [Greenspan-Casulli, 1988] GREENSPAN, D. e CASULLI, V. (1988): *Numerical Analysis for Applied, Science and Engineering*, Addison-Wesley.
- [Hansselman-Littlefield, 1998] HANSSELMAN, D. e LITTLEFIELD, B. (1998): *Mastering MATLAB 5*, Prentice Hall.
- [Hadamard, 1902] HADAMARD, J. (1902): Sur les problemes aux derivees partieles et leur signification physique, *Bull. Univ. Priceton*, 13, 49-52.
- [Hansen, 1987] HANSEN, C. (1987): The SVD as a method for regularization, *BIT*, 27, 534-553.
- [Harten, 1983] HARTEN, A. (1983): High Resolution Schemes for Hyperbolic Conservation Laws, *J. Comp. Physics*, 49, 357-393.
- [Hestenes-Stiefel, 1952] HESTENES, M. R. e STIEFEL, E. (1952): *Methods of conjugate gradient for solving linear system*, *J. Res. Nat. Stds*, B49:409-436.
- [Isaacson-Keller, 1967] ISAACSON, E. e KELLER, H. B. (1967): *Analysis of Numerical Methods*, John Wiley.
- [Kambe, 1959] KAMKE, E. (1959): *Differential Gleichungen*, Academic Verlag.
- [Keller, 1968] KELLER, H. B. (1968): *Numerical Methods for Two-Point Boundary Value Problems*, Waltham.
- [Lake, 1989] LAKE, L. W. (1989): *Enhanced Oil Recovery*, Prentice Hall Inc.
- [LeVeque, 1990] LEVEQUE, R. (1990): *Numerical Methods for Conservation Laws*, Birkhäuser-Verlag.

- [Linz, 1979] LINZ, P. (1979): *Theoretical Numerical Analysis, an Introduction to Advanced Techniques*, John Wiley.
- [Mitchell-Griffiths, 1980] MITCHELL, A. R. e GRIFFITHS, D. F. (1980): *The Finite Difference Method in Partial Differential Equations*, John Wiley.
- [Noble-Daniel, 1977] NOBLE, B. e DANIEL, J. W. (1977): *Applied Linear Algebra*, 2nd edition, Prentice Hall.
- [Ortega-Rheinboldt, 1970] ORTEGA, J. M. e RHEINBOLDT, W. C. (1970): *Iterative solutions of Nonlinear Equations in Several Variables*, Academic Press.
- [Peaceman, 1977] PEACEMAN, D. W. (1977): *Fundamentals of Numerical Reservoir Simulation*, Elsevier.
- [Prenter, 1975] PRENTER, P. (1975): *Splines and Variational Methods*, John Wiley.
- [Press et al., 1990] PRESS, W. H., FLANNERY, B. P., TEUKOLSKY, S. A. e VETTERLING, W. T. (1990): *Numerical Recipes: the Art of Scientific Computing*, Cambridge Univ. Press.
- [Rall, 1969] RALL, L. B. (1969): *Computational Solution of Nonlinear Operator Equations*, John Wiley.
- [Richtmyer-Morton, 1967] RICHTMYER, R. D. e MORTON, K. W. (1967): *Difference Methods for Initial-Value Problems*, 2nd edition, John Wiley.
- [Roache-Dietrich, 1988] ROACHE P. J. e DIETRICH, D. E. (1988): Evaluation of the filtered leapfrog trapezoidal time integration method, *Num. Heat Trans.* 14, 149-164.
- [Roe, 1985] ROE, P. L. (1985): Some contributions to the modeling of discontinuous flows, *Lect. Notes Appl. Math.*, 22, 163-193.
- [Ruggiero-Lopes, 1997] RUGGIERO, M. A. G. e LOPES, V. L. R. (1997): *Calculo Numérico, Aspectos Teóricos e Computacionais*, 2^a ed., Makron Books.
- [Schultz, 1973] SCHULTZ, M. (1973): *Spline Analysis*, Prentice Hall Inc.
- [Schoenberg, 1946] SCHOENBERG, I. J. (1946): Contribution to the problem of approximation of equidistant data by analytic functions, Parts A and B, *Quart. Appl. Math.*, 4, 45-99 and 112-141.
- [Sod, 1985] SOD, G. (1985): *Numerical Methods for Fluid Dynamics*, Cambridge Univ Press.

- [Strang, 1986] STRANG, W. G. (1986): *Introduction to Applied Mathematics*, Cambridge Press.
- [Struik, 1987] STRUIK, D. (1987): *A Concise History of Mathematics*, Dover.
- [Szegő, 1959] SZEGÖ, G. (1959): *Orthogonal Polynomials*, American Mathematical Society.
- [Trefethen, 1982] TREFETHEN, L. N. (1982): Group velocity in finite difference schemes, *SIAM Reviews*, 24, 113-136.
- [VanLeer, 1974] VANLEER, B. (1974): Towards the ultimate conservative difference scheme II: monotonicity and conservation combined in a second order scheme, *J. Comput. Phys.*, 14, 361-370.
- [Varga, 1962] VARGA, R. S. (1962): *Matrix Iterative Analysis*, Prentice Hall.
- [Vichnevetsky-Bowless, 1982] VICHNEVETSKY, R. e BOWLESS, J. B. (1982): *Fourier Analysis of Numerical Approximation of Hyperbolic Equations*, SIAM.

ANEXO

gracon.m

% gracon.m - Método Gradientes Conjugados: Exemplo 3.3.1

% COMO EXECUTAR : clear;n=15;gracon

% Entrada Externa n=dimensão da matriz

l=6; % Faixa de Zeros

tol=1e-5 % Tolerância

% 1. Definindo o sistema

% 1.1 Matriz: Entrada das Diagonais

for i=2:n-1;a(i,i)=-4;a(i,i+1)=1;a(i,i-1)=1;end % Centrais

a(1,1)=-2;a(n,n)=-2;a(n,n-1)=1;a(1,2)=1;

for i=1:n-1;a(i,i+1)=1;end % Afastada Superior

for i=l+1:n;a(i,i-1)=1;end % Afastada Inferior

% 1.2 Termo Independente para $x(i)=1$, $i=1:n$

for i=2:l;b(i)=-1;end

for i=l+1:n-l;b(i)=0;end

for i=n-l+1:n-1;b(i)=-1;end;

b(1)=0;b(n)=0;

% 2. Aplicação do Método Gradiente Conjugado

% 2.1 Aproximação Inicial

for i=1:n;u(i)=0;sol(i)=1;end;u=u';sol=sol';

% 2.2 Iterações

r=b';p=b';aux=r'*r;

for k=1:200

z=a*p; v=aux/(p'*z);

x=u+v*p; r=r-v*z; aux1=r'*r;

if max(abs(x-sol))<=tol;break;end

w=aux1/aux; aux=aux1; p=r+w*p; u=x;

end

numiteracoes=k

gaunew.m

```
% gaunew.m - Gauss-Newton para Ajuste Não-Linear
% Exemplo 5.5.1
% COMO EXECUTAR - clear;gaunew

% 1.1 Definindo os Dados Usados no Ajuste
x=0:.2:2;x=x';
yx=[.45 .52 .46 .40 .29 .21 .15 .08 .04 .02 .01]';
m=length(x); % Tamanho do Vetor dos Dados

% 1.2 Precisão, Aproximação e Resíduo Iniciais
tol=1e-3; % Precisão
a=[1 0 1]'; % Aproximação Inicial
f=yx-a(1)*exp(-(x+a(2)).^2/a(3)); % Resíduo Inicial

% 2. Iterações do Método
for k=1:10

    % 2.1 Cálculo das Três Colunas da Matriz Jacobiana
    jac(:,1)=-exp(-(x+a(2)).^2/a(3));
    jac(:,2)=2*a(1)*(x+a(2))/a(3).*exp(-(x+a(2)).^2/a(3));
    jac(:,3)=-a(1)*(x+a(2)).^2/a(3)^2.*exp(-(x+a(2)).^2/a(3));

    % 2.2 Solução do Sistema Linear dos Incrementos
    ma=jac'*jac; % Matriz
    b=-jac'*f; % Termo Independente
    p=ma\b; % Solução do Sistema: Incrementos

    % 2.3 Resultados da Presente Iteração
    a=a+p; % Nova Aproximação
    f=yx-a(1)*exp(-(x+a(2)).^2/a(3)); % Resíduos na Iteração k
    somares(k)=sum(f'*f); % Soma dos Resíduos na Iteração k
    if somares(k)<=tol;break;end
end

f=yx-f; % Valores da Função Ajustada
plot(x,f,x,yx,'*')
title('Ajuste Não-Linear Usando Gauss-Newton')
```

poisson.m

```

% poisson.m ~ Solução Numérica da Equação de Poisson
% Exercício 9.3.2
% PARA EXECUTAR : poisson

% 1. Entrada de Dados
N=9;M=9; % Número de Pontos da Malha, Direções x e y
lx=1.;ly=0.5; % Arestas nas Direções x e y

% 2. Cálculos Preliminares
hx=lx/(N+1);hy=ly/(M+1); % Espaçamentos da Malha
for i=1:N;x(i)=i*hx;end % Malha na Direção x
for i=1:M;y(i)=i*hy;end % Malha na Direção y
h=hx*hx;k=hy*hy; % Para Simplificar
n=M*N; % Dimensão do Sistema

% 3. Definindo a Matriz da Discretização
for i=1:n;a(i,i)=-2*(h+k);end % Diagonal Principal
for i=1:n-1;a(i,i+1)=k;end % Diagonal Superior
for i=2:n;a(i,i-1)=k;end % Diagonal Inferior
for i=1:(M-1)*N;a(i,i+N)=h;end % Diagonal Sup. Afastada
for i=N+1:n;a(i,i-N)=h;end % Diagonal Inf. Afastada

% 4. Definindo o Termo Independente do Sistema:
% Considera-se Termo Fonte e Condição de Contorno
kk=0; % Contador
for j=1:M
    for i=1:N
        kk=kk+1;
        b(kk)=-2*h*k*(sin(pi*x(i))*cos(pi*y(j))); % Termo Fonte
        if j==1;b(kk)=b(kk)-h*sin(pi*x(i))/(pi^2);end % Cond. Cont.
        if i==1&j>1;a(kk,kk-1)=0,end
        if i==N&j<M;a(kk,kk+1)=0,end
    end;end

% 5. Encontrando a Solução
v=a\b'; % Solução do Sistema
kk=0; % Contador
for j=1:M % Transformando vetor em Matrix
    for i=1:N
        kk=kk+1;u(j,i)=v(kk); % Solução na Malha
        ue(j,i)=sin(pi*x(i))*cos(pi*y(j))/(pi^2); % Solução Analítica
    end;end

```

% 6. Erro dos Resultados

```
erro=norm(u-ue)           % Erro, Norma 2
erroinf=norm((u-ue),inf)  % Erro, Norma Sup
```

% 7. Traçando Gráficos: Soluções Aproximadas e Exatas

```
xi=linspace(0,1,50);yi=linspace(0,.5,50);
[xxi,yyi]=meshgrid(xi,yi);      % Malha Refinada p/ Solução Analítica
zzi=sin(pi*xxi).*cos(pi*yyi)/(pi^2); % Solução Analítica
[xx,yy]=meshgrid(x,y);          % Malha para Traçado dos Gráficos
plot3(xx,yy,u,'*')              % Gráfico das Aproximações Obtidas
hold on
mesh(xxi,yyi,zzi)               % Gráfico da Solução Exata
hold off
```

calor.m

```
% calor.m - Solução Numérica da Equação de Difusão
% Exemplo 9.4.1
% PARA EXECUTAR : clear;metodo=1;end

% 1. Entrada de Dados
tf=1;           % Tempo Final
h=0.1;          % Espaçamento da Malha
lam=0.5;alfa=1; % Dados da Discretização

% 2. Cálculos Preliminares
x=0:h:1;nx=length(x); % Definindo Malha em x
nt=tf/(lam*h*h/alfa)   % Número de Passos no Tempo
n=nx-2;                % Dimensão das Matrizes
ue=exp(-pi*pi*tf)*sin(pi*x); % Solução Analítica

for i=1:nx;v(i)=sin(pi*x(i));end;ui=v; % Condição Inicial

% 3. Solução Usando Métodos Explícito ( se método=1),
% Implícito (se método=2) e Crank-Nicolson (se método=3)

if metodo==1;           % Método 1: Explícito
    for k=1:nt
        for i=2:nx-1
            u(i)=(1-2*lam)*v(i)+lam*(v(i+1)+v(i-1));
        end;u(1)=0;u(nx)=0;v=u;
    end;uexp=u;          % Solução Método Explícito em tf
    erroexp=norm(uexp-ue)/(nx*max(ue)) % Erro Relativo Explícito
elseif metodo==2;       % Método 2: Implícito
    for i=1:n;d(i)=1+2*lam;end % Diagonal Principal
    for i=1:n-1;c(i)=-lam;end % Diagonal Superior
    for i=2:n;a(i)=-lam;end % Diagonal Inferior
    % Eliminação: Trocar diagonal p/ m(i) p/ usar em b
    for i=1:n-1;m(i+1)=a(i+1)/d(i);d(i+1)=d(i+1)-m(i+1)*c(i);end

    for k=1:nt           % Evoluindo no Tempo
        for i=1:n;b(i)=v(i+1);end % Termo Independente
        for i=1:n-1      % Resolver Sistema Tridiagonal
            b(i+1)=b(i+1)-m(i+1)*b(i);end
            s(n)=b(n)/d(n);
            for i=n-1:-1:1
                s(i)=(b(i)-c(i)*s(i+1))/d(i);end
            end
        end
    end
end
```

```

v(1)=0;v(2:nx-1)=s;v(nx)=0; % Incluir Condições Contorno
end;uimp=v; % Solução em t=tf
erroimp=norm(uimp-ue)/(nx*max(ue)) % Erro Relativo Implícito

elseif metodo==3; % Método 3: Crank-Nicolson
for i=1:n;d(i)=1+lam;end % Diagonal Principal
for i=1:n-1;c(i)=-lam/2;end % Diagonal Superior
for i=2:n;a(i)=-lam/2;end % Diagonal Inferior
% Eliminação: Trocar diagonal p/ m(i) p/ usar em b
for i=1:n-1;m(i+1)=a(i+1)/d(i);d(i+1)=d(i+1)-m(i+1)*c(i);end

for k=1:nt % Evoluir no Tempo
b(1)=(1-lam)*v(2)+lam*v(3)/2; % Termo Independente
for i=2:n-1;
b(i)=(1-lam)*v(i+1)+(v(i)+v(i+2))*lam/2;end
b(n)=lam*v(nx-2)/2+(1-lam)*v(nx-1);
for i=1:n-1 % Resolver Sistema Tridiagonal
b(i+1)=b(i+1)-m(i+1)*b(i);end
s(n)=b(n)/d(n);
for i=n-1:-1:1
s(i)=(b(i)-c(i)*s(i+1))/d(i);end
v(1)=0;v(2:nx-1)=s;v(nx)=0; % Incluir Condições Contorno
end;ucn=v; % Solução em t=tf
errocn=norm(ucn-ue)/(nx*max(ue)) % Erro Rel. Crank-Nicolson
end

% 7. Traçado de Gráficos
if metodo==1;
plot(x,uexp,x,ue,'.');
title('Método Explícito e Solução Exata (.)')
elseif metodo==2;
plot(x,uimp,x,ue,'.');
title('Método Implícito e Solução Exata (.)')
elseif metodo==3;
plot(x,ucn,x,ue,'.');
title('Método Crank-Nicolson e Solução Exata (.)')
end

```

uplax.m

```
% uplax.m - Discretização Equação Advecção Linear - Exemplo 10.3.1
% Métodos: Upwind, Lax-Friedrichs, Lax-Wendroff, Beam-Warming
% COMO EXECUTAR : clear;uplax

% Entrada de Dados
a=2; % Equação:  $du/dt + a \cdot du/dx = 0$ 
tf=.3; % Tempo Final
h=1/200; x=0:h:1; n=length(x); % Discretização em x
lam=.1; %  $\lambda = \Delta t / \Delta x$ 
dt=lam*h; k1=tf/dt; % Passos no Tempo

u0(1)=1; for i=2:n; u0(i)=2; end % Condição Inicial

% Solução exata no Tempo tf
for i=1:n; if x(i) <= 2*tf; ue(i)=1; else; ue(i)=2; end; end

% Cálculos Para os Quatro Métodos
for km=1:4 % Contador do Método
    u=u0;
    if km==1; for k=1:k1; for i=2:n-1 % Upwind
        v(i)=u(i)-lam*a*(u(i)-u(i-1)); end
        v(1)=1; v(n)=v(n-1); u=v; end; uup=u;
    elseif km==2; for k=1:k1; for i=2:n-1 % Lax-Friedrichs
        v(i)=.5*(u(i-1)+u(i+1))-0.5*lam*a*(u(i+1)-u(i-1)); end
        v(1)=1; v(n)=v(n-1); u=v; end; ulf=u;
    elseif km==3; for k=1:k1; for i=2:n-1 % Lax-Wendroff
        v(i)=u(i)-.5*lam*a*(u(i+1)-u(i-1));
        v(i)=v(i)+.5*a*lam*(u(i-1)-2*u(i)+u(i+1)); end
        v(1)=1; v(n)=v(n-1); u=v; end; ulw=u;
    elseif km==4; for k=1:k1; for i=3:n-1 % Beam-Warming
        v(i)=u(i)-.5*lam*a*(3*u(i)-4*u(i-1)+u(i-2));
        v(i)=v(i)+.5*a*lam*(u(i-1)-2*u(i)+u(i+1)); end
        v(1)=1; v(2)=1; v(n)=v(n-1); u=v; end; ubw=u;
    end
end

% Traçado de Gráficos
subplot(2,2,1); plot(x,ue,x,uup,'.',0,.5,0,2.5); title('Upwind')
subplot(2,2,2); plot(x,ue,x,ulf,'.',0,.5,0,2.5); title('Lax-Friedrichs')
subplot(2,2,3); plot(x,ue,x,ulw,'.',0,.5,0,2.5); title('Lax-Wendroff')
subplot(2,2,4); plot(x,ue,x,ubw,'.',0,.5,0,2.5); title('Beam-Warming')
```

burger.m

```
% burger.m - Discretização Burger
% Métodos Não Conservativo e Conservativo
% Exemplo 10.5.1
% COMO EXECUTAR : clear;burger

% Entrada de Dados
h=1/100;x=0:h:1;n=length(x);      % Malha em x
lam=.1;
tfi=.6;                             % Tempo Final
dt=lam*h;k1=tfi/dt;                % Passos no Tempo

ue=2;ud=.5;                         % Estados à esquerda, à direita
mc=(ue+ud)/2;                       % Velocidade do choque
u0(1)=ue;for i=2:n;u0(i)=ud;end     % Condição Inicial

% Solução exata
for i=1:n;if x(i)<=mc*tfi;uex(i)=ue;else;uex(i)=ud;end;end

for km=1:2
    u=u0;
    if km==1;for k=1:k1;for i=2:n-1    % Upwind Não Conservativo
        v(i)=u(i)-lam*u(i)*(u(i)-u(i-1));end
        v(1)=ue;v(n)=v(n-1);u=v;end;unc=u;
    elseif km==2;for k=1:k1;for i=2:n-1 % Upwind Conservativo
        v(i)=u(i)-.5*lam*(u(i)^2-u(i-1)^2);end
        v(1)=ue;v(n)=v(n-1);u=v;end;uc=u;
    end
end

plot(x,uex,x,unc,'.',x,uc,0,0,0,2.5)
title('Burger: Conservativo(-) e Não Conservativo (.)')
xlabel('Distância, x')
ylabel('Soluções, u(x,tfi)')
```

```
% bltvd.m - Equação Buckley-Leverett
% Método TVD; Referência: Fynlaysson
% Exercício 10.6.1
% COMO EXECUTAR: clear;bltvd

% Entrada de Dados
h=.01;x=0:h:1;n=length(x);      % Malha em x
tf=.4;                            % Tempo Final
lam=.05;m=tf/(lam*h);            % Passos no Tempo
for i=1:n;u(i)=0;end;u(1)=1;    % Condição Inicial

% Evolução no Tempo
for j=1:m
    f=u.^3./(u.^3+2*(1-u).^3/3); % Função Fluxo
    delta(1)=0;delta(n)=0;
    for i=2:n-1
        aux1=u(i)-u(i-1);
        aux2=min([abs(aux1) (u(i+1)-u(i))*sign(aux1)]);
        aux2=max([0 aux2]);
        delta(i)=sign(aux1)*aux2;
    end

    umed=u+.5*delta;              % Cálculo de umed=u(i+1/2)
    fmed=umed.^3./(umed.^3+2*(1-umed).^3/3);
    du=diff(umed)/h;du(n)=0;

    us(1)=u(1);us(n)=u(n);       % Cálculo de us=u*(i+1/2)
    for i=2:n-1;us(i)=u(i)-lam*(fmed(i)-fmed(i-1));end
    for i=2:n-1
        aux1=us(i)-us(i-1);
        aux2=min([abs(aux1) (us(i+1)-us(i))*sign(aux1)]);
        aux2=max([0 aux2]);
        delta(i)=sign(aux1)*aux2;
    end

    umed=us+.5*delta;
    fsmed=umed.^3./(umed.^3+2*(1-umed).^3/3);
    du=diff(umed)/h;du(n)=0;
    for i=2:n-1;                % Cálculo Final da Solução
        u(i)=u(i)-.5*lam*(fmed(i)-fmed(i-1)+fsmed(i)-fsmed(i-1));
    end;u(1)=1;u(n)=0;
```

```

% Escolhendo Alguns Níveis do Tempo para Gráficos
if j==200;u1=u;elseif j==400;u2=u;elseif j==600;u3=u;
elseif j==800;u4=u;end
end

plot(x,u1,x,u2,x,u3,x,u4,x,u)
title('Solução da Equação Buckley-Leverett: TVD')
xlabel('Distância, x')
ylabel('Perfis de Saturação')

```
